

1. 创建前端项目

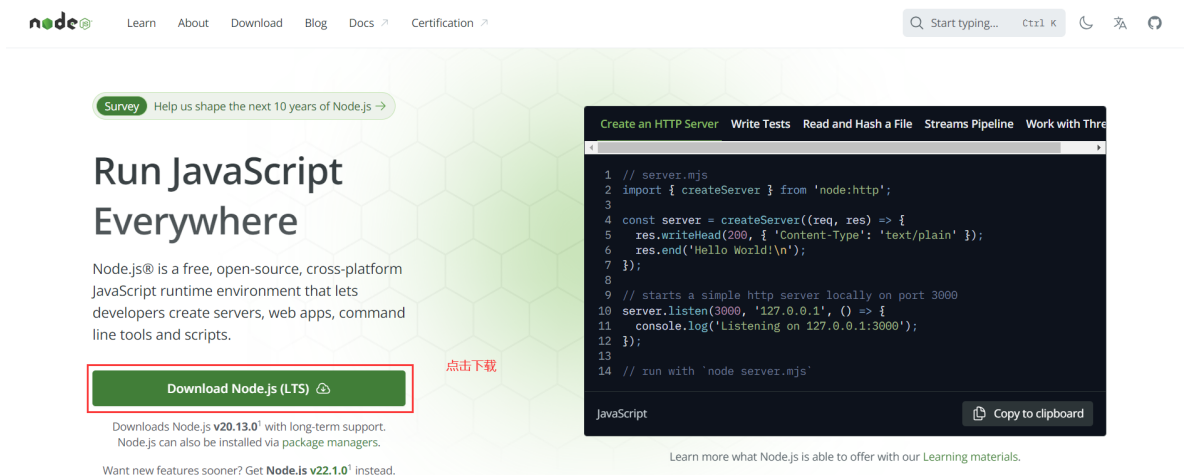
需要具备的技能储备：

```
vue3
elementui Plus
ts
```

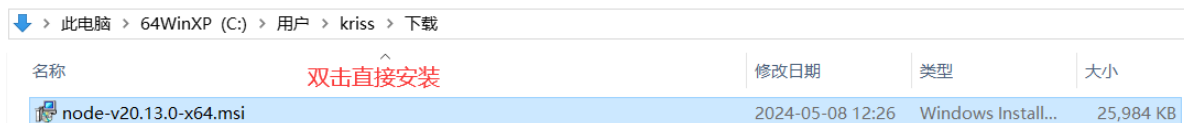
1.1 准备nodejs环境

我们使用vite创建项目，而vite需要nodejs的环境，建议大家nodejs的版本大于等于16以上。大家如果没有安装nodejs，先安装nodejs环境。nodejs环境安装如下：

第一步：打开nodejs官网(<https://nodejs.org/en/>)，并下载nodejs的安装包。

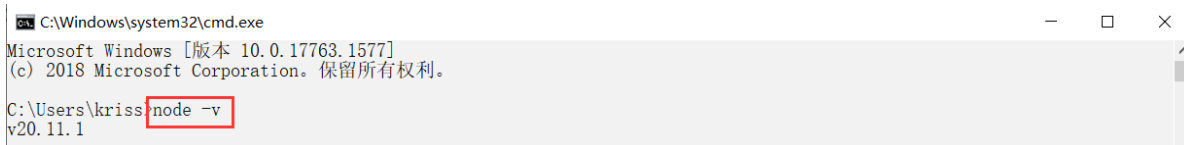


第二步：安装nodejs



第三步：查看按照版本

打开cmd，输入node -v,查看版本信息：



1.2 新建项目

创建存放项目目录，cmd 进入到存放目录，执行如下命令创建项目：

```
npm init vite@latest
```

```
npm init vite@latest
Microsoft Windows [版本 10.0.17763.1577]
(c) 2018 Microsoft Corporation。保留所有权利。

C:\Users\kriss\Desktop\springboot+vue项目\前端>npm init vite@latest
Need to install the following packages:
create-vite@5.2.3
Ok to proceed? (y)
```

如下图所示，输入 y 回车：

```
Microsoft Windows [版本 10.0.17763.1577]
(c) 2018 Microsoft Corporation。保留所有权利。

C:\Users\kriss\Desktop\springboot+vue项目\前端>npm init vite@latest
Need to install the following packages:
create-vite@5.2.3
Ok to proceed? (y) y
? Project name: » vite-project
```

输入项目名称 gymnasium-project：

```
C:\Users\kriss\Desktop\springboot+vue项目\前端>npm init vite@latest
Need to install the following packages:
create-vite@5.2.3
Ok to proceed? (y) y
? Project name: ... gymnasium-project
? Select a framework: » - Use arrow-keys. Return to submit.
> Vanilla
  Vue
  React
  Preact
  Lit
  Svelte
  Solid
  Qwik
  Others
```

选择vue框架：

```
✓ Project name: ... gymnasium-project
? Select a framework: » - Use arrow-keys. Return to submit.
> Vanilla
  Vue
  React
  Preact
  Lit
  Svelte
  Solid
  Qwik
  Others
```

```
C:\Users\kriss\Desktop\springboot+vue项目\前端>npm init vite@latest
✓ Project name: ... gymnasium-project
✓ Select a framework: » Vue
? Select a variant: » - Use arrow-keys. Return to submit.
> TypeScript
  JavaScript
  Customize with create-vue ↗
  Nuxt ↗
```

最终结果如下：

```
C:\Users\kriss\Desktop\springboot+vue项目\前端>npm init vite@latest
✓ Project name: ... gymnasium-project
✓ Select a framework: » Vue
✓ Select a variant: » TypeScript

Scaffolding project in C:\Users\kriss\Desktop\springboot+vue项目\前端\gymnasium-project...

Done. Now run:

cd gymnasium-project
npm install
npm run dev

C:\Users\kriss\Desktop\springboot+vue项目\前端>
```

cd 进入刚刚创建的项目,执行如下命令:

```
C:\Users\kriss\Desktop\springboot+vue项目\前端>cd gymnasium-project
C:\Users\kriss\Desktop\springboot+vue项目\前端\gymnasium-project>npm install
[ sill idealTree:gymnasium-project: sill idealTree buildDeps

C:\Users\kriss\Desktop\springboot+vue项目\前端>cd gymnasium-project
C:\Users\kriss\Desktop\springboot+vue项目\前端\gymnasium-project>npm install
added 43 packages, and audited 44 packages in 12s

5 packages are looking for funding
  run `npm fund` for details

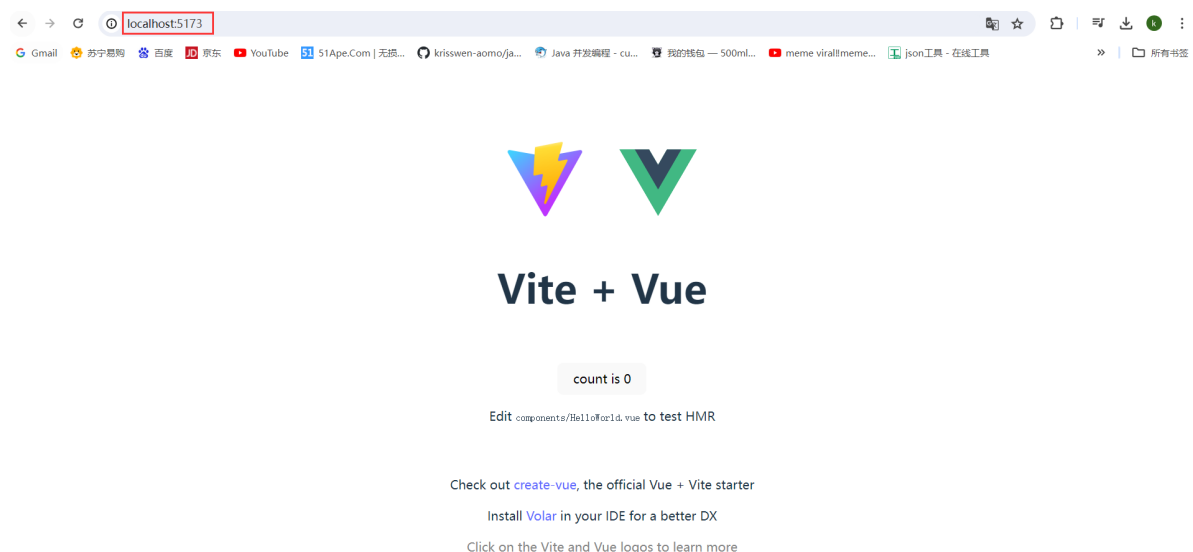
found 0 vulnerabilities

C:\Users\kriss\Desktop\springboot+vue项目\前端\gymnasium-project>npm run dev
> gymnasium-project@0.0.0 dev
> vite

VITE v5.2.11 ready in 656 ms
  ➜ Local:   http://localhost:5173/
  ➜ Network: use --host to expose
  ➜ press h + enter to show help
```

1.3 运行项目

当我们运行完npm run dev之后, 可以看到访问前端工程的地址: <http://localhost:5173/>。在浏览器输入并访问:



1.4 安装路由

1.4.1 项目配置别名

1.4.1.1 vite.config.js配置文件, 添加如下配置

```
server: {
  host: '0.0.0.0', // 任何ip都可以访问
  port: 8080, // 项目端口号
  hmr: true, //开启热加载
  open: true //自动打开浏览器
}
```

```
vite.config.ts > | default > server
1  import { defineConfig } from 'vite'
2  import vue from '@vitejs/plugin-vue'
3
4  // https://vitejs.dev/config/
5  export default defineConfig({
6    plugins: [vue()],
7    server: {
8      host: '0.0.0.0', // 任何ip都可以访问
9      port: 8080, // 项目端口号
10     hmr: true, // 开启热加载
11     open: true // 自动打开浏览器
12   }
13 })
```

1.4.1.2 vite 配置别名

安装所需要的相关依赖：

```
npm install @types/node --save-dev
```

在vite.config.ts里面定义：

```
resolve: {
  alias: [
    {
      find: '@',
      replacement: resolve(__dirname, 'src')
    }
  ]
}
```

```
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'
import { resolve } from 'path'

// https://vitejs.dev/config/
export default defineConfig({
  plugins: [vue()],
  server: {
    host: '0.0.0.0', // 任何ip都可以访问
    port: 8080, // 项目端口号
    hmr: true, // 开启热加载
    open: true // 自动打开浏览器
  },
  resolve: {
    alias: [
      {
        find: '@',
        replacement: resolve(__dirname, 'src')
      }
    ]
  }
})
```

1.4.1.3 tsconfig.json里面添加如下代码

添加baseUrl和paths


```

"baseUrl": ".", // paths 路径解析起点
"paths": {
  "@/*": [ // 别名路径设置
    "src/*"
  ]
}

```

```

{
  "compilerOptions": {
    "target": "ES2020",
    "useDefineForClassFields": true,
    "module": "ESNext",
    "lib": ["ES2020", "DOM", "DOM.Iterable"],
    "skipLibCheck": true,

    /* Bundler mode */
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
    "resolveJsonModule": true,
    "isolatedModules": true,
    "noEmit": true,
    "jsx": "preserve",

    /* Linting */
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noFallthroughCasesInSwitch": true,
    "baseUrl": ".", // paths 路径解析起点
    "paths": {
      "@/*": [ // 别名路径设置
        "src/*"
      ]
    }
  },
  "include": ["src/**/*.ts", "src/**/*.tsx", "src/**/*.vue"],
  "references": [{ "path": "./tsconfig.node.json" }]
}

```

1.4.2 安装路由

1.4.2.1 安装路由所需要的依赖

```
npm install vue-router@4
```

1.4.2.2 新建路由文件

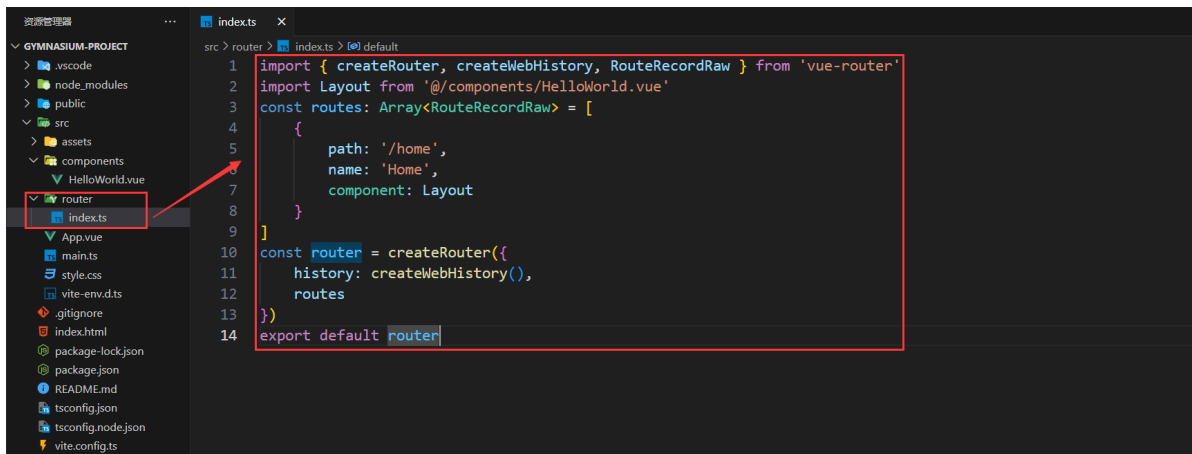
src目录下新建router文件夹，然后新建index.ts文件

```

import { createRouter, createWebHistory, RouteRecordRaw } from
'vue-router'
import Layout from '@components/Helloworld.vue'
const routes: Array<RouteRecordRaw> = [
  {
    path: '/home',
    name: 'Home',
    component: Layout
  }
]

```

```
const router = createRouter({
  history: createWebHistory(),
  routes
})
export default router
```



1.4.2.3 main.ts里面引入路由

```
import { createApp } from 'vue'
import App from './App.vue'
import router from './router/index'
const app = createApp(App);
app.use(router)
app.mount('#app')
```

1.4.2.4 修改App.vue HelloWorld.vue为如下所示

App.vue

```
<script setup lang="ts">

</script>

<template>
  <router-view/>
</template>

<style scoped>

</style>
```

HelloWorld.vue

```
<script setup lang="ts">

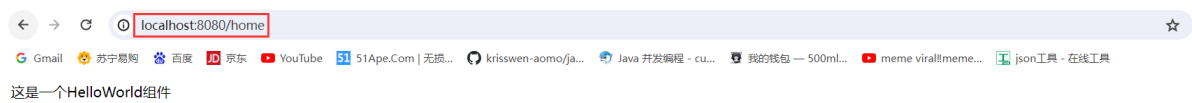
</script>

<template>
  <div>这是一个HelloWorld组件</div>
</template>

<style scoped>

</style>
```

启动项目并测试：



1.5 安装element-plus

官网： <https://element-plus.org/zh-CN/#/zh-CN>

1.5.1 安装element-plus

```
npm install element-plus --save
npm install @element-plus/icons-vue
```

```
PS C:\Users\kriss\Desktop\springboot+vue项目\前端\gymnasium-project> npm install element-plus --save
added 23 packages, and audited 71 packages in 28s

11 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\kriss\Desktop\springboot+vue项目\前端\gymnasium-project> npm install @element-plus/icons-vue
up to date, audited 71 packages in 2s

11 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

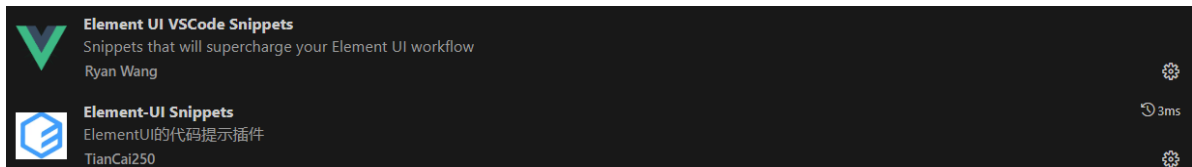
1.5.2 在main.ts中引入elementui-plus

```

import { createApp } from 'vue'
import App from './App.vue'
import router from '@router/index'
//引入element plus
import ElementPlus from 'element-plus'
import 'element-plus/dist/index.css'
import * as ElementPlusIconsVue from '@element-plus/icons-vue'
const app = createApp(App);
app.use(ElementPlus).use(router).mount('#app')
//全局注册图标组件
for (const [key, component] of
Object.entries(ElementPlusIconsVue)) {
  app.component(key, component)
}

```

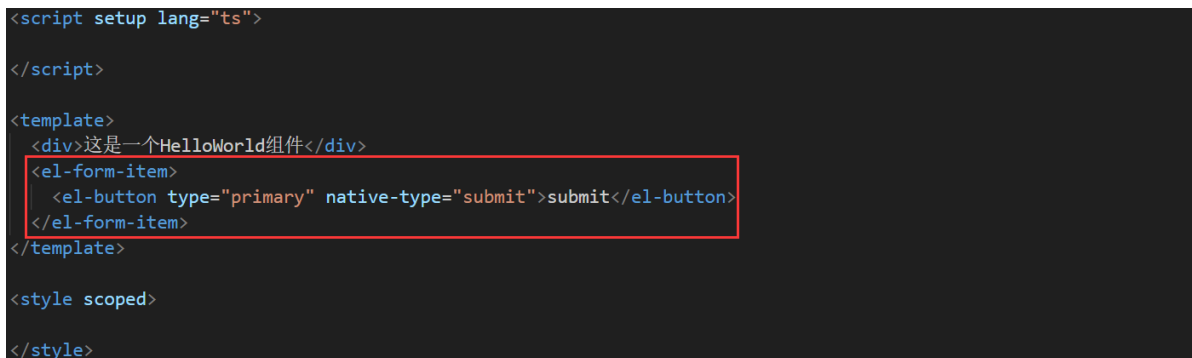
1.5.3 安装Element UI相关的插件



安装插件之后，我们写element-plus相关的代码才会有提示。

现在我们来测试一下在项目中是否成功引入了elementUI-plus的环境。

在HelloWorld.vue中输入如下代码(直接elb回车):

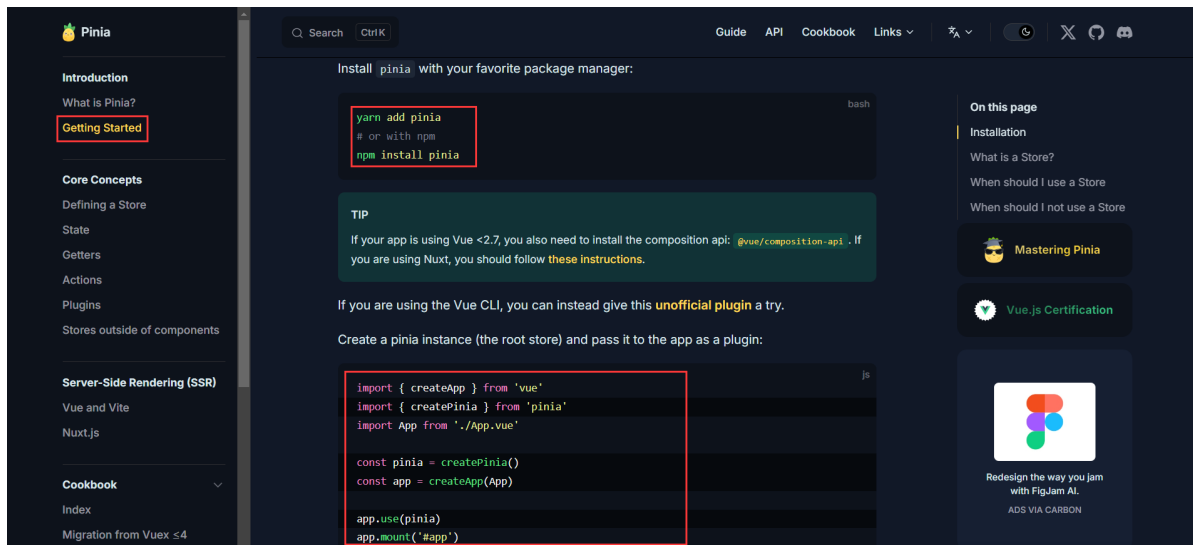


再次访问浏览器：



1.6 初识Pinia与安装

Pinia官网: <https://pinia.vuejs.org/>



1.6.1 安装依赖

```
npm install pinia
```

1.6.2 在main.ts引入pinia

```
//引入Pinia构造函数
import { createPinia } from 'pinia'
// 实例化 Pinia
const pinia = createPinia()
const app = createApp(App);
app.use(ElementPlus).use(router).use(pinia).mount('#app')
```

```
import { createApp } from 'vue'
import App from './App.vue'
import router from '@router/index'
//引入element plus
import ElementPlus from 'element-plus'
import 'element-plus/dist/index.css'
import * as ElementPlusIconsVue from '@element-plus/icons-vue'
//引入Pinia构造函数
import { createPinia } from 'pinia'
// 实例化 Pinia
const pinia = createPinia()
const app = createApp(App);
app.use(ElementPlus).use(router).use(pinia).mount('#app')
//全局注册图标组件
for (const [key, component] of Object.entries(ElementPlusIconsVue)) {
  app.component(key, component)
}
```

1.6.3 使用Pinia

1.6.3.1 定义仓库

在src下新建store,然后store下新建test目录, 并建index.ts文件

```
import { defineStore } from 'pinia'
//定义store
export const testStore = defineStore('testStore', {
  state: () => {
    return {
      count: 0
    }
  },
  getters: {
    getCount(state) {
      return state.count
    }
  },
  actions: {
    setCount(count: number) {
      console.log(count)
      this.count = count;
    }
  }
})
```

1.6.3.2 修改HelloWorld.vue

```
<template>
  <div>这是一个HelloWorld组件</div>
  <el-form-item>
    <el-button type="primary" native-type="submit"
    @click="addBtn">按钮1</el-button>
  </el-form-item>
  <h1>{{ store.count }}</h1>
</template>

<script setup lang="ts">
import { testStore } from '@/store/test/index'
const store = testStore();
const addBtn = () => {
  //store.count++;
  store.setCount(++store.count)
}
```

```
</script>

<style scoped>

</style>@/store/test/index
```

```
<template>
  <div>这是一个HelloWorld组件</div>
  <el-form-item>
    <el-button type="primary" native-type="submit" @click="addBtn">按钮1</el-button>
  </el-form-item>
  <h1>{{ store.count }}</h1>
</template>

<script setup lang="ts">
import { testStore } from '@/store/test/index'
const store = testStore();
const addBtn = () => {
  //store.count++;
  store.setCount(++store.count)
}
</script>

<style scoped>

</style>@/store/test/index
```

然后访问页面，点击按钮，查看数据变化效果。

1.7 主界面布局

1.7.1 安装sass

Sass (Syntactically Awesome Style Sheets) 是一种CSS预处理器，用于增强CSS的功能和灵活性。它扩展了CSS，并引入了许多有用的功能，如变量、嵌套、混合、继承以及模块化的结构。

```
npm install --save-dev sass
```

1.7.2 找到index.html添加如下style里面的样式

```
<style>
  html,body,#app{
    padding: 0px;
    margin: 0px;
    height: 100%;
    box-sizing: border-box;
  }
</style>
```

1.7.3 src目录下新建layout目录，并新建Index.vue主页面组件

从elementUI plus官网上拷贝相关代码：



```
<template>
  <el-container class="layout">
    <el-aside width="200px" class="asside">Aside</el-aside>
    <el-container>
      <el-header class="header">Header</el-header>
      <el-main class="main">Main</el-main>
    </el-container>
  </el-container>
</template>
<script></script>
<style lang="scss">
.layout {
  height: 100%;
  .asside {
    background-color: blueviolet;
  }
  .header {
    background-color: chocolate;
  }
  .main {
    background-color: darkcyan;
  }
}
</style>
```

App.vue修改为如下所示:


```

<script setup lang="ts">

</script>

<template>
  <router-view/>
</template>

<style lang="scss">

</style>

```

lang="scss": 这表示该 `<style>` 标签中使用的是SCSS语法编写的样式。SCSS是Sass（Syntactically Awesome Stylesheets）的一种语法，它是一种CSS预处理器，提供了许多有用的功能，如变量、嵌套规则、混合宏等，这些功能可以使CSS的编写更加简洁和强大。lang="scss"属性告诉Vue的构建系统（如Webpack），在将这个组件编译为最终的HTML和CSS之前，需要先用一个SCSS编译器处理这里的内容。

1.7.4 在router中引入主页面组件

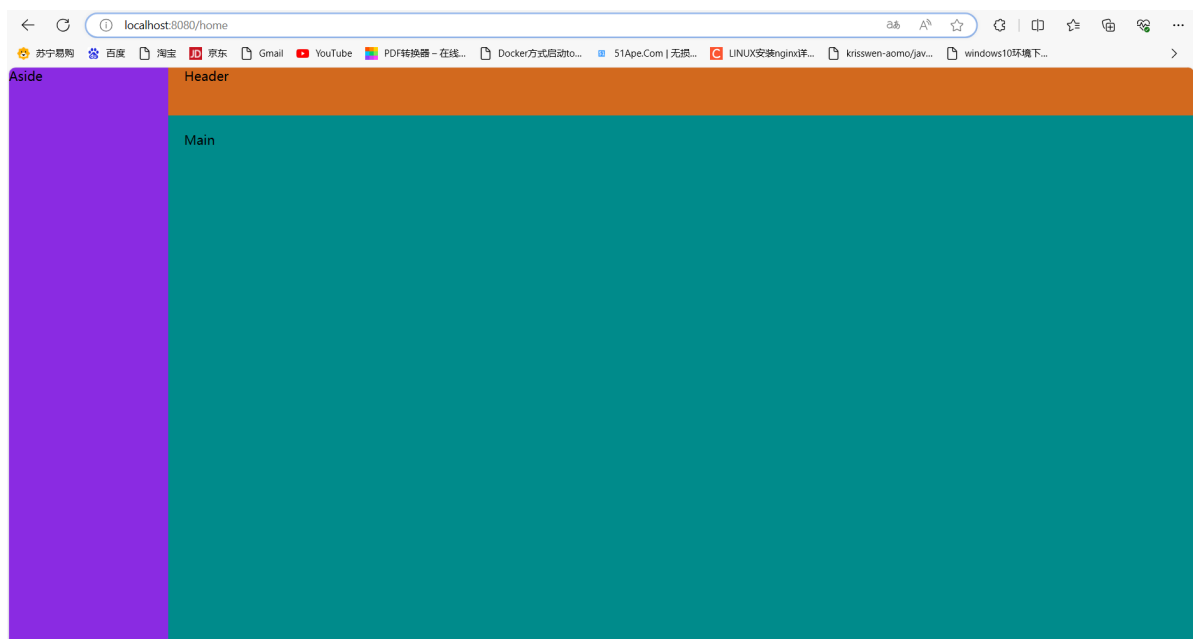
在router目录下的index.ts中修改Layout组件路径。

```

import { createRouter, createWebHistory, RouteRecordRaw } from 'vue-router'
import Layout from '@layout/Index.vue'
const routes: Array<RouteRecordRaw> = [
  {
    path: '/home',
    name: 'Home',
    component: Layout
  }
]
const router = createRouter({
  history: createWebHistory(),
  routes
})
export default router

```

启动项目，查看效果：



1.8 左侧导航菜单制作

elementUI plus导航菜单对应官网: <https://element-plus.org/zh-CN/component/menu.html>

1.8.1 创建Header组件

layout下新建header目录, 然后新建Header.vue组件

```
<template>
  <div>头部</div>
</template>
<script setup lang="ts">
import { ref, reactive } from 'vue'

</script>
<style scoped lang='scss'>

</style>
```

1.8.2 创建MenuBar.vue和MenuItem.vue组件

在layout下新建menu目录, 然后新建MenuBar.vue和MenuItem.vue组件

(1) MenuBar.vue组件

```
<template>
  <el-menu
    default-active="2"
    class="el-menu-vertical-demo"
    :collapse="isCollapse"
    unique-opened
    background-color="#304156"
  >
    <menu-item :menuList="menuList"></menu-item>
  </el-menu>
</template>
<script setup lang="ts">
import { ref, reactive } from "vue";
import MenuItem from "@layout/menu/MenuItem.vue";
const isCollapse = ref(false);
let menuList = reactive([
  {
    path: "/system",
    component: "Layout",
```

```
name: "system",
meta: {
  title: "系统管理",
  icon: "Setting",
  roles: ["sys:manage"],
},
children: [
  {
    path: "/userList",
    component: "/system/User/UserList",
    name: "userList",
    meta: {
      title: "员工管理",
      icon: "UserFilled",
      roles: ["sys:user"],
    },
  },
  {
    path: "/roleList",
    component: "/system/Role/RoleList",
    name: "roleList",
    meta: {
      title: "角色管理",
      icon: "wallet",
      roles: ["sys:role"],
    },
  },
  {
    path: "/menuList",
    component: "/system/Menu/MenuList",
    name: "menuList",
    meta: {
      title: "菜单管理",
      icon: "Menu",
      roles: ["sys:menu"],
    },
  },
],
},
{
  path: "/memberRoot",
  component: "Layout",
  name: "memberRoot",
  meta: {
    title: "会员管理",
```

```

        icon: "Setting",
        roles: ["sys:memberRoot"],
    },
    children: [
        {
            path: "/cardType",
            component: "/member/CardType",
            name: "cardType",
            meta: {
                title: "会员卡类型",
                icon: "UserFilled",
                roles: ["sys:cardType"],
            },
        },
        {
            path: "/memberList",
            component: "/member/MemberList",
            name: "memberList",
            meta: {
                title: "会员管理",
                icon: "wallet",
                roles: ["sys:memberList"],
            },
        },
        {
            path: "/myFee",
            component: "/system/FeeList",
            name: "myFee",
            meta: {
                title: "我的充值",
                icon: "Menu",
                roles: ["sys:myFee"],
            },
        },
    ],
},
]);
</script>
<style scoped lang="scss">
.el-menu-vertical-demo:not(.el-menu--collapse) {
    width: 230px;
    min-height: 400px;
}
.el-menu {
    border-right: none;

```

```

}
:deep(.el-sub-menu .el-sub-menu__title){
    color: #f4f4f5 !important;
}
:deep(.el-menu .el-menu-item){
    color: #bfc9d9;
}
/* 菜单点中文字的颜色 */
:deep(.el-menu-item.is-active){
    color: #409eff !important;
}
/* 当前打开菜单的所有子菜单颜色 */
:deep(.is-opened .el-menu-item){
    background-color: #1f2d3d !important;
}
/* 鼠标移动菜单的颜色 */
:deep(.el-menu-item:hover){
    background-color: #001528 !important;
}
</style>

```

(2) MenuItem.vue组件

```

<template>
    <template v-for="menu in menuList" :key="menu.path">
        <el-sub-menu v-if="menu.children && menu.children.length
> 0"
            :index="menu.path">
                <template #title>
                    <el-icon>
                        <component :is="menu.meta.icon"></component>
                    </el-icon>
                    <span>{{ menu.meta.title }}</span>
                </template>
                <menu-item :menuList="menu.children"></menu-item>
            </el-sub-menu>
            <el-menu-item style="color: #f4f4f5" v-else
:index="menu.path">
                <el-icon>
                    <component :is="menu.meta.icon"></component>
                </el-icon>
                <template #title>{{ menu.meta.title }}</template>
            </el-menu-item>
        </template>
    </template>

```

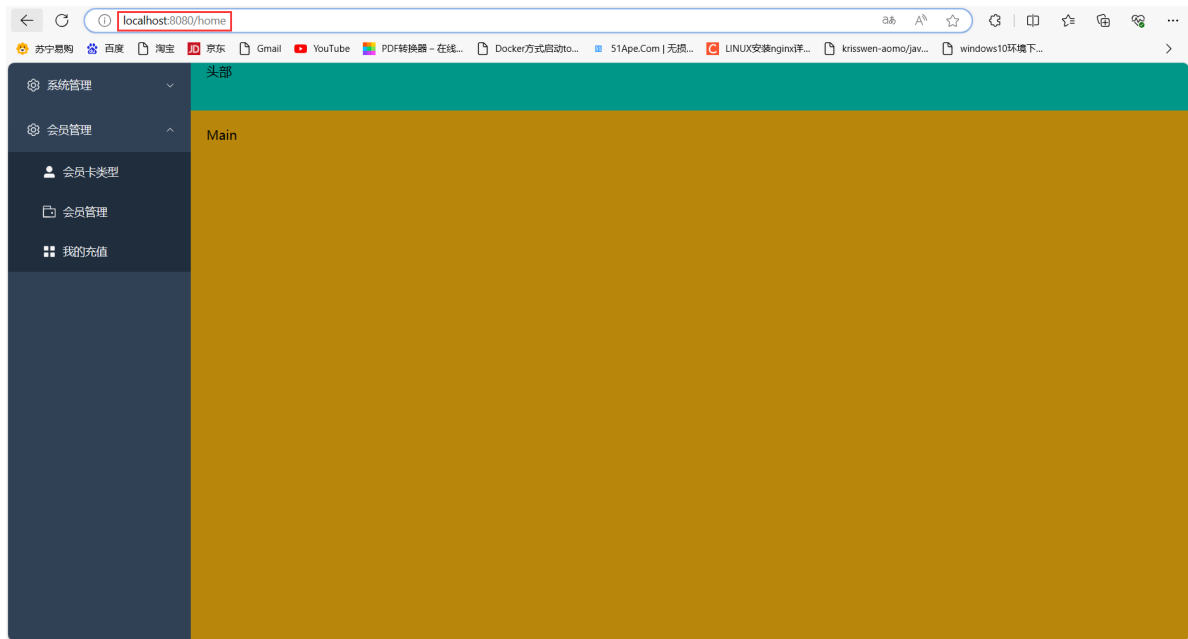
```
<script setup lang="ts">
  defineProps(["menuList"]);
</script>
<style scoped>
</style>
```

1.8.3 修改layout/Index.vue组件

```
<template>
  <div class="full-screen">
    <el-container class="layout">
      <el-aside width="auto" class="asside">
        <menu-bar></menu-bar>
      </el-aside>
      <el-container>
        <el-header class="header">
          <Header></Header>
        </el-header>
        <el-main class="main">Main</el-main>
      </el-container>
    </el-container>
  </div>
</template>
<script setup lang="ts">
import Header from "@/layout/header/Header.vue";
import MenuBar from "@/layout/menu/MenuBar.vue";
</script>
<style lang="scss">
.full-screen {
  height: 100vh; /* 视口高度 */
  width: 100vw; /* 视口宽度 */
}
.layout {
height: 100%;
.asside {
// background-color: blueviolet;
background-color: #304156;
}
.header {
background-color: #009688;
}
.main {
background-color: darkgoldenrod;
}
}
```

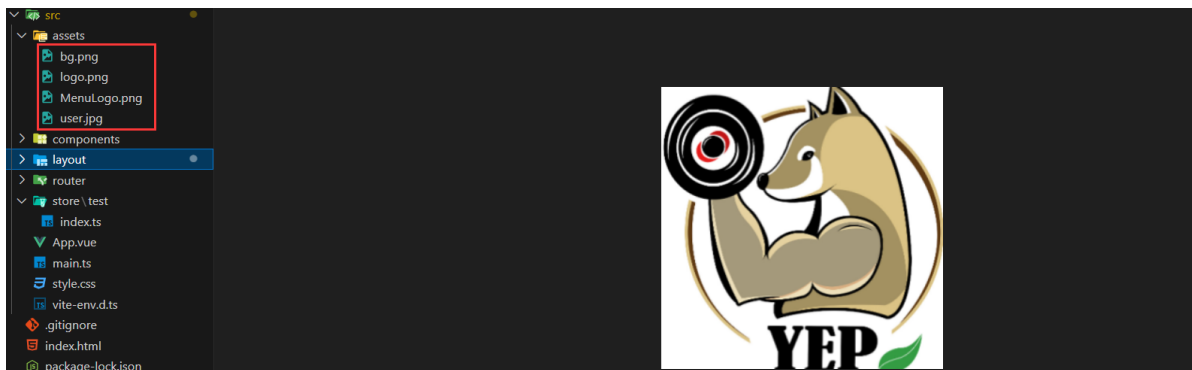
```
</style>
```

最后启动项目查看效果：



1.9 菜单logo制作

1.9.1 项目assets里面加入logo



1.9.2 layout/menu下新建MenuLogo.vue组件

```
<template>
  <div class="logo">
    
    <span class="logo-title">{{ title }}</span>
  </div>
</template>
<script setup lang="ts">
import MenuLogo from '@assets/MenuLogo.png'
import { ref } from "vue";
const title = ref("酷我运动管理系统");
</script>
<style lang="scss" scoped>
```

```

.logo {
  display: flex;
  width: 100%;
  height: 60px;
  line-height: 60px;
  background: #2b2f3a;
  text-align: center;
  cursor: pointer;
  align-items: center;
  img {
    width: 36px;
    height: 36px;
    margin-right: 12px;
    margin-left: 20px;
  }
  .logo-title {
    color: #fff;
    font-weight: 800;
    line-height: 60px;
    font-size: 22px;
    font-family: FangSong;
  }
}
}
</style>

```

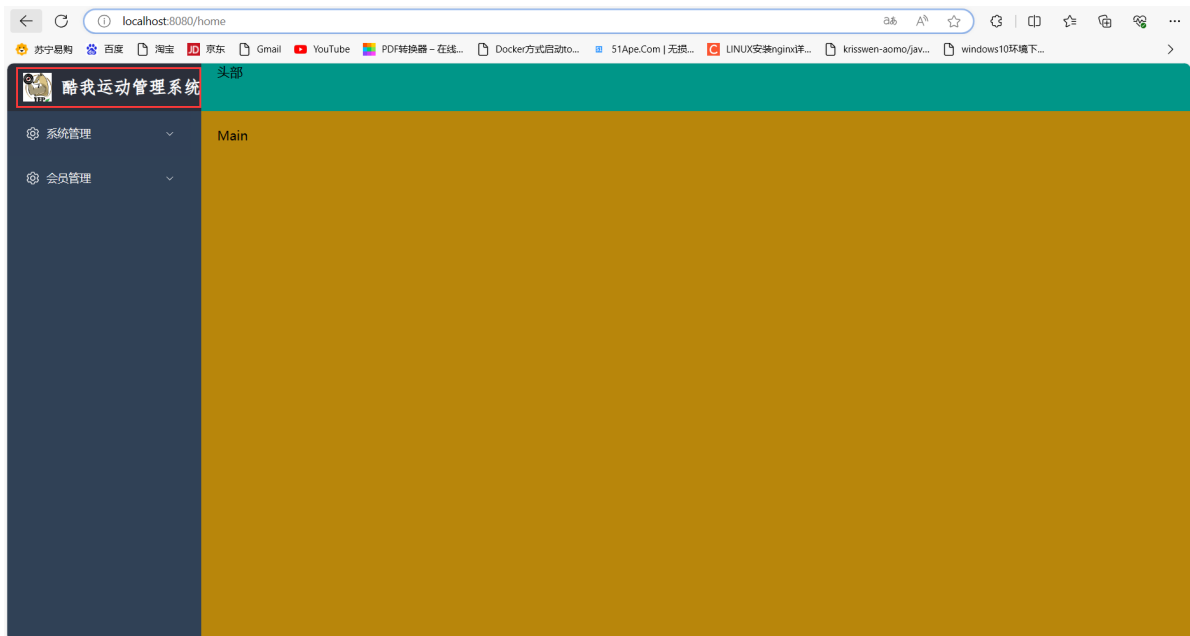
1.9.3 MenuBar.vue里面引入MenuLogo组件

```

<template>
  <menu-logo></menu-logo>
  <el-menu
    default-active="2"
    class="el-menu-vertical-demo"
    :collapse="isCollapse"
    unique-opened
    background-color="#304156"
  >
    <menu-item :menuList="menuList"></menu-item>
  </el-menu>
</template>
<script setup lang="ts">
import { ref, reactive } from "vue";
import MenuItem from "@layout/menu/MenuItem.vue";
import MenuLogo from "@layout/menu/MenuLogo.vue";
const isCollapse = ref(false);

```

启动项目查看效果：



1.10 路由配置与页面创建

需求：点击左侧菜单，能够在内容展示区展示对应页面

1.10.1 修改路由

router/index.ts路由修改为如下

```
import { createRouter, createWebHistory, RouteRecordRaw } from
'vue-router'
import Layout from '@/layout/Index.vue'
const routes: Array<RouteRecordRaw> = [
  {
    path: '/',
    component: Layout,
    redirect: '/dashboard',
    children: [
      {
        path: '/dashboard',
        component: () =>
import('@/layout/dashboard/Index.vue'),
        name: 'dashboard',
        meta: {
          title: '首页',
          icon: '#icondashboard'
        }
      }
    ]
  },
  {
```

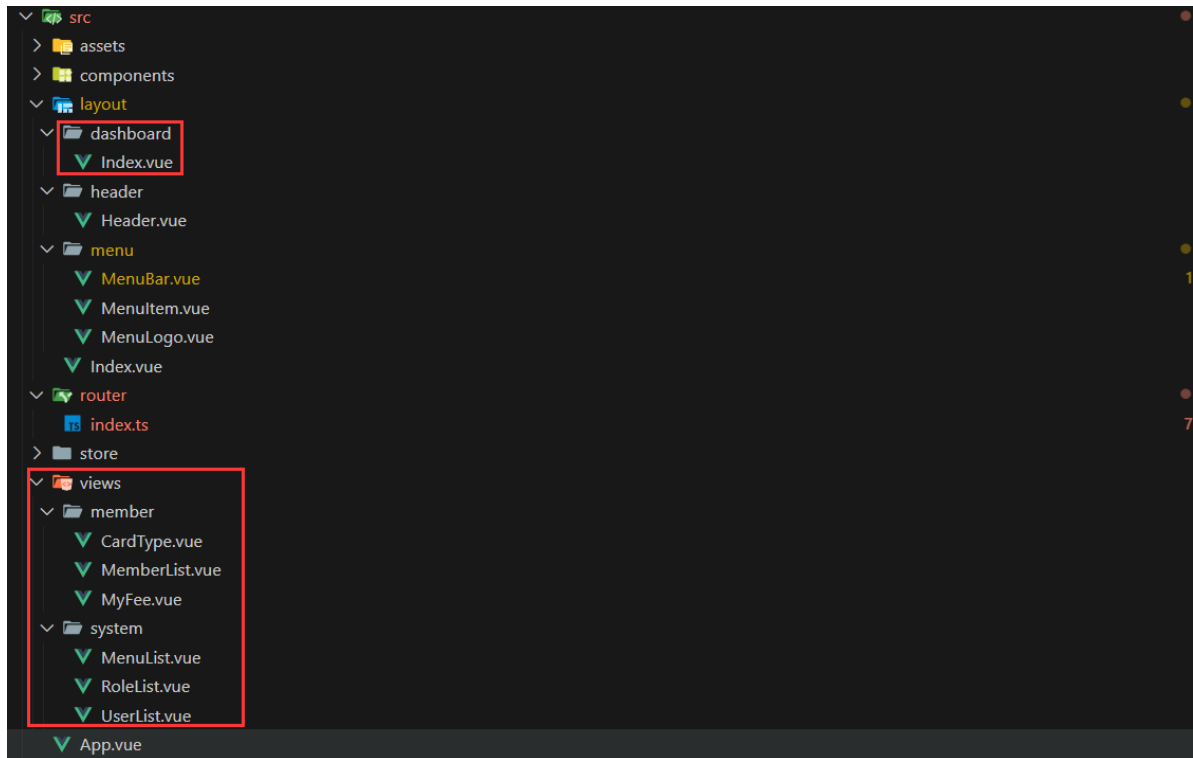
```
path: "/system",
component: Layout,
name: "system",
meta: {
  title: "系统管理",
  icon: "el-icon-menu",
  roles: ["sys:manage"]
},
children: [
  {
    path: "/userList",
    component: () =>
import('@/views/system/UserList.vue'),
    name: "userList",
    meta: {
      title: "员工管理",

      icon: "el-icon-s-custom",
      roles: ["sys:user"]
    },
  },
  {
    path: "/roleList",
    component: () =>
import('@/views/system/RoleList.vue'),
    name: "roleList",
    meta: {
      title: "角色管理",
      icon: "el-icon-s-tools",
      roles: ["sys:role"]
    },
  },
  {
    path: "/menuList",
    component: () =>
import('@/views/system/MenuList.vue'),
    name: "menuList",
    meta: {
      title: "权限管理",
      icon: "el-icon-document",
      roles: ["sys:menu"]
    },
  },
]
},
```

```
{
  path: "/memberRoot",
  component: Layout,
  name: "memberRoot",
  meta: {
    title: "会员管理",
    icon: "Setting",
    roles: ["sys:memberRoot"],
  },
  children: [
    {
      path: "/cardType",
      component: () =>
import('@/views/member/CardType.vue'),
      name: "cardType",
      meta: {
        title: "会员卡类型",
        icon: "UserFilled",
        roles: ["sys:cardType"],
      },
    },
    {
      path: "/memberList",
      component: () =>
import('@/views/member/MemberList.vue'),
      name: "memberList",
      meta: {
        title: "会员管理",
        icon: "wallet",
        roles: ["sys:memberList"],
      },
    },
    {
      path: "/myFee",
      component: () => import('@/views/member/MyFee.vue'),
      name: "myFee",
      meta: {
        title: "我的充值",
        icon: "Menu",
        roles: ["sys:myFee"],
      },
    },
  ],
}
]
```

```
const router = createRouter({
  history: createWebHistory(),
  routes
})
export default router
```

1.10.2 创建路由对应的页面



1.10.3 在MenuBar.vue组件的el-menu添加 router属性

router: 是否启用 vue-router 模式。 启用该模式会在激活导航时以 index 作为 path 进行路由跳转

```
<template>
  <menu-logo></menu-logo>
  <el-menu
    :default-active="activeIndex"
    class="el-menu-vertical-demo"
    unique-opened
    background-color="#304156"
    router
  >
    <menu-item :menuList="menuList"></menu-item>
  </el-menu>
</template>
```

1.10.4 在src/layout/Index.vue的添加路由

```

<template>
  <el-container class="layout">
    <el-aside width="auto" class="asside">
      <menu-bar></menu-bar>
    </el-aside>
    <el-container>
      <el-header class="header">
        <Header></Header>
      </el-header>
      <el-main class="main">
        <router-view></router-view>
      </el-main>
    </el-container>
  </el-container>
</template>

```

1.10.5 MenuBar.vue设置当前激活的菜单

default-active 在MenuBar.vue添加如下代码

```

import { ref, reactive, computed } from "vue";
import { useRoute } from 'vue-router';
const route = useRoute();
//获取激活的菜单
const activeIndex = computed(()=>{
  const {path} = route;
  return path;
})

```

MenuBar.vue代码最后修改如下:

```

<template>
  <menu-logo></menu-logo>
  <el-menu
    :default-active="activeIndex"
    class="el-menu-vertical-demo"
    unique-opened
    background-color="#304156"
    router
  >
    <menu-item :menuList="menuList"></menu-item>
  </el-menu>
</template>
<script setup lang="ts">
import MenuItem from '@/layout/menu/MenuItem.vue'
import MenuLogo from '@/layout/menu/MenuLogo.vue'
import { ref, reactive, computed } from "vue";
import { useRoute } from 'vue-router';
const route = useRoute();
//获取激活的菜单
const activeIndex = computed(()=>{
  const {path} = route;
  return path;
})
//树形的菜单数据
//reactive:定义响应式的数据（复杂类型，对象）

```

//ref:定义响应式的数据（基本类型） let count = ref(0)

```
let menuList = reactive([
  {
    path: "/system",
    component: "Layout",
    name: "system",
    meta: {
      title: "系统管理",
      icon: "Setting",
      roles: ["sys:manage"],
    },
    children: [
      {
        path: "/userList",
        component: "/system/User/UserList",
        name: "userList",
        meta: {
          title: "员工管理",
          icon: "UserFilled",
          roles: ["sys:user"],
        },
      },
    ],
      {
        path: "/roleList",
        component: "/system/Role/RoleList",
        name: "roleList",
        meta: {
          title: "角色管理",
          icon: "wallet",
          roles: ["sys:role"],
        },
      },
    ],
      {
        path: "/menuList",
        component: "/system/Menu/MenuList",
        name: "menuList",
        meta: {
          title: "菜单管理",
          icon: "Menu",
          roles: ["sys:menu"],
        },
      },
    ],
  },
  {

```

```
path: "/memberRoot",
component: "Layout",
name: "memberRoot",
meta: {
  title: "会员管理",
  icon: "Setting",
  roles: ["sys:memberRoot"],
},
children: [
  {
    path: "/cardType",
    component: "/member/CardType",
    name: "cardType",
    meta: {
      title: "会员卡类型",
      icon: "UserFilled",
      roles: ["sys:cardType"],
    },
  },
  {
    path: "/memberList",
    component: "/member/MemberList",
    name: "memberList",
    meta: {
      title: "会员管理",
      icon: "wallet",
      roles: ["sys:memberList"],
    },
  },
  {
    path: "/myFee",
    component: "/system/MyFee",
    name: "myFee",
    meta: {
      title: "我的充值",
      icon: "Menu",
      roles: ["sys:myFee"],
    },
  },
],
},
]);
</script>
<style scoped lang="scss">
.el-menu-vertical-demo:not(.el-menu--collapse) {
```

```

width: 230px;
min-height: 400px;
}
.el-menu {
border-right: none;
}
:deep(.el-sub-menu .el-sub-menu__title){
color: #f4f4f5 !important;
}
:deep(.el-menu .el-menu-item){
color: #bfcabd9;
}
/* 菜单点中文字的颜色 */
:deep(.el-menu-item.is-active){
color: #409eff !important;
}
/* 当前打开菜单的所有子菜单颜色 */
:deep(.is-opened .el-menu-item){
background-color: #1f2d3d !important;
}
/* 鼠标移动菜单的颜色 */
:deep(.el-menu-item:hover){
background-color: #001528 !important;
}
</style>

```

1.11 菜单收缩制作

效果:



1.11.1 store下新建collapse目录，然后新建index.ts

```

import { defineStore } from 'pinia'
//定义store
export const collapseStore = defineStore('collapseStore',{
state:()=>{
return{
collapse:false
}
},

```



```

getters:{
  getCollapse(state){
    return state.collapse
  }
},
actions:{
  setCollapse(collapse:boolean){
    this.collapse = collapse;
  }
}
})

```

1.11.2 layout/header下新建Collapse.vue组件

```

<template>
  <el-icon class="icons" @click="setCollapse">
    <component :is="status ? Fold : Expand" />
  </el-icon>
</template>
<script setup lang="ts">
  import { computed } from "vue";
  import { Fold, Expand } from "@element-plus/icons-vue";
  import { collapseStore } from "@/store/collapse/index";
  //获取store
  const store = collapseStore();
  //获取collapse
  const status = computed(() => {
    return !store.getCollapse;
  });
  //改变collapse
  const setCollapse = ()=>{
    store.setCollapse(!store.getCollapse)
  }
</script>
<style scoped lang="scss">
  .icons {
    color: #fff;
    font-size: 24px;
  }
</style>

```

1.11.3 修改Header.vue组件

在Header.vue中引入Collapse.vue组件

```
<template>
  <Collapse></Collapse>
</template>
<script setup lang="ts">
  import collapse from './Collapse.vue';
</script>
<style scoped lang='scss'>

</style>
```

1.11.4 修改layout下的Index.vue的header样式

layout下的Index.vue的header样式修改为如下：

```
.header {
  display: flex;
  align-items: center;
  background-color: #009688;
}
```

1.11.5 MenuBar.vue组件的el-menu添加 :collapse属性

查看官方文档，collapse的属性意思如下：

属性名	说明	类型	可选值	默认值
mode	菜单展示模式	string	horizontal / vertical	vertical
collapse	是否水平折叠收起菜单（仅在 mode 为 vertical 时可用）	boolean	—	false
ellipsis	是否省略多余的子项（仅在横向模式生效）	boolean	—	true

```
<template>
  <menu-logo :isCollapsed="isCollapse"></menu-logo>
  <el-menu
    :collapse="isCollapse"
    :default-active="activeIndex"
    class="el-menu-vertical-demo"
    unique-opened
    background-color="#304156"
    router
  >
    <menu-item :menuList="menuList"></menu-item>
  </el-menu>
</template>
```

最终修改完整代码如下：

```

<template>
  <menu-logo :isCollapsed="isCollapse"></menu-logo>
  <el-menu
    :collapse="isCollapse"
    :default-active="activeIndex"
    class="el-menu-vertical-demo"
    unique-opened
    background-color="#304156"
    router
  >
    <menu-item :menuList="menuList"></menu-item>
  </el-menu>
</template>
<script setup lang="ts">
import { ref, reactive, computed } from "vue";
import MenuItem from "@/layout/menu/MenuItem.vue";
import MenuLogo from "@/layout/menu/MenuLogo.vue";
import { useRoute } from "vue-router";
import { collapseStore } from "@/store/collapse/index";
const colstore = collapseStore();
const isCollapse = computed(() => {
  return colstore.getCollapse;
});
const route = useRoute();
//获取激活的菜单
const activeIndex = computed(() => {
  const { path } = route;
  return path;
});
//树形的菜单数据
//reactive:定义响应式的数据（复杂类型，对象）
//ref:定义响应式的数据（基本类型） let count = ref(0)
let menuList = reactive([
  {
    path: "/system",
    component: "Layout",
    name: "system",
    meta: {
      title: "系统管理",
      icon: "Setting",
      roles: ["sys:manage"],
    },
    children: [
      {
        path: "/userList",

```

```
        component: "/system/User/UserList",
        name: "userList",
        meta: {
            title: "员工管理",
            icon: "UserFilled",
            roles: ["sys:user"],
        },
    },
    {
        path: "/roleList",
        component: "/system/Role/RoleList",
        name: "roleList",
        meta: {
            title: "角色管理",
            icon: "wallet",
            roles: ["sys:role"],
        },
    },
    {
        path: "/menuList",
        component: "/system/Menu/MenuList",
        name: "menuList",
        meta: {
            title: "菜单管理",
            icon: "Menu",
            roles: ["sys:menu"],
        },
    },
],
},
{
    path: "/memberRoot",
    component: "Layout",
    name: "memberRoot",
    meta: {
        title: "会员管理",
        icon: "Setting",
        roles: ["sys:memberRoot"],
    },
    children: [
        {
            path: "/cardType",
            component: "/member/CardType",
            name: "cardType",
            meta: {
```

```

        title: "会员卡类型",
        icon: "UserFilled",
        roles: ["sys:cardType"],
      },
    ],
    {
      path: "/memberList",
      component: "/member/MemberList",
      name: "memberList",
      meta: {
        title: "会员管理",
        icon: "wallet",
        roles: ["sys:memberList"],
      },
    },
    {
      path: "/myFee",
      component: "/system/MyFee",
      name: "myFee",
      meta: {
        title: "我的充值",
        icon: "Menu",
        roles: ["sys:myFee"],
      },
    },
  ],
},
]);
</script>
<style scoped lang="scss">
.el-menu-vertical-demo: not(.el-menu--collapse) {
  width: 230px;
  min-height: 400px;
}
.el-menu {
  border-right: none;
}

:deep(.el-sub-menu .el-sub-menu__title) {
  color: #f4f4f5 !important;
}

:deep(.el-menu .el-menu-item) {
  color: #bfcabd9;
}

```

```

/* 菜单点中文字的颜色 */

:deep(.el-menu-item.is-active) {
  color: #409eff !important;
}

/* 当前打开菜单的所有子菜单颜色 */

:deep(.is-opened .el-menu-item) {
  background-color: #1f2d3d !important;
}

/* 鼠标移动菜单的颜色 */

:deep(.el-menu-item:hover) {
  background-color: #001528 !important;
}
</style>

```

1.11.6 修改MenuLogo.vue

修改的目的是将Logo上的文字也会随之MenuBar的伸缩表现出显示与隐藏的效果。

```

<template>
  <div class="logo">
    
    <span v-if="show" class="logo-title">{{ title }}</span>
  </div>
</template>
<script setup lang="ts">
  import { ref, watch } from "vue";
  import MenuLogo from "@assets/MenuLogo.png";
  const title = ref("健身房管理系统");
  const props = defineProps({
    isCollapsed: {
      require: true,
      type: Boolean,
    },
  });
  const show = ref(true);
  watch(
    () => props.isCollapsed,
    (collapsed: boolean) => {
      if (!collapsed) {
        setTimeout(() => {
          show.value = !collapsed;
        }, 300);
      } else {
        show.value = !collapsed;
      }
    }
  );
</script>

```

完整代码如下:

```

<template>
  <div class="logo">
    
    <span v-if="show" class="logo-title">{{ title }}</span>
  </div>
</template>

```

```

    </div>
</template>
<script setup lang="ts">
  import { ref, watch } from "vue";
  import MenuLogo from "@/assets/MenuLogo.png";
  const title = ref("健身房管理系统");
  const props = defineProps({
    isCollapsed: {
      require: true,
      type: Boolean,
    },
  });
  const show = ref(true);
  watch(
    () => props.isCollapsed,
    (collapsed: boolean) => {
      if (!collapsed) {
        setTimeout(() => {
          show.value = !collapsed;
        }, 300);
      } else {
        show.value = !collapsed;
      }
    }
  );
</script>
<style scoped lang="scss">
.logo {
  display: flex;
  width: 100%;
  height: 60px;
  line-height: 60px;
  background: #2b2f3a;
  text-align: center;
  cursor: pointer;
  align-items: center;
  img {
    width: 36px;
    height: 36px;
    margin-right: 12px;
    margin-left: 20px;
  }
  .logo-title {
    color: #fff;
    font-weight: 800;
  }
}

```

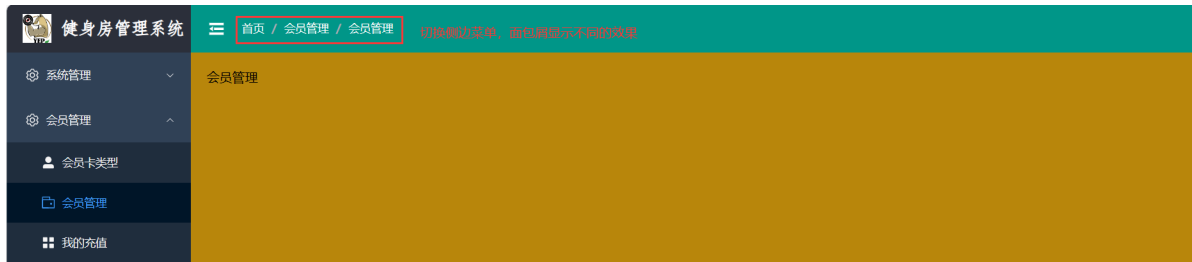
```

    line-height: 60px;
    font-size: 22px;
    font-family: FangSong;
  }
}
</style>

```

1.12 面包屑导航制作

面包屑实现效果：



面包屑代码实现参考官网：<https://element-plus.org/zh-CN/component/breadcrumb.html#breadcrumb-attributes>

1.12.1 header目录下新建BreadCrumb.vue组件

```

<template>
  <el-breadcrumb class="bred" separator="/">
    <el-breadcrumb-item v-for="item in tabs">{{ item.meta.title }}</el-breadcrumb-item>
  </el-breadcrumb>
</template>

<script setup lang="ts">
import { ref, watch, Ref } from "vue";
import { useRouter, RouteLocationMatched } from "vue-router";
//面包屑数据
const tabs: Ref<RouteLocationMatched[]> = ref([]);
//获取当前路由
const route = useRouter();
const getBredcrumb = () => {
  //从路由里面获取所有有meta和title
  let mached = route.matched.filter((item) => item.meta &&
item.meta.title);
  //console.log(mached)
  //判断第一个是否是首页,不是,构造一个
  const first = mached[0];
  if (first.path !== "/dashboard") {

```



```

    mached = [{ path: "/dashboard", meta: { title: "首页" } } as
any].concat(mached);
  }
  tabs.value = mached;
};
getBredcrumb();
watch(
  () => route.path,
  () => getBredcrumb()
);
</script>

<style scoped lang="scss">
//修改字体颜色
:deep(.el-breadcrumb__inner) {
  color: #fff !important;
}
.bred {
  margin-left: 20px;
}
// 修改字体大小
// :deep(.el-breadcrumb__item) {
//   font-size: 16px !important;
// }
</style>

```

1.12.2 修改Header.vue组件

在Header.vue组件里面引入面包屑组件：

```

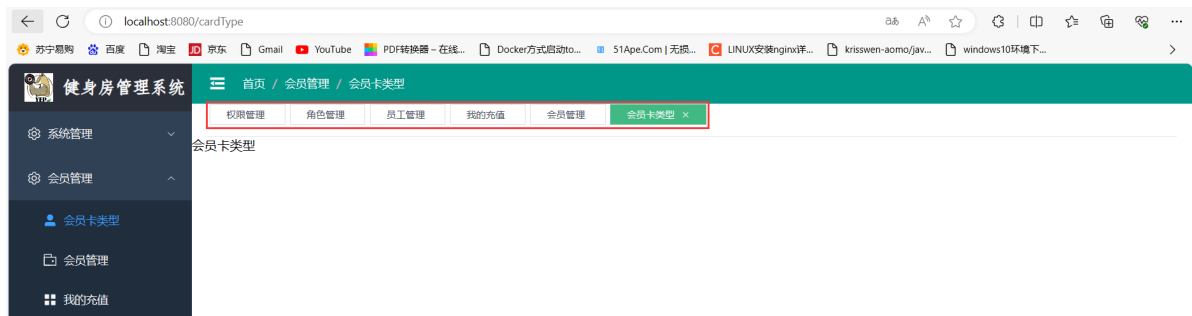
<template>
  <Collapse></Collapse>
  <bread-crumb></bread-crumb>
</template>
<script setup lang="ts">
  import Collapse from './Collapse.vue';
  import BreadCrumb from './BreadCrumb.vue';
</script>
<style scoped lang='scss'>

</style>

```

1.13 tabs选项卡制作

效果:



1.13.1 store下新建tabs目录

在tabs目录下面创建index.ts,定义如下:

```
import { defineStore } from 'pinia'
//定义Tab类型
export type Tab = {
  title: string;
  path: string;
}
//定义state类型
export type TabState = {
  tabList: Tab[]
}
//定义store
export const tabStore = defineStore('tabStore', {
  state: (): TabState => {
    return {
      tabList: []
    }
  },
  getters: {
    getTabs(state) {
      return state.tabList
    }
  },
  actions: {
    addTab(tab: Tab) {
      //判断是否已经存在, 如果不存在, 才放入
      if (this.tabList.some(item => item.path ===
tab.path)) return;
      this.tabList.push(tab);
    }
  }
})
```

```
} )
```

1.13.2 layout目录新建tabs目录，然后新建Tabs.vue

```
<template>
  <el-tabs
    v-model="activeTab"
    @tab-click="clickBtn"
    type="card"
    closable
    @tab-remove="removeTab"
  >
    <el-tab-pane
      v-for="item in tabsList"
      :key="item.path"
      :label="item.title"
      :name="item.path"
    ></el-tab-pane>
  </el-tabs>
</template>
<script setup lang="ts">
import { ref, computed, watch, onMounted } from "vue";
import { tabStore } from "@/store/tabs";
import { useRoute, useRouter } from "vue-router";
import { Tab } from "@/store/tabs/index";
import { TabPanelName } from "element-plus";
const route = useRoute();
const router = useRouter();
const store = tabStore();
//获取tabs数据
const tabsList = computed(() => {
  // return store.getters['getTabs']
  return store.getTabs;
});
//当前激活的选项卡，他跟当前激活的路由是一样的；
const activeTab = ref("");
const setActiveTab = () => {
  activeTab.value = route.path;
};
//删除选项卡
const removeTab = (targetName: TabPanelName) => {
  if (targetName === "/dashboard") return;
  //选项卡数据列表
  const tabs = tabsList.value;
  //当前激活的选项卡
```

```

let activeName = activeTab.value;
if (activeName === targetName) {
  tabs.forEach((tab: Tab, index: number) => {
    if (tab.path === targetName) {
      const nextTab = tabs[index + 1] || tabs[index - 1];
      if (nextTab) {
        activeName = nextTab.path;
      }
    }
  });
}
//重新设置当前激活的选项卡
activeTab.value = activeName;
//重新设置选项卡数据
store.tabList = tabs.filter((tab: Tab) => tab.path !==
targetName);
//跳转路由
router.push({ path: activeName });
};
//添加选项卡
const addTab = () => {
  //从当前路由获取path和title
  const { path, meta } = route;
  //通过vuex设置
  const tab: Tab = {
    path: path,
    title: meta.title as string,
  };
  store.addTab(tab);
};
//监听路由的变化
watch(
  () => route.path,
  () => {
    //设置激活的选项卡
    setActiveTab();
    //把当前路由添加到选项卡数据
    addTab();
  }
);
//解决刷新数据丢失的问题
const beforeRefresh = () => {
  if (route.path !== "/login") {
    window.addEventListener("beforeunload", () => {

```

```

        sessionStorage.setItem("tabView",
JSON.stringify(tabsList.value));
    });
    let tabSession = sessionStorage.getItem("tabView");
    if (tabSession) {
        let oldTabs = JSON.parse(tabSession);
        if (oldTabs.length > 0) {
            store.tabList = oldTabs;
        }
    }
}
};
onMounted(() => {
    //解决刷新选项卡数据丢失的问题
    beforeRefresh();
    //设置激活的选项卡
    setActiveTab();
    //把当前路由添加到选项卡数据
    addTab();
});
//选项卡点击事件
const clickBtn = (tab: any) => {
    console.log(tab);
    const { props } = tab;
    console.log(props);
    //跳转路由
    router.push({ path: props.name });
};
</script>
<style scoped lang="scss">
:deep(.el-tabs__header) {
    margin: 0px;
}
:deep(.el-tabs__item) {
    height: 26px !important;
    line-height: 26px !important;
    text-align: center !important;
    border: 1px solid #d8dce5 !important;
    margin: 0px 3px !important;
    color: #495060;
    font-size: 12px !important;
    padding: 0px 10px !important;
}
:deep(.el-tabs__nav) {
    border: none !important;

```

```

}
:deep(.is-active) {
  border-bottom: 1px solid transparent !important;
  border: 1px solid #42b983 !important;
  background-color: #42b983 !important;
  color: #fff !important;
}
:deep(.el-tabs__item:hover) {
  color: #495060 !important;
}
:deep(.is-active:hover) {
  color: #fff !important;
}
:deep(.el-tabs__nav-next){
  line-height: 26px !important;
}
:deep(.el-tabs__nav-prev){
  line-height: 26px !important;
}
}
</style>

```

1.13.3 修改layout/Index.vue

```

<template>
  <el-container class="layout">
    <el-aside width="auto" class="asside">
      <menu-bar></menu-bar>
    </el-aside>
    <el-container>
      <el-header height="50px" class="header">
        <Header></Header>
      </el-header>
      <el-main class="main">
        <Tabs style="padding: 0px 20px"></Tabs>
        <router-view></router-view>
      </el-main>
    </el-container>
  </el-container>
</template>
<script setup lang="ts">
  //vue3里面，组件引入，可以直接使用。不需要注册
  import Header from "@/layout/header/Header.vue";
  import MenuBar from "@/layout/menu/MenuBar.vue";
  import Tabs from "@/tabs/Tabs.vue";
</script>

```

```

<style scoped lang="scss">
.layout {
  height: 100%;
  .asside {
    // background-color: blueviolet;
    background-color: #304156;
  }
  .header {
    display: flex;
    align-items: center;
    background-color: #009688;
  }
  .main {
    padding-top: 1px;
    padding-left: 0px !important;
    padding-right: 0px !important;
  }
}
</style>

```

1.14 项目安装axios

首先在项目中安装axios。

```
npm install axios
```

在src目录下面创建http目录，在目录里面定义index.ts。

```

import axios, { AxiosInstance, AxiosRequestConfig, AxiosResponse,
AxiosRequestHeaders } from "axios";
import { ElMessage } from 'element-plus';

//axios请求配置
const config = {
  baseURL: 'http://localhost:8089', //真实请求接口的地址,真实上线是一个域名
  timeout: 10000
}
//定义返回值类型
export interface Result<T = any> {
  code: number;
  msg: string;
  data: T;
}

```

```

class Http {
  //axios实例
  private instance: AxiosInstance;
  //构造函数里面初始化
  constructor(config: AxiosRequestConfig) {
    this.instance = axios.create(config)
    //定义拦截器
    this.interceptors()
  }
  //拦截器
  private interceptors() {
    //axios发送请求之前的处理
    this.instance.interceptors.request.use((config:
AxiosRequestConfig) => {
      //在请求头部携带token
      // let token = sessionStorage.getItem('token');
      let token =
'eyJhbGciOiJIUzUxMiJ9.eyJleHAiOjE2NzE5ODMwMzksInN1YiI6ImFkbWluciw
iaWF0IjoxNjUzOTgzMDM5NTQzfQ.Ai_Lcq0tE7-
GurWqjng86gGn_WWVWN9jOx6yHeueP9nqxDDWft_h5SeopsQIiUewYbZVCiKSJKUP
_E8bMRdwxw';
      if (token) {
        config.headers!['token'] = token
        //把token放到headers里面
        // (config.headers as AxiosRequestHeaders).token
= token
      }
      console.log(config)
      return config;
    }, (error: any) => {
      error.data = {}
      error.data.msg = '服务器异常，请联系管理员!'
      return error;
    })
    //axios请求返回之后的处理
    //请求返回之后的处理
    this.instance.interceptors.response.use((res:
AxiosResponse) => {
      // console.log(res.data)
      if (res.data.code !== 200) {
        ElMessage.error(res.data.msg || '服务器出错!')
        return Promise.reject(res.data.msg || '服务器出错')
      } else {
        return res.data
      }
    })
  }
}

```



```
    }  
  }, (error) => {  
    console.log('进入错误')  
    error.data = {};  
    if (error && error.response) {  
      switch (error.response.status) {  
        case 400:  
          error.data.msg = '错误请求';  
          ErrorMessage.error(error.data.msg)  
          break  
        case 401:  
          error.data.msg = '未授权, 请重新登录';  
          ErrorMessage.error(error.data.msg)  
          break  
        case 403:  
          error.data.msg = '拒绝访问';  
          ErrorMessage.error(error.data.msg)  
          break  
        case 404:  
          error.data.msg = '请求错误, 未找到该资源';  
          ErrorMessage.error(error.data.msg)  
          break  
        case 405:  
          error.data.msg = '请求方法未允许';  
          ErrorMessage.error(error.data.msg)  
          break  
        case 408:  
          error.data.msg = '请求超时';  
          ErrorMessage.error(error.data.msg)  
          break  
        case 500:  
          error.data.msg = '服务器端出错';  
          ErrorMessage.error(error.data.msg)  
          break  
        case 501:  
          error.data.msg = '网络未实现';  
          ErrorMessage.error(error.data.msg)  
          break  
        case 502:  
          error.data.msg = '网络错误';  
          ErrorMessage.error(error.data.msg)  
          break  
        case 503:  
          error.data.msg = '服务不可用';  
          ErrorMessage.error(error.data.msg)  
        }  
      }  
    }  
  }  
}
```

```

        break
    case 504:
        error.data.msg = '网络超时';
        ErrorMessage.error(error.data.msg)
        break
    case 505:
        error.data.msg = 'http版本不支持该请求';
        ErrorMessage.error(error.data.msg)
        break
    default:
        error.data.msg = `连接错误
${error.response.status}`;
        ErrorMessage.error(error.data.msg)
    }
} else {
    error.data.msg = "连接到服务器失败";
    ErrorMessage.error(error.data.msg)
}
return Promise.reject(error)
})
}

// restful API的封装
/* GET 方法 */
get<T = Result>(url: string, params?: object): Promise<T> {
    return this.instance.get(url, { params })
}

/* POST 方法 */
post<T = Result>(url: string, data?: object): Promise<T> {
    return this.instance.post(url, data)
}

/* PUT 方法 */
put<T = Result>(url: string, data?: object): Promise<T> {
    return this.instance.put(url, data)
}

/* DELETE 方法 */
delete<T = Result>(url: string): Promise<T> {
    return this.instance.delete(url)
}
}

export default new Http(config)

```

1.15 弹框组件封装

弹框组件对应官网地址: <https://element-plus.org/zh-CN/component/dialog.html>

1.15.1 components下新建SysDialog.vue

```
<template>
  <el-dialog
    :title="title"
    :model-value="visible"
    :before-close="onClose"
    append-to-body
    :width="width + 'px'"
    :close-on-click-modal="false"
  >
    <div class="container" :style="{ height: height + 'px' }">
      <slot name="content"></slot>
    </div>
    <template #footer>
      <span class="dialog-footer">
        <el-button type="danger" @click="onClose">取消</el-
button>
        <el-button type="primary" @click="onConfirm">确定</el-
button>
      </span>
    </template>
  </el-dialog>
</template>
<script setup lang="ts">
interface DialogProps{
  title:string,
  visible:boolean,
  width:number,
  height:number
}
const props = withDefaults(defineProps<DialogProps>(), {
  title: '标题',
  visible:false,
  width:630,
  height:280
});
const emit = defineEmits(["onClose", "onConfirm"]);
//定义弹框的关闭
const onClose = () => {
  emit("onClose");
}
```

```
};  
//定义弹框的确定  
const onConfirm = () => {  
  emit("onConfirm");  
};  
  
</script>  
<style lang="scss" scope>  
.container {  
  overflow-x: initial;  
  overflow-y: auto;  
}  
.el-dialog {  
  border-top-left-radius: 7px !important;  
  border-top-right-radius: 7px !important;  
  .el-dialog__header {  
    margin-right: 0px;  
    border-top-left-radius: 7px !important;  
    border-top-right-radius: 7px !important;  
    background-color: #009688 !important;  
    .el-dialog__title {  
      color: #fff;  
      font-size: 16px;  
      font-weight: 600;  
    }  
  }  
  .el-dialog__headerbtn {  
    .el-dialog__close {  
      color: #fff;  
    }  
  }  
  .el-dialog__body {  
    padding: 10px;  
  }  
  .el-dialog__footer {  
    border-top: 1px solid #e8eaec !important;  
    padding: 10px;  
  }  
}  
</style>
```

1.5.2 封装弹框属性

src下新建type，然后新建BaseType.ts

```
//弹框属性类型
export type DialogModel = {
  title: string,
  visible: boolean,
  height: number,
  width: number
}
```

src下新建hooks目录，然后新建useDialog.ts

```
import { DialogModel } from '@/type/BaseType'
import { reactive } from 'vue'
export default function useDialog() {

  //定义弹框属性
  const dialog = reactive<DialogModel>({
    title: '',
    visible: false,
    width: 630,
    height: 280
  })

  //展示
  const onShow = () => {
    dialog.visible = true;
  }

  //关闭
  const onClose = () => {
    dialog.visible = false;
  }

  //确定
  const onConfirm = () => {
    dialog.visible = false;
  }

  return {
    dialog,
    onShow,
    onClose,
  }
}
```

```
        onConfirm
      }
    }
  }
```

1.5.3 在UserList.vue里面使用弹框组件

在views/system/UserList.vue里面使用弹框组件：

```
<template>
  <div>
    <el-button type="primary" size="default" @click="addBtn">新增</el-button>
    <sys-dialog
      :visible="dialog.visible"
      @onclose="onClose"
      @onconfirm="onConfirm"
    >
      <template v-slot:content>
        <div>我是弹框</div>
      </template>
    </sys-dialog>
  </div>
</template>

<script setup lang="ts">
import SysDialog from "@/components/SysDialog.vue";
import useDialog from "@/hooks/useDialog";

//弹框属性
const {dialog,onClose,onConfirm} = useDialog()

const addBtn = () => {
  dialog.visible = true;
};

</script>

<style scoped></style>
```

2 数据表准备

基于系统管理模块我们分析出来的数据表如下：

- 菜单表(sys_menu)
- 角色表(sys_role)

- 菜单表和角色表的中间表(sys_role_menu)
- 员工表(sys_user)
- 员工表和角色表的中间表(sys_user_role)

-- 菜单表

```
create table sys_menu(  
    menu_id int not null auto_increment comment '菜单id',  
    parent_id int not null comment '父级id',  
    title varchar(64) comment '菜单名称',  
    `code` VARCHAR(64) COMMENT '权限字段',  
    `name` VARCHAR(64) comment '路由名称',  
    path varchar(64) COMMENT '路由path',  
    url varchar(128) comment '组件路径',  
    `type` varchar(2) comment '类型(0目录 1菜单 2按钮)',  
    icon varchar(64) comment '图标',  
    parent_name varchar(64) comment '上级菜单名称',  
    order_num int comment '序号',  
    create_time datetime comment '创建时间',  
    update_time datetime comment '更新时间',  
    primary key(menu_id)  
)
```

-- 角色表

```
create table sys_role(  
    role_id int not null auto_increment comment '角色id',  
    role_name varchar(64) comment '角色名称',  
    remark varchar(64) comment '备注',  
    create_time datetime comment '创建时间',  
    update_time datetime comment '更新时间',  
    primary key(role_id)  
)
```

-- 菜单和角色之间的中间表

```
create table sys_role_menu(  
    role_menu_id int not null auto_increment comment '主键',  
    role_id int comment '角色id',  
    menu_id int comment '菜单id',  
    primary key (role_menu_id)  
)
```

-- 用户表

```
create table sys_user(  
    user_id int not null auto_increment comment '员工id',  
    username varchar(64) comment '账户(员工编号)',  
    `password` varchar(128) comment '密码',
```

```

phone varchar(15) comment '电话',
email varchar(36) comment '邮箱',
sex varchar(2) comment '性别 0:男 1:女',
salary decimal(18,2) comment '工资',
user_type varchar(2) comment '类型(1:员工 2:教练)',
`status` VARCHAR(2) comment '状态(0:停用 1:启用)',
is_admin varchar(2) comment '是否是管理员(1:是 0:否)',
is_account_non_expired tinyint(4) comment '账户是否过期(1:未过期
0:已过期)',
is_account_non_locked TINYINT(4) COMMENT '账户是否被锁定(1:未锁
定 0: 已锁定)',
is_credentials_non_expired TINYINT(4) COMMENT '密码是否过期(1:未
过期 0:已过期)',
is_enabled TINYINT(4) COMMENT '账户是否可用(1 可用 0不可用)',
nick_name varchar(64) comment '姓名',
create_time datetime comment '创建时间',
update_time datetime comment '更新时间',
primary key(user_id)
)

-- 用户表和角色表的中间表
create table sys_user_role(
    user_role_id int not null auto_increment comment '主键',
    user_id int comment '员工id',
    role_id int comment '角色id',
    primary key(user_role_id)
)

```

3 创建后端项目

开发环境配置：

- 1、JDK安装配置 1.8
- 2、IDEA安装
- 3、maven安装
- 4、MySQL安装5.7以上
- 5、MySQL可视化工具安装，比如sqlYog或Navicat或datagrip

项目结构：

gym-project-parent	父模块，pom类型、统一管理子模块
gym-common	公共模块、如通用工具的封装等
gym-service-web	前端接口模块，提供api接口

其中父工程gym-project-parent打包类型为pom。gym-common和gym-service-web打包为jar包。

3.1 创建父工程

在父工程中引入如下依赖：

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>2.4.4</version>
  <relativePath></relativePath>
</parent>

<properties>
  <java.version>1.8</java.version>
  <swagger2.version>3.0.0</swagger2.version>
  <lombok.version>1.18.12</lombok.version>
  <mybatis-plus.version>3.4.1</mybatis-plus.version>
  <druid.version>1.2.1</druid.version>
  <kaptcha.version>2.3.2</kaptcha.version>
  <fastjson.version>1.2.69</fastjson.version>
  <commons-lang.version>2.6</commons-lang.version>
  <commons-collections.version>3.2.2</commons-
collections.version>
  <commons-io.version>2.6</commons-io.version>
  <mysql-connection.version>8.0.21</mysql-connection.version>
  <jwt.version>3.10.3</jwt.version>
  <easypoi.version>4.2.0</easypoi.version>
</properties>

<dependencyManagement>
  <dependencies>
    <!-- Lombok依赖 -->
    <dependency>
      <groupId>org.projectlombok</groupId>
      <artifactId>lombok</artifactId>
      <version>${lombok.version}</version>
      <scope>provided</scope>
    </dependency>
    <dependency>
      <groupId>mysql</groupId>
      <artifactId>mysql-connector-java</artifactId>
      <version>${mysql-connection.version}</version>
      <scope>runtime</scope>
```

```
</dependency>
<!--druid连接池-->
<dependency>
    <groupId>com.alibaba</groupId>
    <artifactId>druid-spring-boot-starter</artifactId>
    <version>${druid.version}</version>
</dependency>
<!--mybatis-plus依赖-->
<dependency>
    <groupId>com.baomidou</groupId>
    <artifactId>mybatis-plus-boot-starter</artifactId>
    <version>${mybatis-plus.version}</version>
</dependency>
<!-- kaptcha 图形验证码 -->
<dependency>
    <groupId>com.github.penggle</groupId>
    <artifactId>kaptcha</artifactId>
    <version>${kaptcha.version}</version>
</dependency>

<!-- JSON转换工具类依赖 -->
<dependency>
    <groupId>com.alibaba</groupId>
    <artifactId>fastjson</artifactId>
    <version>${fastjson.version}</version>
</dependency>

<dependency>
    <groupId>commons-lang</groupId>
    <artifactId>commons-lang</artifactId>
    <version>${commons-lang.version}</version>
</dependency>
<dependency>
    <groupId>commons-collections</groupId>
    <artifactId>commons-collections</artifactId>
    <version>${commons-collections.version}</version>
</dependency>
<dependency>
    <groupId>commons-io</groupId>
    <artifactId>commons-io</artifactId>
    <version>${commons-io.version}</version>
</dependency>
<dependency>
    <groupId>io.springfox</groupId>
    <artifactId>springfox-boot-starter</artifactId>
```

```

        <version>${swagger2.version}</version>
    </dependency>
    <dependency>
        <groupId>com.auth0</groupId>
        <artifactId>java-jwt</artifactId>
        <version>${jwt.version}</version>
    </dependency>
    <dependency>
        <groupId>cn.afterturn</groupId>
        <artifactId>easypoi-spring-boot-starter</artifactId>
        <version>${easypoi.version}</version>
    </dependency>
</dependencies>
</dependencyManagement>

```

3.2 创建子模块

3.2.1 创建gym-common

创建子模块，并引入如下依赖：

```

<dependencies>
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
    </dependency>
    <!-- swagger api文档 -->
    <dependency>
        <groupId>io.springfox</groupId>
        <artifactId>springfox-boot-starter</artifactId>
    </dependency>
    <!-- 工具类依赖 -->
    <dependency>
        <groupId>com.alibaba</groupId>
        <artifactId>fastjson</artifactId>
    </dependency>
</dependencies>

```

3.2.2 创建gym-service-web

创建子模块，并引入如下依赖：

```

<dependencies>
    <dependency>
        <groupId>com.xq</groupId>

```

```
        <artifactId>gym-common</artifactId>
        <version>1.0-SNAPSHOT</version>
    </dependency>
    <!--web启动器,对springmvc,serlvet等支持-->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-aop</artifactId>
    </dependency>

    <!--数据库依赖-->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-jdbc</artifactId>
    </dependency>
    <dependency>
        <groupId>mysql</groupId>
        <artifactId>mysql-connector-java</artifactId>
    </dependency>
    <!--mybatis-plus启动器-->
    <dependency>
        <groupId>com.baomidou</groupId>
        <artifactId>mybatis-plus-boot-starter</artifactId>
    </dependency>
    <!--druid连接池-->
    <dependency>
        <groupId>com.alibaba</groupId>
        <artifactId>druid-spring-boot-starter</artifactId>
    </dependency>
    <!-- swagger api文档 -->
    <dependency>
        <groupId>io.springfox</groupId>
        <artifactId>springfox-boot-starter</artifactId>
    </dependency>
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
    </dependency>
    <dependency>
        <groupId>org.apache.commons</groupId>
        <artifactId>commons-compress</artifactId>
        <version>1.18</version>
```

```

</dependency>
<dependency>
    <groupId>commons-lang</groupId>
    <artifactId>commons-lang</artifactId>
</dependency>
<dependency>
    <groupId>com.auth0</groupId>
    <artifactId>java-jwt</artifactId>
</dependency>
<dependency>
    <groupId>cn.afterturn</groupId>
    <artifactId>easypoi-spring-boot-starter</artifactId>
</dependency>
</dependencies>
<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
            <version>2.1.4.RELEASE</version>
        </plugin>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <version>3.1</version>
            <configuration>
                <source>${java.version}</source>
                <target>${java.version}</target>
            </configuration>
        </plugin>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-surefire-plugin</artifactId>
            <version>2.19.1</version>
            <configuration>
                <skipTests>true</skipTests>    <!--打包过程默认关掉单
元测试 -->
            </configuration>
        </plugin>
    </plugins>
</build>

```

3.3 在项目中整合mybatis-plus

3.3.1 新建application.yml文件和启动类

在gym-service-web子模块中创建application.yml配置文件

(1) 创建application-dev.yml

```
#端口号配置
server:
  port: 8089
#数据库连接配置
spring:
  datasource:
    type: com.alibaba.druid.pool.DruidDataSource
    driver-class-name: com.mysql.jdbc.Driver
    url: jdbc:mysql://192.168.10.140:3306/gymnasium
    username: root
    password: Admin123!

#mybatis plus配置
mybatis-plus:
  configuration:
    log-impl: org.apache.ibatis.logging.stdout.StdoutImpl
  global-config:
    db-config:
      #配置mybatis plus 在更新时只更新非空和非NULL的字段
      update-strategy: not_empty

# 定义日志文件的输出格式
logging:
  pattern:
    console: '%d{yyyy-MM-dd} [%thread] %-5level %logger- %msg%n'
```

(2) 创建application.yml

```
spring:
  profiles:
    active: dev
```

(3) 创建启动类

```

import org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class GYMServiceWebApplication {
    public static void main(String[] args) {

        SpringApplication.run(GYMServiceWebApplication.class,args);
    }
}

```

3.3.2 配置MyBatis Plus配置文件

```

package com.xq.config;

import com.baomidou.mybatisplus.annotation.DbType;
import
com.baomidou.mybatisplus.extension.plugins.MybatisPlusInterceptor
;
import
com.baomidou.mybatisplus.extension.plugins.inner.PaginationInnerI
nterceptor;
import org.mybatis.spring.annotation.MapperScan;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
@Configuration
@MapperScan("com.xq.mapper")
public class MyBatisPlusConfig {
    // 最新版
    @Bean
    public MybatisPlusInterceptor mybatisPlusInterceptor() {
        MybatisPlusInterceptor interceptor = new
MybatisPlusInterceptor();
        interceptor.addInnerInterceptor(new
PaginationInnerInterceptor(DbType.MYSQL));
        return interceptor;
    }
}

```

3.3.3 封装返回值

在gym-common中定义如下：

```
package com.xq.utils;

@Data
@AllArgsConstructor
public class ResultVo<T> {
    private String msg;
    private int code;
    private T data;
}
```

3.3.4 返回值状态码定义

```
package com.xq.status;

/**
 * 返回状态码
 */
public class StatusCode {
    //返回成功
    public static final int SUCCESS_CODE = 200;
    //错误状态码
    public static final int ERROR_CODE = 500;
    //无权限
    public static final int NO_LOGIN = 600;
    public static final int NO_AUTH = 700;
}
```

3.3.5 返回工具类的封装

```
package com.xq.utils;

import com.xq.status.StatusCode;

/**
 * 数据返回工具类
 */
public class ResultUtils {
    /**
     * 无参数返回
     * @return
     */
}
```



```

    */
    public static ResultVo success() {
        return Vo(null, StatusCode.SUCCESS_CODE, null);
    }
    public static ResultVo success(String msg){
        return Vo(msg,StatusCode.SUCCESS_CODE,null);
    }
    /**
     * 返回带参数
     * @param msg
     * @param data
     * @return
     */
    public static ResultVo success(String msg,Object data){
        return Vo(msg,StatusCode.SUCCESS_CODE,data);
    }
    public static ResultVo success(String msg,int code,Object
data){
        return Vo(msg,code,data);
    }
    public static ResultVo Vo(String msg, int code, Object data)
{
        return new ResultVo(msg, code, data);
    }

    /**
     * 错误返回
     * @return
     */
    public static ResultVo error(){
        return Vo(null,StatusCode.ERROR_CODE,null);
    }
    public static ResultVo error(String msg){
        return Vo(msg,StatusCode.ERROR_CODE,null);
    }
    public static ResultVo error(String msg,int code,Object data)
{
        return Vo(msg,code,data);
    }
    public static ResultVo error(String msg,int code){
        return Vo(msg,code,null);
    }
    public static ResultVo error(String msg,Object data){
        return Vo(msg, StatusCode.ERROR_CODE,data);
    }
}

```

4 Swagger

4.1 Swagger简介

接口文档对于前后端开发人员都十分重要。尤其近几年流行前后端分离后接口文档又变成重中之重。接口文档固然重要，但是由于项目周期等原因后端人员经常出现无法及时更新，导致前端人员抱怨接口文档和实际情况不一致。

很多人员会抱怨别人写的接口文档不规范，不及时更新。当自己写的时候确实最烦去写接口文档。这种痛苦只有亲身经历才会牢记于心。如果接口文档可以实时动态生成就不会出现上面问题。Swagger可以完美的解决上面的问题。

4.1.1 OpenAPI规范

Open API 规范(OpenAPISpecification)以前叫做 Swagger 规范，是 REST API 的 API 描述格式。Open API 文件允许描述整个API，包括：

- 每个访问地址的类型。POST或GET。
- 每个操作的参数。包括输入输出参数。
- 认证方法。
- 连接信息，声明，使用团队和其他信息。

Open API 规范可以使用**YAML或JSON格式**进行编写。这样更利于我们和机器进行阅读。OpenAPI 规范（OAS）为REST API 定义了一个与语言无关的标准接口，允许人和计算机发现和理解服务的功能，而无需访问源代码，文档或通过网络流量检查。正确定义后，消费者可以使用最少量的实现逻辑来理解远程服务并与之交互。

然后，文档生成工具可以使用OpenAPI定义来显示API，使用各种编程语言生成服务器和客户端的代码生成工具，测试工具以及许多其他用例。

4.1.2 Swagger简介

Swagger 是一套围绕OpenAPI规范构建的开源工具，可以帮助设计，构建，记录和使用RESTAPI。

Swagger 工具包括的组件：

- Swagger Editor：基于浏览器编辑器，可以在里面编写OpenAPI 规范。类似Markdown具有实时预览描述文件的功能。
- Swagger UI：将 OpenAPI 规范呈现为交互式API文档。用可视化 UI 展示描述文件。
- Swagger Codegen：将 OpenAPI 规范生成成为服务器存根和客户端库。通过SwaggerCodegen可以将描述文件生成html格式和cwiki形式的接口文档，

同时也可以生成多种言语的客户端和服务端代码。

- Swagger Inspector: 和 Swagger UI 有点类似, 但是可以返回更多信息, 也会保存请求的实际参数数据。
- Swagger Hub: 集成了上面所有项目的各个功能, 你可以以项目和版本为单位, 将你的描述文件上传到SwaggerHub中。在Swagger Hub 中可以完成上面项目的所有工作, 需要注册账号, 分免费版和收费版

使用Swagger, 就是把相关的信息存储在它定义的描述文件里面 (yaml或json格式), 再通过维护这个描述文件可以去更新接口文档, 以及生成各端代码。

4.2 Springfox

使用Swagger 时如果碰见版本更新或迭代时, 只需要更改 Swagger 的描述文件即可。但是在频繁的更新项目版本时很多开发人员认为即使修改描述文件 (yaml或json) 也是一定的工作负担, 久而久之就直接修改代码, 而不去修改描述文件了, 这样基于描述文件生成接口文档也失去了意义。

Marty Pitt 编写了一个基于 Spring 的组件 swagger-springmvc。Spring-fox 就是根据这个组件发展而来的全新项目。

Spring-fox 是根据代码生成接口文档, 所以正常的进行更新项目版本, 修改代码即可, 而不需要跟随修改描述文件。Spring-fox 利用自身 AOP 特性, 把Swagger集成进来, 底层还是 Swagger。但是使用起来确实方便很多。所以在实际开发中, 都是直接使用spring-fox。

官网地址: <http://springfox.github.io/springfox/>

官方源码: <https://github.com/springfox/springfox>

4.3 Swagger的使用

4.3.1 创建SpringBoot项目, 并配置Swagger

编写SpringBoot 项目, 项目中controller 中包含一个处理器方法, 测试项目, 保证程序可以正确运行。

(1) 引入相关依赖

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>2.1.6.RELEASE</version>
</parent>

<dependencies>
```

```

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
    <groupId>io.springfox</groupId>
    <artifactId>springfox-swagger2</artifactId>
    <version>2.9.2</version>
</dependency>
<dependency>
    <groupId>io.springfox</groupId>
    <artifactId>springfox-swagger-ui</artifactId>
    <version>2.9.2</version>
</dependency>
</dependencies>

```

在项目的pom.xml中导入了Spring-fox依赖。目前最新版本为2.9.2，所以导入的依赖也是这个版本。其中springfox-swagger2是核心内容的封装。springfox-swagger-ui 是对 swagger-ui 的封装。

(2) 编写启动类和配置文件

```

@SpringBootApplication
@EnableSwagger2
public class App {
    public static void main(String[] args) {
        SpringApplication.run(App.class, args);
    }
}

```

在SpringBoot 的启动类中添加@EnableSwagger2注解，添加此注解后表示对当前项目中全部控制器进行扫描。

```

server:
    port: 8888

```

(3) 编写控制器相关代码

```

package com.xq.pojo;

public class Person {
    private Integer id;
    private String username;
    private String address;
}

```

```

    public Integer getId() {
        return id;
    }

    public void setId(Integer id) {
        this.id = id;
    }

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getAddress() {
        return address;
    }

    public void setAddress(String address) {
        this.address = address;
    }
}

```

```

package com.xq.controller;

import com.xq.pojo.Person;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("person")
public class PersonController {

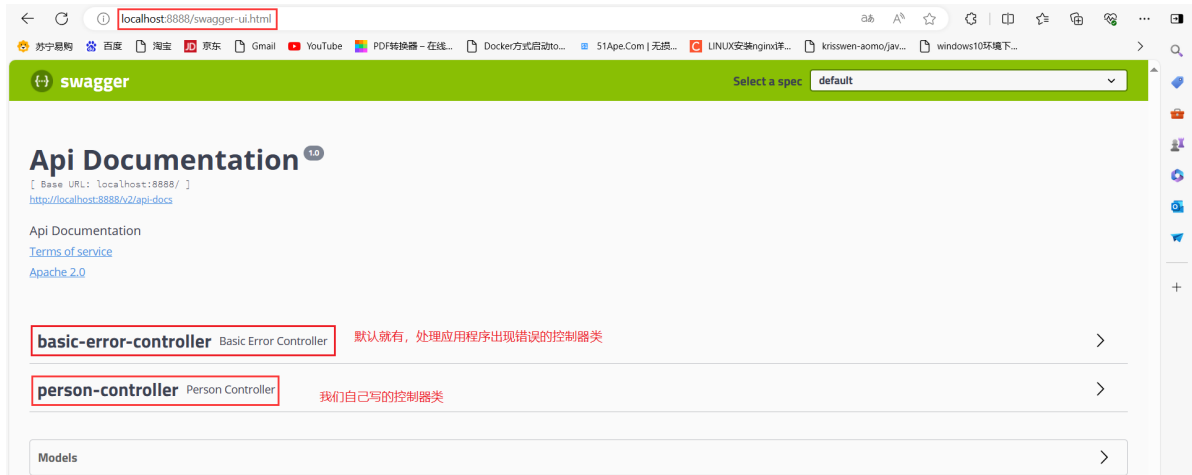
    @RequestMapping("findPerson")
    public Person findPerson(@RequestParam("id") Integer id,
        @RequestParam("username") String username){
        Person person = new Person();
        person.setId(id);
        person.setUsername(username);
        person.setAddress("USA");
        return person;
    }
}

```

```
}
```

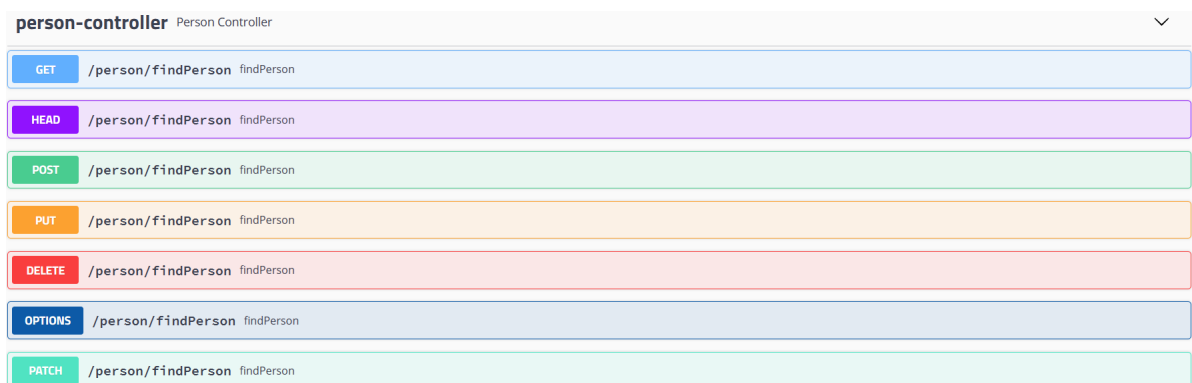
4.3.2 使用Swagger

启动项目后在浏览器中输入<http://ip:port/swagger-ui.html>即可以访问到swagger-ui 页面，在页面中可以可视化的进行操作项目中所有接口。

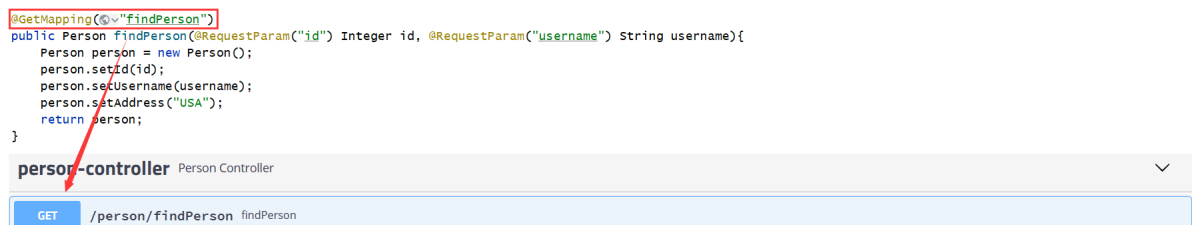


访问swagger-ui.html 后可以在页面中看到所有需要生成接口文档的控制器名称。

每个控制器中间包含多所有控制器方法的各种访问方式。如果使用的是 @RequestMapping 进行映射，将显示下面的所有请求方式。



如果使用@GETtMapping 将只有Post方式可以能访问，下面也就只显示GET的一个。



我们还还原到@RequestMapping注解修饰控制器方法，点击某个请求方式中try it out, 我们以GET请求为例:

GET

/person/findPerson findPerson

Try it out

Parameters

Name	Description
id * required integer (\$int32) (query)	id
username * required string (query)	username

会出现界面要求输入的值。输入完成后点击Execute按钮:

GET

/person/findPerson findPerson

Cancel

Parameters

Name	Description
id * required integer (\$int32) (query)	id 10086
username * required string (query)	username LebronJames

Execute

下面会出现RequestURL已经不同状态码相应回来的结果。

Code

Details

200

Response body

```
{
  "id": 10086,
  "username": "LebronJames",
  "address": "USA"
}
```

Download

Response headers

```
content-type: application/json;charset=UTF-8
date: Mon, 13 May 2024 04:19:09 GMT
transfer-encoding: chunked
```

Responses

Code	Description
200	OK Example Value Model <pre>{ "address": "string", "id": 0, "username": "string" }</pre>

4.4 Swagger配置

4.4.1 配置基本信息

我们可以在项目中创建SwaggerConfig，进行配置文档内容。

```
import com.xq.annotation.NotIncludeSwagger;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import springfox.documentation.builders.ApiInfoBuilder;
import springfox.documentation.builders.PathSelectors;
import springfox.documentation.builders.RequestHandlerSelectors;
import springfox.documentation.service.ApiInfo;
import springfox.documentation.service.Contact;
import springfox.documentation.spi.DocumentationType;
```

```

import springfox.documentation.spring.web.plugins.Docket;

import static com.google.common.base.Predicates.not;
import static com.google.common.base.Predicates.or;
import static
springfox.documentation.builders.RequestHandlerSelectors.withMethodAnnotation;

@Configuration
public class SwaggerConfig {
    /**
     * 创建Docket类型的对象,并使用Spring容器管理
     * Docket是Swagger中的全局配置对象
     * @return
     */
    @Bean
    public Docket getDocket() {
        return new Docket(DocumentationType.SWAGGER_2)
            .apiInfo(swaggerDemoApiInfo())
            .select()
            .build();
    }

    private ApiInfo swaggerDemoApiInfo(){
        return new ApiInfoBuilder()
            .contact(new Contact("JAVA学习社区",
"https://www.qq.com",
"eric@qq.com")) //配置文档主体内容
            //文档标题
            .title("这里是Swagger的标题")
            //文档描述
            .description("这里是Swagger的描述")
            //文档版本
            .version("1.0.0")
            .build();
    }
}

```

显示效果如下：



这里是Swagger的标题 ^{1.0.0}
[Base URL: localhost:8888/]
<http://localhost:8888/v2/api-docs>
 这里是Swagger的描述
[JAVA学习社区 - Website](#)
[Send email to JAVA学习社区](#)

4.4.2 配置包扫描

有时候我们希望不要展示 basic-error-controller 相关的接口，可以通过apis()方法设置哪个包中内容被扫描。

```
@Bean
public Docket getDocket() {
    return new Docket(DocumentationType.SWAGGER_2)
        .apiInfo(swaggerDemoApiInfo()) Docket
        .select() ApiSelectorBuilder
        .apis(RequestHandlerSelectors.basePackage("com.xq.controller"))
        .build();
}
```

这里是Swagger的标题 ^{1.0.0}

[Base URL: localhost:8888/]
<http://localhost:8888/y2/api-docs>

这里是Swagger的描述

[JAVA学习社区 - Website](#)

[Send email to JAVA学习社区](#)

只有我们自己定义的控制类了

person-controller Person Controller >

Models

Person > {...}

4.4.3 自定义注解设置不需要生成接口文档的方法

有时候我们不希望某些控制器方法生成接口文档。我们可以结语自定义注解设置实现。

(1) 自定义注解

```
@Target({ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)
public @interface NotIncludesSwagger {
}
```

扩展：自定义注解所需要的元注解

- @Retention：定义注解的保留策略。
- @Retention(RetentionPolicy.SOURCE) //注解仅存在于源码中，在class字节码文件中不包含。
- @Retention(RetentionPolicy.CLASS) //默认的保留策略，注解会在class字节码文件中存在，但运行时无法获得。
- @Retention(RetentionPolicy.RUNTIME) //注解会在class字节码文件中存在，在运行时可以通过反射获取到。
- @Target：指定被修饰的Annotation可以放置的位置(被修饰的目标)。
- @Target(ElementType.TYPE) //接口、类
- @Target(ElementType.FIELD) //属性
- @Target(ElementType.METHOD) //方法
- @Target(ElementType.PARAMETER) //方法参数

- @Target(ElementType.CONSTRUCTOR) //构造函数
 - @Target(ElementType.LOCAL_VARIABLE) //局部变量
 - @Target(ElementType.ANNOTATION_TYPE) //注解
 - @Target(ElementType.PACKAGE) //包
- 注：可以指定多个位置，例如：
- @Target({ElementType.METHOD, ElementType.TYPE})，也就是此注解可以在方法和类上面使用。
- @Inherited：指定被修饰的Annotation将具有继承性。
 - @Documented：指定被修饰的该Annotation可以被javadoc工具提取成文档。

(2) 配置自定义注解，让其生效

```
@Bean
public Docket getDocket() {
    return new Docket(DocumentationType.SWAGGER_2)
        .apiInfo(swaggerDemoApiInfo()) Docket
        .select() ApiSelectorBuilder
        .apis(RequestHandlerSelectors.basePackage("com.xq.controller"))
        .apis(not(withMethodAnnotation(NotIncludeSwagger.class)))
        .build();
}
```

(3) 添加NotIncludeSwagger 注解

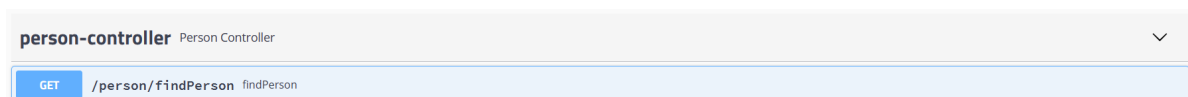
在不需要生成接口文档的方法上面添加@NotIncludeSwagger注解后，该方法将不会被Swagger进行生成在接口文档中。

```
@RestController
@RequestMapping("/person")
public class PersonController {

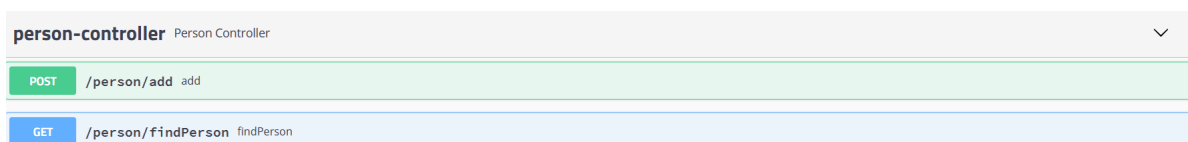
    no usages
    @GetMapping("/findPerson")
    public Person findPerson(@RequestParam("id") Integer id, @RequestParam("username") String username){
        Person person = new Person();
        person.setId(id);
        person.setUsername(username);
        person.setAddress("USA");
        return person;
    }

    @PostMapping("/add")
    @NotIncludeSwagger //不需要生成对应控制器方法的接口文档
    public Person add(@RequestBody Person person){
        return person;
    }
}
```

重启项目查看效果：



我们解开注释，再次查看效果：

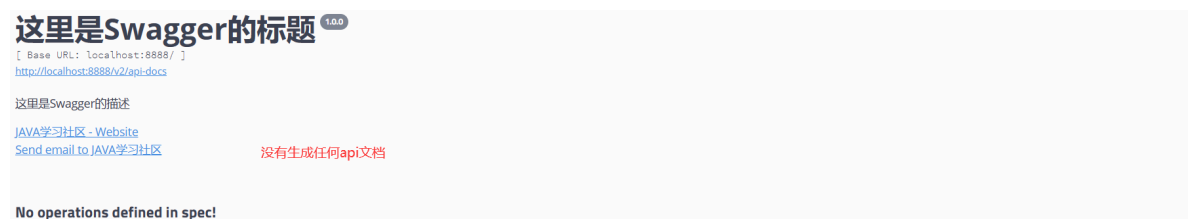


4.4.4 设置范围

我们可以设置满足什么样规则的url被生成接口文档。可以使用正则表达式进行匹配。下面例子中表示只有以/user/开头的url才能被swagger生成接口文档。

```
@Bean
public Docket getDocket() {
    return new Docket(DocumentationType.SWAGGER_2)
        .apiInfo(swaggerDemoApiInfo()) Docket
        .select() ApiSelectorBuilder
        .apis(RequestHandlerSelectors.basePackage("com.xq.controller"))
        .apis(not(withMethodAnnotation(NotIncludeSwagger.class)))
        // 只有路径是/user或者/person开头的控制器方法生成接口文档
        .paths(or(PathSelectors.regex( pathRegex: "/user/.*"),PathSelectors.regex( pathRegex: "/people/.*")))
        .build();
}
```

重启项目再次打开文档：



4.5 Swagger2常用注解

4.5.1 @Api

@Api 是类上注解。控制整个类生成接口信息的内容。 tags：类的名称。可以有多值，多个值表示多个副本。 description:描述，已过时。

代码示例：

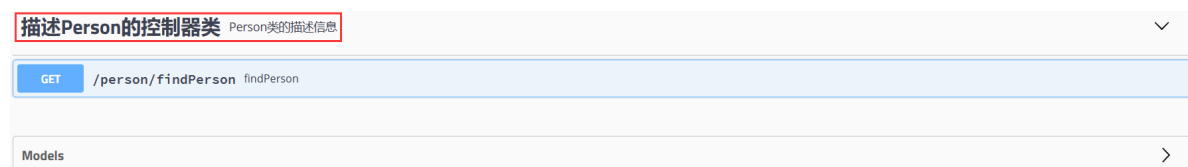
```
@RestController
@RequestMapping("/person")
@Api(tags = {"描述Person的控制器类"},description = "Person类的描述信息")
public class PersonController {

    no usages

    @GetMapping("/findPerson")
    public Person findPerson(@RequestParam("id") Integer id, @RequestParam("username") String username){
        Person person = new Person();
        person.setId(id);
        person.setUsername(username);
        person.setAddress("USA");
        return person;
    }

    @PostMapping("/add")
    @NotIncludeSwagger //不需要生成对应控制器方法的接口文档
    public Person add(@RequestBody Person person){
        return person;
    }
}
```

在swagger-ui.html 中显示效果。



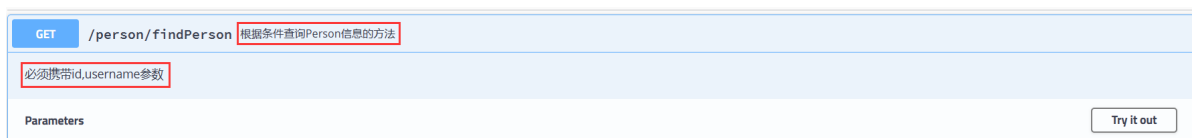
4.5.2 @ApiOperation

@ApiOperation 写在方法上，对方法进行总体描述。value：接口描述。notes：提示信息

代码示例：

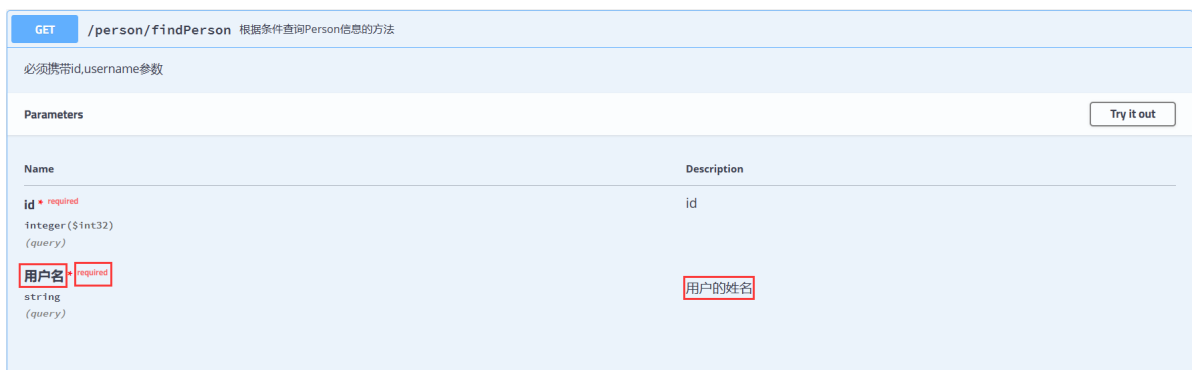
```
no usages
@GetMapping(value = "/person/findPerson")
@ApiOperation(value = "根据条件查询Person信息的方法", notes = "必须携带id,username参数")
public Person findPerson(@RequestParam("id") Integer id, @RequestParam("username") String username){
    Person person = new Person();
    person.setId(id);
    person.setUsername(username);
    person.setAddress("USA");
    return person;
}
```

在swagger-ui.html 中显示效果。



4.5.3 @ApiParam

@ApiParam 写在方法参数前面。用于对参数进行描述或说明是否为必填项等说明。
name：参数名称 value：参数描述 required：是否是必须



4.5.4 @ApiModelProperty

@ApiModelProperty 是类上注解，主要应用Model，也就是说这个注解一般都是写在实体类上。value：名称 description：描述

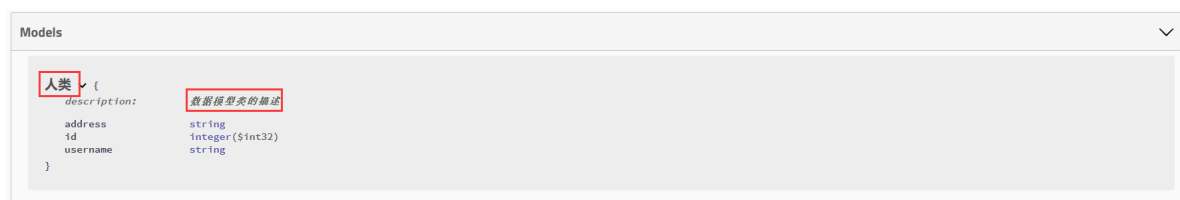
代码示例：

```
import io.swagger.annotations.ApiModel;

6 usages
@ApiModel(value = "人类", description = "数据模型类的描述")
public class Person {
    2 usages
    private Integer id;
    2 usages
    private String username;
    2 usages
    private String address;

    no usages
    public Integer getId() {
        return id;
    }
}
```

在swagger-ui.html 中显示效果。



4.5.5 @ApiModelProperty

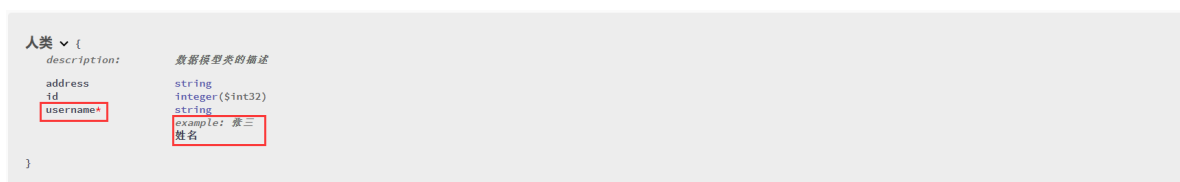
@ApiModelProperty 可以用在方法或属性上。用于当对象作为参 数时定义这个字段的内容。

value: 描述 name: 重写属性名 required: 是否是必须的 example: 示例内容 hidden: 是否隐藏。

```
@ApiModelProperty(value = "人类",description = "数据模型类的描述")
public class Person {
    2 usages
    private Integer id;

    2 usages
    @ApiModelProperty(value = "姓名",name = "username",required = true,example = "张三")
    private String username;
    2 usages
    private String address;
```

在swagger-ui.html 中显示效果。



4.5.6 @ApiIgnore

@ApiIgnore 用于方法或类或参数上，表示这个方法或类被忽略,不生成相关的API文档。和之前讲解的自定义注解@NotIncludeSwagger效果类似。只是这个注解是Swagger内置的注解，而@NotIncludeSwagger是我们自定义的注解。

```
@PostMapping(🔗"add")
//@NotIncludeSwagger //不需要生成对应控制器方法的接口文档
@ApiIgnore
public Person add(@RequestBody Person person){
    return person;
}
```

4.5.7 @ApiImplicitParam

@ApiImplicitParam 用在方法上，表示单独的请求参数，总体功能 和@ApiParam 类似。

name: 属性名 value: 描述 required: 是否是必须的 paramType: 属性类型
dataType: 数据类型

代码示例:

```

@GetMapping(value = "/findPerson")
@ApiOperation(value = "根据条件查询Person信息的方法", notes = "必须携带id,username参数")
@ApiImplicitParam(name = "address", value = "地址", required = true, paramType = "query", dataType = "string")
public Person findPerson(@RequestParam("id") Integer id,
    @RequestParam("username")
    @ApiParam(name = "用户名", value = "用户的姓名", required = true) String username,
    @RequestParam("address") String address){
    Person person = new Person();
    person.setId(id);
    person.setUsername(username);
    person.setAddress("USA");
    return person;
}

```

在swagger-ui.html 中显示效果。



5 角色管理模块接口开发

为了方便我们使用swagger测试接口，在项目中首先我们引入swagger相关的依赖。注意：我们把项目素材中swagger3的依赖去除掉，我们使用课程中给大家讲解的swagger2的版本。

在父工程的pom.xml中注释掉swagger3的版本。

```

<dependency>
  <groupId>commons-io</groupId>
  <artifactId>commons-io</artifactId>
  <version>${commons-io.version}</version>
</dependency>
<!--<dependency>
  <groupId>io.springfox</groupId>
  <artifactId>springfox-boot-starter</artifactId>
  <version>${swagger2.version}</version>
</dependency>-->

```

注释掉，使用swagger2的版本，否则访问api页面报错

在web模块的pom.xml中引入swagger2的依赖：

```

<!-- springfox api文档 -->
<!--<dependency>
  <groupId>io.springfox</groupId>
  <artifactId>springfox-boot-starter</artifactId>
</dependency>-->

<dependency>
  <groupId>io.springfox</groupId>
  <artifactId>springfox-swagger2</artifactId>
  <version>2.9.2</version>

```

```
</dependency>
<dependency>
    <groupId>io.springfox</groupId>
    <artifactId>springfox-swagger-ui</artifactId>
    <version>2.9.2</version>
</dependency>
```

5.1 后端功能代码开发

5.1.1 创建实体类

```
package com.xq.web.sys_role.entity;

import com.baomidou.mybatisplus.annotation.IdType;
import com.baomidou.mybatisplus.annotation.TableId;
import com.baomidou.mybatisplus.annotation.TableName;
import lombok.Data;
import java.util.Date;
@Data
@TableName("sys_role")
public class SysRole {
    @TableId(type = IdType.AUTO)
    private Long roleId;
    private String roleName;
    private String remark;
    private Date createTime;
    private Date updateTime;
}
```

为了方便分页查询，我们还定义一个封装分页查询的数据模型。

```
package com.xq.web.sys_role.entity;

import lombok.Data;
@Data
public class RoleParam {
    private Long currentPage;//当前页码
    private Long pageSize;//页面数据容量
    private String roleName;//角色名称
}
```

5.1.2 创建mapper

```
public interface SysRoleMapper extends BaseMapper<SysRole> {  
  
}
```

新建映射文件,在resources目录下面新建mapper目录, 然后新建 SysRoleMapper.xml文件

```
<!DOCTYPE mapper  
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"  
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">  
<mapper namespace="com.xq.web.sys_role.mapper.SysRoleMapper">  
  
</mapper>
```

修改包扫描规则:

```
import com.baomidou.mybatisplus.annotation.DbType;  
import com.baomidou.mybatisplus.extension.plugins.MybatisPlusInterceptor;  
import com.baomidou.mybatisplus.extension.plugins.inner.PaginationInnerInterceptor;  
import org.mybatis.spring.annotation.MapperScan;  
import org.springframework.context.annotation.Bean;  
import org.springframework.context.annotation.Configuration;  
no usages  
@Configuration  
@MapperScan("com.xq.*.*.mapper")  
public class MyBatisPlusConfig {  
    // 最新版  
    no usages  
    @Bean  
    public MybatisPlusInterceptor mybatisPlusInterceptor() {  
        MybatisPlusInterceptor interceptor = new MybatisPlusInterceptor();  
        interceptor.addInnerInterceptor(new PaginationInnerInterceptor(DbType.MYSQL));  
        return interceptor;  
    }  
}
```

5.1.3 创建Service业务类

我们定义一个通用service接口:

```
import com.baomidou.mybatisplus.core.metadata.IPage;  
import com.baomidou.mybatisplus.extension.service.IService;  
import com.xq.web.sys_role.entity.RoleParam;  
import com.xq.web.sys_role.entity.SysRole;  
public interface SysRoleService extends IService<SysRole> {  
    IPage<SysRole> list(RoleParam param);  
}
```

接下来我们定义业务接口的实现类:

```
package com.xq.web.sys_role.service.impl;
```



```

import
com.baomidou.mybatisplus.core.conditions.query.QueryWrapper;
import com.baomidou.mybatisplus.core.metadata.IPage;
import
com.baomidou.mybatisplus.extension.plugins.pagination.Page;
import
com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
import com.xq.web.sys_role.entity.RoleParam;
import com.xq.web.sys_role.entity.SysRole;
import com.xq.web.sys_role.mapper.SysRoleMapper;
import com.xq.web.sys_role.service.SysRoleService;
import org.apache.commons.lang.StringUtils;
import org.springframework.stereotype.Service;

@Service
public class SysRoleServiceImpl extends
ServiceImpl<SysRoleMapper, SysRole> implements SysRoleService {

    @Override
    public IPage<SysRole> list(RoleParam param) {
        //构造分页对象
        IPage<SysRole> page = new Page<>();
        page.setSize(param.getPageSize());
        page.setCurrent(param.getCurrentPage());
        //构造查询条件
        QueryWrapper<SysRole> query = new QueryWrapper<>();
        if(StringUtils.isNotEmpty(param.getRoleName())){

            query.lambda().like(SysRole::getRoleName,param.getRoleName());
        }
        return this.baseMapper.selectPage(page,query);
    }
}

```

5.1.4 定义控制器类

定义控制器类，用来接收并处理前端发送过来的请求：

```

package com.xq.web.sys_role.controller;

import com.baomidou.mybatisplus.core.metadata.IPage;
import com.xq.utils.ResultUtils;
import com.xq.utils.ResultVo;
import com.xq.web.sys_role.entity.RoleParam;
import com.xq.web.sys_role.entity.SysRole;

```

```
import com.xq.web.sys_role.service.SysRoleService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.Date;

@RestController
@RequestMapping("/api/role")
public class SysRoleController {
    @Autowired
    private SysRoleService sysRoleService;

    //新增角色
    @PostMapping
    public ResultVo addRole(@RequestBody SysRole role){
        role.setCreateTime(new Date());
        boolean save = sysRoleService.save(role);
        if(save){
            return ResultUtils.success("新增成功!");
        }
        return ResultUtils.error("新增失败!");
    }

    //编辑角色
    @PutMapping
    public ResultVo editRole(@RequestBody SysRole role){
        role.setUpdateTime(new Date());
        boolean save = sysRoleService.updateById(role);
        if(save){
            return ResultUtils.success("编辑成功!");
        }
        return ResultUtils.error("编辑失败!");
    }

    //删除角色
    @DeleteMapping("/{roleId}")
    public ResultVo deleteRole(@PathVariable("roleId") Long
roleId){
        boolean b = sysRoleService.removeById(roleId);
        if(b){
            return ResultUtils.success("删除成功!");
        }
        return ResultUtils.error("删除失败!");
    }

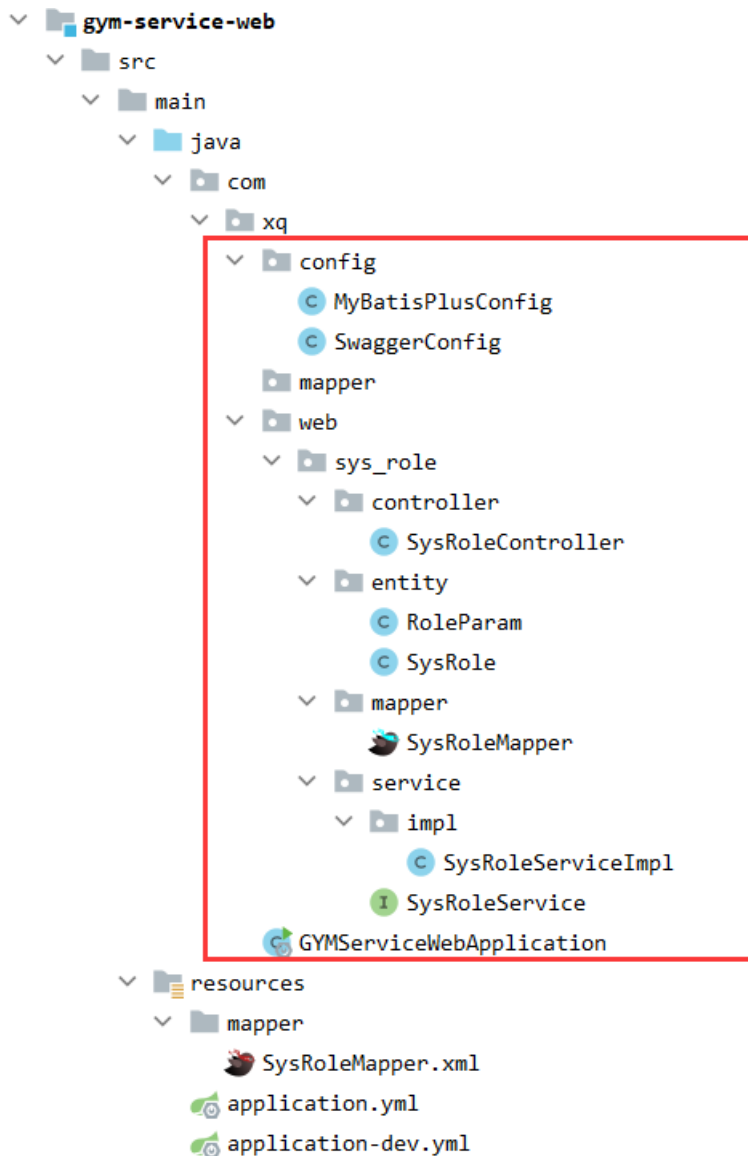
    //角色列表
    @GetMapping("/list")
```

```

    public ResultVo getList(RoleParam param){
        IPage<SysRole> list = sysRoleService.list(param);
        return ResultUtils.success("查询成功",list);
    }
}

```

整个代码目录结构如下：



5.1.5 使用swagger进行测试

第一步：引入swagger所需要的配置类

```

package com.xq.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import springfox.documentation.builders.ApiInfoBuilder;
import springfox.documentation.service.ApiInfo;
import springfox.documentation.service.Contact;
import springfox.documentation.spi.DocumentationType;

```

```

import springfox.documentation.spring.web.plugins.Docket;
@Configuration
public class SwaggerConfig{

    /**
     * 创建Docket类型的对象,并使用Spring容器管理
     * Docket是Swagger中的全局配置对象
     * @return
     */
    @Bean
    public Docket getDocket() {
        return new Docket(DocumentationType.SWAGGER_2)
            .apiInfo(swaggerDemoApiInfo())
            .select()
            .build();
    }

    private ApiInfo swaggerDemoApiInfo(){
        return new ApiInfoBuilder()
            .contact(new Contact("JAVA学习社区",
                "https://www.qq.com",
                "eric@qq.com")) //配置文档主体内容
            //文档标题
            .title("这里是Swagger的标题")
            //文档描述
            .description("这里是Swagger的描述")
            //文档版本
            .version("1.0.0")
            .build();
    }
}

```

第二步：访问Swagger UI页面进行测试：

Swagger UI

sys-role-controller Sys Role Controller	
POST	/api/role addRole
PUT	/api/role editRole
DELETE	/api/role/{roleId} deleteRole
GET	/api/role/list getList

5.2 角色管理功能前端代码

5.2.1 定义数据类型

经过分析，我们需要定义角色数据和分页查询数据的数据模型。在src下新建api，然后新建role目录，并建RoleModel.ts

```
// 角色数据类型
export type AddRoleModel={
  type:string,
  roleId:string,
  roleName:string,
  remark:string
}

//列表查询参数类型
export type ListParm = {
  roleName:string,
  currentPage:number,
  pageSize:number,
  total:number
}
```

5.2.2 封装对角色模块进行业务操作的js代码

在src下新建composables目录，然后新建role目录，并建useRole.ts、useTable.ts

(1) useRole.ts

```
import { AddRoleModel } from '@api/role/RoleModel'
import { reactive } from 'vue'
export default function useRole() {
  //新增
  const addBtn = () => {
  }
  //编辑
  const editBtn = () => {
  }
  //删除
  const deleteBtn = () => {
  }
  return {
    addBtn,
    editBtn,
    deleteBtn
  }
}
```

```
}  
}
```

(2) useTable.ts

```
import { reactive } from "vue";  
import { ListParm } from "@/api/role/RoleModel";  
export default function useTable(){  
  //列表参数  
  const listParm = reactive<ListParm>({  
    roleName: '',  
    currentPage:1,  
    pageSize:10,  
    total:0  
  })  
  //分页  
  //列表查询  
  //搜索  
  const searchBtn = ()=>{  
  }  
  //重置  
  const resetBtn = ()=>{  
  }  
  return{  
    listParm,  
    searchBtn,  
    resetBtn  
  }  
}
```

5.2.3 封装RoleList.vue组件

```
<template>  
  <el-main>  
    <!-- 搜索栏 -->  
    <el-form :model="listParm" :inline="true" size="default">  
      <el-form-item>  
        <el-input  
          placeholder="请输入角色名称"  
          v-model="listParm.roleName"  
        ></el-input>  
      </el-form-item>  
      <el-form-item>  
        <el-button @click="searchBtn" :icon="Search">查询</el-  
button>
```

```

        <el-button @click="resetBtn" plain :icon="Close"
type="danger">重置</el-button>
        <el-button type="primary" :icon="Plus" @click="addBtn">
新增</el-button>
    </el-form-item>
</el-form>
</el-main>
</template>
<script setup lang="ts">
    import { Search, Edit, Plus, Close, Delete } from '@element-
plus/icons-vue';
    import useTable from "@/composables/role/useTable";
    import useRole from "@/composables/role/useRole";
    //表格相关操作
    const { listParm, searchBtn, resetBtn } = useTable();
    //新增、编辑、删除
    const { addBtn, editBtn, deleteBtn } = useRole();
</script>
<style scoped></style>

```

5.3 新增角色前端页面联调

5.3.1 system/role目录下新建RoleAdd.vue组件

```

<template>
    <sys-dialog
        :title="dialog.title"
        :width="dialog.width"
        :height="dialog.height"
        :visible="dialog.visible"
        @onClose="onClose"
        @onConfirm="commit"
    >
        <template v-slot:content>
            <el-form
                :model="addModel"
                ref="addFormRef"
                :rules="rules"
                label-width="80px"
                size="default"
            >
                <!-- el-row代表一行，24等分，平均分为 12分 -->
                <el-row>
                    <el-col :span="12" :offset="0">

```

```

        <el-form-item prop="rolename" label="角色名称">
            <el-input v-model="addModel.rolename"></el-input>
        </el-form-item>
    </el-col>
    <el-col :span="12" :offset="0">
        <el-form-item prop="remark" label="备注">
            <el-input v-model="addModel.remark"></el-input>
        </el-form-item>
    </el-col>
</el-row>
</el-form>
</template>
</sys-dialog>
</template>

```

```

<script setup lang="ts">
import { AddRoleModel } from "@api/role/RoleModel";
import SysDialog from "@components/SysDialog.vue";
import useDialog from "@hooks/useDialog";
import { reactive, ref } from "vue";
import { FormInstance } from "element-plus";
import { addApi } from "@api/role/index";
//表单的ref属性
const addFormRef = ref<FormInstance>();
//弹框属性
const { dialog, onClose, onConfirm } = useDialog();
//定义show给父组件调用
const show = () => {
    dialog.height = 150;
    dialog.width = 630;
    dialog.title = "新增";
    dialog.visible = true;
};
//暴露子组件的方法给外部使用（父组件）
defineExpose({
    show,
});
//定义表单绑定的属性
const addModel = reactive<AddRoleModel>({
    type: "",
    roleId: "",
    roleName: "",
    remark: "",
});
//表单验证规则

```



```

const rules = reactive({
  roleName: [
    {
      required: true,
      trigger: "change",
      message: "请填写角色名称",
    },
  ],
});
//表单提交
const commit = () => {
  //表单验证规则 1: 表单需要添加ref属性, 2 : item上面需要添加prop属性
  3: 定义表单验证规则
  addFormRef.value?.validate(async (valid) => {
    if (valid) {
      let res = await addApi(addModel);
      if (res && res.code == 200) {
        dialog.visible = false;
      }
    }
  });
};
</script>

<style scoped></style>

```

5.3.2 修改useRole.ts

```

import { ref } from "vue"
export default function useRole() {
  //新增弹框的ref属性(子组件将方法暴露出去, 父组件就可以调用子组件的方法)
  const addRef = ref<{ show: () => void }>()
  //新增
  const addBtn = () => {
    addRef.value?.show()
  }
  //编辑
  const editBtn = () => {

  }
  //删除
  const deleteBtn = () => {

  }
  return {

```

```

        addBtn,
        editBtn,
        deleteBtn,
        addRef
    }
}

```

5.3.3 src/api/role目录， 并建index.ts

```

import http from "@/http";
import { AddRoleModel } from "../RoleModel";
//新增
export const addApi = (param:AddRoleModel)=>{
    return http.post("/api/role",param)
}

```

5.3.4 RoleList.vue组件中引入RoleAdd.vue组件

```

<template>
  <el-main>
    <!-- 搜索栏 -->
    <el-form :model="listParm" :inline="true" size="default">
      <el-form-item>
        <el-input
          placeholder="请输入角色名称"
          v-model="listParm.roleName"
        ></el-input>
      </el-form-item>
      <el-form-item>
        <el-button @click="searchBtn" :icon="Search">查询</el-
button>
        <el-button @click="resetBtn" plain :icon="Close"
type="danger">重置</el-button>
        <el-button type="primary" :icon="Plus" @click="addBtn">
新增</el-button>
      </el-form-item>
    </el-form>
    <!-- 新增、编辑弹框 -->
    <role-add ref="addRef" ></role-add>
  </el-main>
</template>
<script setup lang="ts">
  import { Search, Edit, Plus, Close, Delete } from '@element-
plus/icons-vue';

```

```

import useTable from "@/composables/role/useTable";
import useRole from "@/composables/role/useRole";
import RoleAdd from "@/RoleAdd.vue";
//表格相关操作
const { listParm, searchBtn, resetBtn } = useTable();
//新增、编辑、删除
const { addBtn, editBtn, deleteBtn, addRef } = useRole();
</script>
<style scoped></style>

```

5.3.5 解决跨域问题

我们再后端代码中定义一个配置类，解决跨域问题。

```

package com.xq.config;

import org.springframework.context.annotation.Configuration;
import
org.springframework.web.servlet.config.annotation.CorsRegistry;
import
org.springframework.web.servlet.config.annotation.WebMvcConfigure
r;

/**
 * 解决跨域问题
 */
@Configuration
public class WebMvcConfig implements WebMvcConfigurer {
    /**
     * 跨域配置
     * @param registry
     */
    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/**")
            .allowedOriginPatterns("*")
            .allowedMethods("*")
            .allowedHeaders("*")
            .maxAge(3600)
            .allowCredentials(true);
    }
}

```

5.4 角色列表前端对接

5.4.1 定义查询请求

在api/role/index.ts添加getListApi

```
import http from "@/http";
import { AddRoleModel } from "../RoleModel";
//新增
export const addApi = (param:AddRoleModel)=>{
    return http.post("/api/role",param)
}

//查询列表
export const getListApi = (parm: ListParam) => {
    return http.get("/api/role/list", parm)
}
```

5.4.2 修改useTable.ts

useTable.ts修改为如下:

```
import { nextTick, onMounted, reactive, ref } from "vue";
import { ListParam } from '@api/role/RoleModel'
import { getListApi } from "@api/role";
export default function useTable(){
    //定义表格高度
    const tableHeight = ref(0)
    //定义表格数据
    const tableList = reactive({
        list: []
    })
    //列表参数
    const listParm = reactive<ListParam>({
        roleName: '',
        currentPage:1,
        pageSize:10,
        total:0
    })

    //列表
    const getList = async () => {
        let res = await getListApi(listParm)
        if (res && res.code == 200) {
            console.log(res)
        }
    }
}
```

```
        //设置表格数据
        tableList.list = res.data.records
        //设置分页的总条数
        listParm.total = res.data.total
    }
}
//搜索
const searchBtn = () => {
    getList()
}
//重置
const resetBtn = () => {
    listParm.roleName = ''
    getList()
}
//页容量改变时触发
const sizeChange = (val: number) => {
    listParm.pageSize = val
    getList()
}
//页数改变时触发
const currentChange = (val: number) => {
    listParm.currentPage = val
    getList()
}
//刷新列表
const refresh = ()=>{
    getList()
}
onMounted(() => {
    getList()
    nextTick(() => {
        tableHeight.value = window.innerHeight - 230
    })
})
return {
    listParm,
    getList,
    searchBtn,
    resetBtn,
    tableList,
    sizeChange,
    currentChange,
    tableHeight,
    refresh
}
```

```
}  
}
```

5.4.3 修改RoleList.vue组件

```
<template>  
  <el-main>  
    <!-- 搜索栏 -->  
    <el-form :model="listParm" :inline="true" size="default">  
      <el-form-item>  
        <el-input  
          v-model="listParm.roleName"  
          placeholder="请输入角色名称"  
        ></el-input>  
      </el-form-item>  
      <el-form-item>  
        <el-button @click="searchBtn" :icon="Search">搜索</el-  
button>  
        <el-button @click="resetBtn" :icon="Close" type="danger"  
plain>重置</el-button>  
        <el-button type="primary" @click="addBtn" :icon="Plus">新  
增</el-button>  
      </el-form-item>  
    </el-form>  
    <!-- 表格 -->  
    <el-table :height="tableHeight" :data="tableList.list" border  
stripe>  
      <el-table-column prop="roleName" label="角色名称"></el-table-  
column>  
      <el-table-column prop="remark" label="角色描述"></el-table-  
column>  
      <el-table-column label="操作" width="220" align="center">  
        <template #default="scope">  
          <el-button type="primary" :icon="Edit" size="default"  
@click="editBtn">编辑</el-button>  
          <el-button type="danger" :icon="Delete"  
size="default"  
@click="editBtn">删除</el-button>  
        </template>  
      </el-table-column>  
    </el-table>  
    <!-- 分页 -->  
    <el-pagination  
      @size-change="sizeChange"  
      @current-change="currentChange"
```

```

        :current-page.sync="listParm.currentPage"
        :page-sizes="[10,20, 40, 80, 100]"
        :page-size="listParm.pageSize"
        layout="total, sizes, prev, pager, next, jumper"
        :total="listParm.total" background>
    </el-pagination>

    <!-- 新增、编辑弹框 -->
    <role-add ref="addRef" @refresh="refresh"></role-add>
</el-main>
</template>
<script setup lang="ts">
import RoleAdd from "../RoleAdd.vue";
import { Plus, Edit, Delete, Search, Close } from "@element-plus/icons-vue";
import useTable from "@/hooks/useTable";
import useRole from "@/composables/role/useRole";
//表格相关的业务
const { listParm, getList, searchBtn,
resetBtn,tableList,sizeChange
,currentChange,tableHeight,refresh} = useTable();
//增删改查业务
const { addBtn, editBtn, deleteBtn,addRef } = useRole();
</script>
<style scoped></style>

```

5.4.4 新增成功之后，刷新列表

原理：子组件调用父组件的方法刷新列表

修改RoleAdd.vue

```

<template>
  <sys-dialog
    :title="dialog.title"
    :width="dialog.width"
    :height="dialog.height"
    :visible="dialog.visible"
    @onClose="onClose"
    @onConfirm="commit"
  >
    <template v-slot:content>
      <el-form
        :model="addModel"
        ref="addFormRef"

```

```

        :rules="rules"
        label-width="80px"
        size="default"
    >
    <!-- el-row代表一行，24等分，平均分为 12分 -->
    <el-row>
        <el-col :span="12" :offset="0">
            <el-form-item prop="roleName" label="角色名称">
                <el-input v-model="addModel.roleName"></el-input>
            </el-form-item>
        </el-col>
        <el-col :span="12" :offset="0">
            <el-form-item prop="remark" label="备注">
                <el-input v-model="addModel.remark"></el-input>
            </el-form-item>
        </el-col>
    </el-row>
</el-form>
</template>
</sys-dialog>
</template>

<script setup lang="ts">
import { AddRoleModel } from "@api/role/RoleModel";
import SysDialog from "@components/SysDialog.vue";
import useDialog from "@hooks/useDialog";
import { reactive, ref } from "vue";
import { ElMessage, FormInstance } from "element-plus";
import { addApi } from "@api/role/index";
//表单的ref属性
const addFormRef = ref<FormInstance>();
//弹框属性
const { dialog, onClose, onConfirm } = useDialog();
//定义show给父组件调用
const show = () => {
    dialog.height = 150;
    dialog.width = 630;
    dialog.title = "新增";
    dialog.visible = true;
    addFormRef.value?.resetFields()
};
//暴露子组件的方法给外部使用（父组件）
defineExpose({
    show,
});

```



```

//定义表单绑定的属性
const addModel = reactive<AddRoleModel>({
  type: "",
  roleId: "",
  roleName: "",
  remark: "",
});
//表单验证规则
const rules = reactive({
  roleName: [
    {
      required: true,
      trigger: "change",
      message: "请填写角色名称",
    },
  ],
});
//注册事件
const emit = defineEmits(['refresh'])
//表单提交
const commit = () => {
  //表单验证规则 1: 表单需要添加ref属性, 2 : item上面需要添加prop属性
  3: 定义表单验证规则
  addFormRef.value?.validate(async (valid) => {
    if (valid) {
      let res = await addApi(addModel);
      if (res && res.code == 200) {
        ElMessage.success(res.msg)
        //调用父组件的方法刷新列表
        emit('refresh')
        dialog.visible = false;
      }
    }
  });
};
</script>

<style scoped></style>

```

5.5 编辑、删除前端对接

5.5.1 信息确认框封装

(1) src目录下新建utils目录，然后新建myconfirm.ts

```
/**
 * 信息确定提示框
 */
import { ElMessageBox } from 'element-plus'
export default function myconfirm(text: string) {
  return new Promise((resolve, reject) => {
    ElMessageBox.confirm(text, '系统提示', {
      confirmButtonText: '确定',
      cancelButtonText: '取消',
      type: 'warning',
    })
      .then(() => { //确定按钮
        resolve(true)
      })
      .catch(() => { //取消
        reject(false)
      })
  }).catch(() => {
    return false;
  })
}
```

(2) utils下新建objCorry.ts

```
/**
 * 对象快速复制: obj1复制到obj2
 * @param obj1
 * @param obj2
 */
export default function objCorry(obj1:any,obj2:any){
  Object.keys(obj2).forEach(key =>{
    obj2[key] = obj1[key]
  })
}
```

(3) main.ts全局挂载

```
import { createApp } from 'vue'
import App from './App.vue'
import router from '@/router/index'
//引入element plus
import ElementPlus from 'element-plus'
```

```

import 'element-plus/dist/index.css'
import * as ElementPlusIconsVue from '@element-plus/icons-vue'
//引入Pinia构造函数
import { createPinia } from 'pinia'
// 实例化 Pinia
const pinia = createPinia()
//国际化
import zhCn from 'element-plus/es/locale/lang/zh-cn'
import objCoppY from './utils/objCoppY'
//信息确认框
import myconfirm from './utils/myconfirm'
const app = createApp(App);
app.use(router).use(ElementPlus, {
  locale: zhCn,
}).use(pinia)
app.mount('#app')
//挂载全局属性
app.config.globalProperties.$objCoppY = objCoppY;
app.config.globalProperties.$myconfirm = myconfirm;
//全局注册图标组件
for (const [key, component] of
Object.entries(ElementPlusIconsVue)) {
  app.component(key, component)
}

```

5.5.2 角色编辑、删除接口对接

(1) 在hooks下新建useInstance工具，获取全局挂载属性

```

import { getCurrentInstance, ComponentInternalInstance } from
"vue";
export default function useInstance(){
  const {appContext,proxy} = getCurrentInstance() as
ComponentInternalInstance
  const global = appContext.config.globalProperties;
  return{
    proxy,
    global
  }
}

```

(2) type下新建BaseEnum.ts和BaseType.ts

- BaseEnum.ts

```

export enum EditType{
    ADD = '0', //代表新增
    EDIT = '1' //代表编辑
}

//定义弹框的标题
export enum Title {
    ADD = '新增',
    EDIT = '编辑'
}

export enum UserType {
    ADMIN = '1',
    STUDENT = '2',
}

```

- **BaseType.ts**

```

//弹框属性类型
export type DialogModel = {
    title: string,
    visible: boolean,
    height: number,
    width: number
}

//通用函数类型
export type FuncList = () => any

```

(3) api/role新增editApi()和deleteApi()方法

```

import http from "@/http";
import { AddRoleModel } from "../RoleModel";
//新增
export const addApi = (param:AddRoleModel)=>{
    return http.post("/api/role",param)
}

//查询列表
export const getListApi = (parm: ListParam) => {
    return http.get("/api/role/list", parm)
}

//删除
export const deleteApi = (roleId:string)=>{
    return http.delete(`/api/role/${roleId}`)
}

```

```
//编辑
export const editApi = (parm: AddRoleModel) => {
  return http.put("/api/role", parm)
}
```

(4) 调整useRole.ts

```
import { AddRoleModel } from '@api/role/RoleModel'
import { EditType } from '@type/BaseEnum'
import { FuncList } from '@type/BaseType'
import { ref } from 'vue'
import { deleteApi } from '@api/role'
import { ElMessage } from 'element-plus'
import useInstance from '@hooks/useInstance'
export default function useRole(getList: FuncList) {
  const { global } = useInstance()
  //新增、编辑组件ref属性
  const addRef = ref<{ show: (type: string, row?: AddRoleModel)
=> void }>()
  //新增
  const addBtn = () => {
    addRef.value?.show(EditType.ADD)
  }
  //编辑
  const editBtn = (row: AddRoleModel) => {
    addRef.value?.show(EditType.EDIT, row);
  }
  //删除
  const deleteBtn = async (row: AddRoleModel) => {
    let confirm = await global.$myconfirm('确定删除该数据吗?')
    if (confirm) {
      let res = await deleteApi(row.roleId)
      if (res && res.code == 200) {
        ElMessage.success(res.msg)
        getList()
      }
    }
  }

  return {
    addBtn,
    editBtn,
    deleteBtn,
    addRef
  }
}
```

```
}  
}
```

(5) 调整RoleAdd.vue

```
<template>  
  <sys-dialog  
    :title="dialog.title"  
    :width="dialog.width"  
    :height="dialog.height"  
    :visible="dialog.visible"  
    @onClose="onClose"  
    @onConfirm="commit"  
  >  
    <template v-slot:content>  
      <el-form  
        :model="addModel"  
        ref="addFormRef"  
        :rules="rules"  
        label-width="80px"  
        size="default"  
      >  
        <el-form-item prop="roleName" label="角色名称">  
          <el-input v-model="addModel.roleName"></el-input>  
        </el-form-item>  
        <el-form-item prop="roleName" label="角色类型">  
          <el-select  
            style="width: 100%"  
            v-model="addModel.types"  
            class="m-2"  
            placeholder="请选择角色类型"  
            size="default"  
          >  
            <el-option  
              v-for="item in options"  
              :key="item.value"  
              :label="item.label"  
              :value="item.value"  
            />  
          </el-select>  
        </el-form-item>  
        <el-form-item prop="remark" label="备注">  
          <el-input v-model="addModel.remark"></el-input>  
        </el-form-item>  
      </el-form>  
    </template>  
  </sys-dialog>  
</template>
```

```

    </template>
  </sys-dialog>
</template>

<script setup lang="ts">
import { AddRoleModel } from "@/api/role/RoleModel";
import SysDialog from "@/components/SysDialog.vue";
import useDialog from "@/hooks/useDialog";
import { nextTick, reactive, ref } from "vue";
import { ElMessage, FormInstance } from "element-plus";
import { addApi, editApi } from "@/api/role/index";
import { EditType, Title } from "@/type/BaseEnum";
import useInstance from "@/hooks/useInstance";
const { global } = useInstance();
//表单的ref属性
const addFormRef = ref<FormInstance>();
//定义表单绑定的属性
const addModel = reactive<AddRoleModel>({
  type: "",
  roleId: "",
  roleName: "",
  remark: "",
  // types: "",
});
//角色类型
const options = ref([
  {
    value: "1",
    label: "员工类型",
  },
  {
    value: "2",
    label: "会员类型",
  },
]);

//弹框属性
const { dialog, onClose } = useDialog();
//定义show给父组件调用
const show = (type: string, row?: AddRoleModel) => {
  dialog.height = 150;
  dialog.width = 630;
  //设置标题
  type == EditType.ADD ? (dialog.title = Title.ADD) :
(dialog.title = Title.EDIT);

```

```

//如果是编辑，需要回显数据
if (type == EditType.EDIT) {
  nextTick(() => {
    global.$objCoppY(row, addModel);
  });
}
//区分是新增还是编辑
addModel.type = type;
dialog.visible = true;
addFormRef.value?.resetFields();
};
//暴露子组件的方法给外部使用（父组件）
defineExpose({
  show,
});

//表单验证规则
const rules = reactive({
  roleName: [
    {
      required: true,
      trigger: "change",
      message: "请填写角色名称",
    },
  ],
});
//注册事件
const emit = defineEmits(["refresh"]);
//表单提交
const commit = () => {
  //表单验证规则 1： 表单需要添加ref属性， 2： item上面需要添加prop属性
  3: 定义表单验证规则
  addFormRef.value?.validate(async (valid) => {
    if (valid) {
      let res = null;
      if (addModel.type == EditType.ADD) {
        //新增
        res = await addApi(addModel);
      } else {
        res = await editApi(addModel);
      }
      if (res && res.code == 200) {
        ElMessage.success(res.msg);
        //调用父组件的方法刷新列表
        emit("refresh");
      }
    }
  });
}

```



```

        dialog.visible = false;
    }
}
});
};
</script>

<style scoped></style>

```

(6) 调整RoleList.vue

```

<template>
  <el-main>
    <!-- 搜索栏 -->
    <el-form :model="listParm" :inline="true" size="default">
      <el-form-item>
        <el-input
          v-model="listParm.roleName"
          placeholder="请输入角色名称"
        ></el-input>
      </el-form-item>
      <el-form-item>
        <el-button @click="searchBtn" :icon="Search">搜索</el-
button>
        <el-button @click="resetBtn" :icon="Close" type="danger"
plain>重置</el-button>
        <el-button type="primary" @click="addBtn" :icon="Plus">新
增</el-button>
      </el-form-item>
    </el-form>
    <!-- 表格 -->
    <el-table :height="tableHeight" :data="tableList.list" border
stripe>
      <el-table-column prop="roleName" label="角色名称"></el-table-
column>
      <el-table-column prop="remark" label="角色描述"></el-table-
column>
      <el-table-column label="操作" width="220" align="center">
        <template #default="scope">
          <el-button type="primary" :icon="Edit" size="default"
@click="editBtn(scope.row)">编辑</el-button>
          <el-button type="danger" :icon="Delete"
size="default" @click="deleteBtn(scope.row)">删除</el-button>
        </template>
      </el-table-column>

```

```

</el-table>
<!-- 分页 -->
<el-pagination
  @size-change="sizeChange"
  @current-change="currentChange"
  :current-page.sync="listParm.currentPage"
  :page-sizes="[10,20, 40, 80, 100]"
  :page-size="listParm.pageSize"
  layout="total, sizes, prev, pager, next, jumper"
  :total="listParm.total" background>
</el-pagination>

<!-- 新增、编辑弹框 -->
<role-add ref="addRef" @refresh="refresh"></role-add>
</el-main>
</template>
<script setup lang="ts">
import RoleAdd from "./RoleAdd.vue";
import { Plus, Edit, Delete, Search, Close } from "@element-
plus/icons-vue";
import useTable from "@/hooks/useTable";
import useRole from "@/composables/role/useRole";
//表格相关的业务
const { listParm, getList, searchBtn,
resetBtn,tableList,sizeChange
,currentChange,tableHeight,refresh} = useTable();
//增删改查业务
const { addBtn, editBtn, deleteBtn,addRef } = useRole(getList);
</script>
<style scoped></style>

```

6 用户管理模块接口开发

6.1 后端功能代码开发

6.1.1 新建实体类

新建com.xq.web.sys_user.entity包，然后新建SysUser实体类

```

package com.xq.web.sys_user.entity;

import com.baomidou.mybatisplus.annotation.IdType;
import com.baomidou.mybatisplus.annotation.TableField;
import com.baomidou.mybatisplus.annotation.TableId;

```

```

import com.baomidou.mybatisplus.annotation.TableName;
import lombok.Data;

import java.math.BigDecimal;
import java.util.Date;

@Data
@TableName("sys_user")
public class SysUser {
    @TableId(type = IdType.AUTO)
    private Long userId;
    //表明roleId字段不属于sys_user表，需要排除
    @TableField(exist = false)
    private Long roleId;
    private String username;
    private String password;
    private String phone;
    private String email;
    private String sex;
    private String isAdmin;
    //类型（1: 员工 2: 教练）
    private String userType;
    //状态 0: 停用 1: 启用
    private String status;
    private BigDecimal salary;
    //帐户是否过期(1 未过期, 0已过期)
    private boolean isAccountNonExpired = true;
    //帐户是否被锁定(1 未锁定, 0已锁定)
    private boolean isAccountNonLocked = true;
    //密码是否过期(1 未过期, 0已过期)
    private boolean isCredentialsNonExpired = true;
    //帐户是否可用(1 可用, 0 删除用户)
    private boolean isEnabled = true;
    private String nickName;
    //创建时间
    private Date createTime;
    //更新时间
    private Date updateTime;
}

```

分页查询参数封装实体：

```
@Data
public class PageParam {
    private Long currentPage;
    private Long pageSize;
    private String phone;
    private String nickName;
}
```

因为在项目中我们要操作用户表和角色表的中间表，所以我们也提供中间表(sys_user_role)对象的实体。

新建com.xq.web.sys_user_role.entity包，然后新建SysUserRole实体类

```
@Data
@TableName("sys_user_role")
public class SysUserRole {
    @TableId(type = IdType.AUTO)
    private Long userRoleId;
    private Long userId;
    private Long roleId;
}
```

6.1.2 新建mapper接口

新建com.xq.web.sys_user.mapper包，然后新建SysUserMapper接口

```
public interface SysUserMapper extends BaseMapper<SysUser> {
}
```

新建com.xq.web.sys_user_role.mapper包，然后新建SysUserRoleMapper接口

```
public interface SysUserRoleMapper extends
BaseMapper<SysUserRole> {
}
```

6.1.3 新建mapper接口映射文件

在resources目录下新建mapper目录，然后新建SysUserMapper.xml文件和SysUserRoleMapper.xml文件。

```
<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="com.xq.web.sys_user.mapper.SysUserMapper">

</mapper>
```

```
<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper
namespace="com.xq.web.sys_user_role.mapper.SysUserRoleMapper">

</mapper>
```

6.1.4 新建service业务层

新建com.xq.web.sys_user.service包，然后新建SysUserService接口

```
package com.xq.web.sys_user.service;

import com.baomidou.mybatisplus.core.metadata.IPage;
import com.baomidou.mybatisplus.extension.service.IService;
import com.xq.web.sys_user.entity.PageParam;
import com.xq.web.sys_user.entity.SysUser;

public interface SysUserService extends IService<SysUser> {
    IPage<SysUser> list(PageParam param);
}
```

新建com.xq.web.sys_user_role.service包，然后新建SysUserRoleService接口

```
package com.xq.web.sys_user_role.service;

import com.baomidou.mybatisplus.extension.service.IService;
import com.xq.web.sys_user_role.entity.SysUserRole;

public interface SysUserRoleService extends IService<SysUserRole>
{
}
```

新建com.xq.web.sys_user.service.impl包，然后新建SysUserService接口的实现类

```

package com.xq.web.sys_user.service.impl;

import
com.baomidou.mybatisplus.core.conditions.query.QueryWrapper;
import com.baomidou.mybatisplus.core.metadata.IPage;
import
com.baomidou.mybatisplus.extension.plugins.pagination.Page;
import
com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
import com.xq.web.sys_user.entity.PageParam;
import com.xq.web.sys_user.entity.SysUser;
import com.xq.web.sys_user.mapper.SysUserMapper;
import com.xq.web.sys_user.service.SysUserService;
import org.apache.commons.lang.StringUtils;
import org.springframework.stereotype.Service;

@Service
public class SysUserServiceImpl extends
ServiceImpl<SysUserMapper, SysUser> implements SysUserService {

    @Override
    public IPage<SysUser> list(PageParam parm) {
        //构造分页对象
        IPage<SysUser> page = new Page<>();
        page.setSize(parm.getPageSize());
        page.setCurrent(parm.getCurrentPage());
        QueryWrapper<SysUser> query = new QueryWrapper<>();
        //构造查询条件
        if(StringUtils.isNotEmpty(parm.getNickName())){

            query.lambda().like(SysUser::getNickName,parm.getNickName());
        }
        if(StringUtils.isNotEmpty(parm.getPhone())){

            query.lambda().like(SysUser::getPhone,parm.getPhone());
        }
        return this.baseMapper.selectPage(page,query);
    }
}

```

新建com.xq.web.sys_user_role.service.impl包，然后新建SysUserRoleService接口的实现类

```

import
com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
import com.xq.web.sys_user_role.entity.SysUserRole;
import com.xq.web.sys_user_role.mapper.SysUserRoleMapper;
import com.xq.web.sys_user_role.service.SysUserRoleService;
import org.springframework.stereotype.Service;

@Service
public class SysUserRoleServiceImpl extends
ServiceImpl<SysUserRoleMapper, SysUserRole> implements
SysUserRoleService {
}

```

6.1.5 定义控制器类

新建com.xq.web.sys_user.controller包，然后新建SysUserController控制器类

```

package com.xq.web.sys_user.controller;

import
com.baomidou.mybatisplus.core.conditions.query.QueryWrapper;
import com.baomidou.mybatisplus.core.metadata.IPage;
import com.xq.utils.ResultUtils;
import com.xq.utils.ResultVo;
import com.xq.web.sys_user.entity.PageParam;
import com.xq.web.sys_user.entity.SysUser;
import com.xq.web.sys_user.service.SysUserService;
import com.xq.web.sys_user_role.entity.SysUserRole;
import com.xq.web.sys_user_role.service.SysUserRoleService;
import org.apache.commons.lang.StringUtils;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.util.DigestUtils;
import org.springframework.web.bind.annotation.*;

import java.util.Date;

@RestController
@RequestMapping("/api/user")
public class SysUserController {
    @Autowired
    private SysUserService sysUserService;

    @Autowired

```

```

private SysUserRoleService sysUserRoleService;

//新增员工
@PostMapping
public ResultVo addUser(@RequestBody SysUser sysUser){
    //判断用户名是否存在
    QueryWrapper<SysUser> query = new QueryWrapper<>();

    query.lambda().eq(SysUser::getUsername,sysUser.getUsername());
    SysUser one = sysUserService.getOne(query);
    if(one != null){
        return ResultUtils.error("账户已经被占用");
    }
    //密码加密
    if(StringUtils.isNotEmpty(sysUser.getPassword())){
        sysUser.setPassword(DigestUtils.md5DigestAsHex(sysUser.getPassword().getBytes()));
    }
    sysUser.setIsAdmin("0");
    sysUser.setCreateTime(new Date());
    //存入数据库
    boolean save = sysUserService.save(sysUser);
    if(save){
        return ResultUtils.success("新增用户成功!");
    }
    return ResultUtils.error("新增用户失败!");
}

//编辑员工
@PutMapping
public ResultVo editUser(@RequestBody SysUser sysUser){
    //判断用户名是否存在
    QueryWrapper<SysUser> query = new QueryWrapper<>();

    query.lambda().eq(SysUser::getUsername,sysUser.getUsername());
    SysUser one = sysUserService.getOne(query);
    if(one != null && one.getUserId() != sysUser.getUserId())
    {
        return ResultUtils.error("账户已经被占用");
    }
    //密码加密
    if(StringUtils.isNotEmpty(sysUser.getPassword())) {

```



```

        sysUser.setPassword(DigestUtils.md5DigestAsHex(sysUser.getPassword().getBytes()));
    }
    sysUser.setUpdateTime(new Date());
    //存入数据库
    boolean save = sysUserService.updateById(sysUser);
    if(save){
        return ResultUtils.success("编辑用户成功!");
    }
    return ResultUtils.error("编辑用户失败!");
}

//删除用户
@DeleteMapping("/{userId}")
public ResultVo deleteUser(@PathVariable("userId") Long
userId){
    boolean remove = sysUserService.removeById(userId);
    if(remove){
        return ResultUtils.success("删除用户成功!");
    }
    return ResultUtils.error("删除用户失败!");
}

//用户列表
@GetMapping("/list")
public ResultVo getList(PageParam param){
    IPage<SysUser> list = sysUserService.list(param);
    //密码不显示
    list.getRecords().stream().forEach(item ->{
        item.setPassword("");
    });
    return ResultUtils.success("查询成功",list);
}

//根据用户id查询角色id
@GetMapping("/role")
public ResultVo getRole(Long userId){
    QueryWrapper<SysUserRole> query = new QueryWrapper<>();
    query.lambda().eq(SysUserRole::getUserId,userId);
    SysUserRole one = sysUserRoleService.getOne(query);
    return ResultUtils.success("查询成功",one);
}
}

```

6.2 新增员工角色获取

6.2.1 在sys_role模块添加如下实体类

在sys_role/entity目录下创建实体类：

```
package com.xq.web.sys_role.entity;

import lombok.Data;

@Data
public class SelectType {
    private Long value;
    private String label;
}
```

6.2.2 添加控制器方法

在SysRoleController控制器添加getListSelect()方法

```
@GetMapping("/getSelect")
public ResultVo getListSelect(){
    List<SysRole> list = sysRoleService.list();
    List<SelectType> selectTypeList = new ArrayList<>();
    if(list.size() > 0){
        list.stream().forEach(item ->{
            SelectType type = new SelectType();
            type.setValue(item.getRoleId());
            type.setLabel(item.getRoleName());
            selectTypeList.add(type);
        });
    }
    return ResultUtils.success("查询成功",selectTypeList);
}
```

6.2.3 在api目录中添加user目录,然后新建UserModel.ts和index.ts

- UserModel.ts中添加：

```
//定义角色类型
export type RoleType = {
  value: string,
  label: string
}
export type SelectRole = {
  list: RoleType[]
}
```

- index.ts:

```
import http from "@http";
//查询角色列表
export const getSelectApi = ()=>{
  return http.get("/api/role/getSelect")
}
```

6.2.4 composables目录新建user目录，然后新建useSelectRole.ts

```
import { reactive } from "vue";
import { getSelectApi } from "@api/user";
import { selectRole } from "@api/user/UserModel";
export default function useSelectRole() {
  //定义角色数据
  const roleData = reactive<SelectRole>({
    list: []
  })
  //获取列表
  const listRole = async () => {
    let res = await getSelectApi()
    if (res && res.code == 200) {
      console.log(res)
      roleData.list = res.data;
    }
  }
  return {
    roleData,
    listRole
  }
}
```

6.3 新增用户页面制作与对接

6.3.1 在api/user的UserModel.ts添加如下代码

```
//定义角色类型
export type RoleType = {
  value: string,
  label: string
}

export type SelectRole = {
  list: RoleType[]
}

//列表查询参数类型
export type ListParam = {
  phone: string,
  nickName: string,
  currentPage: number,
  pageSize: number,
  total: number
}

//用户数据类型
export type AddUserModel = {
  userId: string,
  type: string,
  roleId: string,
  username: string,
  password: string,
  phone: string,
  email: string,
  sex: string,
  userType: string,
  status: string,
  salary: string,
  nickName: string
}
```

6.3.2 api/user/index.ts添加addApi

```
import http from "@/http";
import { AddUserModel, ListParam } from "../UserModel";
//获取角色列表
export const getSelectApi = ()=>{
```

```

    return http.get("/api/role/getSelect")
  }
  //新增
  export const addApi = (param:AddUserModel)=>{
    return http.post("/api/user",param)
  }
  //用户列表
  export const getListApi = (param:ListParam)=>{
    return http.get("/api/user/list",param)
  }
  //编辑
  export const editApi = (param:AddUserModel)=>{
    return http.put('/api/user',param)
  }
  //删除
  export const deleteApi = (userId:string)=>{
    return http.delete(`/api/user/${userId}`)
  }
  //根据用户id查询角色
  export const getRoleApi = (userId:string)=>{
    return http.get("/api/user/role",{userId:userId})
  }
}

```

6.3.3 composables下新建user目录，然后新建useTable.ts和useUser.ts

- useTable.ts

```

import { ListParam } from "@api/user/UserModel";
import { nextTick, onMounted, reactive, ref } from "vue";
import { getListApi } from "@api/user";
export default function useTable() {
  //查询参数
  const listParam = reactive<ListParam>({
    phone: '',
    pageSize: 10,
    currentPage: 1,
    nickName: '',
    total: 0
  })
  //列表
  const getList = async()=>{

  }
  //搜索

```

```

const searchBtn = ()=>{

}
//重置
const resetBtn = ()=>{

}
//页容量改变时触发
const sizeChange = (size:number)=>{

}
//页数改变时触发
const currentChange = (page:number)=>{

}
//刷新列表
const refresh = ()=>{

}
onMounted(()=>{

})
return{
    tableList,
    listParam,
    getList,
    searchBtn,
    resetBtn,
    sizeChange,
    currentChange,
    tableHeight,
    refresh
}
}

```

- useUser.ts

```

import { AddUserModel } from "@/api/user/UserModel"
import { EditType } from "@/type/BaseEnum"
import { ref } from "vue"
import { deleteApi } from "@/api/user"
import { ElMessage } from "element-plus"
import { FuncList } from "@/type/BaseType"
import useInstance from "@/hooks/useInstance"

```

```

export default function useUser(){
  //新增弹框属性
  const addRef = ref<{show:()=>void}>()
  //新增
  const addBtn = ()=>{
    addRef.value?.show()
  }
  //编辑
  const editBtn = ()=>{

  }
  //删除
  const deleteBtn = ()=>{

  }
  return{
    addBtn,
    editBtn,
    deleteBtn,
    addRef
  }
}

```

6.4 用户列表页面制作与对接

6.4.1 api/user/index.ts添加查询接口

```

import http from "@/http";
import { AddUserModel, ListParam } from "@/UserModel";
//获取角色列表
export const getSelectApi =()=>{
  return http.get("/api/role/getSelect")
}
//新增
export const addApi = (param:AddUserModel)=>{
  return http.post("/api/user",param)
}
//用户列表
export const getListApi = (param:ListParam)=>{
  return http.get("/api/user/list",param)
}

```

6.4.2 修改composables/user/useTable.ts

```
import { ListParam } from "@/api/user/UserModel";
import { nextTick, onMounted, reactive, ref } from "vue";
import { getListApi } from "@/api/user";
export default function useTable() {
  //表格高度
  const tableHeight = ref(0)
  //表格数据
  const tableList = reactive({
    list:[]
  })
  //查询参数
  const listParam = reactive<ListParam>({
    phone: '',
    pageSize: 10,
    currentPage: 1,
    nickName: '',
    total: 0
  })
  //列表
  const getList = async()=>{
    let res = await getListApi(listParam)
    if(res && res.code == 200){
      //设置表格的数据
      console.log(res.data)
      tableList.list = res.data.records
      //设置分页总条数
      listParam.total = res.data.total
    }
  }
  //搜索
  const searchBtn = ()=>{
    getList()
  }
  //重置
  const resetBtn = ()=>{
    listParam.nickName = ''
    listParam.phone = ''
    getList()
  }
  //页容量改变时触发
  const sizeChange = (size:number)=>{
    listParam.pageSize = size;
    getList()
  }
}
```



```

    }
    //页数改变时触发
    const currentChange = (page:number)=>{
      listParam.currentPage = page;
      getList()
    }
    //刷新列表
    const refresh = ()=>{
      getList()
    }
    onMounted(()=>{
      getList()
      nextTick(()=>{
        tableHeight.value = window.innerHeight - 230
      })
    })
    return{
      tableList,
      listParam,
      getList,
      searchBtn,
      resetBtn,
      sizeChange,
      currentChange,
      tableHeight,
      refresh
    }
  }
}

```

6.4.3 修改UserList.vue组件

```

<template>
  <el-main>
    <!-- 搜索栏 -->
    <el-form :model="listParam" :inline="true" size="default">
      <el-form-item>
        <el-input v-model="listParam.phone" placeholder="请输入电话"></el-input>
      </el-form-item>
      <el-form-item>
        <el-input
          v-model="listParam.nickName"
          placeholder="请输入姓名"
        ></el-input>
      </el-form-item>
    </el-form>
  </el-main>
</template>

```

```

        <el-form-item>
            <el-button @click="searchBtn" :icon="Search">搜索</el-
button>
            <el-button @click="resetBtn" type="danger" plain
:icon="Close">重置</el-button>
            <el-button v-permission="['sys:user:add']" type="primary"
@click="addBtn" :icon="Plus">新增</el-button>
        </el-form-item>
    </el-form>
    <!-- 表格 -->
    <el-table :height="tableHeight" :data="tableList.list" border
stripe>
        <el-table-column prop="nickName" label="姓名"></el-table-
column>
        <el-table-column prop="username" label="登录账户"></el-table-
column>
        <el-table-column prop="sex" label="性别">
            <template #default="scope">
                <el-tag v-if="scope.row.sex == '0'" type="success"
size="default">男</el-tag>
                <el-tag v-if="scope.row.sex == '1'" type="danger"
size="default">女</el-tag>
            </template>
        </el-table-column>
        <el-table-column prop="phone" label="电话"></el-table-
column>
        <el-table-column prop="email" label="邮箱"></el-table-
column>
        <el-table-column prop="userType" label="类型">
            <template #default="scope">
                <el-tag v-if="scope.row.userType == '1'"
type="success" size="default">员工</el-tag>
                <el-tag v-if="scope.row.userType == '2'"
type="danger" size="default">教练</el-tag>
            </template>
        </el-table-column>
        <el-table-column prop="salary" label="薪水"></el-table-
column>
        <el-table-column prop="status" label="状态">
            <template #default="scope">
                <el-tag v-if="scope.row.status == '1'" type="success"
size="default">启用</el-tag>
                <el-tag v-if="scope.row.status == '0'" type="danger"
size="default">停用</el-tag>
            </template>

```

```

        </el-table-column>
        <el-table-column label="操作" align="center" width="320">
          <template #default="scope">
            <el-button v-permission="['sys:user:edit']"
type="primary" :icon="Edit" size="default"
@click="editBtn(scope.row)">编辑</el-button>
            <el-button v-permission="['sys:user:delete']"
type="danger" :icon="Delete" size="default"
@click="deleteBtn(scope.row)">删除</el-button>
            <el-button v-permission="['sys:user:resetPassword']"
type="warning" :icon="Edit" size="default"
@click="resetPasBtn(scope.row)">重置密码</el-button>
          </template>
        </el-table-column>
      </el-table>
      <!-- 分页 -->
      <el-pagination
        @size-change="sizeChange"
        @current-change="currentChange"
        :current-page.sync="listParam.currentPage"
        :page-sizes="[10,20, 40, 80, 100]"
        :page-size="listParam.pageSize"
        layout="total, sizes, prev, pager, next, jumper"
        :total="listParam.total" background>
      </el-pagination>

      <!-- 新增、编辑弹框 -->
      <add-user ref="addRef" @refresh="refresh"></add-user>
    </el-main>
  </template>

  <script setup lang="ts">
import AddUser from "../AddUser.vue";
import { Plus, Edit, Delete, Search, Close } from "@element-
plus/icons-vue";
import useTable from "@/composables/user/useTable";
import useUser from "@/composables/user/useUser";
//表格
const { listParam,
getList,searchBtn,resetBtn,tableList,sizeChange,currentChange,tab
leHeight,refresh } = useTable();
//新增、编辑
const { addBtn, editBtn, deleteBtn,addRef } = useUser(getList);
</script>

```

```
<style scoped></style>
```

6.4.4 定义AddUser.vue组件

```
<template>
  <SysDialog
    :title="dialog.title"
    :height="dialog.height"
    :width="dialog.width"
    :visible="dialog.visible"
    @onClose="onClose"
    @onConfirm="commit"
  >
    <template v-slot:content>
      <el-form
        :model="addModel"
        ref="addFormRef"
        :rules="rules"
        label-width="80px"
        size="default"
      >
        <el-row>
          <el-col :span="12" :offset="0">
            <el-form-item prop="nickName" label="姓名">
              <el-input v-model="addModel.nickName"></el-input>
            </el-form-item>
          </el-col>
          <el-col :span="12" :offset="0">
            <el-form-item prop="sex" label="性别">
              <el-radio-group v-model="addModel.sex">
                <el-radio :label="'0'">男</el-radio>
                <el-radio :label="'1'">女</el-radio>
              </el-radio-group>
            </el-form-item>
          </el-col>
        </el-row>
        <el-row>
          <el-col :span="12" :offset="0">
            <el-form-item prop="phone" label="电话">
              <el-input v-model="addModel.phone"></el-input>
            </el-form-item>
          </el-col>
          <el-col :span="12" :offset="0">
            <el-form-item prop="email" label="邮箱">
              <el-input v-model="addModel.email"></el-input>
            </el-form-item>
          </el-col>
        </el-row>
      </el-form>
    </template>
  </SysDialog>
</template>
```

```
        </el-form-item>
      </el-col>
    </el-row>
    <el-row>
      <el-col :span="12" :offset="0">
        <el-form-item prop="salary" label="薪水">
          <el-input v-model="addModel.salary"></el-input>
        </el-form-item>
      </el-col>
      <el-col :span="12" :offset="0">
        <el-form-item prop="userType" label="类型">
          <el-radio-group v-model="addModel.userType">
            <el-radio :label="'1'">员工</el-radio>
            <el-radio :label="'2'">教练</el-radio>
          </el-radio-group>
        </el-form-item>
      </el-col>
    </el-row>
    <el-row>
      <el-col :span="12" :offset="0">
        <el-form-item prop="roleId" label="角色">
          <el-select
            v-model="addModel.roleId"
            class="m-2"
            placeholder="请选择角色"
            size="default"
          >
            <el-option
              v-for="item in roleData.list"
              :key="item.value"
              :label="item.label"
              :value="item.value"
            />
          </el-select>
        </el-form-item>
      </el-col>
      <el-col :span="12" :offset="0">
        <el-form-item prop="status" label="状态">
          <el-radio-group v-model="addModel.status">
            <el-radio :label="'0'">停用</el-radio>
            <el-radio :label="'1'">启用</el-radio>
          </el-radio-group>
        </el-form-item>
      </el-col>
    </el-row>
```

```

        <el-row>
          <el-col :span="12" :offset="0">
            <el-form-item prop="username" label="账户">
              <el-input v-model="addModel.username"></el-input>
            </el-form-item>
          </el-col>
          <el-col v-if="addModel.type == '0'" :span="12"
:offset="0">
            <el-form-item prop="password" label="密码">
              <el-input v-model="addModel.password"></el-input>
            </el-form-item>
          </el-col>
        </el-row>
      </el-form>
    </template>
  </SysDialog>
</template>

<script setup lang="ts">
import { AddUserModel } from "@/api/user/UserModel";
import SysDialog from "@/components/SysDialog.vue";
import useDialog from "@/hooks/useDialog";
import { nextTick, reactive, ref } from "vue";
import useSelectRole from "@/composables/user/useSelectRole";
import { ElMessage, FormInstance } from "element-plus";
import { addApi, editApi } from "@/api/user/index";
import { EditType, Title, UserType } from "@/type/BaseEnum";
import useInstance from "@/hooks/useInstance";
const { global } = useInstance();
//表单ref属性
const addFormRef = ref<FormInstance>();
//角色
const { roleData, listRole, roleId, getRole } = useSelectRole();
//弹框属性
const { dialog, onClose, onShow } = useDialog();
//显示弹框
const show = async (type: string, row?: AddUserModel) => {
  roleId.value = "";
  addModel.roleId = "";
  dialog.height = 270;
  addModel.type = type;
  //获取角色数据
  await listRole(UserType.ADMIN);
  await getRole(row!?.userId);
  type == EditType.ADD

```

```

    ? (dialog.title = Title.ADD)
    : (dialog.title = Title.EDIT);
if (type == EditType.EDIT) {
    //回显数据
    nextTick(() => {
        global.$objCoppY(row, addModel);
        addModel.roleId = roleId.value;
    });
}

onShow();
//清空数据
addFormRef.value?.resetFields();
};
//暴露出去给父组件调用
defineExpose({
    show,
});
//表单绑定的对象
const addModel = reactive<AddUserModel>({
    userId: "",
    type: "",
    roleId: "",
    username: "",
    password: "",
    phone: "",
    email: "",
    sex: "",
    userType: "",
    status: "",
    salary: "",
    nickName: "",
});
//表单验证规则
const rules = reactive({
    nickName: [
        {
            required: true,
            trigger: "change",
            message: "请输入姓名",
        },
    ],
    phone: [
        {
            required: true,

```

```
        trigger: "change",
        message: "请输入电话",
    },
],
sex: [
    {
        required: true,
        trigger: "change",
        message: "请输入选择性别",
    },
],
userType: [
    {
        required: true,
        trigger: "change",
        message: "请输入选择类型",
    },
],
status: [
    {
        required: true,
        trigger: "change",
        message: "请输入选择状态",
    },
],
salary: [
    {
        required: true,
        trigger: "change",
        message: "请输入薪水",
    },
],
username: [
    {
        required: true,
        trigger: "change",
        message: "请输入账户",
    },
],
password: [
    {
        required: true,
        trigger: "change",
        message: "请输入密码",
    },
],
```



```

    ],
    roleId: [
      {
        required: true,
        trigger: "change",
        message: "请选择角色",
      },
    ],
  ],
});
//注册事件
const emits = defineEmits(["refresh"]);
//表单提交
const commit = () => {
  addFormRef.value?.validate(async (valid) => {
    if (valid) {
      let res = null;
      if (addModel.type === EditType.ADD) {
        res = await addApi(addModel);
      } else {
        res = await editApi(addModel);
      }
      if (res && res.code === 200) {
        ElMessage.success(res.msg);
        //刷新表格
        emits("refresh");
        onClose();
      }
    }
  });
};
</script>

<style scoped></style>

```

6.5 员工编辑、删除接口对接

6.5.1 SysUserController添加getRole()方法

```
//根据用户id查询角色id
@GetMapping("/role")
public ResultVo getRole(Long userId){
    QueryWrapper<SysUserRole> query = new QueryWrapper<>();
    query.lambda().eq(SysUserRole::getUserId,userId);
    SysUserRole one = sysUserRoleService.getOne(query);
    return ResultUtils.success("查询成功",one);
}
```

6.5.2 api/user/index.ts添加editApi()、deleteApi()、getRoleApi()方法

```
import http from "@/http";
import { AddUserModel, ListParam } from "../UserModel";
//获取角色列表
export const getSelectApi =()=>{
    return http.get("/api/role/getSelect")
}
//新增
export const addApi = (param:AddUserModel)=>{
    return http.post("/api/user",param)
}
//用户列表
export const getListApi = (param:ListParam)=>{
    return http.get("/api/user/list",param)
}
//编辑
export const editApi = (param:AddUserModel)=>{
    return http.put('/api/user',param)
}
//删除
export const deleteApi = (userId:string)=>{
    return http.delete(`/api/user/${userId}`)
}
//根据用户id查询角色
export const getRoleApi = (userId:string)=>{
    return http.get("/api/user/role",{userId:userId})
}
```

6.5.3 useSelectRole.ts添加getRole()方法

```
import { SelectRole } from "@api/user/UserModel";
import { reactive, ref } from "vue";
import { getSelectApi, getRoleApi } from "@api/user";
export default function useSelectRole() {
  const roleId = ref('')
  //定义角色列表数据
  const roleData = reactive<SelectRole>({
    list: []
  })
  //获取数据
  const listRole = async()=>{
    let res = await getSelectApi()
    if(res && res.code == 200){
      roleData.list = res.data
    }
  }
  //根据用户id查询角色
  const getRole = async(userId:string)=>{
    roleId.value = ''
    let res = await getRoleApi(userId)
    if(res && res.code == 200 && res.data){
      roleId.value = res.data.roleId
    }
  }
  return{
    roleData,
    listRole,
    getRole,
    roleId
  }
}
```

6.5.4 useUser.ts

```
import { AddUserModel } from "@api/user/UserModel"
import { EditType } from "@type/BaseEnum"
import { ref } from "vue"
import { deleteApi } from "@api/user"
import { ElMessage } from "element-plus"
import { FuncList } from "@type/BaseType"
import useInstance from "@hooks/useInstance"
export default function useUser(getList: FuncList) {
```

```

const { global } = useInstance()
//新增组件的ref属性
const addRef = ref<{ show: (type: string, row?: AddUserModel)
=> void }>()
//新增
const addBtn = () => {
    //父组件调用子组件的show方法
    addRef.value?.show(EditType.ADD)
}
//编辑
const editBtn = (row: AddUserModel) => {
    //父组件调用子组件的show方法
    addRef.value?.show(EditType.EDIT, row)
}
//删除
const deleteBtn = async (row: AddUserModel) => {
    const confrim = await global.$myconfirm('确定删除该数据吗?')
    if (confrim) {
        let res = await deleteApi(row.userId)
        if (res && res.code == 200) {
            ElMessage.success(res.msg)
            //刷新表格
            getList()
        }
    }
}
return {
    addBtn,
    editBtn,
    deleteBtn,
    addRef
}
}

```

7 菜单管理模块功能开发

7.1 菜单管理模块后台接口开发

7.1.1 创建实体类

在com.xq.web下面创建sys_menu目录。然后新建如下实体类：

```
package com.xq.web.sys_menu.entity;
```

```

import com.baomidou.mybatisplus.annotation.IdType;
import com.baomidou.mybatisplus.annotation.TableField;
import com.baomidou.mybatisplus.annotation.TableId;
import com.baomidou.mybatisplus.annotation.TableName;
import lombok.Data;

import java.util.ArrayList;
import java.util.Date;
import java.util.List;

@Data
@TableName("sys_menu")
public class SysMenu {
    @TableId(type = IdType.AUTO)
    private Long menuId;
    private Long parentId;
    private String title;
    private String code;
    private String name;
    private String path;
    private String url;
    private String type;
    private String icon;
    @TableField(exist = false)
    private Boolean open;
    private String parentName;
    private Integer orderNum;
    private Date createTime;
    private Date updateTime;

    @TableField(exist = false)
    private List<SysMenu> children = new ArrayList<>();
}

```

描述路由信息的实体类：

```

@Data
@JsonInclude(JsonInclude.Include.NON_EMPTY)
public class RouterVO {
    private String path;
    private String component;
    private String name;
    private String redirect;
    private Meta meta;
}

```

```

@Data
@AllArgsConstructor
public class Meta {
    private String title;
    private String icon;
    private Object[] roles;
}
private List<RouterVO> children =new ArrayList<>();
}

```

7.1.2 新建树形构造工具

```

package com.xq.web.sys_menu.entity;

import org.springframework.beans.BeanUtils;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

public class MakeMenuTree {
    //构造菜单树
    public static List<SysMenu> makeTree(List<SysMenu> menuList,
Long pid) {
        List<SysMenu> list = new ArrayList<>();
        Optional.ofNullable(menuList).orElse(new ArrayList<>())
            .stream()
            .filter(item -> item != null &&
item.getParentId().equals(pid))
            .forEach(item -> {
                SysMenu menu = new SysMenu();
                BeanUtils.copyProperties(item, menu);
                //递归查找下级,自己调用自己
                List<SysMenu> children = makeTree(menuList,
item.getMenuId());
                menu.setChildren(children);
                list.add(menu);
            });
        return list;
    }
    //构造路由数据

```

```

    public static List<RouterVO> makeRourer(List<SysMenu>
menuList, Long pid){
    //构建存放路由数据的容器
    List<RouterVO> list = new ArrayList<>();
    Optional.ofNullable(menuList).orElse(new ArrayList<>())
        .stream()
        .filter(item -> item != null &&
item.getParentId().equals(pid))
        .forEach(item ->{
            RouterVO router = new RouterVO();
            router.setName(item.getName());
            router.setPath(item.getPath());
            //设置children 递归调用: 自己调用自己
            List<RouterVO> children =
makeRourer(menuList, item.getMenuId());
            router.setChildren(children);
            if(item.getParentId() == 0L){//如果是上级是0,那
么他的component是Layout
                router.setComponent("Layout");
                //判断该数据是目录还是菜单
                if(item.getType().equals("1")){ //如果一级
菜单是 菜单类型, 单独处理
                    router.setRedirect(item.getPath());
                    //菜单需要设置children
                    List<RouterVO> listChild = new
ArrayList<>();

                    RouterVO child = new RouterVO();
                    child.setName(item.getName());
                    child.setPath(item.getPath());
                    child.setComponent(item.getUrl());
                    child.setMeta(child.new Meta(
                        item.getTitle(),
                        item.getIcon(),
                        item.getCode().split(",")
                    ));
                    listChild.add(child);
                    router.setChildren(listChild);
                    router.setPath(item.getPath()
+"parent");

                    router.setName(item.getName()+"parent");
                }
            }else{
                router.setComponent(item.getUrl());
            }
        })
    }
}

```

```

        router.setMeta(router.new Meta(
            item.getTitle(),
            item.getIcon(),
            item.getCode().split(",")
        ));
        list.add(router);
    });
    return list;
}
}

```

7.1.3 新建mapper层

```

public interface SysMenuMapper extends BaseMapper<SysMenu> {
}

```

7.1.4 创建映射文件

```

<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="com.xq.web.sys_menu.mapper.SysMenuMapper">

</mapper>

```

7.1.5 新建service层

```

public interface SysMenuService extends IService<SysMenu> {
    List<SysMenu> getParent();
}

```

```

package com.xq.web.sys_menu.service.impl;

import
com.baomidou.mybatisplus.core.conditions.query.QueryWrapper;
import
com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
import com.xq.web.sys_menu.entity.MakeMenuTree;
import com.xq.web.sys_menu.entity.SysMenu;
import com.xq.web.sys_menu.mapper.SysMenuMapper;
import com.xq.web.sys_menu.service.SysMenuService;
import org.springframework.stereotype.Service;

```



```

import java.util.Arrays;
import java.util.List;

@Service
public class SysMenuServiceImpl extends
ServiceImpl<SysMenuMapper, SysMenu> implements SysMenuService {
    @Override
    public List<SysMenu> getParent() {
        //查询目录和菜单
        String[] type = {"0","1"};
        List<String> strings = Arrays.asList(type);
        //构造查询条件
        QueryWrapper<SysMenu> query = new QueryWrapper<>();

        query.lambda().in(SysMenu::getType, strings).orderByAsc(SysMenu::
getOrderNum);
        List<SysMenu> menus = this.baseMapper.selectList(query);
        //组装顶级菜单（默认）
        SysMenu menu = new SysMenu();
        menu.setMenuId(0L);
        menu.setParentId(-1L);
        menu.setTitle("顶级菜单");
        menus.add(menu);
        //组装树数据
        return MakeMenuTree.makeTree(menus, -1L);
    }
}

```

7.1.6 新建控制器

```

package com.xq.web.sys_menu.controller;

import
com.baomidou.mybatisplus.core.conditions.query.QueryWrapper;
import com.xq.utils.ResultUtils;
import com.xq.utils.ResultVo;
import com.xq.web.sys_menu.entity.MakeMenuTree;
import com.xq.web.sys_menu.entity.SysMenu;
import com.xq.web.sys_menu.service.SysMenuService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.Date;
import java.util.List;

```

```
@RestController
@RequestMapping("/api/menu")
public class SysMenuController {
    @Autowired
    private SysMenuService sysMenuService;

    //新增
    @PostMapping
    public ResultVo add(@RequestBody SysMenu sysMenu){
        sysMenu.setCreateTime(new Date());
        if(sysMenuService.save(sysMenu)){
            return ResultUtils.success("新增成功!");
        }
        return ResultUtils.error("新增失败!");
    }

    //编辑
    @PutMapping
    public ResultVo edit(@RequestBody SysMenu sysMenu){
        sysMenu.setUpdateTime(new Date());
        if(sysMenuService.updateById(sysMenu)){
            return ResultUtils.success("编辑成功!");
        }
        return ResultUtils.error("编辑失败!");
    }

    //删除
    @DeleteMapping("/{menuId}")
    public ResultVo delete(@PathVariable("menuId") Long menuId){
        if(sysMenuService.removeById(menuId)){
            return ResultUtils.success("删除成功!");
        }
        return ResultUtils.error("删除失败!");
    }

    @GetMapping("/list")
    public ResultVo list(){
        QueryWrapper<SysMenu> query = new QueryWrapper<>();
        query.lambda().orderByAsc(SysMenu::getOrderNum);
        List<SysMenu> list = sysMenuService.list(query);
        //树形数据组装
        List<SysMenu> menuList = MakeMenuTree.makeTree(list, 0L);
        return ResultUtils.success("查询成功", menuList);
    }
}
```

```

//查询上级树
@GetMapping("/parent")
public ResultVo getParent(){
    List<SysMenu> parent = sysMenuService.getParent();
    return ResultUtils.success("查询成功",parent);
}

}

```

7.2 新增菜单页面制作

7.2.1 定义菜单数据模型

在api下新建menu目录，新建MenuModel.ts

```

//菜单数据类型
export type MenuType = {
    editType:string,
    menuId:string,
    parentId:string,
    title:string,
    code:string,
    name:string,
    path:string,
    url:string,
    type:string,
    icon:string,
    parentName:string,
    orderNum:string
}

```

7.2.2 composables下新建menu目录，新建useMenu.ts

```

import { ref } from "vue"

export default function useMenu() {
    const addRef = ref<{show:()=>void}>()
    //新增
    const addBtn = () => {
        addRef.value?.show()
    }
    //编辑
    const editBtn = () => {

```

```

    }
    //删除
    const deleteBtn = () => {

    }
    return {
      addBtn,
      editBtn,
      deleteBtn,
      addRef
    }
  }
}

```

7.2.3 新建AddMenu.vue组件

在views/system目录下面创建AddMenu.vue组件

```

<template>
  <SysDialog
    :title="dialog.title"
    :height="dialog.height"
    :width="dialog.width"
    :visible="dialog.visible"
    @onClose="onClose"
    @onConfirm="commit"
  >
    <template v-slot:content>
      <el-form
        :model="addModel"
        ref="addFormRef"
        :rules="rules"
        label-width="80px"
        size="default"
      >
        <el-form-item prop="type" label="菜单类型">
          <el-radio-group v-model="addModel.type">
            <el-radio :label="'0'">目录</el-radio>
            <el-radio :label="'1'">菜单</el-radio>
            <el-radio :label="'2'">按钮</el-radio>
          </el-radio-group>
        </el-form-item>
        <el-row>
          <el-col :span="12" :offset="0">
            <el-form-item label="上级菜单" prop="parentId">

```

```
        <el-input v-model="addModel.parentId"></el-input>
      </el-form-item>
    </el-col>
    <el-col :span="12" :offset="0">
      <el-form-item label="菜单名称" prop="title">
        <el-input v-model="addModel.title"></el-input>
      </el-form-item>
    </el-col>
  </el-row>
  <el-row v-if="addModel.type !== '2'">
    <el-col :span="12" :offset="0">
      <el-form-item label="菜单图标" prop="icon">
        <el-input v-model="addModel.icon"></el-input>
      </el-form-item>
    </el-col>
    <el-col :span="12" :offset="0">
      <el-form-item label="路由名称" prop="name">
        <el-input v-model="addModel.name"></el-input>
      </el-form-item>
    </el-col>
  </el-row>
  <el-row>
    <el-col :span="12" :offset="0">
      <el-form-item label="菜单序号" prop="orderNum">
        <el-input v-model="addModel.orderNum"></el-input>
      </el-form-item>
    </el-col>
    <el-col :span="12" :offset="0">
      <el-form-item label="权限字段" prop="code">
        <el-input v-model="addModel.code"></el-input>
      </el-form-item>
    </el-col>
  </el-row>
  <el-row>
    <el-col v-if="addModel.type !== '2'" :span="12"
:offset="0">
      <el-form-item label="路由地址" prop="path">
        <el-input v-model="addModel.path"></el-input>
      </el-form-item>
    </el-col>
    <el-col v-if="addModel.type === '1'" :span="12"
:offset="0">
      <el-form-item label="组件路径" prop="url">
        <el-input v-model="addModel.url"></el-input>
      </el-form-item>
    </el-col>
  </el-row>
</el-form>
```

```

        </el-col>
      </el-row>
    </el-form>
  </template>
</SysDialog>
</template>

<script setup lang="ts">
import SysDialog from "@/components/SysDialog.vue";
import useDialog from "@/hooks/useDialog";
import { MenuType } from "@api/menu/MenuModel";
import { reactive, ref } from "vue";
import { FormInstance } from "element-plus";
const addFormRef = ref<FormInstance>()
//弹框属性
const { dialog, onClose, onConfirm, onShow } = useDialog();
//弹框显示
const show = () => {
  dialog.width = 680;
  onShow();
  //清空表单
  addFormRef.value?.resetFields()
};
//暴露出去，给父组件调用
defineExpose({
  show,
});
//表单绑定的数据
const addModel = reactive<MenuType>({
  editType: "",
  menuId: "",
  parentId: "",
  title: "",
  code: "",
  name: "",
  path: "",
  url: "",
  type: "",
  icon: "",
  parentName: "",
  orderNum: "",
});
//表单验证规则
const rules = reactive({
  parentId: [

```

```
{
  required: true,
  trigger: "change",
  message: "请选择上级菜单",
},
],
title: [
  {
    required: true,
    trigger: "change",
    message: "请填写菜单名称",
  },
],
code: [
  {
    required: true,
    trigger: "change",
    message: "请填写权限字段",
  },
],
name: [
  {
    required: true,
    trigger: "change",
    message: "请填写路由名称",
  },
],
path: [
  {
    required: true,
    trigger: "change",
    message: "请填写路由地址",
  },
],
url: [
  {
    required: true,
    trigger: "change",
    message: "请填写组件路径",
  },
],
type: [
  {
    required: true,
    trigger: "change",
```

```

        message: "请选择菜单类型",
      },
    ],
  });
  //表单提交
  const commit = ()=>{
    addFormRef.value?.validate((valid)=>{
      if(valid){
        onClose()
      }
    })
  }
</script>

<style scoped></style>

```

7.2.4 在MenuList组件中导入AddMenu组件

```

<template>
  <el-main>
    <!--新增 -->
    <el-button type="primary" :icon="Plus" size="default"
    @click="addBtn"
    >新增</el-button>
    <
    <!-- 新增、编辑 -->
    <add-menu ref="addRef"></add-menu>
  </el-main>
</template>

<script setup lang="ts">
  import AddMenu from "../AddMenu.vue";
  import { Plus, Edit, Delete, Search, Close } from "@element-
  plus/icons-vue";
  import useMenu from "@/composables/menu/useMenu";
  //新增、编辑
  const { addBtn, editBtn, deleteBtn ,addRef} = useMenu();
</script>

<style scoped></style>

```


7.3 上级菜单制作

7.3.1 定义查询上级菜单的接口

api/menu新建index.ts

```
import http from "@/http";  
// 查询上级菜单  
export const getParentApi = ()=>{  
  return http.get("/api/menu/parent")  
}
```

7.3.2 MenuModel.ts修改为如下

```
// 菜单数据类型  
export type MenuType = {  
  editType: string,  
  menuId: string,  
  parentId: string | number,  
  title: string,  
  code: string,  
  name: string,  
  path: string,  
  url: string,  
  type: string,  
  icon: string,  
  parentName: string,  
  orderNum: string,  
  open: boolean  
}  
// 选中上级菜单数据类型  
export type SelectNode = {  
  parentId: string | number,  
  parentName: string  
}
```

7.3.3 新增ParentMenu.vue组件

在views/system目录下面创建ParentMenu组件

```
<template>  
  <SysDialog  
    :title="dialog.title"  
    :height="dialog.height"
```

```

:width="dialog.width"
:visible="dialog.visible"
@onClose="onClose"
@onConfirm="confirm"
>
<template v-slot:content>
  <el-tree
    ref="parentTree"
    :data="treeData.list"
    node-key="menuId"
    default-expand-all
    :props="defaultProps"
    :highlight-current="true"
    :expand-on-click-node="false"
    @node-click="handleNodeClick"
  >
    <template #default="{ node, data }">
      <div class="custom-tree-container">
        <!-- 长度为0说明没有下级 -->
        <span v-if="data.children.length == 0">
          <DocumentRemove
            style="
              width: 1.3em;
              height: 1.3em;
              margin-right: 5px;
              color: #8c8c8c;
            "
          ></DocumentRemove>
        </span>
        <!-- 点击展开和关闭 -->
        <span v-else @click.stop="openBtn(data)">
          <component
            style="
              width: 1.1em;
              height: 1.1em;
              margin-right: 5px;
              color: #8c8c8c;
            "
            :is="data.open ? Plus : Minus"
          />
        </span>
        <span>{{ node.label }}</span>
      </div>
    </template>
  </el-tree>

```

```

    </template>
  </SysDialog>
</template>

<script setup lang="ts">
  import {DocumentRemove, Plus, Minus} from '@element-plus/icons-vue'
  import sysDialog from "@/components/SysDialog.vue";
  import useDialog from "@/hooks/useDialog";
  import { getParentApi } from "@api/menu";
  import { reactive, ref } from "vue";
  import { MenuType, SelectNode } from "@api/menu/MenuModel";
  import { ElMessage, ElTree } from "element-plus";
  //树的ref属性
  const parentTree = ref<InstanceType<typeof ElTree>>();
  //树属性配置
  const defaultProps = {
    children: "children",
    label: "title",
  };
  //上级树数据
  const treeData = reactive({
    list: [],
  });
  //选中数据
  const selectNode = reactive<SelectNode>({
    parentId: "",
    parentName: "",
  });
  //弹框属性
  const { dialog, onClose, onShow } = useDialog();
  //显示弹框
  const showParent = async () => {
    //查询树数据
    await getParent();
    dialog.width = 300;
    dialog.height = 450;
    onShow();
  };
  defineExpose({
    showParent,
  });
  //查询上级菜单数据
  const getParent = async () => {
    let res = await getParentApi();

```

```

    if (res && res.code == 200) {
      treeData.list = res.data;
    }
  };
  //树点击事件
  const handleClick = (node: MenuType) => {
    console.log(node);
    selectNode.parentId = node.menuId;
    selectNode.parentName = node.title;
  };
  //注册事件
  const emits = defineEmits(["selectParent"]);
  //弹框确定
  const confirm = () => {
    emits("selectParent", selectNode);
    onClose()
  };
  //加号和减号的点击事件
  const openBtn = (data: MenuType) => {
    console.log(data);
    //设置展开或者关闭
    data.open = !data.open;
    if (parentTree.value) {
      parentTree.value.store.nodesMap[data.menuId].expanded =
!data.open;
    }
  };
</script>

<style lang="scss">
.el-tree {
  // 将每一行的设置为相对定位 方便后面before after 使用绝对定位来固定位置
  .el-tree-node {
    position: relative;
    padding-left: 10px;
  }
  // 子集像右偏移 给数线留出距离
  .el-tree-node__children {
    padding-left: 20px;
  }
  //这是竖线
  .el-tree-node :last-child:before {
    height: 40px;
  }
  .el-tree > .el-tree-node:before {

```

```

border-left: none;
}
.el-tree > .el-tree-node:after {
border-top: none;
}
//这自定义的线 的公共部分
.el-tree-node:before,
.el-tree-node:after {
content: "";
left: -4px;
position: absolute;
right: auto;
border-width: 1px;
}
.tree :first-child .el-tree-node:before {
border-left: none;
}
// 竖线
.el-tree-node:before {
border-left: 1px dotted #d9d9d9;
bottom: 0px;
height: 100%;
top: -25px;
width: 1px;
}
//横线
.el-tree-node:after {
border-top: 1px dotted #d9d9d9;
height: 20px;
top: 14px;
width: 24px;
}
.el-tree-node__expand-icon.is-leaf {
width: 8px;
}
//去掉elementui自带的展开按钮 一个向下的按钮,打开时向右
.el-tree-node__content > .el-tree-node__expand-icon {
display: none;
}
//每一行的高度
.el-tree-node__content {
line-height: 30px;
height: 30px;
padding-left: 10px !important;
}

```

```

}
//去掉最上级的before after 即是去电最上层的连接线
.el-tree > div {
  &::before {
    display: none;
  }
  &::after {
    display: none;
  }
}
</style>

```

7.3.4 修改AddMenu.vue组件

```

<template>
  <SysDialog
    :title="dialog.title"
    :height="dialog.height"
    :width="dialog.width"
    :visible="dialog.visible"
    @onClose="onClose"
    @onConfirm="commit"
  >
    <template v-slot:content>
      <el-form
        :model="addModel"
        ref="addFormRef"
        :rules="rules"
        label-width="80px"
        size="default"
      >
        <el-form-item prop="type" label="菜单类型">
          <el-radio-group v-model="addModel.type">
            <el-radio :label="'0'">目录</el-radio>
            <el-radio :label="'1'">菜单</el-radio>
            <el-radio :label="'2'">按钮</el-radio>
          </el-radio-group>
        </el-form-item>
        <el-row>
          <el-col :span="12" :offset="0">
            <el-form-item label="上级菜单" prop="parentName">
              <el-input type="hidden" v-
model="addModel.parentId"></el-input>
              <el-input readonly @click="selectOpen" v-
model="addModel.parentName"></el-input>

```

```
        </el-form-item>
      </el-col>
      <el-col :span="12" :offset="0">
        <el-form-item label="菜单名称" prop="title">
          <el-input v-model="addModel.title"></el-input>
        </el-form-item>
      </el-col>
    </el-row>
    <el-row v-if="addModel.type != '2'">
      <el-col :span="12" :offset="0">
        <el-form-item label="菜单图标" prop="icon">
          <el-input v-model="addModel.icon"></el-input>
        </el-form-item>
      </el-col>
      <el-col :span="12" :offset="0">
        <el-form-item label="路由名称" prop="name">
          <el-input v-model="addModel.name"></el-input>
        </el-form-item>
      </el-col>
    </el-row>
    <el-row>
      <el-col :span="12" :offset="0">
        <el-form-item label="菜单序号" prop="orderNum">
          <el-input v-model="addModel.orderNum"></el-input>
        </el-form-item>
      </el-col>
      <el-col :span="12" :offset="0">
        <el-form-item label="权限字段" prop="code">
          <el-input v-model="addModel.code"></el-input>
        </el-form-item>
      </el-col>
    </el-row>
    <el-row>
      <el-col v-if="addModel.type != '2'" :span="12"
:offset="0">
        <el-form-item label="路由地址" prop="path">
          <el-input v-model="addModel.path"></el-input>
        </el-form-item>
      </el-col>
      <el-col v-if="addModel.type == '1'" :span="12"
:offset="0">
        <el-form-item label="组件路径" prop="url">
          <el-input v-model="addModel.url"></el-input>
        </el-form-item>
      </el-col>
    </el-row>
  </div>
</div>
```

```

        </el-row>
      </el-form>
    </template>
  </SysDialog>
  <!-- 上级菜单弹框 -->
  <ParentMenu ref="parentRef" @selectParent="selectParent">
</ParentMenu>
</template>

<script setup lang="ts">
import SysDialog from "@/components/SysDialog.vue";
import ParentMenu from "../ParentMenu.vue";
import useDialog from "@/hooks/useDialog";
import { MenuType, SelectNode } from "@api/menu/MenuModel";
import { reactive, ref } from "vue";
import { FormInstance } from "element-plus";
const addFormRef = ref<FormInstance>();
const parentRef = ref<{ showParent: () => void }>();
//弹框属性
const { dialog, onClose, onConfirm, onShow } = useDialog();
//弹框显示
const show = () => {
  dialog.width = 680;
  onShow();
  //清空表单
  addFormRef.value?.resetFields();
};
//暴露出去，给父组件调用
defineExpose({
  show,
});
//表单绑定的数据
const addModel = reactive<MenuType>({
  editType: "",
  menuId: "",
  parentId: "",
  title: "",
  code: "",
  name: "",
  path: "",
  url: "",
  type: "",
  icon: "",
  parentName: "",
  orderNum: "",

```



```
    open: false
  });
  // 表单验证规则
  const rules = reactive({
    parentName: [
      {
        required: true,
        trigger: "change",
        message: "请选择上级菜单",
      },
    ],
    title: [
      {
        required: true,
        trigger: "change",
        message: "请填写菜单名称",
      },
    ],
    code: [
      {
        required: true,
        trigger: "change",
        message: "请填写权限字段",
      },
    ],
    name: [
      {
        required: true,
        trigger: "change",
        message: "请填写路由名称",
      },
    ],
    path: [
      {
        required: true,
        trigger: "change",
        message: "请填写路由地址",
      },
    ],
    url: [
      {
        required: true,
        trigger: "change",
        message: "请填写组件路径",
      },
    ],
  });
```

```

    ],
    type: [
      {
        required: true,
        trigger: "change",
        message: "请选择菜单类型",
      },
    ],
  ],
});
//表单提交
const commit = () => {
  addFormRef.value?.validate((valid) => {
    if (valid) {
      onClose();
    }
  });
};
//选择上级菜单
const selectOpen = ()=>{
  parentRef.value?.showParent()
}
//上级菜单选择事件
const selectParent = (node:SelectNode)=>{
  addModel.parentId = node.parentId
  addModel.parentName = node.parentName
}
</script>

<style scoped></style>

```

7.4 新增菜单数据对接

7.4.1 api/menu/index.ts 添加 addApi() 方法

```

import http from "@/http";
import { MenuType } from "../MenuModel";
// 查询上级菜单
export const getParentApi = ()=>{
  return http.get("/api/menu/parent")
}

// 新增
export const addApi = (parm:MenuType)=>{
  return http.post("/api/menu",parm)
}

```

7.4.2 修改AddMenu组件

```

<template>
  <SysDialog
    :title="dialog.title"
    :height="dialog.height"
    :width="dialog.width"
    :visible="dialog.visible"
    @onClose="onClose"
    @onConfirm="commit"
  >
    <template v-slot:content>
      <el-form
        :model="addModel"
        ref="addFormRef"
        :rules="rules"
        label-width="80px"
        size="default"
      >
        <el-form-item prop="type" label="菜单类型">
          <el-radio-group v-model="addModel.type">
            <el-radio :label="'0'">目录</el-radio>
            <el-radio :label="'1'">菜单</el-radio>
            <el-radio :label="'2'">按钮</el-radio>
          </el-radio-group>
        </el-form-item>
        <el-row>
          <el-col :span="12" :offset="0">
            <el-form-item label="上级菜单" prop="parentName">
              <el-input type="hidden" v-
model="addModel.parentId"></el-input>
              <el-input

```

```
        @click="selectOpen"
        readonly
        v-model="addModel.parentName"
    ></el-input>
</el-form-item>
</el-col>
<el-col :span="12" :offset="0">
    <el-form-item label="菜单名称" prop="title">
        <el-input v-model="addModel.title"></el-input>
    </el-form-item>
</el-col>
</el-row>
<el-row v-if="addModel.type !== '2'">
    <el-col :span="12" :offset="0">
        <el-form-item label="菜单图标" prop="icon">
            <el-input v-model="addModel.icon"></el-input>
        </el-form-item>
    </el-col>
    <el-col :span="12" :offset="0">
        <el-form-item label="路由名称" prop="name">
            <el-input v-model="addModel.name"></el-input>
        </el-form-item>
    </el-col>
</el-row>
<el-row>
    <el-col :span="12" :offset="0">
        <el-form-item label="菜单序号" prop="orderNum">
            <el-input v-model="addModel.orderNum"></el-input>
        </el-form-item>
    </el-col>
    <el-col :span="12" :offset="0">
        <el-form-item label="权限字段" prop="code">
            <el-input v-model="addModel.code"></el-input>
        </el-form-item>
    </el-col>
</el-row>
<el-row>
    <el-col v-if="addModel.type !== '2'" :span="12"
:offset="0">
        <el-form-item label="路由地址" prop="path">
            <el-input v-model="addModel.path"></el-input>
        </el-form-item>
    </el-col>
    <el-col v-if="addModel.type === '1'" :span="12"
:offset="0">
```

```

        <el-form-item label="组件路径" prop="url">
            <el-input v-model="addModel.url"></el-input>
        </el-form-item>
    </el-col>
</el-row>
</el-form>
</template>
</SysDialog>
<!-- 上级菜单 -->
<ParentMenu ref="parentRef" @selectParent="selectParent">
</ParentMenu>
</template>

<script setup lang="ts">
import ParentMenu from "../ParentMenu.vue";
import SysDialog from "@components/SysDialog.vue";
import useDialog from "@hooks/useDialog";
import { MenuType, SelectNode } from "@api/menu/MenuModel";
import { reactive, ref } from "vue";
import { ElMessage, FormInstance } from "element-plus";
import { addApi } from "@api/menu";
import { EditType, Title } from "@type/BaseEnum";
const parentRef = ref<{ showParent: () => void }>();
const addFormRef = ref<FormInstance>();
//弹框属性
const { dialog, onClose, onConfirm, onShow } = useDialog();
//弹框显示
const show = (type:string,row?:MenuType) => {
    dialog.width = 680;
    type == EditType.ADD ? dialog.title = Title.ADD :
dialog.title = Title.EDIT
    onShow();
    //清空表单
    addFormRef.value?.resetFields();
    addModel.editType = type;
};
//暴露出去，给父组件调用
defineExpose({
    show,
});
//表单绑定的数据
const addModel = reactive<MenuType>({
    editType: "",
    menuId: "",
    parentId: "",

```

```
title: "",
code: "",
name: "",
path: "",
url: "",
type: "",
icon: "",
parentName: "",
orderNum: "",
open: true,
});
//表单验证规则
const rules = reactive({
  parentName: [
    {
      required: true,
      trigger: "change",
      message: "请选择上级菜单",
    },
  ],
  title: [
    {
      required: true,
      trigger: "change",
      message: "请填写菜单名称",
    },
  ],
  code: [
    {
      required: true,
      trigger: "change",
      message: "请填写权限字段",
    },
  ],
  name: [
    {
      required: true,
      trigger: "change",
      message: "请填写路由名称",
    },
  ],
  path: [
    {
      required: true,
      trigger: "change",

```

```

        message: "请填写路由地址",
      },
    ],
    url: [
      {
        required: true,
        trigger: "change",
        message: "请填写组件路径",
      },
    ],
    type: [
      {
        required: true,
        trigger: "change",
        message: "请选择菜单类型",
      },
    ],
  ],
});
//表单提交
const commit = () => {
  addFormRef.value?.validate(async (valid) => {
    if (valid) {
      let res = null;
      if (addModel.editType == EditType.ADD) {
        res = await addApi(addModel);
      }
      if(res && res.code == 200){
        ElMessage.success(res.msg)
        onClose();
      }
    }
  });
};
//选择上级菜单事件
const selectOpen = () => {
  parentRef.value?.showParent();
};
//选择回调
const selectParent = (node: SelectNode) => {
  addModel.parentId = node.parentId;
  addModel.parentName = node.parentName;
};
</script>

```

```
<style scoped></style>
```

7.4.3 修改composables/menu/useMenu.ts

```
import { ref } from 'vue'

export default function useMenu(){

  const addRef = ref<{show:()=>void}>()

  //新增
  const addBtn = ()=>{
    addRef.value?.show() //展示新增菜单的对话框
  }

  //编辑
  const editBtn = ()=>{

  }

  //删除
  const deleteBtn = ()=>{

  }

  return {
    addBtn,
    editBtn,
    deleteBtn,
    addRef
  }
}
```

7.5 菜单管理列表制作与对接

7.5.1 api/menu/index.ts添加getListApi()方法

```
import http from "@/http";
import { MenuType } from "../MenuModel";
//查询上级菜单
export const getParentApi = ()=>{
  return http.get("/api/menu/parent")
}

//新增
export const addApi = (parm:MenuType)=>{
```



```

    return http.post("/api/menu", parm)
  }

  //列表
  export const getListApi = ()=>{
    return http.get("/api/menu/list")
  }

```

7.5.2 composables/menu添加useMenuTable.ts

```

import { onMounted, reactive, ref, nextTick } from "vue";
import { getListApi } from "@api/menu";
export default function useMenuTable() {
  //表格高度
  const tableHeight = ref(0)
  //定义表格数据
  const tableList = reactive({
    list: []
  })
  //获取表格数据
  const getList = async () => {
    let res = await getListApi()
    if (res && res.code == 200) {
      tableList.list = res.data
    }
  }
  //刷新列表
  const refresh = ()=>{
    getList()
  }
  onMounted(() => {
    getList()
    nextTick(() => {
      tableHeight.value = window.innerHeight - 200
    })
  })
  return {
    tableList,
    getList,
    tableHeight,
    refresh
  }
}

```

7.5.3 修改MunuList.vue组件

```
<template>
  <el-main>
    <!-- 新增 -->
    <el-button style="margin-bottom:20px;" type="primary"
:icon="Plus" size="default" @click="addBtn"
    >新增</el-button>
  >
  <!-- 表格数据 -->
  <el-table
    :height="tableHeight"
    :data="tableList.list"
    row-key="menuId"
    :tree-props="{ children: 'children', hasChildren:
'hasChildren' }"
    border
    default-expand-all
    stripe
  >
    <el-table-column prop="title" label="菜单名称"></el-table-
column>
    <el-table-column prop="type" label="类型">
      <template #default="scope">
        <el-tag v-if="scope.row.type == '0'" size="default">目
录</el-tag>
        <el-tag v-if="scope.row.type == '1'" type="success"
size="default"
          >菜单</el-tag>
        >
        <el-tag v-if="scope.row.type == '2'" type="danger"
size="default"
          >按钮</el-tag>
        >
      </template>
    </el-table-column>
    <el-table-column prop="icon" label="图标">
      <template #default="scope">
        <el-icon>
          <!-- 动态组件的方式显示图标 -->
          <component :is="scope.row.icon"></component>
        </el-icon>
      </template>
    </el-table-column>
```

```

        <el-table-column prop="name" label="路由名称"></el-table-
column>
        <el-table-column prop="path" label="路由地址"></el-table-
column>
        <el-table-column prop="url" label="组件路径"></el-table-
column>
        <el-table-column prop="code" label="权限字段"></el-table-
column>
        <el-table-column label="操作" width="220" align="center">
          <template #default="scope">
            <el-button type="success" :icon="Edit" size="default"
@click="editBtn">编辑</el-button>
            <el-button type="danger" :icon="Delete"
size="default" @click="deleteBtn">删除</el-button>
          </template>
        </el-table-column>
      </el-table>

      <!-- 新增、编辑 -->
      <add-menu ref="addRef" @refresh="refresh"></add-menu>
    </el-main>
  </template>

  <script setup lang="ts">
    import AddMenu from "./AddMenu.vue";
    import { Plus, Edit, Delete, Search, Close } from "@element-
plus/icons-vue";
    import useMenu from "@composables/menu/useMenu";
    import useMenuTable from "@composables/menu/useMenuTable";
    //表格数据
    const { tableList, tableHeight, refresh } = useMenuTable();
    //新增、编辑
    const { addBtn, editBtn, deleteBtn, addRef } = useMenu();
  </script>

  <style scoped></style>

```

7.6 编辑、删除菜单接口对接

7.6.1 api/menu/index.ts添加editApi()和deleteApi()方法

```

//编辑
export const editApi = (parm:MenuType)=>{
  return http.put("/api/menu",parm)
}
//删除
export const deleteApi = (menuId:string | number)=>{
  return http.delete(`/api/menu/${menuId}`)
}

```

7.6.2 编辑composables/menu/useMenu.ts

```

import { MenuType } from "@api/menu/MenuModel"
import { EditType } from "@type/BaseEnum"
import { ref } from "vue"
import useInstance from "@hooks/useInstance";
import { deleteApi } from "@api/menu";
import { ElMessage } from "element-plus";
import { FuncList } from "@type/BaseType";
export default function useMenu(getList:FuncList) {
  const { global } = useInstance();
  const addRef = ref<{ show: (type: string, row?: MenuType) =>
void }>()
  //新增
  const addBtn = () => {
    addRef.value?.show(EditType.ADD)
  }
  //编辑
  const editBtn = (row: MenuType) => {
    addRef.value?.show(EditType.EDIT, row)
  }
  //删除
  const deleteBtn = async (row: MenuType) => {
    let confirm = await global.$myconfirm('确定删除该数据吗? ')
    if(confirm){
      let res = await deleteApi(row.menuId)
      if(res && res.code == 200){
        ElMessage.success(res.msg)
        getList()
      }
    }
  }
  return {
    addBtn,
    editBtn,

```

```

        deleteBtn,
        addRef
    }
}

```

7.6.3 修改AddMenu.vue组件

```

<template>
  <SysDialog
    :title="dialog.title"
    :height="dialog.height"
    :width="dialog.width"
    :visible="dialog.visible"
    @onClose="onClose"
    @onConfirm="commit"
  >
    <template v-slot:content>
      <el-form
        :model="addModel"
        ref="addFormRef"
        :rules="rules"
        label-width="80px"
        size="default"
      >
        <el-form-item prop="type" label="菜单类型">
          <el-radio-group v-model="addModel.type">
            <el-radio :label="'0'">目录</el-radio>
            <el-radio :label="'1'">菜单</el-radio>
            <el-radio :label="'2'">按钮</el-radio>
          </el-radio-group>
        </el-form-item>
        <el-row>
          <el-col :span="12" :offset="0">
            <el-form-item label="上级菜单" prop="parentName">
              <el-input type="hidden" v-
model="addModel.parentId"></el-input>
              <el-input
                @click="selectOpen"
                readonly
                v-model="addModel.parentName"
              ></el-input>
            </el-form-item>
          </el-col>
          <el-col :span="12" :offset="0">
            <el-form-item label="菜单名称" prop="title">

```

```
        <el-input v-model="addModel.title"></el-input>
      </el-form-item>
    </el-col>
  </el-row>
  <el-row v-if="addModel.type !== '2'">
    <el-col :span="12" :offset="0">
      <el-form-item label="菜单图标" prop="icon">
        <el-input v-model="addModel.icon"></el-input>
      </el-form-item>
    </el-col>
    <el-col :span="12" :offset="0">
      <el-form-item label="路由名称" prop="name">
        <el-input v-model="addModel.name"></el-input>
      </el-form-item>
    </el-col>
  </el-row>
  <el-row>
    <el-col :span="12" :offset="0">
      <el-form-item label="菜单序号" prop="orderNum">
        <el-input v-model="addModel.orderNum"></el-input>
      </el-form-item>
    </el-col>
    <el-col :span="12" :offset="0">
      <el-form-item label="权限字段" prop="code">
        <el-input v-model="addModel.code"></el-input>
      </el-form-item>
    </el-col>
  </el-row>
  <el-row>
    <el-col v-if="addModel.type !== '2'" :span="12"
:offset="0">
      <el-form-item label="路由地址" prop="path">
        <el-input v-model="addModel.path"></el-input>
      </el-form-item>
    </el-col>
    <el-col v-if="addModel.type === '1'" :span="12"
:offset="0">
      <el-form-item label="组件路径" prop="url">
        <el-input v-model="addModel.url"></el-input>
      </el-form-item>
    </el-col>
  </el-row>
</el-form>
</template>
</SysDialog>
```

```

    <!-- 上级菜单 -->
    <ParentMenu ref="parentRef" @selectParent="selectParent">
  </ParentMenu>
</template>

<script setup lang="ts">
import ParentMenu from "../ParentMenu.vue";
import SysDialog from "@components/SysDialog.vue";
import useDialog from "@hooks/useDialog";
import { MenuType, SelectNode } from "@api/menu/MenuModel";
import { nextTick, reactive, ref } from "vue";
import { ElMessage, FormInstance } from "element-plus";
import { addApi, editApi } from "@api/menu";
import { EditType, Title } from "@type/BaseEnum";
import useInstance from "@hooks/useInstance";
const { global } = useInstance();
const parentRef = ref<{ showParent: () => void }>();
const addFormRef = ref<FormInstance>();
//弹框属性
const { dialog, onClose, onConfirm, onShow } = useDialog();
//弹框显示
const show = (type: string, row?: MenuType) => {
  dialog.width = 680;
  type == EditType.ADD
    ? (dialog.title = Title.ADD)
    : (dialog.title = Title.EDIT);
  if (type == EditType.EDIT) {
    //把要编辑的数据设置到表单绑定的对象里面
    nextTick(() => {
      global.$objCopy(row, addModel);
    });
  }
  onShow();
  //清空表单
  addFormRef.value?.resetFields();
  addModel.editType = type;
};
//暴露出去，给父组件调用
defineExpose({
  show,
});
//表单绑定的数据
const addModel = reactive<MenuType>({
  editType: "",
  menuId: "",

```

```
    parentId: "",
    title: "",
    code: "",
    name: "",
    path: "",
    url: "",
    type: "",
    icon: "",
    parentName: "",
    orderNum: "",
    open: true,
  });
  // 表单验证规则
  const rules = reactive({
    parentName: [
      {
        required: true,
        trigger: "change",
        message: "请选择上级菜单",
      },
    ],
    title: [
      {
        required: true,
        trigger: "change",
        message: "请填写菜单名称",
      },
    ],
    code: [
      {
        required: true,
        trigger: "change",
        message: "请填写权限字段",
      },
    ],
    name: [
      {
        required: true,
        trigger: "change",
        message: "请填写路由名称",
      },
    ],
    path: [
      {
        required: true,
```



```

        trigger: "change",
        message: "请填写路由地址",
    },
],
url: [
    {
        required: true,
        trigger: "change",
        message: "请填写组件路径",
    },
],
type: [
    {
        required: true,
        trigger: "change",
        message: "请选择菜单类型",
    },
],
});
//注册事件
const emits = defineEmits(["refresh"]);
//表单提交
const commit = () => {
    addFormRef.value?.validate(async (valid) => {
        if (valid) {
            let res = null;
            if (addModel.editType == EditType.ADD) {
                res = await addApi(addModel);
            } else {
                res = await editApi(addModel)
            }
            if (res && res.code == 200) {
                ElMessage.success(res.msg);
                //刷新表格
                emits("refresh");
                onClose();
            }
        }
    });
};
//选择上级菜单事件
const selectOpen = () => {
    parentRef.value?.showParent();
};
//选择回调

```

```

const selectParent = (node: SelectNode) => {
  addModel.parentId = node.parentId;
  addModel.parentName = node.parentName;
};
</script>

<style scoped></style>

```

7.6.4 修改MenuList.vue组件

```

<template>
  <el-main style="padding-top: 10px">
    <!-- 新增 -->
    <div style="display: flex; align-items: center;">
      <el-button
        style="margin-bottom: 10px"
        type="primary"
        :icon="Plus"
        size="default"
        @click="addBtn"
      >新增</el-button>
    </div>
    <!-- 表格数据 -->
    <el-table
      :height="tableHeight"
      :data="tableList.list"
      row-key="menuId"
      :tree-props="{ children: 'children', hasChildren:
'hasChildren' }"
      border
      default-expand-all
      stripe
    >
      <el-table-column prop="title" label="菜单名称"></el-table-
column>
      <el-table-column prop="type" label="类型">
        <template #default="scope">
          <el-tag v-if="scope.row.type == '0'" size="default">目
录</el-tag>
          <el-tag v-if="scope.row.type == '1'" type="success"
size="default"
            >菜单</el-tag>
        </template>
      </el-table-column>
    </el-table>
  </el-main>
</template>

```

```

        <el-tag v-if="scope.row.type == '2'" type="danger"
size="default"
        >按钮</el-tag
    >
    </template>
</el-table-column>
<el-table-column prop="icon" label="图标">
    <template #default="scope">
        <el-icon>
            <!-- 动态组件的方式显示图标 -->
            <component :is="scope.row.icon"></component>
        </el-icon>
    </template>
</el-table-column>
<el-table-column prop="name" label="路由名称"></el-table-
column>
<el-table-column prop="path" label="路由地址"></el-table-
column>
<el-table-column prop="url" label="组件路径"></el-table-
column>
<el-table-column prop="code" label="权限字段"></el-table-
column>
<el-table-column label="操作" width="220" align="center">
    <template #default="scope">
        <el-button type="success" :icon="Edit" size="default"
@click="editBtn(scope.row)"
        >编辑</el-button
    >
    <el-button
        type="danger"
        :icon="Delete"
        size="default"
        @click="deleteBtn(scope.row)"
        >删除</el-button
    >
    </template>
</el-table-column>
</el-table>

<!-- 新增、编辑 -->
<add-menu ref="addRef" @refresh="refresh"></add-menu>
</el-main>
</template>

<script setup lang="ts">

```

```

import AddMenu from "../AddMenu.vue";
import { Plus, Edit, Delete, Search, Close } from "@element-plus/icons-vue";
import useMenu from "@/composables/menu/useMenu";
import useMenuTable from "@/composables/menu/useMenuTable";
//表格数据
const { tableList, tableHeight, refresh,getList } =
useMenuTable();
//新增、编辑
const { addBtn, editBtn, deleteBtn, addRef } =
useMenu(getList);
</script>

<style scoped></style>

```

8 会员卡管理模块功能开发

8.1 相关数据表设计

```

create table member_card
(
    card_id          int not null auto_increment comment '主
    键',
    title            varchar(128) comment '标题',
    card_type        varchar(5) comment '卡类型 1:天卡 2: 周卡
    3: 月卡 4: 年卡',
    price            decimal(18,2) comment '价格',
    card_day         int comment '天数',
    status           varchar(2) comment '状态 1: 启用 0: 停用',
    primary key (card_id)
);

```

8.2 会员卡管理后端接口开发

8.2.1 新建实体类

在com/xq/web下面创建member_card/entity目录。

```

package com.xq.web.member_card.entity;

import com.baomidou.mybatisplus.annotation.IdType;
import com.baomidou.mybatisplus.annotation.TableId;
import com.baomidou.mybatisplus.annotation.TableName;
import lombok.Data;

```

```
import java.math.BigDecimal;

@Data
@TableName("member_card")
public class MemberCard {
    @TableId(type = IdType.AUTO)
    private Long cardId;
    private String title;
    private String cardType;
    private BigDecimal price;
    private Integer cardDay;
    private String status;
}
```

```
package com.xq.web.member_card.entity;

import lombok.Data;

@Data
public class ListCard {
    private Long currentPage;
    private Long pageSize;
    private String title;
}
```

8.2.2 创建mapper层

```
package com.xq.web.member_card.mapper;
import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.xq.web.member_card.entity.MemberCard;

public interface MemberCardMapper extends BaseMapper<MemberCard>
{
}
}
```

8.2.3 创建映射文件

在resource/mapper目录下面创建映射文件MemberCardMapper.xml

```

<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper
namespace="com.xq.web.member_card.mapper.MemberCardMapper">

</mapper>

```

8.2.4 新建service层

```

package com.xq.web.member_card.service;

import com.baomidou.mybatisplus.extension.service.IService;
import com.xq.web.member_card.entity.MemberCard;

public interface MemberCardService extends IService<MemberCard> {
}

```

创建业务实现类：

```

package com.xq.web.member_card.service.impl;

import
com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
import com.xq.web.member_card.entity.MemberCard;
import com.xq.web.member_card.mapper.MemberCardMapper;
import com.xq.web.member_card.service.MemberCardService;
import org.springframework.stereotype.Service;

@Service
public class MemberCardServiceImpl extends
ServiceImpl<MemberCardMapper, MemberCard> implements
MemberCardService {
}

```

8.2.5 新建控制器

```

package com.xq.web.member_card.controller;

import
com.baomidou.mybatisplus.core.conditions.query.QueryWrapper;
import com.baomidou.mybatisplus.core.metadata.IPage;

```

```
import
com.baomidou.mybatisplus.extension.plugins.pagination.Page;
import com.xq.utils.ResultUtils;
import com.xq.utils.ResultVo;
import com.xq.web.member_card.entity.ListCard;
import com.xq.web.member_card.entity.MemberCard;
import com.xq.web.member_card.service.MemberCardService;
import org.apache.commons.lang.StringUtils;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/memberCard")
public class MemberCardController {
    @Autowired
    private MemberCardService memberCardService;

    //新增
    @PostMapping
    public ResultVo add(@RequestBody MemberCard memberCard){
        if(memberCardService.save(memberCard)){
            return ResultUtils.success("新增成功!");
        }
        return ResultUtils.error("新增失败!");
    }

    //编辑
    @PutMapping
    public ResultVo edit(@RequestBody MemberCard memberCard){
        if(memberCardService.updateById(memberCard)){
            return ResultUtils.success("编辑成功!");
        }
        return ResultUtils.error("编辑失败!");
    }

    //删除
    @DeleteMapping("/{cardId}")
    public ResultVo delete(@PathVariable("cardId") Long cardId){
        if(memberCardService.removeById(cardId)){
            return ResultUtils.success("删除成功!");
        }
        return ResultUtils.error("删除失败!");
    }

    //列表
    @GetMapping("/list")
```

```

    public ResultVo list(ListCard listCard){
        //构造分页对象
        IPage<MemberCard> page = new Page<>
(listCard.getCurrentPage(),listCard.getPageSize());
        //构造查询条件
        QueryWrapper<MemberCard> query = new QueryWrapper<>();
        if(StringUtils.isNotEmpty(listCard.getTitle())){

            query.lambda().like(MemberCard::getTitle,listCard.getTitle());
        }
        IPage<MemberCard> list = memberCardService.page(page,
query);
        return ResultUtils.success("查询成功",list);
    }
}

```

8.3 新增会员卡页面制作与数据对接

8.3.1 定义数据类型和api接口

在api下新建member_card目录，并建index.ts和MemberModel.ts

- MemberModel.ts

```

//列表查询参数类型
export type ListParm = {
    title:string,
    currentPage:number,
    pageSize:number,
    total:number
}
//表单数据类型
export type CardType = {
    type:string,
    title:string,
    cardType:string,
    cardId:string,
    price:string,
    cardDay:number,
    status:string
}

```

- index.ts


```
import http from "@/http";
import { CardType } from "../MemberModel";
//新增
export const addApi = (parm:CardType)=>{
    return http.post("/api/memberCard",parm)
}
```

8.3.2 在composables目录新建member_card，并新建useMemberTable.ts和useMember.ts

- useMemberTable.ts

```
import { ListParm } from "@/api/member_card/MemberModel";
import { reactive } from "vue";

export default function useMemberTable() {
    //表格数据
    //查询参数
    const listParm = reactive<ListParm>({
        title: '',
        pageSize: 10,
        currentPage: 1,
        total: 0
    })
    //查询列表
    const getList = ()=>{

    }
    //搜索
    const searchBtn = ()=>{

    }
    //重置
    const resetBtn = ()=>{

    }
    return{
        listParm,
        getList,
        searchBtn,
        resetBtn
    }
}
```

- useMember.ts

```
import { ref } from "vue"

export default function useMember(){
  //弹框ref属性
  const addRef = ref<{show:()=>void}>()
  //新增
  const addBtn = ()=>{
    addRef.value?.show()
  }
  //编辑
  const editBtn = ()=>{

  }
  //删除
  const deleteBtn = ()=>{

  }
  return{
    addBtn,
    editBtn,
    deleteBtn,
    addRef
  }
}
```

8.3.2 修改CardType.vue组件

在view/member目录下执行修改操作：

```
<template>
  <el-main>
    <!-- 搜索栏 -->
    <el-form :model="listParm" :inline="true" size="default">
      <el-form-item>
        <el-input v-model="listParm.title" placeholder="请输入标题"></el-input>
      </el-form-item>
      <el-form-item>
        <el-button :icon="Search" @click="searchBtn">搜索</el-button>
        <el-button :icon="Close" type="danger" plain @click="resetBtn">重置</el-button>
      </el-form-item>
    </el-form>
  </el-main>
</template>
```

```

        >
        <el-button :icon="Plus" type="primary" @click="addBtn"
        >新增</el-button>
        >
        </el-form-item>
    </el-form>
    <!-- 新增、编辑 -->
    <AddCard ref="addRef"></AddCard>
</el-main>
</template>

<script setup lang="ts">
import AddCard from "../AddCard.vue";
import { Plus, Edit, Delete, Search, Close } from "@element-
plus/icons-vue";
import useMemberTable from
"@/composables/member_card/useMemberTable";
import useMember from "@/composables/member_card/useMember";
//表格操作
const { listParm, getList, resetBtn, searchBtn } =
useMemberTable();
//新增、编辑
const { addBtn, editBtn, deleteBtn, addRef } = useMember();
</script>

<style scoped></style>

```

8.3.3 新建AddCard.vue组件

```

<template>
  <SysDialog
    :title="dialog.title"
    :width="dialog.width"
    :height="dialog.height"
    :visible="dialog.visible"
    @onClose="onClose"
    @onConfirm="commit"
  >
    <template v-slot:content>
      <el-form
        :model="addModel"
        ref="addFormRef"
        :rules="rules"
        label-width="80px"
        size="default"

```

```
>
<el-row>
  <el-col :span="24" :offset="0">
    <el-form-item prop="cardType" label="类型">
      <el-radio-group v-model="addModel.cardType">
        <el-radio :label="'1'">天卡</el-radio>
        <el-radio :label="'2'">周卡</el-radio>
        <el-radio :label="'3'">月卡</el-radio>
        <el-radio :label="'4'">年卡</el-radio>
      </el-radio-group>
    </el-form-item>
  </el-col>
</el-row>
<el-row>
  <el-col :span="12" :offset="0">
    <el-form-item prop="title" label="标题">
      <el-input v-model="addModel.title"></el-input>
    </el-form-item>
  </el-col>
  <el-col :span="12" :offset="0">
    <el-form-item prop="cardDay" label="天数">
      <el-input v-model="addModel.cardDay"></el-input>
    </el-form-item>
  </el-col>
</el-row>
<el-row>
  <el-col :span="12" :offset="0">
    <el-form-item prop="price" label="价格">
      <el-input v-model="addModel.price"></el-input>
    </el-form-item>
  </el-col>
  <el-col :span="12" :offset="0">
    <el-form-item prop="status" label="状态">
      <el-radio-group v-model="addModel.status">
        <el-radio :label="'0'">停用</el-radio>
        <el-radio :label="'1'">启用</el-radio>
      </el-radio-group>
    </el-form-item>
  </el-col>
</el-row>
</el-form>
</template>
</SysDialog>
</template>
```

```
<script setup lang="ts">
import SysDialog from "@/components/SysDialog.vue";
import useDialog from "@/hooks/useDialog";
import { CardType } from "@api/member_card/MemberModel";
import { reactive, ref } from "vue";
import { addApi } from "@api/member_card/index";
import { ElMessage, FormInstance } from "element-plus";
//表单ref属性
const addFormRef = ref<FormInstance>();
//弹框属性
const { dialog, onClose, onConfirm, onShow } = useDialog();
//显示弹框
const show = () => {
  dialog.height = 200;
  onShow();
};
//暴露出去，给父组件调用
defineExpose({
  show,
});
//表单数据
const addModel = reactive<CardType>({
  type: "",
  title: "",
  cardType: "",
  cardId: "",
  price: "",
  cardDay: 0,
  status: "",
});
//表单验证规则
const rules = reactive({
  title: [
    {
      required: true,
      message: "请填写标题",
      trigger: "change",
    },
  ],
  cardType: [
    {
      required: true,
      message: "请选择类型",
      trigger: "change",
    },
  ],
}
```

```

    ],
    price: [
      {
        required: true,
        message: "请填写价格",
        trigger: "change",
      },
    ],
    cardDay: [
      {
        required: true,
        message: "请填写天数",
        trigger: "change",
      },
    ],
    status: [
      {
        required: true,
        message: "请选择状态",
        trigger: "change",
      },
    ],
  ]
});
//表单提交
const commit = () => {
  addFormRef.value?.validate(async (valid) => {
    if (valid) {
      let res = await addApi(addModel);
      if (res && res.code == 200) {
        ElMessage.success(res.msg);
        onClose();
      }
    }
  });
};
</script>

<style scoped></style>

```

8.4 会员卡管理列表制作与对接

8.4.1 添加getListApi()方法

```
import http from "@/http";
import { CardType, ListParm } from "../MemberModel";
//新增
export const addApi = (parm:CardType)=>{
    return http.post("/api/memberCard",parm)
}
//列表
export const getListApi = (parm:ListParm)=>{
    return http.get("/api/memberCard/list",parm)
}
```

8.4.2 composables/member_card/useMemberTable.ts修改为如下

```
import { ListParm } from "@/api/member_card/MemberModel";
import { nextTick, onMounted, reactive, ref } from "vue";
import { getListApi } from "@/api/member_card";
export default function useMemberTable() {
    //表格高度
    const tableHeight = ref(0)
    //表格数据
    const tableList =reactive({
        list:[]
    })
    //查询参数
    const listParm = reactive<ListParm>({
        title: '',
        pageSize: 10,
        currentPage: 1,
        total: 0
    })
    //查询列表
    const getList = async()=>{
        let res = await getListApi(listParm)
        if(res && res.code == 200){
            console.log(res)
            tableList.list = res.data.records
            listParm.total = res.data.total
        }
    }
    //搜索
    const searchBtn = ()=>{
```

```
        getList()
    }
    //重置
    const resetBtn = ()=>{
        listParm.title = ''
        getList()
    }
    //页容量改变时触发
    const sizeChange = (size:number)=>{
        listParm.pageSize = size
        getList()
    }
    //页数改变时触发
    const currentChange = (page:number)=>{
        listParm.currentPage = page
        getList()
    }
    //刷新表格
    const refresh = ()=>{
        getList()
    }
    onMounted(()=>{
        getList()
        nextTick(()=>{
            tableHeight.value = window.innerHeight - 230
        })
    })
    return{
        listParm,
        getList,
        searchBtn,
        resetBtn,
        tableList,
        sizeChange,
        currentChange,
        tableHeight,
        refresh
    }
}
```


8.4.3 修改CardType.vue组件

```
<template>
  <el-main>
    <!-- 搜索栏 -->
    <el-form :model="listParm" :inline="true" size="default">
      <el-form-item>
        <el-input v-model="listParm.title" placeholder="请输入标题"></el-input>
      </el-form-item>
      <el-form-item>
        <el-button :icon="Search" @click="searchBtn">搜索</el-button>
        <el-button :icon="Close" type="danger" plain @click="resetBtn">重置</el-button>
        <el-button :icon="Plus" type="primary" @click="addBtn">新增</el-button>
      </el-form-item>
    </el-form>
    <!-- 表格 -->
    <el-table :height="tableHeight" :data="tableList.list" border stripe>
      <el-table-column prop="title" label="标题"></el-table-column>
      <el-table-column prop="cardType" label="类型">
        <template #default="scope">
          <el-tag v-if="scope.row.cardType == '1'" size="default">天卡</el-tag>
          <el-tag v-if="scope.row.cardType == '2'" type="success" size="default">周卡</el-tag>
          <el-tag v-if="scope.row.cardType == '3'" type="danger" size="default">月卡</el-tag>
          <el-tag v-if="scope.row.cardType == '4'" type="danger" size="default">年卡</el-tag>
        </template>
      </el-table-column>
    </el-table>
  </el-main>
</template>
```

```

        <el-table-column prop="cardDay" label="天数"></el-table-
column>
        <el-table-column prop="price" label="价格(元)"></el-table-
column>
        <el-table-column prop="status" label="状态">
            <template #default="scope">
                <el-tag v-if="scope.row.status == '1'"
type="success" size="default">启用</el-tag>
                <el-tag v-if="scope.row.status == '0'"
type="danger" size="default">停用</el-tag>
            </template>
        </el-table-column>
        <el-table-column label="操作" align="center" width="220">
            <template #default="scope">
                <el-button type="primary" :icon="Edit" size="default"
@click="editBtn">编辑</el-button>
                <el-button type="danger" :icon="Delete"
size="default" @click="deleteBtn">删除</el-button>
            </template>
        </el-table-column>
    </el-table>
    <!-- 分页 -->
    <el-pagination
        @size-change="sizeChange"
        @current-change="currentChange"
        :current-page.sync="listParm.currentPage"
        :page-sizes="[10,20, 40, 80, 100]"
        :page-size="listParm.pageSize"
        layout="total, sizes, prev, pager, next, jumper"
        :total="listParm.total" background>
    </el-pagination>

    <!-- 新增、编辑 -->
    <AddCard ref="addRef" @refresh="refresh"></AddCard>
</el-main>
</template>

<script setup lang="ts">
    import AddCard from "@/AddCard.vue";
    import { Plus, Edit, Delete, Search, Close } from "@element-
plus/icons-vue";
    import useMemberTable from
"@/composables/member_card/useMemberTable";
    import useMember from "@/composables/member_card/useMember";
    // 表格操作

```

```

    const { listParm, getList, resetBtn,
searchBtn,tableList,sizeChange,currentChange,tableHeight,refresh
} = useMemberTable();
    //新增、编辑
    const { addBtn, editBtn, deleteBtn,addRef } = useMember();
</script>

<style scoped></style>

```

8.4.4 修改AddCard.vue组件

```

<template>
  <SysDialog
    :title="dialog.title"
    :width="dialog.width"
    :height="dialog.height"
    :visible="dialog.visible"
    @onClose="onClose"
    @onConfirm="commit"
  >
    <template v-slot:content>
      <el-form
        :model="addModel"
        ref="addFormRef"
        :rules="rules"
        label-width="80px"
        size="default"
      >
        <el-row>
          <el-col :span="24" :offset="0">
            <el-form-item prop="cardType" label="类型">
              <el-radio-group v-model="addModel.cardType">
                <el-radio :label="'1'">天卡</el-radio>
                <el-radio :label="'2'">周卡</el-radio>
                <el-radio :label="'3'">月卡</el-radio>
                <el-radio :label="'4'">年卡</el-radio>
              </el-radio-group>
            </el-form-item>
          </el-col>
        </el-row>
        <el-row>
          <el-col :span="12" :offset="0">
            <el-form-item prop="title" label="标题">
              <el-input v-model="addModel.title"></el-input>
            </el-form-item>
          </el-col>
        </el-row>
      </template>
    </template>
  </SysDialog>
</template>

```

```

    </el-col>
    <el-col :span="12" :offset="0">
      <el-form-item prop="cardDay" label="天数">
        <el-input v-model="addModel.cardDay"></el-input>
      </el-form-item>
    </el-col>
  </el-row>
  <el-row>
    <el-col :span="12" :offset="0">
      <el-form-item prop="price" label="价格">
        <el-input v-model="addModel.price"></el-input>
      </el-form-item>
    </el-col>
    <el-col :span="12" :offset="0">
      <el-form-item prop="status" label="状态">
        <el-radio-group v-model="addModel.status">
          <el-radio :label="'0'">停用</el-radio>
          <el-radio :label="'1'">启用</el-radio>
        </el-radio-group>
      </el-form-item>
    </el-col>
  </el-row>
</el-form>
</template>
</SysDialog>
</template>

```

```

<script setup lang="ts">
import SysDialog from "@components/SysDialog.vue";
import useDialog from "@hooks/useDialog";
import { CardType } from "@api/member_card/MemberModel";
import { reactive, ref } from "vue";
import { addApi } from "@api/member_card/index";
import { ElMessage, FormInstance } from "element-plus";
// 表单ref属性
const addFormRef = ref<FormInstance>();
// 弹框属性
const { dialog, onClose, onConfirm, onShow } = useDialog();
// 显示弹框
const show = () => {
  dialog.height = 200;
  onShow();
};
// 暴露出去，给父组件调用
defineExpose({

```

```
    show,
  });
// 表单数据
const addModel = reactive<CardType>({
  type: "",
  title: "",
  cardType: "",
  cardId: "",
  price: "",
  cardDay: 0,
  status: "",
});
// 表单验证规则
const rules = reactive({
  title: [
    {
      required: true,
      message: "请填写标题",
      trigger: "change",
    },
  ],
  cardType: [
    {
      required: true,
      message: "请选择类型",
      trigger: "change",
    },
  ],
  price: [
    {
      required: true,
      message: "请填写价格",
      trigger: "change",
    },
  ],
  cardDay: [
    {
      required: true,
      message: "请填写天数",
      trigger: "change",
    },
  ],
  status: [
    {
      required: true,
```

```

        message: "请选项状态",
        trigger: "change",
      },
    ],
  });
  //注册事件
  const emits = defineEmits(['refresh'])
  //表单提交
  const commit = () => {
    addFormRef.value?.validate(async (valid) => {
      if (valid) {
        let res = await addApi(addModel);
        if (res && res.code == 200) {
          ElMessage.success(res.msg);
          emits('refresh')
          onClose();
        }
      }
    });
  };
</script>

<style scoped></style>

```

8.5 会员卡编辑、删除接口对接

8.5.1 定义编辑、删除的接口

```

import http from "@/http";
import { CardType, ListParm } from "../MemberModel";
//新增
export const addApi = (parm:CardType)=>{
  return http.post("/api/memberCard",parm)
}
//列表
export const getListApi = (parm:ListParm)=>{
  return http.get("/api/memberCard/list",parm)
}
//编辑
export const editApi = (parm:CardType)=>{
  return http.put("/api/memberCard",parm)
}
//删除
export const deleteApi = (cardId:string)=>{

```

```
    return http.delete(`/api/memberCard/${cardId}`)  
  }  
}
```

8.5.2 修改useMember.ts

```
import { CardType } from "@api/member_card/MemberModel"  
import { EditType } from "@type/BaseEnum"  
import { ref } from "vue"  
import useInstance from "@hooks/useInstance";  
import { deleteApi } from "@api/member_card/index";  
import { ElMessage } from "element-plus";  
import { FuncList } from "@type/BaseType";  
export default function useMember(getList:FuncList){  
  const { global } = useInstance();  
  //弹框ref属性  
  const addRef = ref<{show:(type:string,row?:CardType)=>void}>  
(  
    //新增  
    const addBtn = ()=>{  
      addRef.value?.show(EditType.ADD)  
    }  
    //编辑  
    const editBtn = (row:CardType)=>{  
      addRef.value?.show(EditType.EDIT,row)  
    }  
    //删除  
    const deleteBtn = async(row:CardType)=>{  
      let confirm = await global.$myconfirm('确定删除该数据吗?')  
      if(confirm){  
        let res = await deleteApi(row.cardId)  
        if(res && res.code == 200){  
          ElMessage.success(res.msg)  
          //刷新表格  
          getList()  
        }  
      }  
    }  
  )  
  return{  
    addBtn,  
    editBtn,  
    deleteBtn,  
    addRef  
  }  
}
```

8.5.3 修改AddCard.vue组件

```
<template>
  <SysDialog
    :title="dialog.title"
    :width="dialog.width"
    :height="dialog.height"
    :visible="dialog.visible"
    @onClose="onClose"
    @onConfirm="commit"
  >
    <template v-slot:content>
      <el-form
        :model="addModel"
        ref="addFormRef"
        :rules="rules"
        label-width="80px"
        size="default"
      >
        <el-row>
          <el-col :span="24" :offset="0">
            <el-form-item prop="cardType" label="类型">
              <el-radio-group v-model="addModel.cardType">
                <el-radio :label="'1'">天卡</el-radio>
                <el-radio :label="'2'">周卡</el-radio>
                <el-radio :label="'3'">月卡</el-radio>
                <el-radio :label="'4'">年卡</el-radio>
              </el-radio-group>
            </el-form-item>
          </el-col>
        </el-row>
        <el-row>
          <el-col :span="12" :offset="0">
            <el-form-item prop="title" label="标题">
              <el-input v-model="addModel.title"></el-input>
            </el-form-item>
          </el-col>
          <el-col :span="12" :offset="0">
            <el-form-item prop="cardDay" label="天数">
              <el-input v-model="addModel.cardDay"></el-input>
            </el-form-item>
          </el-col>
        </el-row>
        <el-row>
          <el-col :span="12" :offset="0">
```



```

        <el-form-item prop="price" label="价格">
            <el-input v-model="addModel.price"></el-input>
        </el-form-item>
    </el-col>
    <el-col :span="12" :offset="0">
        <el-form-item prop="status" label="状态">
            <el-radio-group v-model="addModel.status">
                <el-radio :label="'0'">停用</el-radio>
                <el-radio :label="'1'">启用</el-radio>
            </el-radio-group>
        </el-form-item>
    </el-col>
</el-row>
</el-form>
</template>
</SysDialog>
</template>

```

```

<script setup lang="ts">
import SysDialog from "@/components/SysDialog.vue";
import useDialog from "@/hooks/useDialog";
import { CardType } from "@/api/member_card/MemberModel";
import { nextTick, reactive, ref } from "vue";
import { addApi, editApi } from "@/api/member_card/index";
import { ElMessage, FormInstance } from "element-plus";
import { EditType, Title } from "@/type/BaseEnum";
import useInstance from "@/hooks/useInstance";
const { global } = useInstance();
// 表单ref属性
const addFormRef = ref<FormInstance>();
// 弹框属性
const { dialog, onClose, onConfirm, onShow } = useDialog();
// 显示弹框
const show = (type: string, row?: CardType) => {
    dialog.height = 200;
    type == EditType.ADD
        ? (dialog.title = Title.ADD)
        : (dialog.title = Title.EDIT);
    if (type == EditType.EDIT) {
        // 把要编辑的数据复制到表单对象
        nextTick(() => {
            global.$objCopy(row, addModel);
        });
    }
    addModel.type = type;
}

```

```
    onShow();
};
//暴露出去，给父组件调用
defineExpose({
    show,
});
//表单数据
const addModel = reactive<CardType>({
    type: "",
    title: "",
    cardType: "",
    cardId: "",
    price: "",
    cardDay: 0,
    status: "",
});
//表单验证规则
const rules = reactive({
    title: [
        {
            required: true,
            message: "请填写标题",
            trigger: "change",
        },
    ],
    cardType: [
        {
            required: true,
            message: "请选择类型",
            trigger: "change",
        },
    ],
    price: [
        {
            required: true,
            message: "请填写价格",
            trigger: "change",
        },
    ],
    cardDay: [
        {
            required: true,
            message: "请填写天数",
            trigger: "change",
        },
    ],
}
```

```

    ],
    status: [
      {
        required: true,
        message: "请选项状态",
        trigger: "change",
      },
    ],
  ],
});
//注册事件
const emits = defineEmits(["refresh"]);
//表单提交
const commit = () => {
  addFormRef.value?.validate(async (valid) => {
    if (valid) {
      let res = null;
      if (addModel.type == EditType.ADD) {
        res = await addApi(addModel);
      } else {
        res = await editApi(addModel)
      }
      if (res && res.code == 200) {
        ElMessage.success(res.msg);
        emits("refresh");
        onClose();
      }
    }
  });
};
</script>

<style scoped></style>

```

8.5.4 修改CardType.vue组件

```

<template>
  <el-main>
    <!-- 搜索栏 -->
    <el-form :model="listParm" :inline="true" size="default">
      <el-form-item>
        <el-input v-model="listParm.title" placeholder="请输入标题"></el-input>
      </el-form-item>
    </el-form>
  </el-main>

```

```
<el-button :icon="Search" @click="searchBtn">搜索</el-  
button>  
  <el-button :icon="Close" type="danger" plain  
@click="resetBtn"  
  >重置</el-button  
  >  
  <el-button v-permission="['sys:memberRoot:add']"  
:icon="Plus" type="primary" @click="addBtn"  
  >新增</el-button  
  >  
</el-form-item>  
</el-form>  
<!-- 表格 -->  
<el-table :height="tableHeight" :data="tableList.list" border  
stripe>  
  <el-table-column prop="title" label="标题"></el-table-  
column>  
  <el-table-column prop="cardType" label="类型">  
    <template #default="scope">  
      <el-tag v-if="scope.row.cardType == '1'"  
size="default">天卡</el-tag>  
      <el-tag v-if="scope.row.cardType == '2'" type="success"  
size="default"  
      >周卡</el-tag  
      >  
      <el-tag v-if="scope.row.cardType == '3'" type="danger"  
size="default"  
      >月卡</el-tag  
      >  
      <el-tag v-if="scope.row.cardType == '4'" type="danger"  
size="default"  
      >年卡</el-tag  
      >  
    </template>  
  </el-table-column>  
  <el-table-column prop="cardDay" label="天数"></el-table-  
column>  
  <el-table-column prop="price" label="价格(元)"></el-table-  
column>  
  <el-table-column prop="status" label="状态">  
    <template #default="scope">  
      <el-tag v-if="scope.row.status == '1'" type="success"  
size="default">启用</el-tag>  
      <el-tag v-if="scope.row.status == '0'" type="danger"  
size="default">停用</el-tag>
```

```

        </template>
      </el-table-column>
      <el-table-column label="操作" align="center" width="220">
        <template #default="scope">
          <el-button v-permission="['sys:memberRoot:edit']"
type="primary" :icon="Edit" size="default"
@click="editBtn(scope.row)">编辑</el-button>
          <el-button v-permission="['sys:memberRoot:delete']"
type="danger" :icon="Delete" size="default"
@click="deleteBtn(scope.row)">删除</el-button>
        </template>
      </el-table-column>
    </el-table>
    <!-- 分页 -->
    <el-pagination
      @size-change="sizeChange"
      @current-change="currentChange"
      :current-page.sync="listParm.currentPage"
      :page-sizes="[10,20, 40, 80, 100]"
      :page-size="listParm.pageSize"
      layout="total, sizes, prev, pager, next, jumper"
      :total="listParm.total" background>
    </el-pagination>

    <!-- 新增、编辑 -->
    <AddCard ref="addRef" @refresh="refresh"></AddCard>
  </el-main>
</template>

<script setup lang="ts">
import AddCard from "../AddCard.vue";
import { Plus, Edit, Delete, Search, Close } from "@element-
plus/icons-vue";
import useMemberTable from
"@/composables/member_card/useMemberTable";
import useMember from "@/composables/member_card/useMember";
//表格操作
const { listParm, getList, resetBtn,
searchBtn, tableList, sizeChange, currentChange, tableHeight, refresh
} = useMemberTable();
//新增、编辑
const { addBtn, editBtn, deleteBtn, addRef } = useMember(getList);
</script>

<style scoped></style>

```

9 会员管理模块功能开发

9.1 数据表设计

```
create table member
(
    member_id          int not null auto_increment comment '会员id',
    name               varchar(64) comment '姓名',
    sex                varchar(2) comment '性别',
    phone              varchar(15) comment '电话',
    age                int comment '年龄',
    birthday            varchar(36) comment '生日',
    height              int comment '身高',
    weight              int comment '体重',
    waist              int comment '腰围',
    join_time           varchar(36) comment '加入时间',
    end_time            varchar(36) comment '到期时间',
    username            varchar(64) comment '会员卡号',
    password            varchar(128) comment '密码',
    status              varchar(2) comment '状态 1: 启用 0: 停用',
    primary key (member_id)
);
```

```
create table member_role
(
    member_role_id      int not null auto_increment comment '会员角色id',
    member_id            int comment '会员id',
    role_id              int comment '角色id',
    primary key (member_role_id)
);
```

9.2 会员管理后台接口开发

9.2.1 创建会员实体相关的后端代码

9.2.1.1 创建实体类

在web模块下面创建member/entity目录，并定义如下实体类：

```
package com.xq.web.member.entity;

import com.baomidou.mybatisplus.annotation.IdType;
```

```
import com.baomidou.mybatisplus.annotation.TableId;
import com.baomidou.mybatisplus.annotation.TableName;
import lombok.Data;

@Data
@TableName("member")
public class Member {
    @TableId(type = IdType.AUTO)
    private Long memberId;
    private String name;
    private String sex;
    private String phone;
    private Integer age;
    private String birthday;
    private Integer height;
    private Integer weight;
    private Integer waist;
    private String joinTime;
    private String endTime;
    private String username;
    private String password;
    private String status;
}
```

```
package com.xq.web.member.entity;

import lombok.Data;

@Data
public class PageParm {
    private Long currentPage;
    private Long pageSize;
    private String name;
    private String phone;
    private String username;
}
```

9.2.1.2 创建mapper接口

```

package com.xq.web.member.mapper;

import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.xq.web.member.entity.Member;

public interface MemberMapper extends BaseMapper<Member> {
}

```

9.2.1.3 创建mapper映射文件

```

<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="com.xq.web.member.mapper.MemberMapper">

</mapper>

```

9.2.1.4 创建service业务接口及实现类

```

package com.xq.web.member.service;

import com.baomidou.mybatisplus.extension.service.IService;
import com.xq.web.member.entity.Member;

public interface MemberService extends IService<Member> {
}

```

```

package com.xq.web.member.service.impl;

import
com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
import com.xq.web.member.entity.Member;
import com.xq.web.member.mapper.MemberMapper;
import com.xq.web.member.service.MemberService;
import org.springframework.stereotype.Service;

@Service
public class MembersServiceImpl extends ServiceImpl<MemberMapper,
Member> implements MemberService {

}

```


9.2.1.5 创建controller

```
package com.xq.web.member.controller;

import
com.baomidou.mybatisplus.core.conditions.query.QueryWrapper;
import com.baomidou.mybatisplus.core.metadata.IPage;
import
com.baomidou.mybatisplus.extension.plugins.pagination.Page;
import com.xq.utils.ResultUtils;
import com.xq.utils.ResultVo;
import com.xq.web.member.entity.Member;
import com.xq.web.member.entity.PageParm;
import com.xq.web.member.service.MemberService;
import org.apache.commons.lang.StringUtils;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/member")
public class MemberController {
    @Autowired
    private MemberService memberService;

    //新增
    @PostMapping
    public ResultVo add(@RequestBody Member member){
        if(memberService.save(member)){
            return ResultUtils.success("新增成功!");
        }
        return ResultUtils.error("新增失败!");
    }

    //编辑
    @PutMapping
    public ResultVo edit(@RequestBody Member member){
        if(memberService.updateById(member)){
            return ResultUtils.success("编辑成功!");
        }
        return ResultUtils.error("编辑失败!");
    }

    //删除
    @DeleteMapping("/{memberId}")
```

```

        public ResultVo delete(@PathVariable("memberId") Long
memberId){
            if(memberService.removeById(memberId)){
                return ResultUtils.success("删除成功!");
            }
            return ResultUtils.error("删除失败!");
        }

        //查询
        @GetMapping("/list")
        public ResultVo list(PageParm pageParm){
            //构造分页对象
            IPage<Member> page = new Page<>
(pageParm.getCurrentPage(),pageParm.getPageSize());
            //构造查询条件
            QueryWrapper<Member> query = new QueryWrapper<>();
            if(StringUtils.isNotEmpty(pageParm.getName())){

                query.lambda().like(Member::getName,pageParm.getName());
            }
            if(StringUtils.isNotEmpty(pageParm.getPhone())){

                query.lambda().like(Member::getPhone,pageParm.getPhone());
            }
            if(StringUtils.isNotEmpty(pageParm.getUsername())){

                query.lambda().like(Member::getUsername,pageParm.getUsername());
            }
            IPage<Member> list = memberService.page(page, query);
            return ResultUtils.success("查询成功",list);
        }
    }
}

```

9.2.2 创建会员和角色中间表相关的后端代码

9.2.2.1 创建实体类

创建member_role/entity目录:

```

package com.xq.web.member_role.entity;

import com.baomidou.mybatisplus.annotation.IdType;
import com.baomidou.mybatisplus.annotation.TableId;
import com.baomidou.mybatisplus.annotation.TableName;
import lombok.Data;

```

```
@Data
@TableName("member_role")
public class MemberRole {
    @TableId(type = IdType.AUTO)
    private Long memberRoleId;
    private Long memberId;
    private Long roleId;
}
```

9.2.2.2 创建mapper接口

```
package com.xq.web.member_role.mapper;

import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.xq.web.member_role.entity.MemberRole;

public interface MemberRoleMapper extends BaseMapper<MemberRole>
{
}
```

9.2.2.3 创建mapper映射文件

```
<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper
    namespace="com.xq.web.member_role.mapper.MemberRoleMapper">

</mapper>
```

9.2.2.4 创建service业务接口和实现类

```
package com.xq.web.member_role.service;

import com.baomidou.mybatisplus.extension.service.IService;
import com.xq.web.member_role.entity.MemberRole;

public interface MemberRoleService extends IService<MemberRole> {
}
```

```

package com.xq.web.member_role.service.impl;

import
com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
import com.xq.web.member_role.entity.MemberRole;
import com.xq.web.member_role.mapper.MemberRoleMapper;
import com.xq.web.member_role.service.MemberRoleService;
import org.springframework.stereotype.Service;

@Service
public class MemberRoleServiceImpl extends
ServiceImpl<MemberRoleMapper, MemberRole> implements
MemberRoleService {

}

```

9.3 会员管理模块前端页面制作与数据对接

9.3.1 会员新增页面制作与数据对接

9.3.1.1 api下新建member目录，建MemberModel.ts和index.ts

- MemberModel.ts

```

//定义列表参数类型
export type MemberParam = {
  name:string,
  phone:string,
  username:string,
  currentPage:number,
  pageSize:number,
  total:number
}
//会员数据类型
export type MemberType = {
  type:string,
  memberId:string,
  name:string,
  sex:string,
  phone:string,
  age:string,
  birthday:string,
  height:string,
  weight:string,
  waist:string,

```

```

    joinTime:string,
    endTime:string,
    username:string,
    password:string,
    status:string
  }

```

- index.ts

```

import http from "@http"
import { MemberType } from "../MemberModel"
//新增
export const addApi = (param:MemberType)=>{
  return http.post("/api/member",param)
}

```

9.3.1.2 composables下新建member目录, 新建useMember.ts和useTable.ts

- useMember.ts

```

import { ref } from "vue"

export default function useMember(){
  const addRef = ref<{show:()=>void}>()
  //新增
  const addBtn = ()=>{
    addRef.value?.show()
  }
  //编辑
  const editBtn = ()=>{

  }
  //删除
  const deleteBtn = ()=>{

  }
  return{
    addBtn,
    editBtn,
    deleteBtn,
    addRef
  }
}

```

- useTable.ts

```

import { MemberParam } from "@api/member/MemberModel";
import { reactive } from "vue";

export default function useTable() {
  //参数列表
  const listParam = reactive<MemberParam>({
    name: '',
    phone: '',
    username: '',
    pageSize: 10,
    currentPage: 1,
    total: 0
  })
  //列表
  const getList = ()=>{

  }
  //搜索
  const searchBtn = ()=>{

  }
  //重置
  const resetBtn = ()=>{

  }
  return{
    listParam,
    getList,
    searchBtn,
    resetBtn
  }
}

```

9.3.1.3 新建AddMember.vue组件

在views/member目录下面创建AddMember.vue组件

```

<template>
  <SysDialog
    :title="dialog.title"
    :width="dialog.width"
    :height="dialog.height"
    :visible="dialog.visible"
    @onClose="onClose"
    @onConfirm="commit"
  >

```

```
>
<template v-slot:content>
  <el-form
    :model="addModel"
    ref="addFormRef"
    :rules="rules"
    label-width="80px"
    size="default"
  >
    <el-row>
      <el-col :span="12" :offset="0">
        <el-form-item prop="name" label="姓名">
          <el-input v-model="addModel.name"></el-input>
        </el-form-item>
      </el-col>
      <el-col :span="12" :offset="0">
        <el-form-item prop="sex" label="性别">
          <el-radio-group v-model="addModel.sex">
            <el-radio :label="'0'">男</el-radio>
            <el-radio :label="'1'">女</el-radio>
          </el-radio-group>
        </el-form-item>
      </el-col>
    </el-row>
    <el-row>
      <el-col :span="12" :offset="0">
        <el-form-item prop="phone" label="电话">
          <el-input v-model="addModel.phone"></el-input>
        </el-form-item>
      </el-col>
      <el-col :span="12" :offset="0">
        <el-form-item prop="age" label="年龄">
          <el-input v-model="addModel.age"></el-input>
        </el-form-item>
      </el-col>
    </el-row>
    <el-row>
      <el-col :span="12" :offset="0">
        <el-form-item prop="birthday" label="生日">
          <el-date-picker
            v-model="addModel.birthday"
            type="date"
            placeholder="请选择生日"
            size="default"
            value-format="YYYY-MM-DD"
          >

```

```
        format="YYYY-MM-DD"
      />
    </el-form-item>
  </el-col>
  <el-col :span="12" :offset="0">
    <el-form-item prop="joinTime" label="加入时间">
      <el-date-picker
        v-model="addModel.joinTime"
        type="date"
        placeholder="请选择加入时间"
        size="default"
        format="YYYY-MM-DD"
        value-format="YYYY-MM-DD"
      />
    </el-form-item>
  </el-col>
</el-row>
<el-row>
  <el-col :span="12" :offset="0">
    <el-form-item prop="height" label="身高">
      <el-input v-model="addModel.height"></el-input>
    </el-form-item>
  </el-col>
  <el-col :span="12" :offset="0">
    <el-form-item prop="weight" label="体重">
      <el-input v-model="addModel.weight"></el-input>
    </el-form-item>
  </el-col>
</el-row>
<el-row>
  <el-col :span="12" :offset="0">
    <el-form-item prop="waist" label="腰围">
      <el-input v-model="addModel.waist"></el-input>
    </el-form-item>
  </el-col>
  <el-col :span="12" :offset="0">
    <el-form-item prop="username" label="会员卡号">
      <el-input v-model="addModel.username"></el-input>
    </el-form-item>
  </el-col>
</el-row>
<el-row>
  <el-col :span="12" :offset="0">
    <el-form-item prop="password" label="密码">
      <el-input v-model="addModel.password"></el-input>
```



```

        </el-form-item>
      </el-col>
      <el-col :span="12" :offset="0">
        <el-form-item prop="status" label="状态">
          <el-radio-group v-model="addModel.status">
            <el-radio :label="'0'">停用</el-radio>
            <el-radio :label="'1'">启用</el-radio>
          </el-radio-group>
        </el-form-item>
      </el-col>
    </el-row>
  </el-form>
</template>
</SysDialog>
</template>

<script setup lang="ts">
import SysDialog from "@/components/SysDialog.vue";
import useDialog from "@/hooks/useDialog";
import { reactive, ref } from "vue";
import { MemberType } from "@api/member/MemberModel";
import { ElMessage, FormInstance } from "element-plus";
import { addApi } from "@api/member/index";
const addRormRef = ref<FormInstance>();
//弹框属性
const { dialog, onClose, onConfirm, onShow } = useDialog();
//弹框显示
const show = () => {
  dialog.width = 680;
  dialog.height = 300;
  onShow();
};
//暴露出去，给父组件调用
defineExpose({
  show,
});
//表单绑定的数据对象
const addModel = reactive<MemberType>({
  type: "",
  memberId: "",
  name: "",
  sex: "",
  phone: "",
  age: "",
  birthday: "",

```

```
height: "",
weight: "",
waist: "",
joinTime: "",
endTime: "",
username: "",
password: "",
status: "",
});
//表单验证规则
const rules = reactive({
  name: [
    {
      required: true,
      trigger: "change",
      message: "请填写姓名",
    },
  ],
  sex: [
    {
      required: true,
      trigger: "change",
      message: "请选择性别",
    },
  ],
  phone: [
    {
      required: true,
      trigger: "change",
      message: "请填写电话",
    },
  ],
  joinTime: [
    {
      required: true,
      trigger: "change",
      message: "请填写加入时间",
    },
  ],
  username: [
    {
      required: true,
      trigger: "change",
      message: "请填写会员卡号",
    },
  ],
});
```

```

    ],
    password: [
      {
        required: true,
        trigger: "change",
        message: "请填写密码",
      },
    ],
    status: [
      {
        required: true,
        trigger: "change",
        message: "请选择状态",
      },
    ],
  ],
});
//表单提交
const commit = () => {
  addRormRef.value?.validate(async (valid) => {
    if (valid) {
      let res = await addApi(addModel);
      if (res && res.code == 200) {
        ElMessage.success(res.msg)
        onClose();
      }
    }
  });
};
</script>

<style scoped></style>

```

9.3.1.4 创建MemberList.vue组件

```

<template>
  <el-main>
    <!--搜索栏-->
    <el-form :model="listParam" :inline="true" size="default">
      <el-form-item>
        <el-input v-model="listParam.name" placeholder="请输入姓名"></el-input>
      </el-form-item>
      <el-form-item>
        <el-input v-model="listParam.phone" placeholder="请输入电话"></el-input>

```

```

        </el-form-item>
        <el-form-item>
            <el-input
                v-model="listParam.username"
                placeholder="请输入会员卡号"
            ></el-input>
        </el-form-item>
        <el-form-item>
            <el-button :icon="Search" @click="searchBtn">搜索</el-
button>
            <el-button type="danger" plain :icon="Close"
@click="resetBtn">重置</el-button>
            <el-button :icon="Plus" type="primary" @click="addBtn"
            >新增</el-button>
        >
    </el-form-item>
</el-form>
<!-- 新增、编辑 -->
<AddMember ref="addRef"></AddMember>
</el-main>
</template>

<script setup lang="ts">
    import AddMember from "../AddMember.vue";
    import { Plus, Edit, Delete, Search, Close } from "@element-
plus/icons-vue";
    import useTable from "@composables/member/useTable";
    import useMember from "@composables/member/useMember";
    //表格相关
    const { listParam, getList, resetBtn, searchBtn } =
useTable();
    //新增、编辑操作
    const { addBtn, editBtn, deleteBtn, addRef } = useMember();
</script>

<style scoped></style>

```

9.3.2 会员信息列表展示数据对象

9.3.2.1 在api/member/index.ts新增getListApi()方法

```

import http from "@/http"
import { MemberParm, MemberType } from "@/MemberModel"
//新增
export const addApi = (parm:MemberType)=>{
    return http.post("/api/member",parm)
}
//列表
export const getListApi = (parm:MemberParm)=>{
    return http.get("/api/member/list",parm)
}

```

9.3.2.2 composables/member/useTable.ts修改为如下

```

import { MemberParm } from "@/api/member/MemberModel";
import { nextTick, onMounted, reactive, ref } from "vue";
import { getListApi } from "@/api/member";
export default function useTable() {
    //表格高度
    const tableHeight = ref(0)
    //表格数据定义
    const tableList = reactive({
        list:[]
    })
    //参数列表
    const listParm = reactive<MemberParm>({
        name: '',
        phone: '',
        username: '',
        pageSize: 10,
        currentPage: 1,
        total: 0
    })
    //列表
    const getList = async()=>{
        let res = await getListApi(listParm)
        if(res && res.code == 200){
            console.log(res)
            tableList.list = res.data.records
            listParm.total = res.data.total
        }
    }
    //搜索
    const searchBtn = ()=>{
        getList()
    }
}

```

```
}
//重置
const resetBtn = ()=>{
  listParm.name = ''
  listParm.phone = ''
  listParm.username = ''
  getList()
}
//页容量改变时触发
const sizeChange = (size:number)=>{
  listParm.pageSize = size;
  getList()
}
//页数改变时触发
const currentChange = (page:number)=>{
  listParm.currentPage = page;
  getList()
}
//刷新表格
const refresh = ()=>{
  getList()
}
onMounted(()=>{
  getList()
  nextTick(()=>{
    tableHeight.value = window.innerHeight - 230
  })
})
return{
  listParm,
  getList,
  searchBtn,
  resetBtn,
  tableList,
  sizeChange,
  currentChange,
  tableHeight,
  refresh
}
}
```

9.3.2.3 MemberList.vue组件

```
<template>
  <el-main>
    <!-- 搜索栏 -->
    <el-form :model="listParm" :inline="true" size="default">
      <el-form-item>
        <el-input v-model="listParm.name" placeholder="请输入姓名">
      </el-input>
      </el-form-item>
      <el-form-item>
        <el-input v-model="listParm.phone" placeholder="请输入电
话"></el-input>
      </el-form-item>
      <el-form-item>
        <el-input
          v-model="listParm.username"
          placeholder="请输入会员卡号"
        ></el-input>
      </el-form-item>
      <el-form-item>
        <el-button :icon="Search" @click="searchBtn">搜索</el-
button>
        <el-button type="danger" plain :icon="Close"
@click="resetBtn">重置</el-button>
        <el-button :icon="Plus" type="primary" @click="addBtn"
>新增</el-button>
      >
    </el-form-item>
  </el-form>
  <!-- 表格 -->
  <el-table :height="tableHeight" :data="tableList.list" border
stripe>
    <el-table-column prop="username" label="会员卡号"></el-table-
column>
    <el-table-column prop="name" label="姓名"></el-table-column>
    <el-table-column prop="sex" label="性别">
      <template #default="scope">
        <el-tag v-if="scope.row.sex == '0'" type="success"
size="default">男</el-tag>
        <el-tag v-if="scope.row.sex == '1'" type="danger"
size="default">女</el-tag>
      </template>
    </el-table-column>
    <el-table-column prop="age" label="年龄"></el-table-column>
```

```

        <el-table-column prop="birthday" label="生日"></el-table-
column>
        <el-table-column prop="joinTime" label="加入时间"></el-table-
column>
        <el-table-column prop="height" label="身高"></el-table-
column>
        <el-table-column prop="weight" label="体重"></el-table-
column>
        <el-table-column prop="waist" label="腰围"></el-table-
column>
        <el-table-column prop="status" label="状态">
          <template #default="scope">
            <el-tag v-if="scope.row.status == '1'" type="success"
size="default">启用</el-tag>
            <el-tag v-if="scope.row.status == '0'" type="danger"
size="default">停用</el-tag>
          </template>
        </el-table-column>
        <el-table-column label="操作" width="220" align="center">
          <template #default="scope">
            <el-button type="primary" :icon="Edit" size="default"
@click="editBtn">编辑</el-button>
            <el-button type="danger" :icon="Delete" size="default"
@click="deleteBtn">删除</el-button>
          </template>
        </el-table-column>
      </el-table>
      <!-- 分页 -->
      <el-pagination
        @size-change="sizeChange"
        @current-change="currentChange"
        :current-page.sync="listParm.currentPage"
        :page-sizes="[10,20, 40, 80, 100]"
        :page-size="listParm.pageSize"
        layout="total, sizes, prev, pager, next, jumper"
        :total="listParm.total" background>
      </el-pagination>

      <!-- 新增、编辑 -->
      <AddMember ref="addRef" @refresh="refresh"></AddMember>
    </el-main>
  </template>

  <script setup lang="ts">
  import AddMember from "../AddMember.vue";

```



```

import { Plus, Edit, Delete, Search, Close } from "@element-plus/icons-vue";
import useTable from "@/composables/member/useTable";
import useMember from "@/composables/member/useMember";
//表格相关
const { listParm, getList, resetBtn,
searchBtn, tableList, sizeChange
,currentChange, tableHeight, refresh } = useTable();
//新增、编辑操作
const { addBtn, editBtn, deleteBtn, addRef } = useMember();
</script>

<style scoped></style>

```

9.3.2.4 AddMember.vue组件刷新表格

```

<template>
  <SysDialog
    :title="dialog.title"
    :width="dialog.width"
    :height="dialog.height"
    :visible="dialog.visible"
    @onClose="onClose"
    @onConfirm="commit"
  >
    <template v-slot:content>
      <el-form
        :model="addModel"
        ref="addFormRef"
        :rules="rules"
        label-width="80px"
        size="default"
      >
        <el-row>
          <el-col :span="12" :offset="0">
            <el-form-item prop="name" label="姓名">
              <el-input v-model="addModel.name"></el-input>
            </el-form-item>
          </el-col>
          <el-col :span="12" :offset="0">
            <el-form-item prop="sex" label="性别">
              <el-radio-group v-model="addModel.sex">
                <el-radio :label="'0'">男</el-radio>
                <el-radio :label="'1'">女</el-radio>
              </el-radio-group>
            </el-form-item>
          </el-col>
        </el-row>
      </el-form>
    </template>
  </SysDialog>

```

```
        </el-radio-group>
      </el-form-item>
    </el-col>
  </el-row>
  <el-row>
    <el-col :span="12" :offset="0">
      <el-form-item prop="phone" label="电话">
        <el-input v-model="addModel.phone"></el-input>
      </el-form-item>
    </el-col>
    <el-col :span="12" :offset="0">
      <el-form-item prop="age" label="年龄">
        <el-input v-model="addModel.age"></el-input>
      </el-form-item>
    </el-col>
  </el-row>
  <el-row>
    <el-col :span="12" :offset="0">
      <el-form-item prop="birthday" label="生日">
        <el-date-picker
          v-model="addModel.birthday"
          type="date"
          placeholder="请选择生日"
          size="default"
          value-format="YYYY-MM-DD"
          format="YYYY-MM-DD"
        />
      </el-form-item>
    </el-col>
    <el-col :span="12" :offset="0">
      <el-form-item prop="joinTime" label="加入时间">
        <el-date-picker
          v-model="addModel.joinTime"
          type="date"
          placeholder="请选择加入时间"
          size="default"
          format="YYYY-MM-DD"
          value-format="YYYY-MM-DD"
        />
      </el-form-item>
    </el-col>
  </el-row>
  <el-row>
    <el-col :span="12" :offset="0">
      <el-form-item prop="height" label="身高">
```

```

        <el-input v-model="addModel.height"></el-input>
      </el-form-item>
    </el-col>
    <el-col :span="12" :offset="0">
      <el-form-item prop="weight" label="体重">
        <el-input v-model="addModel.weight"></el-input>
      </el-form-item>
    </el-col>
  </el-row>
  <el-row>
    <el-col :span="12" :offset="0">
      <el-form-item prop="waist" label="腰围">
        <el-input v-model="addModel.waist"></el-input>
      </el-form-item>
    </el-col>
    <el-col :span="12" :offset="0">
      <el-form-item prop="username" label="会员卡号">
        <el-input v-model="addModel.username"></el-input>
      </el-form-item>
    </el-col>
  </el-row>
  <el-row>
    <el-col :span="12" :offset="0">
      <el-form-item prop="password" label="密码">
        <el-input v-model="addModel.password"></el-input>
      </el-form-item>
    </el-col>
    <el-col :span="12" :offset="0">
      <el-form-item prop="status" label="状态">
        <el-radio-group v-model="addModel.status">
          <el-radio :label="'0'">停用</el-radio>
          <el-radio :label="'1'">启用</el-radio>
        </el-radio-group>
      </el-form-item>
    </el-col>
  </el-row>
</el-form>
</template>
</SysDialog>
</template>

<script setup lang="ts">
import SysDialog from "@/components/SysDialog.vue";
import useDialog from "@/hooks/useDialog";
import { reactive, ref } from "vue";

```

```
import { MemberType } from "@/api/member/MemberModel";
import { ElMessage, FormInstance } from "element-plus";
import { addApi } from "@/api/member/index";
const addRormRef = ref<FormInstance>();
//弹框属性
const { dialog, onClose, onConfirm, onShow } = useDialog();
//弹框显示
const show = () => {
  dialog.width = 680;
  dialog.height = 300;
  onShow();
};
//暴露出去，给父组件调用
defineExpose({
  show,
});
//表单绑定的数据对象
const addModel = reactive<MemberType>({
  type: "",
  memberId: "",
  name: "",
  sex: "",
  phone: "",
  age: "",
  birthday: "",
  height: "",
  weight: "",
  waist: "",
  joinTime: "",
  endTime: "",
  username: "",
  password: "",
  status: "",
});
//表单验证规则
const rules = reactive({
  name: [
    {
      required: true,
      trigger: "change",
      message: "请填写姓名",
    },
  ],
  sex: [
    {
```

```

        required: true,
        trigger: "change",
        message: "请选择性别",
    },
],
phone: [
    {
        required: true,
        trigger: "change",
        message: "请填写电话",
    },
],
joinTime: [
    {
        required: true,
        trigger: "change",
        message: "请填写加入时间",
    },
],
username: [
    {
        required: true,
        trigger: "change",
        message: "请填写会员卡号",
    },
],
password: [
    {
        required: true,
        trigger: "change",
        message: "请填写密码",
    },
],
status: [
    {
        required: true,
        trigger: "change",
        message: "请选择状态",
    },
],
});
//注册事件
const emits = defineEmits(['refresh'])
//表单提交
const commit = () => {

```

```

addRormRef.value?.validate(async (valid) => {
  if (valid) {
    let res = await addApi(addModel);
    if (res && res.code == 200) {
      ElMessage.success(res.msg)
      emits('refresh')
      onClose();
    }
  }
});
</script>

<style scoped></style>

```

9.3.3 编辑、删除会员接口对接

9.3.3.1 api/member/index.ts添加editApi()和deleteApi()

```

import http from "@http"
import { MemberParm, MemberType, ApplyCard, Recharge } from
"./MemberModel"
//新增
export const addApi = (parm:MemberType)=>{
  return http.post("/api/member",parm)
}
//列表
export const getListApi = (parm:MemberParm)=>{
  return http.get("/api/member/list",parm)
}
//编辑
export const editApi = (parm:MemberType)=>{
  return http.put("/api/member",parm)
}
//删除
export const deleteApi = (memberId:string)=>{
  return http.delete(`/api/member/${memberId}`)
}

```

9.3.3.2 修改composables/member/useMember.ts

```
import { MemberType } from "@api/member/MemberModel"
import { EditType } from "@type/BaseEnum"
import { ref } from "vue"
import { deleteApi } from "@api/member"
import { ElMessage } from "element-plus"
import { FuncList } from "@type/BaseType"
import useInstance from "@hooks/useInstance"
export default function useMember(getList: FuncList) {
  const { global } = useInstance()
  const addRef = ref<{ show: (type: string, row?: MemberType)
=> void }>()
  //新增
  const addBtn = () => {
    addRef.value?.show(EditType.ADD)
  }
  //编辑
  const editBtn = (row: MemberType) => {
    addRef.value?.show(EditType.EDIT, row)
  }
  //删除
  const deleteBtn = async (row: MemberType) => {
    const confirm = await global.$myconfirm('确定删除该数据吗?')
    if (confirm) {
      let res = await deleteApi(row.memberId)
      if (res && res.code == 200) {
        ElMessage.success(res.msg)
        getList()
      }
    }
  }
  return {
    addBtn,
    editBtn,
    deleteBtn,
    addRef
  }
}
```

9.3.3.3 修改AddMember.vue组件

```
<template>
  <SysDialog
    :title="dialog.title"
```

```

:width="dialog.width"
:height="dialog.height"
:visible="dialog.visible"
@onClose="onClose"
@onConfirm="commit"
>
<template v-slot:content>
  <el-form
    :model="addModel"
    ref="addFormRef"
    :rules="rules"
    label-width="80px"
    size="default"
  >
    <el-row>
      <el-col :span="12" :offset="0">
        <el-form-item prop="name" label="姓名">
          <el-input v-model="addModel.name"></el-input>
        </el-form-item>
      </el-col>
      <el-col :span="12" :offset="0">
        <el-form-item prop="sex" label="性别">
          <el-radio-group v-model="addModel.sex">
            <el-radio :label="'0'">男</el-radio>
            <el-radio :label="'1'">女</el-radio>
          </el-radio-group>
        </el-form-item>
      </el-col>
    </el-row>
    <el-row>
      <el-col :span="12" :offset="0">
        <el-form-item prop="phone" label="电话">
          <el-input v-model="addModel.phone"></el-input>
        </el-form-item>
      </el-col>
      <el-col :span="12" :offset="0">
        <el-form-item prop="age" label="年龄">
          <el-input v-model="addModel.age"></el-input>
        </el-form-item>
      </el-col>
    </el-row>
    <el-row>
      <el-col :span="12" :offset="0">
        <el-form-item prop="birthday" label="生日">
          <el-date-picker

```



```
        v-model="addModel.birthday"
        type="date"
        placeholder="请选择生日"
        size="default"
        value-format="YYYY-MM-DD"
        format="YYYY-MM-DD"
      />
    </el-form-item>
  </el-col>
  <el-col :span="12" :offset="0">
    <el-form-item prop="joinTime" label="加入时间">
      <el-date-picker
        v-model="addModel.joinTime"
        type="date"
        placeholder="请选择加入时间"
        size="default"
        format="YYYY-MM-DD"
        value-format="YYYY-MM-DD"
      />
    </el-form-item>
  </el-col>
</el-row>
<el-row>
  <el-col :span="12" :offset="0">
    <el-form-item prop="height" label="身高">
      <el-input v-model="addModel.height"></el-input>
    </el-form-item>
  </el-col>
  <el-col :span="12" :offset="0">
    <el-form-item prop="weight" label="体重">
      <el-input v-model="addModel.weight"></el-input>
    </el-form-item>
  </el-col>
</el-row>
<el-row>
  <el-col :span="12" :offset="0">
    <el-form-item prop="waist" label="腰围">
      <el-input v-model="addModel.waist"></el-input>
    </el-form-item>
  </el-col>
  <el-col :span="12" :offset="0">
    <el-form-item prop="username" label="会员卡号">
      <el-input v-model="addModel.username"></el-input>
    </el-form-item>
  </el-col>
</el-row>
```

```

    </el-row>
    <el-row>
      <el-col :span="12" :offset="0">
        <el-form-item prop="password" label="密码">
          <el-input v-model="addModel.password"></el-input>
        </el-form-item>
      </el-col>
      <el-col :span="12" :offset="0">
        <el-form-item prop="status" label="状态">
          <el-radio-group v-model="addModel.status">
            <el-radio :label="'0'">停用</el-radio>
            <el-radio :label="'1'">启用</el-radio>
          </el-radio-group>
        </el-form-item>
      </el-col>
    </el-row>
  </el-form>
</template>
</SysDialog>
</template>

```

```

<script setup lang="ts">
import SysDialog from "@/components/SysDialog.vue";
import useDialog from "@/hooks/useDialog";
import { nextTick, reactive, ref } from "vue";
import { MemberType } from "@api/member/MemberModel";
import { ElMessage, FormInstance } from "element-plus";
import { addApi, editApi } from "@api/member/index";
import { EditType, Title } from "@type/BaseEnum";
import useInstance from "@/hooks/useInstance";
const { global } = useInstance();
const addRormRef = ref<FormInstance>();
//弹框属性
const { dialog, onClose, onConfirm, onShow } = useDialog();
//弹框显示
const show = (type: string, row?: MemberType) => {
  dialog.width = 680;
  dialog.height = 300;
  type == EditType.ADD
    ? (dialog.title = Title.ADD)
    : (dialog.title = Title.EDIT);
  if (EditType.EDIT == type) {
    nextTick(() => {
      global.$objCopy(row, addModel);
    });
  }
}

```

```

    }
    addModel.type = type;
    onShow();
    addFormRef.value?.resetFields()
  };
  //暴露出去，给父组件调用
  defineExpose({
    show,
  });
  //表单绑定的数据对象
  const addModel = reactive<MemberType>({
    type: "",
    memberId: "",
    name: "",
    sex: "",
    phone: "",
    age: "",
    birthday: "",
    height: "",
    weight: "",
    waist: "",
    joinTime: "",
    endTime: "",
    username: "",
    password: "",
    status: "",
  });
  //表单验证规则
  const rules = reactive({
    name: [
      {
        required: true,
        trigger: "change",
        message: "请填写姓名",
      },
    ],
    sex: [
      {
        required: true,
        trigger: "change",
        message: "请选择性别",
      },
    ],
    phone: [
      {

```

```

        required: true,
        trigger: "change",
        message: "请填写电话",
    },
],
joinTime: [
    {
        required: true,
        trigger: "change",
        message: "请填写加入时间",
    },
],
username: [
    {
        required: true,
        trigger: "change",
        message: "请填写会员卡号",
    },
],
password: [
    {
        required: true,
        trigger: "change",
        message: "请填写密码",
    },
],
status: [
    {
        required: true,
        trigger: "change",
        message: "请选择状态",
    },
],
});
//注册事件
const emits = defineEmits(["refresh"]);
//表单提交
const commit = () => {
    addRormRef.value?.validate(async (valid) => {
        if (valid) {
            let res = null;
            if(addModel.type == EditType.ADD){
                res = await addApi(addModel);
            }else{
                res = await editApi(addModel)
            }
        }
    })
}

```

```

    }
    if (res && res.code == 200) {
      ElMessage.success(res.msg);
      emits("refresh");
      onClose();
    }
  }
});
};
</script>

<style scoped></style>

```

9.3.3.4 修改MemberList.vue组件

```

<template>
  <el-main>
    <!-- 搜索栏 -->
    <el-form :model="listParm" :inline="true" size="default">
      <el-form-item>
        <el-input v-model="listParm.name" placeholder="请输入姓名">
      </el-input>
      </el-form-item>
      <el-form-item>
        <el-input v-model="listParm.phone" placeholder="请输入电话">
      </el-input>
      </el-form-item>
      <el-form-item>
        <el-input
          v-model="listParm.username"
          placeholder="请输入会员卡号"
        >
      </el-input>
      </el-form-item>
      <el-form-item>
        <el-button :icon="Search" @click="searchBtn">搜索</el-button>
        <el-button type="danger" plain :icon="Close"
          @click="resetBtn">重置</el-button>
        <el-button :icon="Plus" type="primary" @click="addBtn"
          >新增</el-button>
      >
      </el-form-item>
    </el-form>
    <!-- 表格 -->

```

```

<el-table :height="tableHeight" :data="tableList.list" border
stripe>
  <el-table-column prop="username" label="会员卡号"></el-table-
column>
  <el-table-column prop="name" label="姓名"></el-table-column>
  <el-table-column prop="sex" label="性别">
    <template #default="scope">
      <el-tag v-if="scope.row.sex == '0'" type="success"
size="default">男</el-tag>
      <el-tag v-if="scope.row.sex == '1'" type="danger"
size="default">女</el-tag>
    </template>
  </el-table-column>
  <el-table-column prop="age" label="年龄"></el-table-column>
  <el-table-column prop="birthday" label="生日"></el-table-
column>
  <el-table-column prop="joinTime" label="加入时间"></el-table-
column>
  <el-table-column prop="height" label="身高"></el-table-
column>
  <el-table-column prop="weight" label="体重"></el-table-
column>
  <el-table-column prop="waist" label="腰围"></el-table-
column>
  <el-table-column prop="status" label="状态">
    <template #default="scope">
      <el-tag v-if="scope.row.status == '1'" type="success"
size="default">启用</el-tag>
      <el-tag v-if="scope.row.status == '0'" type="danger"
size="default">停用</el-tag>
    </template>
  </el-table-column>
  <el-table-column label="操作" width="220" align="center">
    <template #default="scope">
      <el-button type="primary" :icon="Edit" size="default"
@click="editBtn(scope.row)">编辑</el-button>
      <el-button type="danger" :icon="Delete" size="default"
@click="deleteBtn(scope.row)">删除</el-button>
    </template>
  </el-table-column>
</el-table>
<!-- 分页 -->
<el-pagination
  @size-change="sizeChange"
  @current-change="currentChange"

```

```

        :current-page.sync="listParm.currentPage"
        :page-sizes="[10,20, 40, 80, 100]"
        :page-size="listParm.pageSize"
        layout="total, sizes, prev, pager, next, jumper"
        :total="listParm.total" background>
    </el-pagination>

    <!-- 新增、编辑 -->
    <AddMember ref="addRef" @refresh="refresh"></AddMember>
</el-main>
</template>

<script setup lang="ts">
import AddMember from "../AddMember.vue";
import { Plus, Edit, Delete, Search, Close } from "@element-plus/icons-vue";
import useTable from "@/composables/member/useTable";
import useMember from "@/composables/member/useMember";
//表格相关
const { listParm, getList, resetBtn,
searchBtn,tableList,sizeChange
,currentChange,tableHeight,refresh} = useTable();
//新增、编辑操作
const { addBtn, editBtn, deleteBtn,addRef } = useMember(getList);
</script>

<style scoped></style>

```

9.4 会员角色设置

9.4.1 修改后台代码

9.4.1.1 修改实体类Member

```

@Data
@Table("member")
public class Member {
    no usages
    @TableId(type = IdType.AUTO)
    private Long memberId;
    //排除roleId字段
    no usages
    @TableField(exist = false)
    private Long roleId;
    no usages
    private String name;
    no usages
    private String sex;
}

```

9.4.1.2 修改MemberService接口

```
package com.xq.web.member.service;

import com.baomidou.mybatisplus.extension.service.IService;
import com.xq.web.member.entity.Member;

public interface MemberService extends IService<Member> {

    void addMember(Member member);
    void editMember(Member member);
    void deleteMember(Long memberId);
}
```

修改实现类:

```
package com.xq.web.member.service.impl;

import
com.baomidou.mybatisplus.core.conditions.query.QueryWrapper;
import
com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
import com.xq.web.member.entity.Member;
import com.xq.web.member.mapper.MemberMapper;
import com.xq.web.member.service.MemberService;
import com.xq.web.member_role.entity.MemberRole;
import com.xq.web.member_role.service.MemberRoleService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class MemberServiceImpl extends ServiceImpl<MemberMapper,
Member> implements MemberService {

    @Autowired
    private MemberRoleService memberRoleService;

    @Override
    @Transactional
    public void addMember(Member member) {
        //新增会员
        int insert = this.baseMapper.insert(member);
        //设置会员角色
        if(insert >0){
```



```

        MemberRole role = new MemberRole();
        role.setMemberId(member.getMemberId());
        role.setRoleId(member.getRoleId());
        memberRoleService.save(role);
    }
}

@Override
@Transactional
public void editMember(Member member) {
    int i = this.baseMapper.updateById(member);
    //设置角色 先删除，再插入
    if(i>0){
        //删除
        QueryWrapper<MemberRole> query = new QueryWrapper<>
();

        query.lambda().eq(MemberRole::getMemberId,member.getMemberId());
        memberRoleService.remove(query);
        //重新插入
        MemberRole role = new MemberRole();
        role.setMemberId(member.getMemberId());
        role.setRoleId(member.getRoleId());
        memberRoleService.save(role);
    }
}

@Override
@Transactional
public void deleteMember(Long memberId) {
    //删除
    int i = this.baseMapper.deleteById(memberId);
    if(i > 0){
        QueryWrapper<MemberRole> query = new QueryWrapper<>
();

        query.lambda().eq(MemberRole::getMemberId,memberId);
        memberRoleService.remove(query);
    }
}
}

```

9.4.1.3 修改MemberController

```
import
com.baomidou.mybatisplus.core.conditions.query.QueryWrapper;
import com.baomidou.mybatisplus.core.metadata.IPage;
import
com.baomidou.mybatisplus.extension.plugins.pagination.Page;
import com.xq.utils.ResultUtils;
import com.xq.utils.ResultVo;
import com.xq.web.member.entity.Member;
import com.xq.web.member.entity.PageParm;
import com.xq.web.member.service.MemberService;
import com.xq.web.member_role.entity.MemberRole;
import com.xq.web.member_role.service.MemberRoleService;
import org.apache.commons.lang.StringUtils;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/member")
public class MemberController {
    @Autowired
    private MemberService memberService;
    @Autowired
    private MemberRoleService memberRoleService;

    //新增
    @PostMapping
    public ResultVo add(@RequestBody Member member){
        //判断卡号是否重复
        QueryWrapper<Member> query = new QueryWrapper<>();

        query.lambda().eq(Member::getUsername,member.getUsername());
        Member one = memberService.getOne(query);
        if(one != null){
            return ResultUtils.error("会员卡号被占用!");
        }
        memberService.addMember(member);
        return ResultUtils.success("新增成功!");
    }

    //编辑
    @PutMapping
    public ResultVo edit(@RequestBody Member member){
        //判断卡号是否重复
```

```

        QueryWrapper<Member> query = new QueryWrapper<>();

        query.lambda().eq(Member::getUsername, member.getUsername());
        Member one = memberService.getOne(query);
        if(one != null &&
!one.getMemberId().equals(member.getMemberId())){
            return ResultUtils.error("会员卡号被占用!");
        }
        memberService.editMember(member);
        return ResultUtils.success("编辑成功!");
    }

    //删除
    @DeleteMapping("/{memberId}")
    public ResultVo delete(@PathVariable("memberId") Long
memberId){
        memberService.deleteMember(memberId);
        return ResultUtils.success("删除成功!");
    }

    //查询
    @GetMapping("/list")
    public ResultVo list(PageParm pageParm){
        //构造分页对象
        IPage<Member> page = new Page<>
(pageParm.getCurrentPage(), pageParm.getPageSize());
        //构造查询条件
        QueryWrapper<Member> query = new QueryWrapper<>();
        if(StringUtils.isEmpty(pageParm.getName())){

            query.lambda().like(Member::getName, pageParm.getName());
        }
        if(StringUtils.isEmpty(pageParm.getPhone())){

            query.lambda().like(Member::getPhone, pageParm.getPhone());
        }
        if(StringUtils.isEmpty(pageParm.getUsername())){

            query.lambda().like(Member::getUsername, pageParm.getUsername());
        }
        query.lambda().orderByDesc(Member::getJoinTime);
        IPage<Member> list = memberService.page(page, query);
        return ResultUtils.success("查询成功", list);
    }

    //根据会员id查询角色id

```

```

    @GetMapping("/getRoleByMemberId")
    public ResultVo getRoleByMemberId(Long memberId){
        QueryWrapper<MemberRole> query = new QueryWrapper<>();
        query.lambda().eq(MemberRole::getMemberId, memberId);
        MemberRole one = memberRoleService.getOne(query);
        return ResultUtils.success("查询成功", one);
    }
}

```

9.4.2 修改前端代码

9.4.2.1 api/member/index.ts添加getRoleByMemberIdApi()方法

```

//根据会员id查询角色id
export const getRoleByMemberIdApi = (memberId:string)=>{
    return http.get("/api/member/getRoleByMemberId",
        {memberId:memberId})
}

```

9.4.2.2 修改api/member/MemberModel.ts

```

//会员数据类型
export type MemberType = {
    type:string,
    memberId:string,
    name:string,
    sex:string,
    phone:string,
    age:string,
    birthday:string,
    height:string,
    weight:string,
    waist:string,
    joinTime:string,
    endTime:string,
    username:string,
    password:string,
    status:string,
    roleId:string
}

```

9.4.2.3 composables/user下的useSelectRole修改为如下

```

import { SelectRole } from "@api/user/UserModel";
import { reactive, ref } from "vue";
import { getSelectApi, getRoleApi } from "@api/user";
import { getRoleByMemberIdApi } from '@api/member/index';

export default function useSelectRole() {
    const roleId = ref('')
    const roleMemberId = ref('')
    //定义角色列表数据
    const roleData = reactive<SelectRole>({
        list: []
    })
}

```

```

    })
    //获取数据
    const listRole = async()=>{
        let res = await getSelectApi()
        if(res && res.code == 200){
            roleData.list = res.data
        }
    }
    //根据用户id查询角色
    const getRole = async(userId:string)=>{
        roleId.value = ''
        let res = await getRoleApi(userId)
        if(res && res.code == 200 && res.data){
            roleId.value = res.data.roleId
        }
    }

    //根据用户id查询角色
    const getMemberRole = async (memberId: string) => {
        roleMemberId.value = ''
        let res = await getRoleByMemberIdApi(memberId)
        if (res && res.code == 200 && res.data) {
            roleMemberId.value = res.data.roleId
        }
    }
    return{
        roleData,
        listRole,
        getRole,
        roleId,
        getMemberRole,
        roleMemberId
    }
}

```

9.4.2.4 修改AddMember.vue组件

```

<template>
  <SysDialog
    :title="dialog.title"
    :width="dialog.width"
    :height="dialog.height"
    :visible="dialog.visible"
    @onClose="onClose"
    @onConfirm="commit"
  >

```

```
>
<template v-slot:content>
  <el-form
    :model="addModel"
    ref="addFormRef"
    :rules="rules"
    label-width="80px"
    size="default"
  >
    <el-row>
      <el-col :span="12" :offset="0">
        <el-form-item prop="name" label="姓名">
          <el-input v-model="addModel.name"></el-input>
        </el-form-item>
      </el-col>
      <el-col :span="12" :offset="0">
        <el-form-item prop="sex" label="性别">
          <el-radio-group v-model="addModel.sex">
            <el-radio :label="'0'">男</el-radio>
            <el-radio :label="'1'">女</el-radio>
          </el-radio-group>
        </el-form-item>
      </el-col>
    </el-row>
    <el-row>
      <el-col :span="12" :offset="0">
        <el-form-item prop="phone" label="电话">
          <el-input v-model="addModel.phone"></el-input>
        </el-form-item>
      </el-col>
      <el-col :span="12" :offset="0">
        <el-form-item prop="age" label="年龄">
          <el-input v-model="addModel.age"></el-input>
        </el-form-item>
      </el-col>
    </el-row>
    <el-row>
      <el-col :span="12" :offset="0">
        <el-form-item prop="birthday" label="生日">
          <el-date-picker
            v-model="addModel.birthday"
            type="date"
            placeholder="请选择生日"
            size="default"
            value-format="YYYY-MM-DD"
          >

```

```
        format="YYYY-MM-DD"
      />
    </el-form-item>
  </el-col>
  <el-col :span="12" :offset="0">
    <el-form-item prop="joinTime" label="加入时间">
      <el-date-picker
        v-model="addModel.joinTime"
        type="date"
        placeholder="请选择加入时间"
        size="default"
        format="YYYY-MM-DD"
        value-format="YYYY-MM-DD"
      />
    </el-form-item>
  </el-col>
</el-row>
<el-row>
  <el-col :span="12" :offset="0">
    <el-form-item prop="height" label="身高">
      <el-input v-model="addModel.height"></el-input>
    </el-form-item>
  </el-col>
  <el-col :span="12" :offset="0">
    <el-form-item prop="weight" label="体重">
      <el-input v-model="addModel.weight"></el-input>
    </el-form-item>
  </el-col>
</el-row>
<el-row>
  <el-col :span="12" :offset="0">
    <el-form-item prop="waist" label="腰围">
      <el-input v-model="addModel.waist"></el-input>
    </el-form-item>
  </el-col>
  <el-col :span="12" :offset="0">
    <el-form-item prop="roleId" label="角色">
      <el-select
        v-model="addModel.roleId"
        class="m-2"
        placeholder="请选择角色"
        size="default"
      >
        <el-option
          v-for="item in roleData.list"

```

```

        :key="item.value"
        :label="item.label"
        :value="item.value"
      />
    </el-select>
  </el-form-item>
</el-col>
</el-row>
<el-row>
  <el-col :span="12" :offset="0">
    <el-form-item prop="status" label="状态">
      <el-radio-group v-model="addModel.status">
        <el-radio :label="'0'">停用</el-radio>
        <el-radio :label="'1'">启用</el-radio>
      </el-radio-group>
    </el-form-item>
  </el-col>
  <el-col :span="12" :offset="0">
    <el-form-item prop="username" label="会员卡号">
      <el-input type="number" v-
model="addModel.username"></el-input>
    </el-form-item>
  </el-col>
</el-row>
<el-row>
  <el-col :span="12" :offset="0">
    <el-form-item prop="password" label="密码">
      <el-input v-model="addModel.password"></el-input>
    </el-form-item>
  </el-col>
</el-row>
</el-form>
</template>
</SysDialog>
</template>

```

```

<script setup lang="ts">
import SysDialog from "@/components/SysDialog.vue";
import useDialog from "@/hooks/useDialog";
import { nextTick, reactive, ref } from "vue";
import { MemberType } from "@api/member/MemberModel";
import { ElMessage, FormInstance } from "element-plus";
import { addApi, editApi } from "@api/member/index";
import { EditType, Title } from "@type/BaseEnum";
import useInstance from "@/hooks/useInstance";

```



```

import useSelectRole from "@/composables/user/useSelectRole";
const { global } = useInstance();
const addRormRef = ref<FormInstance>();
//角色
const { roleData, listRole, roleMemberId, getMemberRole } =
useSelectRole();
//弹框属性
const { dialog, onClose, onConfirm, onShow } = useDialog();
//弹框显示
const show = async(type: string, row?: MemberType) => {
  await listRole()
  await getMemberRole(row!?.memberId)
  dialog.width = 680;
  dialog.height = 350;
  type == EditType.ADD
    ? (dialog.title = Title.ADD)
    : (dialog.title = Title.EDIT);
  if (EditType.EDIT == type) {
    nextTick(() => {
      global.$objCoppay(row, addModel);
      addModel.roleId = roleMemberId.value
    });
  }
  addModel.type = type;
  onShow();
  addRormRef.value?.resetFields()
};
//暴露出去，给父组件调用
defineExpose({
  show,
});
//表单绑定的数据对象
const addModel = reactive<MemberType>({
  type: "",
  memberId: "",
  name: "",
  sex: "",
  phone: "",
  age: "",
  birthday: "",
  height: "",
  weight: "",
  waist: "",
  joinTime: "",
  endTime: "",

```

```
username: "",
password: "",
status: "",
roleId: ''
});
//表单验证规则
const rules = reactive({
  name: [
    {
      required: true,
      trigger: "change",
      message: "请填写姓名",
    },
  ],
  sex: [
    {
      required: true,
      trigger: "change",
      message: "请选择性别",
    },
  ],
  phone: [
    {
      required: true,
      trigger: "change",
      message: "请填写电话",
    },
  ],
  joinTime: [
    {
      required: true,
      trigger: "change",
      message: "请填写加入时间",
    },
  ],
  username: [
    {
      required: true,
      trigger: "change",
      message: "请填写会员卡号",
    },
  ],
  password: [
    {
      required: true,
```

```

        trigger: "change",
        message: "请填写密码",
    },
],
status: [
    {
        required: true,
        trigger: "change",
        message: "请选择状态",
    },
],
});
//注册事件
const emits = defineEmits(["refresh"]);
//表单提交
const commit = () => {
    addRormRef.value?.validate(async (valid) => {
        if (valid) {
            let res = null;
            if(addModel.type == EditType.ADD){
                res = await addApi(addModel);
            }else{
                res = await editApi(addModel)
            }
            if (res && res.code == 200) {
                ElMessage.success(res.msg);
                emits("refresh");
                onClose();
            }
        }
    });
};
</script>

<style scoped></style>

```

10 会员充值、办卡功能开发

10.1 数据表设计

- 修改member表字段

```

drop table if exists member;
create table member

```

```
(
    member_id          int not null auto_increment comment '会员
id',
    name               varchar(64) comment '姓名',
    sex                varchar(2) comment '性别',
    phone              varchar(15) comment '电话',
    age                int comment '年龄',
    birthday            varchar(36) comment '生日',
    height              int comment '身高',
    weight              int comment '体重',
    waist              int comment '腰围',
    join_time           varchar(36) comment '加入时间',
    end_time            varchar(36) comment '到期时间',
    username            varchar(64) comment '会员卡号',
    password            varchar(128) comment '密码',
    status              varchar(2) comment '状态 1: 启用 0: 停用',
    money               decimal(18,2) default 0 comment '充值金额',
    card_type           varchar(64) comment '会员类型',
    card_day            int comment '天数',
    price               decimal(18,2) default 0 comment '会员价格',
    primary key (member_id)
);
```

对应实体类也要做修改：

```
package com.xq.web.member.entity;

import com.baomidou.mybatisplus.annotation.IdType;
import com.baomidou.mybatisplus.annotation.TableField;
import com.baomidou.mybatisplus.annotation.TableId;
import com.baomidou.mybatisplus.annotation.TableName;
import lombok.Data;

import java.math.BigDecimal;

@Data
@TableName("member")
public class Member {
    @TableId(type = IdType.AUTO)
    private Long memberId;
    //排除roleId字段
    @TableField(exist = false)
    private Long roleId;
    private String name;
    private String sex;
```

```

private String phone;
private Integer age;
private String birthday;
private Integer height;
private Integer weight;
private Integer waist;
private String joinTime;
private String endTime;
private String username;
private String password;
private String status;

private String cardType;
private Integer cardDay;
private BigDecimal money;
private BigDecimal price;
}

```

- 办卡表(member_apply)

```

create table member_apply
(
    apply_id          int not null auto_increment comment '主
键',
    member_id         int comment '会员id',
    card_type         varchar(64) comment '会员类型',
    card_day          int comment '天数',
    price             decimal(18,2) default 0 comment '价格',
    create_time       datetime comment '创建时间',
    create_user       varchar(64) comment '创建人',
    primary key (apply_id)
);

```

- 充值表(member_recharge)

```

create table member_recharge
(
    recharge_id          int not null auto_increment comment '充值id',
    member_id            int comment '会员id',
    money                decimal(18,2) default 0 comment '充值金额',
    create_time          datetime comment '创建时间',
    create_user          varchar(64) comment '创建人',
    primary key (recharge_id)
);

```

10.2 会员充值、办卡模块接口开发

10.2.1 member_apply模块

10.2.1.1 新建实体类

在web目录下面创建member_apply/entity目录

```

package com.xq.web.member_apply.entity;

import com.baomidou.mybatisplus.annotation.IdType;
import com.baomidou.mybatisplus.annotation.TableId;
import com.baomidou.mybatisplus.annotation.TableName;
import lombok.Data;

import java.math.BigDecimal;
import java.util.Date;

@Data
@TableName("member_apply")
public class MemberApply {
    @TableId(type = IdType.AUTO)
    private Long applyId;
    private Long memberId;
    private String cardType;
    private Integer cardDay;
    private BigDecimal price;
    private Date createTime;
    private String createUser;
}

```

10.2.1.2 创建mapper接口

```
package com.xq.web.member_apply.mapper;

import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.xq.web.member_apply.entity.MemberApply;

public interface MemberApplyMapper extends
BaseMapper<MemberApply> {
}
```

10.2.1.3 创建mapper映射文件

```
<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper
    namespace="com.xq.web.member_apply.mapper.MemberApplyMapper">

</mapper>
```

10.2.1.4 创建service接口

```
package com.xq.web.member_apply.service;

import com.baomidou.mybatisplus.extension.service.IService;
import com.xq.web.member_apply.entity.MemberApply;

public interface MemberApplyService extends IService<MemberApply>
{
}
```

创建接口实现类：

```

package com.xq.web.member_apply.service;

import com.baomidou.mybatisplus.extension.service.IService;
import com.xq.web.member_apply.entity.MemberApply;

@Service
public class MemberApplyServiceImpl extends
ServiceImp<MemberApplyMapper, MemberApply> implements
MemberApplyService {
}

```

10.2.2 member_recharge模块

10.2.2.1 创建实体类

```

package com.xq.web.member_recharge.entity;

import com.baomidou.mybatisplus.annotation.IdType;
import com.baomidou.mybatisplus.annotation.TableId;
import com.baomidou.mybatisplus.annotation.TableName;
import lombok.Data;

import java.math.BigDecimal;
import java.util.Date;

@Data
@TableName("member_recharge")
public class MemberRecharge {
    @TableId(type = IdType.AUTO)
    private Long rechargeId;
    private Long memberId;
    private BigDecimal money;
    private Date createTime;
    private String createUser;
}

```

10.2.2.2 创建mapper接口


```
package com.xq.web.member_recharge.mapper;

import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.xq.web.member_recharge.entity.MemberRecharge;

public interface MemberRechargeMapper extends
BaseMapper<MemberRecharge> {
}
```

10.2.2.3 创建mapper映射文件

```
<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper
    namespace="com.xq.web.member_recharge.mapper.MemberRechargeMapper"
">

</mapper>
```

10.2.2.4 创建service接口

```
package com.xq.web.member_recharge.service;

import com.baomidou.mybatisplus.extension.service.IService;
import com.xq.web.member_recharge.entity.MemberRecharge;

public interface MemberRechargeService extends
IService<MemberRecharge> {
}
```

业务接口实现类：

```

package com.xq.web.member_recharge.service.impl;

import
com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
import com.xq.web.member_recharge.entity.MemberRecharge;
import com.xq.web.member_recharge.mapper.MemberRechargeMapper;
import com.xq.web.member_recharge.service.MemberRechargeService;
import org.springframework.stereotype.Service;

@Service
public class MemberRechargeServiceImpl extends
ServiceImpl<MemberRechargeMapper, MemberRecharge> implements
MemberRechargeService {
}

```

10.2.3 修改member模块

10.2.3.1 member模块entity添加如下两个类

```

package com.xq.web.member.entity;

import lombok.Data;

import java.math.BigDecimal;

@Data
public class RechargeParm {
    private Long userId;
    private Long memberId;
    private BigDecimal money;
}

```

```

package com.xq.web.member.entity;

import lombok.Data;

@Data
public class JoinParm {
    private Long memberId;
    private Long cardId;
}

```

10.2.3.2 MemberMapper添加addMoney()方法

```
public interface MemberMapper extends BaseMapper<Member> {  
  
    int addMoney(@Param("parm") RechargeParm parm);  
}
```

修改对应的映射文件:

```
<mapper namespace="com.xq.web.member.mapper.MemberMapper">  
    <update id="addMoney">  
        update member set money = money + #{parm.money} where  
            member_id =#{parm.memberId}  
    </update>  
</mapper>
```

10.2.3.3 MemberService接口添加如下方法

```
package com.xq.web.member.service;  
  
import com.baomidou.mybatisplus.extension.service.IService;  
import com.xq.web.member.entity.JoinParm;  
import com.xq.web.member.entity.Member;  
import com.xq.web.member.entity.RechargeParm;  
  
import java.text.ParseException;  
  
public interface MemberService extends IService<Member> {  
  
    void joinApply(JoinParm joinParm) throws ParseException;  
  
    void recharge(RechargeParm parm);  
}
```

修改对应的实体类:

```
@Service  
public class MemberServiceImpl extends ServiceImpl<MemberMapper,  
Member> implements MemberService {  
  
    @Resource  
    private MemberCardMapper memberCardMapper;  
  
    @Resource  
    private MemberApplyMapper memberApplyMapper;
```

```

@Autowired
MemberRechargeService memberRechargeService;

@Override
@Transactional
public void joinApply(JoinParm joinParm) throws
ParseException {
    Member select =
this.baseMapper.selectById(joinParm.getMemberId());
    //更新会员
    MemberCard card =
memberCardMapper.selectById(joinParm.getCardId());
    Member member = new Member();
    member.setMemberId((joinParm.getMemberId()));
    member.setCardType(card.getTitle());
    member.setCardDay(card.getCardDay());
    member.setPrice(card.getPrice());
    String endTime = select.getEndTime();
    Calendar calendar = Calendar.getInstance();
    if(StringUtils.isEmpty(endTime)){
        Date date = new SimpleDateFormat("yyyy-MM-
dd").parse(select.getEndTime());
        calendar.setTime(date);
        calendar.add(Calendar.DATE, card.getCardDay());
    }else{
        Date data = new Date();
        calendar.setTime(data);
        calendar.add(Calendar.DATE, card.getCardDay());
    }
    member.setEndTime(new SimpleDateFormat("yyyy-MM-
dd").format(calendar.getTime()));
    this.baseMapper.updateById(member);
    //插入明细
    MemberApply memberApply = new MemberApply();
    memberApply.setCardDay(card.getCardDay());
    memberApply.setCardType(card.getTitle());
    memberApply.setMemberId(joinParm.getMemberId());
    memberApply.setPrice(card.getPrice());
    memberApply.setCreateTime(new Date());
    memberApplyMapper.insert(memberApply);
}

@Override
@Transactional

```

```

    public void recharge(RechargeParm parm) {
        //生成充值明细
        MemberRecharge recharge = new MemberRecharge();
        recharge.setMemberId(parm.getMemberId());
        recharge.setMoney(parm.getMoney());
        boolean save = memberRechargeService.save(recharge);
        if(save){
            //更新会员账号的金额
            this.baseMapper.addMoney(parm);
        }
    }
}

```

10.2.3.4 MemberController添加getCardList()、joinApply()、recharge()方法

```

@RestController
@RequestMapping("/api/member")
public class MemberController {
    @Autowired
    private MemberService memberService;

    @Autowired
    MemberCardService memberCardService;

    //查询会员卡列表
    @GetMapping("/getCardList")
    public ResultVo getCardList(){
        QueryWrapper<MemberCard> query = new QueryWrapper<>();
        query.lambda().eq(MemberCard::getStatus,"1");
        List<MemberCard> list = memberCardService.list(query);
        return ResultUtils.success("查询成功",list);
    }

    //办卡提交
    @PostMapping("/joinApply")
    public ResultVo joinApply(@RequestBody JoinParm joinParm)
    throws ParseException {
        memberService.joinApply(joinParm);
        return ResultUtils.success("办卡成功!");
    }

    //充值
    @PostMapping("/recharge")
    public ResultVo recharge(@RequestBody RechargeParm parm){
        memberService.recharge(parm);
        return ResultUtils.success("充值成功!");
    }
}

```

```
}  
}
```

10.2.4 办卡与充值前端对接

10.2.4.1 api/member/MemberModel.ts添加卡类型和充值类型

```
//办卡数据类型  
export type ApplyCard = {  
  cardId:string,  
  memberId:string  
}  
  
//充值数据类型  
export type Recharge = {  
  memberId:string,  
  money:number,  
  userId:string  
}
```

10.2.4.2 api/member/index.ts添加办卡和充值的请求接口

```
import http from "@/http"  
import { MemberParam, MemberType, ApplyCard, Recharge } from  
"./MemberModel"  
//办卡  
export const applySaveApi = (parm:ApplyCard)=>{  
  return http.post("/api/member/joinApply",parm)  
}  
//充值  
export const rechargeApi = (parm:Recharge)=>{  
  return http.post("/api/member/recharge",parm)  
}
```

10.2.4.3 composables/member新建useJoin.ts

```
import { MemberType } from "@/api/member/MemberModel";  
import { onMounted, reactive, ref } from "vue";  
import { getCardListApi } from "@/api/member";  
import { CardType } from '@/api/member_card/MemberModel'  
export default function useJoin(){  
  //会员卡列表数据  
  const cardList = ref<CardType[]>([])  
  //弹框属性  
  const joinRef = ref<{show:(row:MemberType)=>void}>()  
  //办卡按钮
```

```

const joinBtn = (row:MemberType)=>{
  joinRef.value?.show(row)
}
//获取会员卡列表
const getCardList = async()=>{
  let res = await getCardListApi()
  if(res && res.code == 200){
    cardList.value = res.data
  }
}
onMounted(()=>{
  getCardList()
})
return{
  joinRef,
  joinBtn,
  cardList
}
}

```

10.2.4.4 composables/member新建useRecharge.ts

```

import { MemberType } from "@api/member/MemberModel";
import { ref } from "vue";

export default function useRecharge(){
  //ref属性
  const rechargeRef = ref<{show:(row:MemberType)=>void}>()
  //充值按钮
  const rechargeBtn = (row:MemberType)=>{
    rechargeRef.value?.show(row)
  }
  return{
    rechargeRef,
    rechargeBtn
  }
}

```

10.2.4.5 新建JoinApply.vue组件

```

<template>
  <SysDialog
    :title="dialog.title"
    :height="dialog.height"
    :width="dialog.width"

```

```

        :visible="dialog.visible"
        @onClose="onClose"
        @onConfirm="commit"
    >
    <template v-slot:content>
        <div>请选择会员卡类型</div>
        <el-divider style="margin: 10px 0px" />
        <el-select
            style="margin-top: 10px; width: 100%"
            v-model="parm.cardId"
            class="m-2"
            placeholder="请选择会员卡类型"
            size="large"
        >
            <el-option
                v-for="item in cardList"
                :key="item.cardId"
                :label="item.title"
                :value="item.cardId"
            />
        </el-select>
    </template>
</SysDialog>
</template>

<script setup lang="ts">
    import { MemberType, ApplyCard } from
"@/api/member/MemberModel";
    import SysDialog from "@/components/SysDialog.vue";
    import useDialog from "@/hooks/useDialog";
    import useJoin from "@/composables/member/useJoin";
    import { reactive } from "vue";
    import { ElMessage } from "element-plus";
    import { applySaveApi } from "@/api/member/index";
    //选择的卡类型
    const parm = reactive<ApplyCard>({
        cardId: "",
        memberId: "",
    });
    //弹框属性
    const { dialog, onClose, onConfirm, onShow } = useDialog();
    //会员卡数据
    const { cardList } = useJoin();
    //显示弹框
    const show = (row: MemberType) => {

```



```

        parm.memberId = row.memberId;
        dialog.title = "为【" + row.name + "】办卡";
        onShow();
    };
    //暴露给父组件调用
    defineExpose({
        show,
    });
    const eimits = defineEmits(['refresh'])
    //提交办卡
    const commit = async () => {
        if (!parm.cardId) {
            ElMessage.warning("请选择会员卡类型!");
            return;
        }
        let res = await applySaveApi(parm);
        if (res && res.code == 200) {
            ElMessage.success(res.msg);
            eimits('refresh')
            onClose()
        }
    };
</script>
<style scoped></style>

```

10.2.4.6 新建Recharge.vue组件

```

<template>
  <SysDialog
    :title="dialog.title"
    :height="dialog.height"
    :width="dialog.width"
    :visible="dialog.visible"
    @onClose="onClose"
    @onConfirm="commit"
  >
    <template v-slot:content>
      <el-form
        :model="addModel"
        ref="addFormRef"
        :rules="rules"
        label-width="80px"
        size="default"
      >
        <el-form-item prop="money" label="充值金额">

```

```

        <el-input type="number" v-model="addModel.money">
    </el-input>
    </el-form-item>
  </el-form>
</template>
</SysDialog>
</template>

<script setup lang="ts">
  import { MemberType, Recharge } from
"@/api/member/MemberModel";
  import SysDialog from "@/components/SysDialog.vue";
  import useDialog from "@/hooks/useDialog";
  import { ElMessage, FormInstance } from "element-plus";
  import { reactive, ref } from "vue";
  import { rechargeApi } from "@/api/member/index";
  const addFormRef = ref<FormInstance>();
  //表单绑定值
  const addModel = reactive<Recharge>({
    userId: "",
    memberId: "",
    money: 0,
  });
  const validMoney = (rule: any, value: any, callback: any) =>
{
    if (value <= 0) {
      callback(new Error("充值金额不能小于0"));
    } else {
      callback();
    }
  };
  //表单验证规则
  const rules = reactive({
    money: [
      {
        required: true,
        trigger: "change",
        validator: validMoney,
      },
    ],
  });
  //弹框属性
  const { dialog, onClose, onConfirm, onShow } = useDialog();
  //显示弹框
  const show = (row: MemberType) => {

```

```

        dialog.title = "为【" + row.name + "】充值";
        addModel.memberId = row.memberId
        onShow();
        addFormRef.value?.resetFields();
    };
    //暴露出去
    defineExpose({
        show,
    });
    //注册事件
    const emits = defineEmits(['refresh'])
    //提交表单
    const commit = () => {
        addFormRef.value?.validate(async (valid) => {
            if (valid) {
                let res = await rechargeApi(addModel);
                if (res && res.code == 200) {
                    ElMessage.success(res.msg)
                    emits('refresh')
                    onClose();
                }
            }
        });
    };
</script>

<style scoped></style>

```

10.2.4.7 修改MemberList.vue组件

```

<template>
  <el-main>
    <!--搜索栏 -->
    <el-form :model="listParam" :inline="true" size="default">
      <el-form-item>
        <el-input v-model="listParam.name" placeholder="请输入姓名"></el-input>
      </el-form-item>
      <el-form-item>
        <el-input v-model="listParam.phone" placeholder="请输入电话"></el-input>
      </el-form-item>
      <el-form-item>
        <el-input
          v-model="listParam.username"

```

```

        placeholder="请输入会员卡号"
      ></el-input>
    </el-form-item>
    <el-form-item>
      <el-button :icon="Search" @click="searchBtn">搜索</el-
button>
      <el-button type="danger" plain :icon="Close"
@click="resetBtn"
      >重置</el-button>
    >
    <el-button :icon="Plus" type="primary" @click="addBtn">新
增</el-button>
  </el-form-item>
</el-form>
<!-- 表格 -->
<el-table :height="tableHeight" :data="tableList.list" border
stripe>
  <el-table-column
    prop="username"
    width="150"
    label="会员卡号"
  ></el-table-column>
  <el-table-column prop="name" label="姓名"></el-table-column>
  <el-table-column prop="cardType" label="会员类型"></el-table-
column>
  <el-table-column prop="price" label="会员价格"></el-table-
column>
  <el-table-column prop="money" label="充值金额"></el-table-
column>
  <el-table-column
    prop="endTime"
    width="150"
    label="到期时间"
  ></el-table-column>
  <el-table-column prop="sex" label="性别">
    <template #default="scope">
      <el-tag v-if="scope.row.sex == '0'" type="success"
size="default"
      >男</el-tag>
    >
      <el-tag v-if="scope.row.sex == '1'" type="danger"
size="default"
      >女</el-tag>
    >
  </template>

```

```

</el-table-column>
<el-table-column prop="age" label="年龄"></el-table-column>
<el-table-column
  prop="birthday"
  width="150"
  label="生日"
></el-table-column>
<el-table-column
  prop="joinTime"
  width="150"
  label="加入时间"
></el-table-column>
<el-table-column prop="height" label="身高"></el-table-
column>
<el-table-column prop="weight" label="体重"></el-table-
column>
<el-table-column prop="waist" label="腰围"></el-table-
column>
<el-table-column prop="status" label="状态">
  <template #default="scope">
    <el-tag v-if="scope.row.status == '1'" type="success"
size="default">
      >启用</el-tag>
    >
    <el-tag v-if="scope.row.status == '0'" type="danger"
size="default">
      >停用</el-tag>
    >
  </template>
</el-table-column>
<el-table-column label="操作" width="220" fixed="right"
align="center">
  <template #default="scope">
    <el-button
      type="success"
      :icon="Edit"
      size="default"
      @click="joinBtn(scope.row)"
    >办卡</el-button>
    >
    <el-button type="primary">
      <el-dropdown>
        <span class="el-dropdown-link" style="color:#FFF">
          更多
        <el-icon class="el-icon--right">

```

```

        <arrow-down />
      </el-icon>
    </span>
    <template #dropdown>
      <el-dropdown-menu>
        <el-dropdown-item :icon="ChatLineSquare"
@click="rechargeBtn(scope.row)">充值</el-dropdown-item>
        <el-dropdown-item :icon="Edit"
@click="editBtn(scope.row)">编辑</el-dropdown-item>
        <el-dropdown-item type="danger" :icon="Delete"
@click="deleteBtn(scope.row)">删除</el-dropdown-item>
      </el-dropdown-menu>
    </template>
  </el-dropdown>
</el-button>
</template>
</el-table-column>
</el-table>
<!-- 分页 -->
<el-pagination
  @size-change="sizeChange"
  @current-change="currentChange"
  :current-page.sync="listParam.currentPage"
  :page-sizes="[10, 20, 40, 80, 100]"
  :page-size="listParam.pageSize"
  layout="total, sizes, prev, pager, next, jumper"
  :total="listParam.total"
  background
>
</el-pagination>

<!-- 新增、编辑 -->
<AddMember ref="addRef" @refresh="refresh"></AddMember>
<!-- 办卡弹框 -->
<JoinApply ref="joinRef" @refresh="refresh"></JoinApply>
<!-- 充值弹框 -->
<Recharge ref="rechargeRef" @refresh="refresh"></Recharge>
</el-main>
</template>

<script setup lang="ts">
import JoinApply from "./JoinApply.vue";
import AddMember from "./AddMember.vue";
import Recharge from "./Recharge.vue";

```

```
import { Plus, Edit, Delete, Search, Close, ChatLineSquare } from
"@element-plus/icons-vue";
import useTable from "@/composables/member/useTable";
import useMember from "@/composables/member/useMember";
import useJoin from "@/composables/member/useJoin";
import useRecharge from "@/composables/member/useRecharge";
//表格相关
const {
  listParam,
  getList,
  resetBtn,
  searchBtn,
  tableList,
  sizeChange,
  currentChange,
  tableHeight,
  refresh,
} = useTable();
//新增、编辑操作
const { addBtn, editBtn, deleteBtn, addRef } =
useMember(getList);
//办卡
const { joinRef, joinBtn } = useJoin();
//充值
const { rechargeRef, rechargeBtn } = useRecharge();
</script>

<style scoped></style>
```

11 课程管理模块功能开发

11.1 课程表数据表设计

```

create table course
(
    course_id          int not null auto_increment comment '课程id',
    course_name        varchar(128) comment '课程名称',
    image              varchar(255) comment '封面图',
    teacher_name       varchar(64) comment '授课教师',
    course_hour        int comment '课时',
    course_details      text comment '课程详情',
    course_price        decimal(18,2) comment '课程价格',
    primary key (course_id)
);

```

11.2 课程管理模块后台接口开发

11.2.1 创建实体类

创建web.course.entity包，并新建课程表对应的实体类

```

package com.xq.web.course.entity;

import com.baomidou.mybatisplus.annotation.IdType;
import com.baomidou.mybatisplus.annotation.TableId;
import com.baomidou.mybatisplus.annotation.TableName;
import lombok.Data;

import java.math.BigDecimal;

@Data
@TableName("course")
public class Course {
    @TableId(type = IdType.AUTO)
    private Long courseId;
    private String courseName;
    private String image;
    private String teacherName;
    private Integer courseHour;
    private String courseDetails;
    private BigDecimal coursePrice;
}

```

进行分页查询，封装参数的实体类：


```

package com.xq.web.course.entity;

import lombok.Data;

@Data
public class CourseList {
    private Long currentPage;
    private Long pageSize;
    private String courseName;
    private String teacherName;
}

```

11.2.2 新建mapper层

```

package com.xq.web.course.mapper;

import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.xq.web.course.entity.Course;

public interface CourseMapper extends BaseMapper<Course> {
}

```

11.2.3 创建mapper映射文件

```

<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="com.xq.web.course.mapper.CourseMapper">

</mapper>

```

11.2.4 创建service层

```

package com.xq.web.course.service;

import com.baomidou.mybatisplus.extension.service.IService;
import com.xq.web.course.entity.Course;

public interface CourseService extends IService<Course> {
}

```

创建service业务接口实现类：

```

package com.xq.web.course.service.impl;

import
com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
import com.xq.web.course.entity.Course;
import com.xq.web.course.mapper.CourseMapper;
import com.xq.web.course.service.CourseService;
import org.springframework.stereotype.Service;

@Service
public class CourseServiceImpl extends ServiceImpl<CourseMapper,
Course> implements CourseService {
}

```

11.2.5 创建控制器层

```

package com.xq.web.course.controller;

import
com.baomidou.mybatisplus.core.conditions.query.QueryWrapper;
import com.baomidou.mybatisplus.core.metadata.IPage;
import
com.baomidou.mybatisplus.extension.plugins.pagination.Page;
import com.xq.utils.ResultUtils;
import com.xq.utils.ResultVo;
import com.xq.web.course.entity.Course;
import com.xq.web.course.entity.CourseList;
import com.xq.web.course.service.CourseService;
import org.apache.commons.lang.StringUtils;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/course")
public class CourseController {
    @Autowired
    private CourseService courseService;

    //新增
    @PostMapping
    public ResultVo add(@RequestBody Course course){
        if(courseService.save(course)){
            return ResultUtils.success("新增成功!");
        }
    }
}

```

```

        return ResultUtils.error("新增失败!");
    }

    //编辑
    @PutMapping
    public ResultVo edit(@RequestBody Course course){
        if(courseService.updateById(course)){
            return ResultUtils.success("编辑成功!");
        }
        return ResultUtils.error("编辑失败!");
    }

    //删除
    @DeleteMapping("/{courseId}")
    public ResultVo delete(@PathVariable("courseId") Long
courseId){
        if(courseService.removeById((courseId))){
            return ResultUtils.success("删除成功!");
        }
        return ResultUtils.error("删除失败!");
    }

    //列表
    @GetMapping("/list")
    public ResultVo list(CourseList courseList){
        //构造分页对象
        IPage<Course> page = new Page<>
(courseList.getCurrentPage(),courseList.getPageSize());
        //构造查询条件
        QueryWrapper<Course> query = new QueryWrapper<>();
        if(StringUtils.isNotEmpty(courseList.getCourseName())){

            query.lambda().like(Course::getCourseName,courseList.getCourseName());
        }
        if(StringUtils.isNotEmpty(courseList.getTeacherName())){

            query.lambda().like(Course::getTeacherName,courseList.getTeacherName());
        }
        IPage<Course> list = courseService.page(page, query);
        return ResultUtils.success("查询成功",list);
    }
}

```

11.3 新增课程页面制作与数据对接

11.3.1 修改MenuBar.vue组件

在MenuBar.vue里面添加如下：

```
{
  path: "/courseRoot",
  component: "Layout",
  name: "courseRoot",
  meta: {
    title: "课程管理",
    icon: "ScaleToOriginal",
    roles: ["sys:courseRoot"],
  },
  children: [
    {
      path: "/courseList",
      component: "/course/CourseList",
      name: "courseList",
      meta: {
        title: "课程列表",
        icon: "UserFilled",
        roles: ["sys:courseList"],
      },
    },
    {
      path: "/mycourse",
      component: "/mycourse/mycourse",
      name: "mycourse",
      meta: {
        title: "我的课程",
        icon: "wallet",
        roles: ["sys:mycourse"],
      },
    },
  ],
},
```

11.3.2 修改路由代码

修改router目录下面的index.ts文件：

```
{
  path: "/courseRoot",
```

```

    component: Layout,
    name: "courseRoot",
    meta: {
      title: "课程管理",
      icon: "Document",
      roles: ["sys:courseRoot"],
    },
    children: [
      {
        path: "/courseList",
        component: () =>
import('@views/course/CourseList.vue'),
        name: "courseList",
        meta: {
          title: "课程列表",
          icon: "UserFilled",
          roles: ["sys:courseList"],
        },
      },
      {
        path: "/mycourse",
        component: () =>
import('@views/mycourse/mycourse.vue'),
        name: "mycourse",
        meta: {
          title: "我的课程",
          icon: "wallet",
          roles: ["sys:mycourse"],
        },
      }
    ]
  },

```

11.3.3 创建AddCourse.vue组件

在views/course目录下面创建AddCourse.vue组件

```

<template>
  <SysDialog
    :title="dialog.title"
    :height="dialog.height"
    :width="dialog.width"
    :visible="dialog.visible"
    @onClose="onClose"
    @onConfirm="commit"
  >

```

```

>
<template v-slot:content>
  <el-form
    :model="addModel"
    ref="addFormRef"
    :rules="rules"
    label-width="80px"
    size="default"
  >
    <el-row>
      <el-col :span="12" :offset="0">
        <el-form-item prop="courseName" label="课程名称">
          <el-input v-model="addModel.courseName"></el-
input>
          </el-form-item>
        </el-col>
        <el-col :span="12" :offset="0">
          <el-form-item prop="courseHour" label="课程课时">
            <el-input v-model="addModel.courseHour"></el-
input>
            </el-form-item>
          </el-col>
        </el-row>
        <el-row>
          <el-col :span="12" :offset="0">
            <el-form-item prop="coursePrice" label="课程价格">
              <el-input v-model="addModel.coursePrice"></el-
input>
              </el-form-item>
            </el-col>
            <el-col :span="12" :offset="0">
              <el-form-item prop="teacherName" label="教练">
                <el-input v-model="addModel.teacherName"></el-
input>
                </el-form-item>
              </el-col>
            </el-row>
            <el-form-item prop="image" label="课程图片">
              <el-input v-model="addModel.image"></el-input>
            </el-form-item>
            <el-form-item prop="courseDetails" label="课程详情">
              <el-input v-model="addModel.courseDetails"></el-
input>
              </el-form-item>
            </el-form>

```

```

    </template>
  </SysDialog>
</template>

<script setup lang="ts">
  //import "@wangeditor/editor/dist/css/style.css"; // 引入 css
  //import { Editor, Toolbar } from "@wangeditor/editor-for-vue";
  import { CourseType } from "@api/course/CourseModel";
  import SysDialog from "@components/SysDialog.vue";
  import useDialog from "@hooks/useDialog";
  import { ElMessage, FormInstance } from "element-plus";
  import { nextTick, reactive, ref } from "vue";
  // import useEditor from "@composables/course/useEditor";
  // import useSelectTeacher from
"@composables/course/useSelectTeacher";
  import { EditType, Title } from "@type/BaseEnum";
  import { addApi } from "@api/course/index";
  import useInstance from "@hooks/useInstance";
  const { global } = useInstance();
  //表单的ref属性
  const addFormRef = ref<FormInstance>();
  //弹框属性
  const { dialog, onClose, onShow, onConfirm } = useDialog();
  //教练选择
  //const { teacherData, listTeacher } = useSelectTeacher();
  //图片上传

  //文本编辑器
  //const {
    // editorRef,
    // handleCreated,
    // mode,
    // editorConfig,
    // valueHtml,
    // toolbarConfig,
  // } = useEditor();
  //显示弹框
  const show = async (type: string, row?: CourseType) => {
    //清空图片数据
    //fileList.value = [];
    //获取教练数据列表
    //await listTeacher();
    //弹框属性
    type == EditType.ADD
      ? (dialog.title = Title.ADD)

```

```

        : (dialog.title = Title.EDIT);
dialog.width = 900;
dialog.height = 500;
//清空图片和文本编辑器

addModel.image = "";
if (type == EditType.ADD) {
    //const editor = editorRef.value;
    //if (editor) {
    //    editor.clear();
    //}
}
addModel.courseDetails = "";
//编辑数据回显
if (type == EditType.EDIT) {

}
if (row && row.courseDetails) {
    //文本编辑器的回显
    //valueHtml.value = row.courseDetails;
}
onShow();
//表单清空
addFormRef.value?.resetFields();
addModel.type = type;
};
//暴露出去，给外部组件使用
defineExpose({
    show,
});
//表单绑定的数据对象
const addModel = reactive<CourseType>({
    type: "", //区分 新增 编辑
    courseId: "",
    courseName: "",
    image: "",
    teacherName: "",
    //teacherId: "",
    courseHour: 0,
    courseDetails: "",
    coursePrice: 0,
});
const validateCourseHour = (rule: any, value: any, callback:
any) => {
    if (value === 0 || value < 0) {

```



```

        callback(new Error("请填写课程课时"));
    } else {
        callback();
    }
};

const validateCoursePrice = (rule: any, value: any, callback:
any) => {
    if (value === 0 || value < 0) {
        callback(new Error("请填写课程价格"));
    } else {
        callback();
    }
};

// 表单验证规则
const rules = reactive({
    courseName: [
        {
            required: true,
            trigger: "blur",
            message: "请输入课程名称",
        },
    ],
    image: [
        {
            required: true,
            trigger: "blur",
            message: "请上传课程图片",
        },
    ],
    teacherName: [
        {
            required: true,
            trigger: "blur",
            message: "请选择课程教练",
        },
    ],
    courseDetails: [
        {
            required: true,
            trigger: "blur",
            message: "请填写课程详情",
        },
    ],
    coursePrice: [
        {

```

```

        required: true,
        validator: validateCoursePrice,
        trigger: "blur",
    },
],
courseHour: [
    {
        required: true,
        validator: validateCourseHour,
        trigger: "blur",
    },
],
});
//注册事件
const emits = defineEmits(["reFresh"]);
//表单提交
const commit = () => {
    //封面图地址
    //addModel.image = imgUrl.value;
    //课程详情
    //addModel.courseDetails = valueHtml.value;
    addFormRef.value?.validate(async (valid) => {
        if (valid) {
            let res = null;
            if (addModel.type == EditType.ADD) {
                res = await addApi(addModel);
            }
            if (res && res.code == 200) {
                ElMessage.success(res.msg);
                emits("reFresh");
                onClose();
            }
        }
    });
};
// 选择教练的change
const selectTeach = (val:any)=>{
    // console.log(val)
    // addModel.teacherId = val.value;
    //addModel.teacherName = val.label;
}
</script>

<style scoped></style>

```

在views/mycourse目录下面创建mycourse.vue组件：

```
<template>
  <div>我的课程</div>
</template>
<script>
</script>
<style>
</style>
```

11.3.4 创建CourseList.vue组件

```
<template>
  <el-main>
    <!-- 搜索栏 -->
    <el-form :inline="true" size="default">
      <el-form-item>
        <el-input
          placeholder="课程名称"
        ></el-input>
      </el-form-item>
      <el-form-item>
        <el-input placeholder="教练"></el-input>
      </el-form-item>
      <el-form-item>
        <el-button :icon="Search" @click="">搜索</el-button>
        <el-button :icon="Close" @click="">重置</el-button>
        <el-button
          :icon="Plus"
          v-permission="['sys:courseList:add']"
          type="primary"
          @click="addBtn"
        >新增</el-button>
      </el-form-item>
    </el-form>
    <!-- 表格 -->
    <el-table border stripe>
      <el-table-column prop="image" width="90" label="课程封面"
        align="center">
        <template #default="scope">
          <el-image
            style="width: 60px; height: 60px; border-radius:
50%"
```

```

        :src="scope.row.image"
      ></el-image>
    </template>
  </el-table-column>
  <el-table-column prop="courseName" label="课程名称"></el-
table-column>
  <el-table-column prop="courseHour" label="课时"></el-
table-column>
  <el-table-column prop="coursePrice" label="价格"></el-
table-column>
  <el-table-column prop="teacherName" label="授课教师"></el-
table-column>
  <el-table-column label="操作" align="center" width="290">
    <template #default="scope">
      <el-button
        v-permission="['sys:courseList:edit']"
        type="primary"
        :icon="Edit"
        size="default"
      >编辑</el-button>
    >
    <el-button
      v-permission="['sys:courseList:join']"
      type="success"
      :icon="Edit"
      size="default"
    >报名</el-button>
    >
    <el-button
      v-permission="['sys:courseList:delete']"
      type="danger"
      :icon="Delete"
      size="default"
    >删除</el-button>
    >
  </template>
</el-table-column>
</el-table>
<!-- 分页 -->

<!-- 新增、编辑 -->
<AddCourse ref="addRef" ></AddCourse>
</el-main>
</template>

```

```

<script setup lang="ts">
import AddCourse from "../AddCourse.vue";
import { Plus, Edit, Delete, Search, Close } from "@element-plus/icons-vue";

import useCourse from "@/composables/course/useCourse";
//列表

//新增、编辑
const { addBtn, editBtn, deleteBtn, addRef, joinBtn } =
useCourse();
</script>

<style scoped></style>

```

11.3.5 创建api接口和数据模型文件

在api/course目录下面创建CourseModel.ts

```

export type CourseListParm = {
  courseName:string,
  currentPage:number,
  pageSize:number,
  total:number
}
export type CourseType = {
  type:string,
  courseId:string,
  courseName:string,
  image:string,
  teacherName:string,
  courseHour:number,
  courseDetails:string,
  coursePrice:number
}

```

在api/course目录下面创建index.ts

```
import http from "@http";
import { CourseType } from "../CourseModel";
//新增
export const addApi = (parm:CourseType)=>{
    return http.post("/api/course",parm)
}
```

11.3.6 创建useCourse.ts

在composables目录下面创建Course目录，再新建useCourse.ts文件

```
import { CourseType } from "@api/course/CourseModel"
import { EditType } from "@type/BaseEnum"
import { ref } from "vue"
export default function useCourse() {

    const addRef = ref<{ show: (type: string, row?: CourseType)
=> void }>()
    //新增
    const addBtn = () => {
        addRef.value?.show(EditType.ADD)
    }
    //编辑
    const editBtn = () => {

    }
    //删除
    const deleteBtn = async() => {

    }
    //选课
    const joinBtn = async()=>{

    }
    return {
        addBtn,
        editBtn,
        deleteBtn,
        addRef,
        joinBtn
    }
}
```

11.4 课程图片文件上传

11.4.1 minio的下载和安装

11.4.1.1 minio的基本介绍

MinIO 是一个开源的对象存储服务器。MinIO 提供了一个强大而灵活的对象存储解决方案，适用于各种规模的应用场景。详细介绍可看官网文档：[MinIO对象存储 Windows — MinIO中文文档](#) | [MinIO Windows中文文档](#)

特点:

- 高性能: MinIO 具有出色的性能，能够在分布式环境中快速传输大容量的数据。
- 可扩展性: 它可以轻松地在集群中扩展，适应不断增长的存储需求。
- 兼容性: MinIO 支持 Amazon S3 API，这意味着可以将其作为 Amazon S3 的替代品使用，并且大多数支持 S3 的应用程序都可以与 MinIO 无缝集成。
- 轻量级: MinIO 软件本身非常轻量级，易于部署和管理。
- 安全性: MinIO 支持各种安全特性，如数据加密、访问控制等，以确保数据的保密性和完整性。

用途:

- 对象存储: MinIO 主要用于对象存储，可用于存储大量非结构化数据，如图片、视频、日志文件等。
- 云存储: 可以作为私有云存储解决方案，满足企业对数据安全和隐私的需求。
- 数据湖: MinIO 可以作为构建数据湖的基础，用于存储和处理各种类型的数据，从而支持数据分析和挖掘工作。

minio与阿里云OSS对比:

- **定价和成本**
 - MinIO: MinIO 是开源软件，可以免费使用。您只需要承担运行和维护 MinIO 的成本，通常来说，这些成本相对较低。
 - 阿里云 OSS: 阿里云 OSS 是一项付费的云服务，其定价根据存储的数据量、使用的功能和网络传输等因素进行计费。在一些特定的场景下，OSS 的费用可能会比 MinIO 高一些。
- **性能和可扩展性**
 - MinIO: MinIO 提供了高性能和可扩展性，尤其适用于需要快速处理大容量数据的场景。它可以在标准硬件上运行，并且可以轻松地在集群中扩展。
 - 阿里云 OSS: 阿里云 OSS 也具有良好的性能和可扩展性，但是由于其基于云服务提供，性能和扩展性可能会受到一定程度的限制，特别是在网络传输方面。

- **数据安全和可靠性**

- MinIO: MinIO 提供了各种安全特性，如数据加密、访问控制等，可以确保数据的安全性。此外，由于可以在私有环境中部署，因此用户对数据的控制更加直接。

- 阿里云 OSS: 阿里云 OSS 也提供了多种安全功能，如数据加密、权限管理等。并且阿里云的数据中心和网络基础设施通常拥有高可靠性，能够确保数据的持久性和可用性。

- **生态系统和集成**

- MinIO: MinIO 的生态系统相对较小，但是有一些常用的客户端和工具支持，可以与各种数据管理和存储技术集成。

- 阿里云 OSS: 阿里云 OSS 作为一项成熟的云服务，拥有丰富的生态系统和各种集成方案，可以与阿里云的其他产品和服务无缝配合使用。

- **地域覆盖和网络性能**

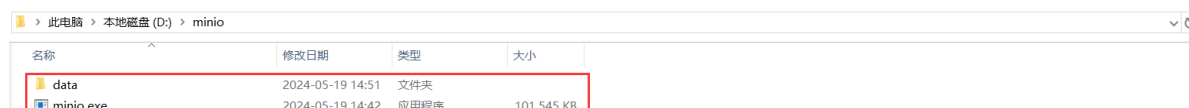
- MinIO: MinIO 的部署地点取决于用户自己的环境，可以选择在本地数据中心或者公有云中进行部署，因此地域覆盖和网络性能受到用户自身环境的影响。

- 阿里云 OSS: 阿里云 OSS 在全球范围内拥有多个数据中心和边缘节点，能够提供更广泛的地域覆盖和更稳定的网络性能。

11.4.1.2 minio的安装

官网下载: <http://minio.org.cn>

(1) 下载Minio安装后，新建1个data文件夹。并在该安装目录cmd 敲命令。注意不要进错目录。输入



名称	修改日期	类型	大小
data	2024-05-19 14:51	文件夹	
minio.exe	2024-05-19 14:42	应用程序	101,545 KB

```
minio.exe server D:\minio\data --console-address ":9001" --address ":9000"
```

注意:

9001: minio后台管理控制台端口号

9000: minio服务本身的端口号


```
D:\minio>minio.exe server D:\minio\data --console-address ":9001" --address ":9000"
Formatting 1st pool, 1 set(s), 1 drives per set.
WARNING: Host local has more than 0 drives of set. A host failure will result in data becoming unavailable.
MinIO Object Storage Server
Copyright: 2015-2024 MinIO, Inc.
License: GNU AGPLv3 <https://www.gnu.org/licenses/agpl-3.0.html>
Version: RELEASE.2024-05-01T01-11-10Z (go1.21.9 windows/amd64)

API: http://192.168.31.173:9000 http://192.168.10.1:9000 http://192.168.25.1:9000 http://127.0.0.1:9000
RootUser: minioadmin
RootPass: minioadmin 登录minio后台管理控制台的用户名和端口号

WebUI: http://192.168.31.173:9001 http://192.168.10.1:9001 http://192.168.25.1:9001 http://127.0.0.1:9001
RootUser: minioadmin
RootPass: minioadmin

CLI: https://min.io/docs/minio/linux/reference/minio-mc.html#quickstart
$ mc alias set 'myminio' 'http://192.168.31.173:9000' 'minioadmin' 'minioadmin'

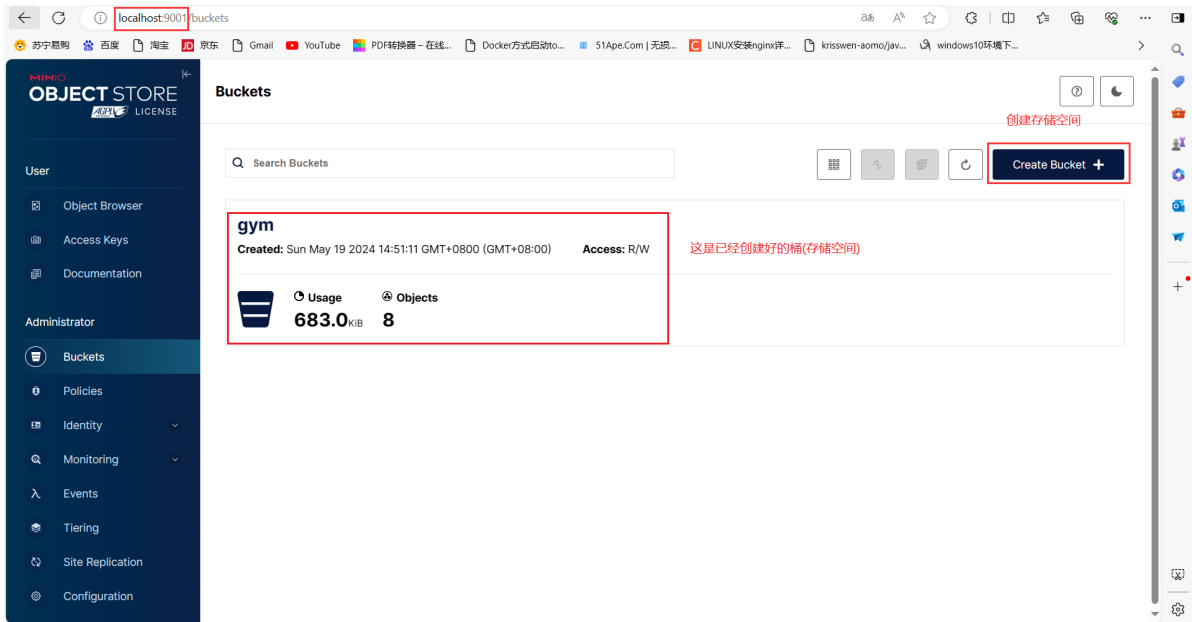
Docs: https://min.io/docs/minio/linux/index.html
Status: 1 Online, 0 Offline.
STARTUP WARNINGS:
  Detected default credentials 'minioadmin:minioadmin', we recommend that you change these values with 'MINIO_ROOT_USER' and 'MINIO_ROOT_PASSWORD' environment variables
  The standard parity is set to 0. This can lead to data loss.

You are running an older version of MinIO released 1 week before the latest release
Update: Run mc admin update ALIAS
```

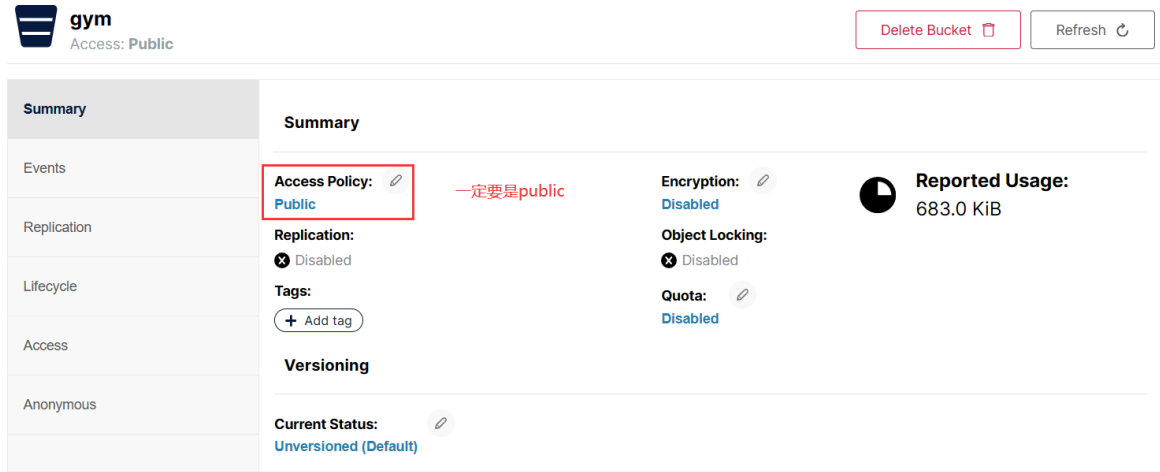
看到这个界面，就说明minio安全完毕。

11.4.1.3 登录Minio网页端

打开<http://127.0.0.1:9001/browser>，初次可能需要登陆，账号密码均输入minioadmin 即可。



注意：bucket创建之后，一定要公开。



11.4.2 springboot整合minio实现文件上传

11.4.2.1 引入minio相关依赖

```
<!--minio-->
<dependency>
    <groupId>com.squareup.okhttp3</groupId>
    <artifactId>okhttp</artifactId>
    <version>4.8.1</version>
</dependency>
<dependency>
    <groupId>com.alibaba</groupId>
    <artifactId>easyexcel</artifactId>
    <version>3.0.5</version>
</dependency>
<dependency>
    <groupId>io.minio</groupId>
    <artifactId>minio</artifactId>
    <version>8.3.9</version>
</dependency>
<dependency>
    <groupId>com.alibaba</groupId>
    <artifactId>fastjson</artifactId>
</dependency>
```

11.4.2.2 定义配置文件

```
spring:
  servlet:
    multipart:
      max-file-size: 200MB #设置单个文件的大小 因为springboot内置
      tomcat的文件传输默认为10MB
      max-request-size: 500MB #设置单次请求的文件总大小
      enabled: true #千万注意要设置该参数，否则不生效
# minio配置
minio:
  endpoint: http://127.0.0.1:9000
  accesskey: minioadmin
  secretkey: minioadmin
```

11.4.2.3 创建 MinioProp 类

这个类的作用是批量读取配置文件中的属性值，配置文件的前缀必须为minio。

```

@Data
@Component
@ConfigurationProperties(prefix = "minio")
public class MinioProp {
    private String endpoint;
    private String accesskey;
    private String secretKey;
}

```

11.4.2.4 创建Minio客户端（配置类）

```

package com.xq.config;

import io.minio.MinioClient;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.context.properties.EnableConfigurationProperties;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
@EnableConfigurationProperties(MinioProp.class)
public class MinioConfig {
    @Autowired
    private MinioProp minioProp;

    @Bean
    public MinioClient minioClient() throws Exception {
        return
        MinioClient.builder().endpoint(minioProp.getEndpoint())
            .credentials(minioProp.getAccesskey(),
            minioProp.getSecretKey()).build();
    }
}

```

11.4.2.5 创建上传文件的工具类 MinioUtils

```

package com.xq.config;

import com.alibaba.fastjson.JSONObject;
import io.minio.BucketExistsArgs;
import io.minio.MakeBucketArgs;
import io.minio.MinioClient;
import io.minio.PutObjectArgs;

```

```
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;
import org.springframework.web.multipart.MultipartFile;

import java.io.InputStream;

@Slf4j
@Component
public class MinioUtils {
    @Autowired
    private MinioClient client;
    @Autowired
    private MinioProp minioProp;

    /**
     * 创建bucket
     */
    public void createBucket(String bucketName) {
        BucketExistsArgs bucketExistsArgs =
BucketExistsArgs.builder().bucket(bucketName).build();
        MakeBucketArgs makeBucketArgs =
MakeBucketArgs.builder().bucket(bucketName).build();
        try {
            if (client.bucketExists(bucketExistsArgs))
                return;
            client.makeBucket(makeBucketArgs);
        } catch (Exception e) {
            log.error("创建桶失败: {}", e.getMessage());
            throw new RuntimeException(e);
        }
    }

    /**
     * @param file 文件
     * @param bucketName 存储桶
     * @return
     */
    public JSONObject uploadFile(MultipartFile file, String
bucketName) throws Exception {
        JSONObject res = new JSONObject();
        res.put("code", 0);
        // 判断上传文件是否为空
        if (null == file || 0 == file.getSize()) {
            res.put("msg", "上传文件不能为空");
        }
    }
}
```

```

        return res;
    }
    // 判断存储桶是否存在
    createBucket(bucketName);
    // 文件名
    String originalFilename = file.getOriginalFilename();
    // 新的文件名 = 存储桶名称_时间戳.后缀名
    String fileName = bucketName + "_" +
System.currentTimeMillis() +
originalFilename.substring(originalFilename.lastIndexOf("."));
    // 开始上传
    InputStream inputStream = file.getInputStream();
    PutObjectArgs args =
PutObjectArgs.builder().bucket(bucketName).object(fileName)

.stream(inputStream, inputStream.available(), -1).build();
    client.putObject(args);
    res.put("code", 1);
    //getEndpoint前缀 + bucketName 桶名称 + fileName文件名称
    res.put("msg", minioProp.getEndpoint() + "/" + bucketName
+ "/" + fileName);
    return res;
}
}

```

11.4.2.6 创建文件上传的Controller

```

package com.xq.web.image;

import com.alibaba.fastjson.JSONObject;
import com.xq.config.MinioUtils;
import com.xq.utils.ResultUtils;
import com.xq.utils.ResultVo;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.multipart.MultipartFile;

@RestController
@RequestMapping("/api/upload/")
@Slf4j
public class ImageUploadController {

```

```

@Autowired
MiniUtils miniUtils;

@RequestMapping("/uploadImage")
public ResultVo uploadImage(@RequestParam("file")
MultipartFile file){
    log.info("触发上传图片controller");
    JSONObject images=null;
    try {
        images = miniUtils.uploadFile(file, "gym"); //这里记得
填你自己的bucket容器
    } catch (Exception e) {
        e.printStackTrace();
    }
    return ResultUtils.success("图片上传成功",images);
}
}

```

11.4.3 文件上传前端页面制作与数据对接

11.4.3.1 修改http模块

修改http目录下的index.ts

```

//图片上传
upload<T = Result>(url: string, params?: object): Promise<T> {
    return this.instance.post(url, params, {
        headers: {
            'Content-Type': 'multipart/form-data'
        }
    })
}
}

```

11.4.3.2 api/course/index.ts添加图片上传方法

```

import http from "@/http";
import { CourseType ,CourseListParm} from "../CourseModel";
//图片上传
export const uploadImageApi = (parm:object)=>{
    return http.upload("/api/upload/uploadImage",parm)
}
//新增
export const addApi = (parm:CourseType)=>{
    return http.post("/api/course",parm)
}
}

```

11.4.3.3 composables/course下新建useUpload.ts

```
import { ref } from 'vue'
import { ElMessage, UploadFile } from 'element-plus'
import { uploadImageApi } from '@/api/course/index'
export default function useUpload() {
  //图片上传组件ref属性
  const uploadRef = ref()
  //图片预览地址
  const dialogImageUrl = ref('')
  //是否点击预览
  const dialogVisible = ref(false)
  const disabled = ref(false)
  //上传的文件
  const fileList = ref<UploadFile[]>([])
  //上传图片的地址
  const imgUrl = ref('');
  //删除图片
  const handleRemove = (file: UploadFile) => {
    console.log(file)
    console.log(fileList.value)
    fileList.value = fileList.value.filter(item => item.name
    !== file.name)
  }
  //点击预览图片
  const handlePictureCardPreview = (file: UploadFile) => {
    dialogImageUrl.value = file.url!
    dialogVisible.value = true
  }
  //图片上传
  const uploadFile = async (file: any) => {
    const typeArr = ["image/png", "image/gif", "image/jpeg",
    "image/jpg"];
    const isImg = typeArr.indexOf(file.raw.type) !== -1;
    const isMore3M = file.size / 1024 / 1024 < 3;
    if (!isImg) {
      ElMessage.warning("只能上传图片类型!");
      uploadRef.value?.clearFiles()
      return;
    }
    if (!isMore3M) {
      ElMessage.warning("只能上传图片类型!");
      uploadRef.value?.clearFiles()
      return;
    }
  }
}
```

```

        const formData = new FormData();
        formData.append("file", file.raw);
        let res = await uploadImageApi(formData);
        if (res && res.code == 200 && res.data) {
            console.log(res.data)
            //imgurl.value = res.data;
            imgurl.value = res.data.msg;
            ElMessage.success("图片上传成功!");
        }
    }

    return {
        dialogImageUrl,
        dialogVisible,
        disabled,
        handleRemove,
        handlePictureCardPreview,
        fileList,
        uploadFile,
        uploadRef,
        imgurl
    }
}

```

11.4.3.4 调整AddCourse.vue组件

```

<template>
  <SysDialog
    :title="dialog.title"
    :height="dialog.height"
    :width="dialog.width"
    :visible="dialog.visible"
    @onClose="onClose"
    @onConfirm="commit"
  >
    <template v-slot:content>
      <el-form
        :model="addModel"
        ref="addFormRef"
        :rules="rules"
        label-width="80px"
        size="default"
      >
        <el-row>
          <el-col :span="12" :offset="0">

```



```

        <el-form-item prop="courseName" label="课程名称">
            <el-input v-model="addModel.courseName"></el-
input>
        </el-form-item>
    </el-col>
    <el-col :span="12" :offset="0">
        <el-form-item prop="courseHour" label="课程课时">
            <el-input v-model="addModel.courseHour"></el-
input>
        </el-form-item>
    </el-col>
</el-row>
<el-row>
    <el-col :span="12" :offset="0">
        <el-form-item prop="coursePrice" label="课程价格">
            <el-input v-model="addModel.coursePrice"></el-
input>
        </el-form-item>
    </el-col>
    <el-col :span="12" :offset="0">
        <el-form-item prop="teacherName" label="教练">
            <el-input v-model="addModel.teacherName"></el-
input>
        </el-form-item>
    </el-col>
</el-row>
<el-form-item label="课程封面">
    <el-upload
        ref="uploadRef"
        action="#"
        :on-change="uploadFile"
        :auto-upload="false"
        :limit="1"
        :file-list="fileList"
        list-type="picture-card"
    >
        <el-icon><Plus /></el-icon>
        <template #file="{ file }">
            <div>
                
            </div>
        </template>
    </el-upload>

```

```

        <span class="el-upload-list__item-actions">
          <span
            class="el-upload-list__item-preview"
            @click="handlePictureCardPreview(file)"
          >
            <el-icon><zoom-in /></el-icon>
          </span>
          <span
            v-if="!disabled"
            class="el-upload-list__item-delete"
            @click="handleRemove(file)"
          >
            <el-icon><Delete /></el-icon>
          </span>
        </span>
      </div>
    </template>
  </el-upload>
  <el-dialog v-model="dialogVisible">
    
  </el-dialog>
</el-form-item>
  <el-form-item prop="courseDetails" label="课程详情">
    <el-input v-model="addModel.courseDetails"></el-
input>
  </el-form-item>
</el-form>
</template>
</SysDialog>
</template>

<script setup lang="ts">
  //import "@wangeditor/editor/dist/css/style.css"; // 引入 css
  //import { Editor, Toolbar } from "@wangeditor/editor-for-vue";
  import { CourseType } from "@/api/course/CourseModel";
  import SysDialog from "@/components/SysDialog.vue";
  import useDialog from "@/hooks/useDialog";
  import { ElMessage, FormInstance, UploadProps } from "element-
plus";
  import { nextTick, reactive, ref } from "vue";
  import useUpload from "@/composables/course/useUpload";
  // import useEditor from "@/composables/course/useEditor";
  // import useSelectTeacher from
"@/composables/course/useSelectTeacher";

```

```

import { EditType, Title } from "@/type/BaseEnum";
import { addApi } from "@/api/course/index";
import useInstance from "@/hooks/useInstance";
const { global } = useInstance();
//表单的ref属性
const addFormRef = ref<FormInstance>();
//弹框属性
const { dialog, onClose, onShow, onConfirm } = useDialog();
//教练选择
//const { teacherData, listTeacher } = useSelectTeacher();
//图片上传
const {
  uploadRef,
  dialogImageUrl,
  dialogvisible,
  disabled,
  handleRemove,
  handlePictureCardPreview,
  fileList,
  uploadFile,
  imgUrl
} = useUpload();
//显示弹框
const show = async (type: string, row?: CourseType) => {
  //清空图片数据
  fileList.value = [];
  //获取教练数据列表
  //await listTeacher();
  //弹框属性
  type == EditType.ADD
    ? (dialog.title = Title.ADD)
    : (dialog.title = Title.EDIT);
  dialog.width = 900;
  dialog.height = 500;
  //清空图片和文本编辑器
  const upload = uploadRef.value;
  if (upload) {
    upload.clearFiles();
  }
  addModel.image = "";
  if (type == EditType.ADD) {
    //const editor = editorRef.value;
    //if (editor) {
    //  editor.clear();
    //}
  }

```

```

    }
    addModel.courseDetails = "";
    //编辑数据回显
    if (type == EditType.EDIT) {

    }
    if (row && row.courseDetails) {
        //文本编辑器的回显
        //valueHtml.value = row.courseDetails;
    }
    onShow();
    //表单清空
    addFormRef.value?.resetFields();
    addModel.type = type;
};
//暴露出去，给外部组件使用
defineExpose({
    show,
});
//表单绑定的数据对象
const addModel = reactive<CourseType>({
    type: "", //区分 新增 编辑
    courseId: "",
    courseName: "",
    image: "",
    teacherName: "",
    //teacherId: "",
    courseHour: 0,
    courseDetails: "",
    coursePrice: 0,
});
const validateCourseHour = (rule: any, value: any, callback:
any) => {
    if (value === 0 || value < 0) {
        callback(new Error("请填写课程课时"));
    } else {
        callback();
    }
};
const validateCoursePrice = (rule: any, value: any, callback:
any) => {
    if (value === 0 || value < 0) {
        callback(new Error("请填写课程价格"));
    } else {
        callback();
    }
};

```

```
    }  
  };  
  //表单验证规则  
  const rules = reactive({  
    courseName: [  
      {  
        required: true,  
        trigger: "blur",  
        message: "请输入课程名称",  
      },  
    ],  
    image: [  
      {  
        required: true,  
        trigger: "blur",  
        message: "请上传课程图片",  
      },  
    ],  
    teacherName: [  
      {  
        required: true,  
        trigger: "blur",  
        message: "请选择课程教练",  
      },  
    ],  
    courseDetails: [  
      {  
        required: true,  
        trigger: "blur",  
        message: "请填写课程详情",  
      },  
    ],  
    coursePrice: [  
      {  
        required: true,  
        validator: validateCoursePrice,  
        trigger: "blur",  
      },  
    ],  
    courseHour: [  
      {  
        required: true,  
        validator: validateCourseHour,  
        trigger: "blur",  
      },  
    ],  
  });
```

```

    ],
  });
  //注册事件
  const emits = defineEmits(["reFresh"]);

  //表单提交
  const commit = () => {
    //课程详情
    addModel.image = imgUrl.value;
    addFormRef.value?.validate(async (valid) => {
      if (valid) {
        let res = null;
        if (addModel.type == EditType.ADD) {
          res = await addApi(addModel);
        }
        if (res && res.code == 200) {
          ElMessage.success(res.msg);
          emits("reFresh");
          onClose();
        }
      }
    });
  };
  // 选择教练的change
  const selectTeach = (val:any)=>{

  }
</script>

<style scoped></style>

```

11.5 富文本编辑器的引入

官网: <https://www.wangeditor.com/>

11.5.1 安装文本编辑器

```

npm install @wangeditor/editor --save
npm install @wangeditor/editor-for-vue@next --save

```

11.5.2 composables/course下新建useEditor.ts

```
import { onBeforeUnmount, ref, shallowRef, onMounted } from 'vue'
import { IEditorConfig } from '@wangeditor/editor'
export default function useEditor() {
  const editorRef = shallowRef()
  const valueHtml = ref('')
  const toolbarConfig = {}
  const editorConfig = { placeholder: '请输入内容...' }

  // 组件销毁时，也及时销毁编辑器
  onBeforeUnmount(() => {
    const editor = editorRef.value
    if (editor == null) return
    editor.destroy()
  })
  const handleCreated = (editor:any) => {
    editorRef.value = editor // 记录 editor 实例，重要!
  }

  return {
    editorRef,
    valueHtml,
    mode: 'default', // 或 'simple'
    toolbarConfig,
    editorConfig,
    handleCreated
  };
}
```

11.5.3 在AddCourse.vue组件中引入富文本编辑器

```
<template>
  <SysDialog
    :title="dialog.title"
    :height="dialog.height"
    :width="dialog.width"
    :visible="dialog.visible"
    @onClose="onClose"
    @onConfirm="commit"
  >
    <template v-slot:content>
      <el-form
```

```

        :model="addModel"
        ref="addFormRef"
        :rules="rules"
        label-width="80px"
        size="default"
    >
    <el-row>
        <el-col :span="12" :offset="0">
            <el-form-item prop="courseName" label="课程名称">
                <el-input v-model="addModel.courseName"></el-
input>
            </el-form-item>
        </el-col>
        <el-col :span="12" :offset="0">
            <el-form-item prop="courseHour" label="课程课时">
                <el-input v-model="addModel.courseHour"></el-
input>
            </el-form-item>
        </el-col>
    </el-row>
    <el-row>
        <el-col :span="12" :offset="0">
            <el-form-item prop="coursePrice" label="课程价格">
                <el-input v-model="addModel.coursePrice"></el-
input>
            </el-form-item>
        </el-col>
        <el-col :span="12" :offset="0">
            <el-form-item prop="teacherName" label="教练">
                <el-input v-model="addModel.teacherName"></el-
input>
            </el-form-item>
        </el-col>
    </el-row>
    <el-form-item label="课程封面">
        <el-upload
            ref="uploadRef"
            action="#"
            :on-change="uploadFile"
            :auto-upload="false"
            :limit="1"
            :file-list="fileList"
            list-type="picture-card"
        >
            <el-icon><Plus /></el-icon>

```



```

<template #file="{ file }">
  <div>
    
    <span class="el-upload-list__item-actions">
      <span
        class="el-upload-list__item-preview"
        @click="handlePictureCardPreview(file)"
      >
        <el-icon><zoom-in /></el-icon>
      </span>
      <span
        v-if="!disabled"
        class="el-upload-list__item-delete"
        @click="handleRemove(file)"
      >
        <el-icon><Delete /></el-icon>
      </span>
    </span>
  </div>
</template>
</el-upload>
<el-dialog v-model="dialogVisible">
  
</el-dialog>
</el-form-item>
<el-form-item prop="courseDetails" label="课程详情">
  <div style="border: 1px solid #ccc">
    <Toolbar
      style="border-bottom: 1px solid #ccc"
      :editor="editorRef"
      :defaultConfig="toolbarConfig"
      :mode="mode"
    />
    <Editor
      style="height: 300px; overflow-y: hidden"
      v-model="valueHtml"
      :defaultConfig="editorConfig"
      :mode="mode"
      @onCreated="handleCreated"

```

```

        />
      </div>
    </el-form-item>
  </el-form>
</template>
</SysDialog>
</template>

<script setup lang="ts">
import "@wangeditor/editor/dist/css/style.css"; // 引入 css
import { Editor, Toolbar } from "@wangeditor/editor-for-vue";
import { CourseType } from "@api/course/CourseModel";
import SysDialog from "@components/SysDialog.vue";
import useDialog from "@hooks/useDialog";
import { ElMessage, FormInstance, UploadProps } from "element-
plus";
import { nextTick, reactive, ref } from "vue";
import useUpload from "@composables/course/useUpload";
import useEditor from "@composables/course/useEditor";
// import useSelectTeacher from
"@composables/course/useSelectTeacher";
import { EditType, Title } from "@type/BaseEnum";
import { addApi } from "@api/course/index";
import useInstance from "@hooks/useInstance";
const { global } = useInstance();
//表单的ref属性
const addFormRef = ref<FormInstance>();
//弹框属性
const { dialog, onClose, onShow, onConfirm } = useDialog();
//教练选择
//const { teacherData, listTeacher } = useSelectTeacher();
//图片上传
const {
  uploadRef,
  dialogImageUrl,
  dialogvisible,
  disabled,
  handleRemove,
  handlePictureCardPreview,
  fileList,
  uploadFile,
  imgUrl
} = useUpload();
//文本编辑器
const {

```

```

editorRef,
handleCreated,
mode,
editorConfig,
valueHtml,
toolbarConfig,
} = useEditor();
//显示弹框
const show = async (type: string, row?: CourseType) => {
    //清空图片数据
    fileList.value = [];
    //获取教练数据列表
    //await listTeacher();
    //弹框属性
    type == EditType.ADD
        ? (dialog.title = Title.ADD)
        : (dialog.title = Title.EDIT);
    dialog.width = 900;
    dialog.height = 500;
    //清空图片和文本编辑器
    const upload = uploadRef.value;
    if (upload) {
        upload.clearFiles();
    }
    addModel.image = "";
    if (type == EditType.ADD) {
        const editor = editorRef.value;
        if (editor) {
            editor.clear();
        }
    }
    addModel.courseDetails = "";
    //编辑数据回显
    if (type == EditType.EDIT) {

    }
    if (row && row.courseDetails) {
        //文本编辑器的回显
        //valueHtml.value = row.courseDetails;
    }
    onShow();
    //表单清空
    addFormRef.value?.resetFields();
    addModel.type = type;
};

```

```

//暴露出去，给外部组件使用
defineExpose({
  show,
});
//表单绑定的数据对象
const addModel = reactive<CourseType>({
  type: "", //区分 新增 编辑
  courseId: "",
  courseName: "",
  image: "",
  teacherName: "",
  //teacherId: "",
  courseHour: 0,
  courseDetails: "",
  coursePrice: 0,
});
const validateCourseHour = (rule: any, value: any, callback:
any) => {
  if (value === 0 || value < 0) {
    callback(new Error("请填写课程课时"));
  } else {
    callback();
  }
};
const validateCoursePrice = (rule: any, value: any, callback:
any) => {
  if (value === 0 || value < 0) {
    callback(new Error("请填写课程价格"));
  } else {
    callback();
  }
};
//表单验证规则
const rules = reactive({
  courseName: [
    {
      required: true,
      trigger: "blur",
      message: "请输入课程名称",
    },
  ],
  image: [
    {
      required: true,
      trigger: "blur",

```

```

        message: "请上传课程图片",
    },
],
teacherName: [
    {
        required: true,
        trigger: "blur",
        message: "请选择课程教练",
    },
],
courseDetails: [
    {
        required: true,
        trigger: "blur",
        message: "请填写课程详情",
    },
],
coursePrice: [
    {
        required: true,
        validator: validateCoursePrice,
        trigger: "blur",
    },
],
courseHour: [
    {
        required: true,
        validator: validateCourseHour,
        trigger: "blur",
    },
],
});
//注册事件
const emits = defineEmits(["reFresh"]);

//表单提交
const commit = () => {
    //课程图片地址
    addModel.image = imgUrl.value;
    //课程详情
    addModel.courseDetails = valueHtml.value;
    addFormRef.value?.validate(async (valid) => {
        if (valid) {
            let res = null;
            if (addModel.type == EditType.ADD) {

```

```

        res = await addApi(addModel);
    }
    if (res && res.code == 200) {
        ElMessage.success(res.msg);
        emits("refresh");
        onClose();
    }
}
});
};
// 选择教练的change
const selectTeach = (val:any)=>{

}
</script>

<style scoped></style>

```

```

<el-form-item prop="courseDetails" label="课程详情">
  <div style="border: 1px solid #ccc">
    <Toolbar
      style="border-bottom: 1px solid #ccc"
      :editor="editorRef"
      :defaultConfig="toolbarConfig"
      :mode="mode"
    />
    <Editor
      style="height: 300px; overflow-y: hidden"
      v-model="valueHtml"
      :defaultConfig="editorConfig"
      :mode="mode"
      @onCreated="handleCreated"
    />
  </div>
</el-form-item>

```

富文本编辑器

```

import "@wangeditor/editor/dist/css/style.css"; // 引入 css
import { Editor, Toolbar } from "@wangeditor/editor-for-vue";
import { CourseType } from "@/api/course/CourseModel";
import SysDialog from "@/components/SysDialog.vue";
import useDialog from "@/hooks/useDialog";

```

11.6 选择课程老师功能实现

11.6.1 在index.html添加如下样式

```

/*定义滚动条高宽及背景 高宽分别对应横竖滚动条的尺寸*/
::-webkit-scrollbar{
  width: 7px;
  height: 5px;
  background-color: #F5F5F5;
}

/*定义滚动条轨道 内阴影+圆角*/
::-webkit-scrollbar-track {
  box-shadow: inset 0 0 6px rgba(0, 0, 0, 0.3);
  -webkit-box-shadow: inset 0 0 6px rgba(0, 0, 0, 0.3);
}

```

```

    border-radius: 8px;
    background-color: #F5F5F5;
}

/*定义滑块 内阴影+圆角*/
::-webkit-scrollbar-thumb{
    border-radius: 8px;
    box-shadow: inset 0 0 6px rgba(0, 0, 0, .1);
    -webkit-box-shadow: inset 0 0 6px rgba(0, 0, 0, .1);
    background-color: #c8c8c8;
}

/*解决element上传组件回显动画问题*/
.el-upload-list__item {
    transition: none !important;
}

```

11.6.2 SysUserController控制器添加getTeacher()方法

```

//查询课程教练
@GetMapping("/getTeacher")
public ResultVo getTeacher(){
    QueryWrapper<SysUser> query = new QueryWrapper<>();
    query.lambda().eq(SysUser::getUserType,"2");
    List<SysUser> list = sysUserService.list(query);
    //组装后的select数据
    List<SelectType> selectTypeList = new ArrayList<>();
    if(list.size() >0){
        list.stream().forEach(item ->{
            SelectType selectType = new SelectType();
            selectType.setLabel(item.getNickName());
            selectType.setValue(item.getRoleId());
            selectTypeList.add(selectType);
        });
    }
    return ResultUtils.success("查询成功",selectTypeList);
}

```

11.6.3 api/course/index.ts添加getTeacherApi()方法

```

//获取课程教练
export const getTeacherApi = ()=>{
    return http.get("/api/user/getTeacher")
}

```

11.6.4 composables/course下添加useSelectTeacher.ts

```
import { getTeacherApi } from "@/api/course/index";
import { reactive, ref } from "vue";
import { SelectRole } from "@/api/user/UserModel";
export default function useSelectTeacher() {
  //定义教练列表数据
  const teacherData = reactive<SelectRole>({
    list: []
  })
  //获取数据
  const listTeacher = async () => {
    let res = await getTeacherApi()
    if (res && res.code == 200) {
      teacherData.list = res.data
    }
  }

  return {
    teacherData,
    listTeacher
  }
}
```

11.6.5 修改AddCourse.vue组件

```
<template>
  <SysDialog
    :title="dialog.title"
    :height="dialog.height"
    :width="dialog.width"
    :visible="dialog.visible"
    @onClose="onClose"
    @onConfirm="commit"
  >
    <template v-slot:content>
      <el-form
        :model="addModel"
        ref="addFormRef"
        :rules="rules"
        label-width="80px"
        size="default"
      >
        <el-row>
```



```

        <el-col :span="12" :offset="0">
            <el-form-item prop="courseName" label="课程名称">
                <el-input v-model="addModel.courseName"></el-
input>
            </el-form-item>
        </el-col>
        <el-col :span="12" :offset="0">
            <el-form-item prop="courseHour" label="课程课时">
                <el-input v-model="addModel.courseHour"></el-
input>
            </el-form-item>
        </el-col>
    </el-row>
    <el-row>
        <el-col :span="12" :offset="0">
            <el-form-item prop="coursePrice" label="课程价格">
                <el-input v-model="addModel.coursePrice"></el-
input>
            </el-form-item>
        </el-col>
        <el-col :span="12" :offset="0">
            <el-form-item prop="teacherName" label="教练">
                <el-select
                    style="width:100%;"
                    v-model="addModel.teacherName"
                    class="m-2"
                    placeholder="请选择教练"
                    size="default"
                >
                    <el-option
                        v-for="item in teacherData.list"
                        :key="item.label"
                        :label="item.label"
                        :value="item.label"
                    />
                </el-select>
            </el-form-item>
        </el-col>
    </el-row>
    <el-form-item label="课程封面">
        <el-upload
            ref="uploadRef"
            action="#"
            :on-change="uploadFile"
            :auto-upload="false"

```

```

        :limit="1"
        :file-list="fileList"
        list-type="picture-card"
      >
      <el-icon><Plus /></el-icon>
      <template #file="{ file }">
        <div>
          
          <span class="el-upload-list__item-actions">
            <span
              class="el-upload-list__item-preview"
              @click="handlePictureCardPreview(file)"
            >
              <el-icon><zoom-in /></el-icon>
            </span>
            <span
              v-if="!disabled"
              class="el-upload-list__item-delete"
              @click="handleRemove(file)"
            >
              <el-icon><Delete /></el-icon>
            </span>
          </span>
        </div>
      </template>
    </el-upload>
    <el-dialog v-model="dialogVisible">
      
  </el-dialog>
</el-form-item>
<el-form-item prop="courseDetails" label="课程详情">
  <div style="border: 1px solid #ccc">
    <Toolbar
      style="border-bottom: 1px solid #ccc"
      :editor="editorRef"
      :defaultConfig="toolbarConfig"
      :mode="mode"
    />
    <Editor

```

```

        style="height: 300px; overflow-y: hidden"
        v-model="valueHtml"
        :defaultConfig="editorConfig"
        :mode="mode"
        @onCreated="handleCreated"
      />
    </div>
  </el-form-item>
</el-form>
</template>
</SysDialog>
</template>

<script setup lang="ts">
import "@wangeditor/editor/dist/css/style.css"; // 引入 css
import { Editor, Toolbar } from "@wangeditor/editor-for-vue";
import { CourseType } from "@api/course/CourseModel";
import SysDialog from "@components/SysDialog.vue";
import useDialog from "@hooks/useDialog";
import { ElMessage, FormInstance, UploadProps } from "element-
plus";
import { nextTick, reactive, ref } from "vue";
import useUpload from "@composables/course/useUpload";
import useEditor from "@composables/course/useEditor";
import useSelectTeacher from
"@composables/course/useSelectTeacher";
import { EditType, Title } from "@type/BaseEnum";
import { addApi } from "@api/course/index";
import useInstance from "@hooks/useInstance";
const { global } = useInstance();
//表单的ref属性
const addFormRef = ref<FormInstance>();
//弹框属性
const { dialog, onClose, onShow, onConfirm } = useDialog();
//教练选择
const { teacherData, listTeacher } = useSelectTeacher();
//图片上传
const {
  uploadRef,
  dialogImageUrl,
  dialogVisible,
  disabled,
  handleRemove,
  handlePictureCardPreview,
  fileList,

```

```

        uploadFile,
        imgUrl
    } = useUpload();
    //文本编辑器
const {
    editorRef,
    handleCreated,
    mode,
    editorConfig,
    valueHtml,
    toolbarConfig,
} = useEditor();
    //显示弹框
const show = async (type: string, row?: CourseType) => {
    //清空图片数据
    fileList.value = [];
    //获取教练数据列表
    await listTeacher();
    //弹框属性
    type == EditType.ADD
        ? (dialog.title = Title.ADD)
        : (dialog.title = Title.EDIT);
    dialog.width = 900;
    dialog.height = 500;
    //清空图片和文本编辑器
    const upload = uploadRef.value;
    if (upload) {
        upload.clearFiles();
    }
    addModel.image = "";
    if (type == EditType.ADD) {
        const editor = editorRef.value;
        if (editor) {
            editor.clear();
        }
    }
    addModel.courseDetails = "";
    //编辑数据回显
    if (type == EditType.EDIT) {

    }

    if (row && row.courseDetails) {
        //文本编辑器的回显
        //valueHtml.value = row.courseDetails;
    }
}

```

```

    onShow();
    //表单清空
    addFormRef.value?.resetFields();
    addModel.type = type;
  };
  //暴露出去，给外部组件使用
  defineExpose({
    show,
  });
  //表单绑定的数据对象
  const addModel = reactive<CourseType>({
    type: "", //区分 新增 编辑
    courseId: "",
    courseName: "",
    image: "",
    teacherName: "",
    //teacherId: "",
    courseHour: 0,
    courseDetails: "",
    coursePrice: 0,
  });
  const validateCourseHour = (rule: any, value: any, callback:
any) => {
    if (value === 0 || value < 0) {
      callback(new Error("请填写课程课时"));
    } else {
      callback();
    }
  };
  const validateCoursePrice = (rule: any, value: any, callback:
any) => {
    if (value === 0 || value < 0) {
      callback(new Error("请填写课程价格"));
    } else {
      callback();
    }
  };
  //表单验证规则
  const rules = reactive({
    courseName: [
      {
        required: true,
        trigger: "blur",
        message: "请输入课程名称",
      },
    ],
  });

```

```

],
image: [
  {
    required: true,
    trigger: "blur",
    message: "请上传课程图片",
  },
],
teacherName: [
  {
    required: true,
    trigger: "blur",
    message: "请选择课程教练",
  },
],
courseDetails: [
  {
    required: true,
    trigger: "blur",
    message: "请填写课程详情",
  },
],
coursePrice: [
  {
    required: true,
    validator: validateCoursePrice,
    trigger: "blur",
  },
],
courseHour: [
  {
    required: true,
    validator: validateCourseHour,
    trigger: "blur",
  },
],
});
//注册事件
const emits = defineEmits(["reFresh"]);

//表单提交
const commit = () => {
  //课程图片地址
  addModel.image = imgurl.value;
  //课程详情

```

```

addModel.courseDetails = valueHtml.value;
addFormRef.value?.validate(async (valid) => {
  if (valid) {
    let res = null;
    if (addModel.type == EditType.ADD) {
      res = await addApi(addModel);
    }
    if (res && res.code == 200) {
      ElMessage.success(res.msg);
      emits("refresh");
      onClose();
    }
  }
});
// 选择教练的change
const selectTeach = (val:any)=>{

}
</script>

<style scoped></style>

```

11.7 富文本编辑器插入图片

11.7.1 配置接口请求的地址为全局配置

```

define:{
  'process.env': {
    'BASE_API': "http://localhost:8089"
  }
}

```

```

export default defineConfig({
  plugins: [vue()],
  server: {
    host: '0.0.0.0', // 任何ip都可以访问
    port: 8080, // 项目端口号
    hmr: false, // 开启热加载
    open: true // 自动打开浏览器
  },
  resolve: {
    alias: [
      {
        find: '@',
        replacement: resolve(__dirname, 'src')
      }
    ]
  },
  define: {
    'process.env': {
      'BASE_API': "http://localhost:8089"
    }
  }
})

```

11.7.2 http的baseUrl修改为如下

```
//axios请求配置
const config = {
  // baseUrl: 'http://localhost:8089', //真实请求接口的地址,真实上线
  //是一个域名
  baseUrl: process.env.BASE_API,
  timeout: 10000
}
```

11.7.3 useEditor.ts添加图片上传的配置

```
import { onBeforeUnmount, ref, shallowRef, onMounted } from 'vue'
import { IEditorConfig } from '@wangeditor/editor'
export default function useEditor() {
  type InsertFnType = (url: string, alt?: string, href?:
string) => void
  const editorRef = shallowRef()
  const valueHtml = ref('')
  const toolbarConfig = {}
  const editorConfig: Partial<IEditorConfig> = { MENU_CONF: {},
placeholder: '请输入内容...' }

  // 上传图片的配置
  editorConfig.MENU_CONF!['uploadImage'] = {
    // form-data fieldName , 默认值 'wangeditor-uploaded-image'
    fieldName: 'file',
    //上传图片后端地址
    server: process.env.BASE_API + '/api/upload/uploadImage',
    // 自定义插入图片
    customInsert(res: any, insertFn: InsertFnType) {
      // res 即服务端的返回结果
      console.log('路径:', process.env.BASE_API +
res.data.msg)
      // 从 res 中找到 url alt href , 然后插图图片
      insertFn(res.data.msg)
    },
  },
}
// 组件销毁时, 也及时销毁编辑器
onBeforeUnmount(() => {
  const editor = editorRef.value
  if (editor == null) return
  editor.destroy()
})
```



```

const handleCreated = (editor:any) => {
  editorRef.value = editor // 记录 editor 实例，重要！
}

return {
  editorRef,
  valueHtml,
  mode: 'default', // 或 'simple'
  toolbarConfig,
  editorConfig,
  handleCreated
};
}

```

11.7.4 解决图片不能回显的问题

在WebMvcConfig中添加如下代码，解决图片不能回显的问题

```

@Override
public void addResourceHandlers(ResourceHandlerRegistry registry)
{
  // MinIO的URL是 http://localhost:9000
  // 静态资源访问路径是 /gymnasium/**
  registry.addResourceHandler("/gymnasium/**")

  .addResourceLocations("http://localhost:9000/gymnasium/");
}

```

11.8 课程列表制作与接口对接

11.8.1 api/course/index添加listApi()方法

```

//列表
export const listApi = (parm:CourseListParm)=>{
  return http.get("/api/course/list",parm)
}

```

11.8.2 composables/course下的useTable定义如下

```

import { nextTick, onMounted, reactive, ref } from "vue";
import { CourseListParm } from '@api/course/CourseModel'
import { listApi } from "@api/course";
export default function useTable() {
  //表格高度

```

```
const tableHeight = ref(0)
//定义表格数据
const tableDate = reactive({
  list:[]
})
//列表查询参数
const listParm = reactive<CourseListParm>({
  courseName:'',
  teacherName:'',
  currentPage:1,
  pageSize:10,
  total:0
})
//查询列表
const getList = async()=>{
  let res = await listApi(listParm)
  if(res && res.code == 200){
    console.log(res)
    tableDate.list = res.data.records;
    listParm.total = res.data.total;
  }
}
//页容量改变时触发
const sizeChange = (size:number)=>{
  listParm.pageSize = size;
  getList()
}
//页数改变时触发
const currentChange = (page:number)=>{
  listParm.currentPage = page;
  getList()
}
//搜索
const searchBtn = ()=>{
  getList()
}
//重置
const resetBtn = ()=>{
  listParm.courseName = ''
  listParm.teacherName = ''
  listParm.currentPage = 1;
  listParm.pageSize = 10
  getList()
}
//刷新表格
```

```

const reFresh = ()=>{
  getList()
}
onMounted(()=>{
  nextTick(()=>{
    tableHeight.value = window.innerHeight - 230
  })
  getList()
})
return{
  listParm,
  tableDate,
  getList,
  sizeChange,
  currentChange,
  tableHeight,
  searchBtn,
  resetBtn,
  reFresh
}
}

```

11.8.3 修改CourseList.vue组件

```

<template>
  <el-main>
    <!-- 搜索栏 -->
    <el-form :model="listParm" :inline="true" size="default">
      <el-form-item>
        <el-input
          v-model="listParm.courseName"
          placeholder="课程名称"
        ></el-input>
      </el-form-item>
      <el-form-item>
        <el-input
          v-model="listParm.teacherName"
          placeholder="教练"
        ></el-input>
      </el-form-item>
      <el-form-item>
        <el-button :icon="Search" @click="searchBtn">搜索</el-
button>
        <el-button :icon="Close" @click="resetBtn">重置</el-
button>

```

```

        <el-button :icon="Plus" type="primary" @click="addBtn"
        >新增</el-button>
    >
    </el-form-item>
</el-form>
<!-- 表格 -->
<el-table :height="tableHeight" :data="tableDate.list" border
stripe>
    <el-table-column prop="courseName" label="课程名称"></el-
table-column>
    <el-table-column prop="courseHour" label="课时"></el-table-
column>
    <el-table-column prop="coursePrice" label="价格"></el-table-
column>
    <el-table-column prop="teacherName" label="授课教师"></el-
table-column>
    <el-table-column label="操作" align="center" width="220">
        <template #default="scope">
            <el-button type="primary" :icon="Edit" size="default"
@click="editBtn(scope.row)">编辑</el-button>
            <el-button type="danger" :icon="Delete" size="default"
@click="deleteBtn(scope.row)">删除</el-button>
        </template>
    </el-table-column>

</el-table>
<!-- 分页 -->
<el-pagination
    @size-change="sizeChange"
    @current-change="currentChange"
    :current-page.sync="listParm.currentPage"
    :page-sizes="[10,20, 40, 80, 100]"
    :page-size="listParm.pageSize"
    layout="total, sizes, prev, pager, next, jumper"
    :total="listParm.total" background>
</el-pagination>

<!-- 新增、编辑 -->
<AddCourse ref="addRef" @reFresh="reFresh"></AddCourse>
</el-main>
</template>

<script setup lang="ts">
import AddCourse from "../AddCourse.vue";

```

```

import { Plus, Edit, Delete, Search, Close } from "@element-plus/icons-vue";
import useTable from "@/composables/course/useTable";
import useCourse from "@/composables/course/useCourse";
//列表
const {
  listParm, tableDate, getList, sizeChange, currentChange, tableHeight, searchBtn, resetBtn, reFresh } = useTable();
//新增、编辑
const { addBtn, editBtn, deleteBtn, addRef } = useCourse();
</script>

<style scoped></style>

```

11.8.4 修改AddCourse.vue组件

```

//注册事件
const emits = defineEmits(["reFresh"]);
//表单提交
const commit = () => {
  //课程图片地址
  addModel.image = imgUrl.value;
  //课程详情
  addModel.courseDetails = valueHtml.value;
  addFormRef.value?.validate(async (valid) => {
    if (valid) {
      let res = null;
      if (addModel.type == EditType.ADD) {
        res = await addApi(addModel);
      }
      if (res && res.code == 200) {
        ElMessage.success(res.msg);
        emits("reFresh");
        onClose();
      }
    }
  });
});

```

11.9 课程编辑删除接口对接

11.9.1 表格添加课程图片

```

<el-table-column prop="image" width="90" label="课程封面"
  align="center">
  <template #default="scope">
    <el-image
      style="width: 60px; height: 60px; border-radius: 50%"
      :src="scope.row.image">
    </el-image>
  </template>
</el-table-column>

```

11.9.2 api/course/index.ts添加editApi() 和 deleteApi()

```
//编辑
export const editApi = (parm:CourseType)=>{
    return http.put("/api/course",parm)
}
//删除
export const deleteApi = (courseId:string)=>{
    return http.delete(`/api/course/${courseId}`)
}
```

11.9.3 修改composables/course目录下的useCourse.ts

```
import { CourseType } from "@api/course/CourseModel"
import { EditType } from "@type/BaseEnum"
import { ref } from "vue"
import { deleteApi } from "@api/course"
import { ElMessage } from "element-plus"
import { FuncList } from "@type/BaseType"
import useInstance from "@hooks/useInstance"
export default function useCourse(getList: FuncList) {
    const { global } = useInstance()
    const addRef = ref<{ show: (type: string, row?: CourseType)
=> void }>()
    //新增
    const addBtn = () => {
        addRef.value?.show(EditType.ADD)
    }
    //编辑
    const editBtn = (row: CourseType) => {
        addRef.value?.show(EditType.EDIT, row)
    }
    //删除
    const deleteBtn = async(row: CourseType) => {
        //信息确定
        let confirm = await global.$myconfirm('确定删除该数据吗?')
        if(confirm){
            let res = await deleteApi(row.courseId)
            if(res && res.code == 200){
                ElMessage.success(res.msg)
                getList()
            }
        }
    }
}
```

```

//选课
const joinBtn = async(row: CourseType)=>{

}
return {
  addBtn,
  editBtn,
  deleteBtn,
  addRef,
  joinBtn
}
}

```

11.9.4 调整AddCourse.vue 和 CourseList.vue组件

- AddCourse.vue

```

<template>
  <SysDialog
    :title="dialog.title"
    :height="dialog.height"
    :width="dialog.width"
    :visible="dialog.visible"
    @onClose="onClose"
    @onConfirm="commit"
  >
    <template v-slot:content>
      <el-form
        :model="addModel"
        ref="addFormRef"
        :rules="rules"
        label-width="80px"
        size="default"
      >
        <el-row>
          <el-col :span="12" :offset="0">
            <el-form-item prop="courseName" label="课程名称">
              <el-input v-model="addModel.courseName"></el-input>
            </el-form-item>
          </el-col>
          <el-col :span="12" :offset="0">
            <el-form-item prop="courseHour" label="课程课时">
              <el-input v-model="addModel.courseHour"></el-input>
            </el-form-item>
          </el-col>
        </el-row>
      </template>
    </template>
  </SysDialog>

```

```

</el-row>
<el-row>
  <el-col :span="12" :offset="0">
    <el-form-item prop="coursePrice" label="课程价格">
      <el-input v-model="addModel.coursePrice"></el-
input>
    </el-form-item>
  </el-col>
  <el-col :span="12" :offset="0">
    <el-form-item prop="teacherName" label="教练">
      <el-select
        style="width:100%;"
        v-model="addModel.teacherName"
        class="m-2"
        placeholder="请选择教练"
        size="default"
      >
        <el-option
          v-for="item in teacherData.list"
          :key="item.label"
          :label="item.label"
          :value="item.label"
        />
      </el-select>
    </el-form-item>
  </el-col>
</el-row>
<el-form-item label="课程封面">
  <el-upload
    ref="uploadRef"
    action="#"
    :on-change="uploadFile"
    :auto-upload="false"
    :limit="1"
    :file-list="fileList"
    list-type="picture-card"
  >
    <el-icon><Plus /></el-icon>
    <template #file="{ file }">
      <div>
        <img
          class="el-upload-list__item-thumbnail"
          ref="imageSrc"
          :src="file.url"
          alt=""

```



```

        />
        <span class="el-upload-list__item-actions">
            <span
                class="el-upload-list__item-preview"
                @click="handlePictureCardPreview(file)"
            >
                <el-icon><zoom-in /></el-icon>
            </span>
            <span
                v-if="!disabled"
                class="el-upload-list__item-delete"
                @click="handleRemove(file)"
            >
                <el-icon><Delete /></el-icon>
            </span>
        </span>
    </div>
</template>
</el-upload>
<el-dialog v-model="dialogVisible">
    
</el-dialog>
</el-form-item>
<el-form-item prop="courseDetails" label="课程详情">
    <div style="border: 1px solid #ccc">
        <Toolbar
            style="border-bottom: 1px solid #ccc"
            :editor="editorRef"
            :defaultConfig="toolbarConfig"
            :mode="mode"
        />
        <Editor
            style="height: 300px; overflow-y: hidden"
            v-model="valueHtml"
            :defaultConfig="editorConfig"
            :mode="mode"
            @onCreated="handleCreated"
        />
    </div>
</el-form-item>
</el-form>
</template>
</SysDialog>
</template>

```

```
<script setup lang="ts">
import "@wangeditor/editor/dist/css/style.css"; // 引入 css
import { Editor, Toolbar } from "@wangeditor/editor-for-vue";
import { CourseType } from "@api/course/CourseModel";
import SysDialog from "@components/SysDialog.vue";
import useDialog from "@hooks/useDialog";
import { ElMessage, FormInstance, UploadProps } from "element-plus";
import { nextTick, reactive, ref } from "vue";
import useUpload from "@composables/course/useUpload";
import useEditor from "@composables/course/useEditor";
import useSelectTeacher from
"@composables/course/useSelectTeacher";
import { EditType, Title } from "@type/BaseEnum";
import { addApi, editApi } from "@api/course/index";
import useInstance from "@hooks/useInstance";
const { global } = useInstance();
//表单的ref属性
const addFormRef = ref<FormInstance>();
//弹框属性
const { dialog, onClose, onShow, onConfirm } = useDialog();
//教练选择
const { teacherData, listTeacher } = useSelectTeacher();
//图片上传
const {
  uploadRef,
  dialogImageUrl,
  dialogVisible,
  disabled,
  handleRemove,
  handlePictureCardPreview,
  fileList,
  uploadFile,
  imgUrl
} = useUpload();
//文本编辑器
const {
  editorRef,
  handleCreated,
  mode,
  editorConfig,
  valueHtml,
  toolbarConfig,
} = useEditor();
```

```

//显示弹框
const show = async (type: string, row?: CourseType) => {
  //清空图片数据
  fileList.value = [];
  //获取教练数据列表
  await listTeacher();
  //弹框属性
  type == EditType.ADD
    ? (dialog.title = Title.ADD)
    : (dialog.title = Title.EDIT);
  dialog.width = 900;
  dialog.height = 500;
  //清空图片和文本编辑器
  const upload = uploadRef.value;
  if (upload) {
    upload.clearFiles();
  }
  addModel.image = "";
  if (type == EditType.ADD) {
    const editor = editorRef.value;
    if (editor) {
      editor.clear();
    }
  }
  addModel.courseDetails = "";
  //编辑数据回显
  if (type == EditType.EDIT) {
    nextTick(() => {
      //设置编辑的数据到表单绑定的数据对象
      global.$objCopy(row, addModel);
      //文本编辑器数据回显
      valueHtml.value = addModel.courseDetails;
      //封面图回显
      if (row?.image) {
        let obj = {
          name: '',
          url: '',
        };
        obj.url = row.image;
        fileList.value.push(obj);
      }
    })
  }
  if (row && row.courseDetails) {
    //文本编辑器的回显
  }
}

```

```

    valueHtml.value = row.courseDetails;
  }
  onShow();
  //表单清空
  addFormRef.value?.resetFields();
  addModel.type = type;
};
//暴露出去，给外部组件使用
defineExpose({
  show,
});
//表单绑定的数据对象
const addModel = reactive<CourseType>({
  type: "", //区分 新增 编辑
  courseId: "",
  courseName: "",
  image: "",
  teacherName: "",
  //teacherId: "",
  courseHour: 0,
  courseDetails: "",
  coursePrice: 0,
});
const validateCourseHour = (rule: any, value: any, callback: any)
=> {
  if (value === 0 || value < 0) {
    callback(new Error("请填写课程课时"));
  } else {
    callback();
  }
};
const validateCoursePrice = (rule: any, value: any, callback:
any) => {
  if (value === 0 || value < 0) {
    callback(new Error("请填写课程价格"));
  } else {
    callback();
  }
};
//表单验证规则
const rules = reactive({
  courseName: [
    {
      required: true,
      trigger: "blur",

```

```

        message: "请输入课程名称",
      },
    ],
    image: [
      {
        required: true,
        trigger: "blur",
        message: "请上传课程图片",
      },
    ],
    teacherName: [
      {
        required: true,
        trigger: "blur",
        message: "请选择课程教练",
      },
    ],
    courseDetails: [
      {
        required: true,
        trigger: "blur",
        message: "请填写课程详情",
      },
    ],
    coursePrice: [
      {
        required: true,
        validator: validateCoursePrice,
        trigger: "blur",
      },
    ],
    courseHour: [
      {
        required: true,
        validator: validateCourseHour,
        trigger: "blur",
      },
    ],
  });
//注册事件
const emits = defineEmits(["reFresh"]);

//表单提交
const commit = () => {
  //课程图片地址

```

```

addModel.image = imgUrl.value;
//课程详情
addModel.courseDetails = valueHtml.value;
addFormRef.value?.validate(async (valid) => {
  if (valid) {
    let res = null;
    if (addModel.type == EditType.ADD) {
      res = await addApi(addModel);
    } else {
      res = await editApi(addModel);
    }
    if (res && res.code == 200) {
      ElMessage.success(res.msg);
      emits("reFresh");
      onClose();
    }
  }
});
};

</script>

<style scoped></style>

```

- CourseList.vue

```

<template>
  <el-main>
    <!-- 搜索栏 -->
    <el-form :model="listParm" :inline="true" size="default">
      <el-form-item>
        <el-input
          v-model="listParm.courseName"
          placeholder="课程名称"
        ></el-input>
      </el-form-item>
      <el-form-item>
        <el-input v-model="listParm.teacherName" placeholder="教
        练"></el-input>
      </el-form-item>
      <el-form-item>
        <el-button :icon="Search" @click="searchBtn">搜索</el-
        button>
        <el-button :icon="Close" @click="resetBtn">重置</el-
        button>
      </el-form-item>
    </el-form>
  </el-main>
</template>

```

```

        <el-button
          :icon="Plus"
          v-permission="['sys:courseList:add']"
          type="primary"
          @click="addBtn"
        >新增</el-button>
      >
    </el-form-item>
  </el-form>
  <!-- 表格 -->
  <el-table :height="tableHeight" :data="tableDate.list" border
stripe>
    <el-table-column prop="image" width="90" label="课程封面"
align="center">
      <template #default="scope">
        <el-image
          style="width: 60px; height: 60px; border-radius: 50%"
          :src="scope.row.image"
        ></el-image>
      </template>
    </el-table-column>
    <el-table-column prop="courseName" label="课程名称"></el-
table-column>
    <el-table-column prop="courseHour" label="课时"></el-table-
column>
    <el-table-column prop="coursePrice" label="价格"></el-table-
column>
    <el-table-column prop="teacherName" label="授课教师"></el-
table-column>
    <el-table-column label="操作" align="center" width="290">
      <template #default="scope">
        <el-button
          v-permission="['sys:courseList:edit']"
          type="primary"
          :icon="Edit"
          size="default"
          @click="editBtn(scope.row)"
        >编辑</el-button>
      >
    </el-table-column>
    <el-button
      v-permission="['sys:courseList:delete']"
      type="danger"
      :icon="Delete"
      size="default"
    >删除</el-button>
  </el-table>

```

```

        @click="deleteBtn(scope.row)"
        >删除</el-button>
    >
</template>
</el-table-column>
</el-table>
<!-- 分页 -->
<el-pagination
    @size-change="sizeChange"
    @current-change="currentChange"
    :current-page.sync="listParm.currentPage"
    :page-sizes="[10, 20, 40, 80, 100]"
    :page-size="listParm.pageSize"
    layout="total, sizes, prev, pager, next, jumper"
    :total="listParm.total"
    background
>
</el-pagination>
<!-- 新增、编辑 -->
<AddCourse ref="addRef" @reFresh="reFresh"></AddCourse>
</el-main>
</template>

<script setup lang="ts">
import AddCourse from "../AddCourse.vue";
import { Plus, Edit, Delete, Search, Close } from "@element-plus/icons-vue";
import useTable from "@/composables/course/useTable";
import useCourse from "@/composables/course/useCourse";
//列表
const {
    listParm,
    tableDate,
    getList,
    sizeChange,
    currentChange,
    tableHeight,
    searchBtn,
    resetBtn,
    reFresh,
} = useTable();
//新增、编辑
const { addBtn, editBtn, deleteBtn, addRef, joinBtn } =
useCourse(getList);
</script>

```



```
<style scoped></style>
```

12 设备管理功能开发

12.1 数据表设计

```
create table material(  
    id                int not null auto_increment comment '器材  
id',  
    name              varchar(128) comment '器材名称',  
    num_total         int comment '数量',  
    details           text comment '简介',  
    primary key (id)  
);
```

12.2 后台代码开发

12.2.1 新建实体类

```
package com.xq.web.equipment.entity;  
  
import com.baomidou.mybatisplus.annotation.IdType;  
import com.baomidou.mybatisplus.annotation.TableId;  
import com.baomidou.mybatisplus.annotation.TableName;  
import lombok.Data;  
  
@Data  
@TableName("material")  
public class Material {  
    @TableId(type = IdType.AUTO)  
    private Long id;  
    private String name;  
    private String details;  
    private Integer numTotal;  
}
```

```
package com.xq.web.equipment.entity;

import lombok.Data;

@Data
public class ListParam {
    private Long currentPage;
    private Long pageSize;
    private String name;
}
```

12.2.2 新建mapper层

```
package com.xq.web.equipment.mapper;

import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.xq.web.equipment.entity.Material;

public interface MaterialMapper extends BaseMapper<Material> {
}
```

12.2.3 创建mapper映射文件

```
<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="com.xq.web.equipment.mapper.MaterialMapper">

</mapper>
```

12.2.4 创建service接口

```
package com.xq.web.equipment.service;

import com.baomidou.mybatisplus.extension.service.IService;
import com.xq.web.equipment.entity.Material;

public interface MaterialService extends IService<Material> {
}
```

编写接口实现类

```

package com.xq.web.equipment.service.impl;

import
com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
import com.xq.web.equipment.entity.Material;
import com.xq.web.equipment.mapper.MaterialMapper;
import com.xq.web.equipment.service.MaterialService;
import org.springframework.stereotype.Service;

@Service
public class MaterialServiceImpl extends
ServiceImpl<MaterialMapper, Material> implements MaterialService
{

}

```

12.2.5 编写controller

```

package com.xq.web.equipment.controller;

import
com.baomidou.mybatisplus.core.conditions.query.QueryWrapper;
import com.baomidou.mybatisplus.core.metadata.IPage;
import
com.baomidou.mybatisplus.extension.plugins.pagination.Page;
import com.xq.utils.ResultUtils;
import com.xq.utils.ResultVo;
import com.xq.web.equipment.entity.ListParam;
import com.xq.web.equipment.entity.Material;
import com.xq.web.equipment.service.MaterialService;
import org.apache.commons.lang.StringUtils;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/material")
public class MaterialController {
    @Autowired
    private MaterialService materialService;

    //新增
    @PostMapping
    public ResultVo add(@RequestBody Material material){

```

```

        if(materialService.save(material)){
            return ResultUtils.success("新增成功!");
        }
        return ResultUtils.error("新增失败!");
    }

    //编辑
    @PutMapping
    public ResultVo edit(@RequestBody Material material){
        if(materialService.updateById(material)){
            return ResultUtils.success("编辑成功!");
        }
        return ResultUtils.error("编辑失败!");
    }

    //删除
    @DeleteMapping("/{id}")
    public ResultVo delete(@PathVariable("id") Long id){
        if(materialService.removeById(id)){
            return ResultUtils.success("删除成功!");
        }
        return ResultUtils.error("删除失败!");
    }

    //列表
    @GetMapping("/list")
    public ResultVo list(ListParam parm){
        //构造分页对象
        IPage<Material> page = new Page<>
(parm.getCurrentPage(),parm.getPageSize());
        //构造查询条件
        QueryWrapper<Material> query = new QueryWrapper<>();
        if(StringUtils.isEmpty(parm.getName())){

        query.lambda().like(Material::getName,parm.getName());
        }
        IPage<Material> list = materialService.page(page, query);
        return ResultUtils.success("查询成功",list);
    }
}

```

12.3 设备管理前端功能实现

12.3.1 设备新增页面制作与数据对接

12.3.1.1 修改MenuBar.vue组件

在MenuBar.vue组件中添加如下代码：

```
{
  path: "/materialRoot",
  component: "Layout",
  name: "materialRoot",
  meta: {
    title: "器材管理",
    icon: "KnifeFork",
    roles: ["sys:materialRoot"],
  },
  children: [
    {
      path: "/materialList",
      component: "/material/MaterialList",
      name: "materialList",
      meta: {
        title: "器材列表",
        icon: "UserFilled",
        roles: ["sys:materialList"],
      },
    },
  ],
},
```

12.3.1.2 修改路由文件

```
{
  path: "/materialRoot",
  component: Layout,
  name: "materialRoot",
  meta: {
    title: "器材管理",
    icon: "Document",
    roles: ["sys:materialRoot"],
  },
  children: [
    {
      path: "/materialList",
```

```

        component: () =>
import('@/views/material/MaterialList.vue'),
        name: "materialList",
        meta: {
            title: "器材列表",
            icon: "UserFilled",
            roles: ["sys:materialList"],
        },
    },
]
},

```

12.3.1.3 api下新建material目录，并建index.ts和MaterialModel.ts

- MaterialModel.ts

```

//列表参数类型
export type ListParm = {
    name:string,
    currentPage:number,
    pageSize:number,
    total:number
}
//器材数据类型
export type MaterialType = {
    id:string,
    name:string,
    details:string,
    numTotal:number,
    type:string
}

```

- index.ts

```

import http from "@/http";
import { MaterialType } from "../MaterialModel";
//新增
export const addApi = (parm:MaterialType)=>{
    return http.post('/api/material',parm)
}

```

12.3.1.4 composables目录新建material目录，新建如下文件

- useTable.ts

```
import { ListParm } from "@api/material/MaterialModel";
import { reactive } from "vue";

export default function useTable(){
  //列表查询参数
  const listParm = reactive<ListParm>({
    name:'',
    currentPage:1,
    pageSize:10,
    total:10
  })
  //搜索
  const searchBtn = ()=>{

  }
  //重置
  const resetBtn = ()=>{

  }
  return{
    listParm,
    searchBtn,
    resetBtn
  }
}
```

- useMaterial.ts

```
import { ref } from "vue"
export default function useMaterial(){
  //新增弹框的ref属性
  const addRef = ref<{show:()=>void}>()
  //新增
  const addBtn = ()=>{
    addRef.value?.show()
  }
  //编辑
  const editBtn = ()=>{

  }
  //删除
```

```

const deleteBtn = ()=>{

}

return{
  addBtn,
  etidBtn,
  deleteBtn,
  addRef
}
}

```

12.3.1.5 新增MaterialList.vue组件和AddMaterial.vue组件

在views/material目录下面创建两个组件。

- AddMaterial.vue

```

<template>
  <SysDialog
    :title="dialog.title"
    :height="dialog.height"
    :width="dialog.width"
    :visible="dialog.visible"
    @onConfirm="commit"
    @onClose="onClose"
  >
    <template v-slot:content>
      <el-form
        :model="addModel"
        ref="addFormRef"
        :rules="rules"
        label-width="80px"
        size="default"
      >
        <el-form-item prop="name" label="器材名称">
          <el-input v-model="addModel.name"></el-input>
        </el-form-item>
        <el-form-item prop="numTotal" label="器材数量">
          <el-input v-model="addModel.numTotal"></el-input>
        </el-form-item>
        <el-form-item prop="details" label="简介">
          <el-input type="textarea" v-model="addModel.details"
        />
      </el-form-item>
    </el-form>

```



```

    </template>
  </SysDialog>
</template>

<script setup lang="ts">
import SysDialog from "@/components/SysDialog.vue";
import useDialog from "@/hooks/useDialog";
import { MaterialType } from "@api/material/MaterialModel";
import { reactive, ref } from "vue";
import { ElMessage, FormInstance } from "element-plus";
import { addApi } from "@api/material/index";
//表单的ref属性
const addFormRef = ref<FormInstance>();
//弹框属性
const { dialog, onClose, onConfirm, onShow } = useDialog();
//定义方法给外部使用
const show = () => {
  onShow();
  //清空表单
  addFormRef.value?.resetFields()
};
//暴露出去给外地调用
defineExpose({
  show,
});
//弹框绑定的对象
const addModel = reactive<MaterialType>({
  type: "",
  name: "",
  numTotal: 0,
  details: "",
  id: "",
});
const validateNum = (rule: any, value: any, callback: any) => {
  if (value === 0 || value < 0) {
    callback(new Error("请填写数量"));
  } else {
    callback();
  }
};
//表单验证规则
const rules = reactive({
  name: [
    {
      required: true,

```

```

        message: "请输入器材名称",
        trigger: "blur",
      },
    ],
    numTotal: [
      {
        required: true,
        validator: validateNum,
        trigger: "blur",
      },
    ],
  });
  //表单提交
  const commit = () => {
    addFormRef.value?.validate(async (valid) => {
      if (valid) {
        let res = await addApi(addModel);
        if (res && res.code == 200) {
          //信息提示
          ElMessage.success(res.msg)
          //刷新表格
          onClose();
        }
      }
    });
  };
</script>

<style scoped></style>

```

- MaterialList.vue

```

<template>
  <el-main>
    <!-- 搜索栏 -->
    <el-form :model="listParm" :inline="true" size="default">
      <el-form-item>
        <el-input
          v-model="listParm.name"
          placeholder="请输入器材名称"
        ></el-input>
      </el-form-item>
      <el-form-item>
        <el-button :icon="Search" @click="searchBtn">搜索</el-
button>

```

```

        <el-button :icon="Close" type="danger" plain
@click="resetBtn">重置</el-button>
        <el-button :icon="Plus" type="primary" @click="addBtn">
新增</el-button>
    </el-form-item>
</el-form>
<!-- 新增、编辑 -->
    <AddMaterial ref="addRef"></AddMaterial>
</el-main>
</template>

<script setup lang="ts">
import AddMaterial from "./AddMaterial.vue";
import { Plus, Edit, Delete, Search, Close } from "@element-
plus/icons-vue";
import useTable from "@/composables/material/useTable";
import useMaterial from "@/composables/material/useMaterial";
//表格
const { listParm ,searchBtn,resetBtn} = useTable();
//新增、编辑
const {addBtn,etidBtn,deleteBtn,addRef} = useMaterial()
</script>

<style scoped></style>

```

12.3.2 设备列表展示数据对接

12.3.2.1 api/material/index.ts添加getListApi()方法

```

import http from "@/http";
import { MaterialType ,ListParm} from "./MaterialModel";
//新增
export const addApi = (parm:MaterialType)=>{
    return http.post('/api/material',parm)
}
//列表
export const getListApi = (parm:ListParm)=>{
    return http.get("/api/material/list",parm)
}

```

12.3.2.2 composables/material/useTable.ts修改为如下

```

import { ListParm } from "@/api/material/MaterialModel";
import { nextTick, onMounted, reactive, ref } from "vue";
import { getListApi } from "@/api/material";

```

```
export default function useTable() {
  //表格高度
  const tableHeight = ref(0)
  //定义表格数据
  const tableData = reactive({
    list:[]
  })
  //列表查询参数
  const listParm = reactive<ListParm>({
    name: '',
    currentPage: 1,
    pageSize: 10,
    total: 10
  })
  //获取表格数据
  const getList = async () => {
    let res = await getListApi(listParm)
    if(res && res.code == 200){
      console.log(res)
      tableData.list = res.data.records;
      listParm.total = res.data.total;
    }
  }
  //搜索
  const searchBtn = () => {
    getList()
  }
  //重置
  const resetBtn = () => {
    listParm.name = ''
    listParm.currentPage = 1
    getList()
  }
  //页容量改变时触发
  const sizeChange = (size:number)=>{
    listParm.pageSize = size;
    getList()
  }
  //页数改变时触发
  const currentChange = (page:number)=>{
    listParm.currentPage = page;
    getList()
  }
  //刷新列表
  const reFresh = ()=>{
```

```

        getList()
    }
    onMounted(()=>{
        nextTick(()=>{
            tableHeight.value = window.innerHeight - 230
        })
        getList()
    })
    return {
        listParm,
        searchBtn,
        resetBtn,
        tableData,
        sizeChange,
        currentChange,
        tableHeight,
        reFresh
    }
}

```

12.3.2.3 MaterialList.vue组件

```

<template>
  <el-main>
    <!-- 搜索栏 -->
    <el-form :model="listParm" :inline="true" size="default">
      <el-form-item>
        <el-input
          v-model="listParm.name"
          placeholder="请输入器材名称"
        ></el-input>
      </el-form-item>
      <el-form-item>
        <el-button :icon="Search" @click="searchBtn">搜索</el-
button>
        <el-button :icon="Close" type="danger" plain
@click="resetBtn">重置</el-button>
        <el-button :icon="Plus" type="primary" @click="addBtn">新
增</el-button>
      </el-form-item>
    </el-form>
    <!-- 表格 -->
    <el-table :height="tableHeight" :data="tableData.list" border
stripe>

```

```

        <el-table-column prop="name" label="器材名称"></el-table-
column>
        <el-table-column prop="numTotal" label="器材数量"></el-table-
column>
        <el-table-column label="操作" align="center" width="220">
          <template #default="scope">
            <el-button
              type="primary"
              :icon="Edit"
              size="default"
              @click="editBtn(scope.row)"
            >编辑</el-button>
          >
            <el-button
              type="danger"
              :icon="Delete"
              size="default"
              @click="deleteBtn(scope.row)"
            >删除</el-button>
          >
        </template>
      </el-table-column>
    </el-table>
    <!-- 分页 -->
    <el-pagination
      @size-change="sizeChange"
      @current-change="currentChange"
      :current-page.sync="listParm.currentPage"
      :page-sizes="[10,20, 40, 80, 100]"
      :page-size="listParm.pageSize"
      layout="total, sizes, prev, pager, next, jumper"
      :total="listParm.total" background>
    </el-pagination>

    <!-- 新增、编辑 -->
    <AddMaterial ref="addRef" @reFresh="reFresh"></AddMaterial>
  </el-main>
</template>

<script setup lang="ts">
import AddMaterial from "./AddMaterial.vue";
import { Plus, Edit, Delete, Search, Close } from "@element-
plus/icons-vue";
import useTable from "@/composables/material/useTable";
import useMaterial from "@/composables/material/useMaterial";

```

```
//表格
const { listParm
,searchBtn,resetBtn,tableData,sizeChange,currentChange,tableHeight,reFresh} = useTable();
//新增、编辑
const {addBtn,editBtn,deleteBtn,addRef} = useMaterial()
</script>

<style scoped></style>
```

12.3.2.4 AddMaterial.vue组件

```
<template>
  <SysDialog
    :title="dialog.title"
    :height="dialog.height"
    :width="dialog.width"
    :visible="dialog.visible"
    @onConfirm="commit"
    @onClose="onClose"
  >
    <template v-slot:content>
      <el-form
        :model="addModel"
        ref="addFormRef"
        :rules="rules"
        label-width="80px"
        size="default"
      >
        <el-form-item prop="name" label="器材名称">
          <el-input v-model="addModel.name"></el-input>
        </el-form-item>
        <el-form-item prop="numTotal" label="器材数量">
          <el-input v-model="addModel.numTotal"></el-input>
        </el-form-item>
        <el-form-item prop="details" label="简介">
          <el-input type="textarea" v-model="addModel.details" />
        </el-form-item>
      </el-form>
    </template>
  </SysDialog>
</template>

<script setup lang="ts">
```

```

import SysDialog from "@/components/SysDialog.vue";
import useDialog from "@/hooks/useDialog";
import { MaterialType } from "@/api/material/MaterialModel";
import { reactive, ref } from "vue";
import { ElMessage, FormInstance } from "element-plus";
import { addApi } from "@/api/material/index";
//表单的ref属性
const addFormRef = ref<FormInstance>();
//弹框属性
const { dialog, onClose, onConfirm, onShow } = useDialog();
//定义方法给外部使用
const show = () => {
  onShow();
  //清空表单
  addFormRef.value?.resetFields()
};
//暴露出去给外地调用
defineExpose({
  show,
});
//弹框绑定的对象
const addModel = reactive<MaterialType>({
  type: "",
  name: "",
  numTotal: 0,
  details: "",
  id: "",
});
const validateNum = (rule: any, value: any, callback: any) => {
  if (value === 0 || value < 0) {
    callback(new Error("请填写数量"));
  } else {
    callback();
  }
};
//表单验证规则
const rules = reactive({
  name: [
    {
      required: true,
      message: "请输入器材名称",
      trigger: "blur",
    },
  ],
  numTotal: [

```



```

    {
      required: true,
      validator: validateNum,
      trigger: "blur",
    },
  ],
});
//注册事件
const emits = defineEmits(['reFresh'])
//表单提交
const commit = () => {
  addFormRef.value?.validate(async (valid) => {
    if (valid) {
      let res = await addApi(addModel);
      if (res && res.code == 200) {
        emits('reFresh')
        //信息提示
        ElMessage.success(res.msg)
        //刷新表格
        onClose();
      }
    }
  });
};
</script>

<style scoped></style>

```

12.3.3 器材编辑、删除接口对接

12.3.3.1 api/material/index.ts添加editApi()和deleteApi()方法

```

import http from "@/http";
import { MaterialType, ListParm } from "../MaterialModel";
//新增
export const addApi = (parm:MaterialType)=>{
  return http.post('/api/material',parm)
}
//列表
export const getListApi = (parm:ListParm)=>{
  return http.get("/api/material/list",parm)
}
//编辑
export const editApi = (parm:MaterialType)=>{
  return http.put('/api/material',parm)
}

```

```

}
//删除
export const deleteApi = (id:string)=>{
  return http.delete(`/api/material/${id}`)
}

```

12.3.3.2 composables/material下的useMaterial.ts改为如下

```

import { MaterialType } from "@api/material/MaterialModel"
import { EditType } from "@type/BaseEnum"
import { ref } from "vue"
import { deleteApi } from "@api/material"
import useInstance from "@hooks/useInstance"
import { ElMessage } from "element-plus"
import { FuncList } from "@type/BaseType"
export default function useMaterial(getList:FuncList) {
  const { global } = useInstance()
  //新增弹框的ref属性
  const addRef = ref<{ show: (type: string, row?: MaterialType)
=> void }>()
  //新增
  const addBtn = () => {
    addRef.value?.show(EditType.ADD)
  }
  //编辑
  const editBtn = (row: MaterialType) => {
    addRef.value?.show(EditType.EDIT, row)
  }
  //删除
  const deleteBtn = async(row: MaterialType) => {
    const confirm = await global.$myconfirm('确定删除该数据吗?')
    if(confirm){
      let res = await deleteApi(row.id)
      if(res && res.code == 200){
        ElMessage.success(res.msg)
        getList()
      }
    }
  }
  return {
    addBtn,
    editBtn,
    deleteBtn,
    addRef
  }
}

```

```
}
```

12.3.3.3 AddMaterial.vue组件

```
<template>
  <SysDialog
    :title="dialog.title"
    :height="dialog.height"
    :width="dialog.width"
    :visible="dialog.visible"
    @onConfirm="commit"
    @onClose="onClose"
  >
    <template v-slot:content>
      <el-form
        :model="addModel"
        ref="addFormRef"
        :rules="rules"
        label-width="80px"
        size="default"
      >
        <el-form-item prop="name" label="器材名称">
          <el-input v-model="addModel.name"></el-input>
        </el-form-item>
        <el-form-item prop="numTotal" label="器材数量">
          <el-input v-model="addModel.numTotal"></el-input>
        </el-form-item>
        <el-form-item prop="details" label="简介">
          <el-input type="textarea" v-model="addModel.details" />
        </el-form-item>
      </el-form>
    </template>
  </SysDialog>
</template>

<script setup lang="ts">
import SysDialog from "@components/SysDialog.vue";
import useDialog from "@hooks/useDialog";
import { MaterialType } from "@api/material/MaterialModel";
import { nextTick, reactive, ref } from "vue";
import { ElMessage, FormInstance } from "element-plus";
import { addApi, editApi } from "@api/material/index";
import { EditType, Title } from "@type/BaseEnum";
import useInstance from "@hooks/useInstance";
const { global } = useInstance();
```

```

//表单的ref属性
const addFormRef = ref<FormInstance>();
//弹框属性
const { dialog, onClose, onConfirm, onShow } = useDialog();
//定义方法给外部使用
const show = (type: string, row?: MaterialType) => {
  //设置弹框属性
  type == EditType.ADD
    ? (dialog.title = Title.ADD)
    : (dialog.title = Title.EDIT);
  //如果是编辑，设置数据回显
  if (type == EditType.EDIT) {
    //把当前编辑的数据复制到表单对象
    nextTick(() => {
      global.$objCoppay(row, addModel);
    });
  }
  onShow();
  //清空表单
  addFormRef.value?.resetFields();
  addModel.type = type;
};
//暴露出去给外地调用
defineExpose({
  show,
});
//弹框绑定的对象
const addModel = reactive<MaterialType>({
  type: "",
  name: "",
  numTotal: 0,
  details: "",
  id: "",
});
const validateNum = (rule: any, value: any, callback: any) => {
  if (value === 0 || value < 0) {
    callback(new Error("请填写数量"));
  } else {
    callback();
  }
};
//表单验证规则
const rules = reactive({
  name: [
    {

```

```

        required: true,
        message: "请输入器材名称",
        trigger: "blur",
    },
],
numTotal: [
    {
        required: true,
        validator: validateNum,
        trigger: "blur",
    },
],
});
//注册事件
const emits = defineEmits(["reFresh"]);
//表单提交
const commit = () => {
    addFormRef.value?.validate(async (valid) => {
        if (valid) {
            let res = null;
            if (addModel.type == EditType.ADD) {
                res = await addApi(addModel);
            } else {
                res = await editApi(addModel);
            }
            if (res && res.code == 200) {
                emits("reFresh");
                //信息提示
                ElMessage.success(res.msg);
                //刷新表格
                onClose();
            }
        }
    });
};
</script>

<style scoped></style>

```

13 商品模块功能开发

13.1 数据表设计

```
create table goods
(
    goods_id          int not null auto_increment comment '商品
id',
    name              varchar(200) comment '名称',
    price              decimal(18,2) comment '价格',
    store              int comment '库存',
    image              varchar(220) comment '图片',
    details            text comment '描述',
    unit               varchar(36) comment '单位',
    specs              varchar(64) comment '规格',
    primary key (goods_id)
);
```

13.2 商品管理模块后台代码实现

13.2.1 新建实体类

```
import com.baomidou.mybatisplus.annotation.TableName;
import lombok.Data;

import java.math.BigDecimal;

@Data
@TableName("goods")
public class Goods {
    @TableId(type = IdType.AUTO)
    private Long goodsId;
    private String name;
    private String image;
    private String details;
    private String unit;
    private String specs;
    private BigDecimal price;
    private Integer store;
}
```

```
package com.xq.web.goods.entity;

import lombok.Data;

@Data
public class GoodsParm {
    private Long currentPage;
    private Long pageSize;
    private String name;
}
```

13.2.2 创建mapper接口和映射文件

```
package com.xq.web.goods.mapper;

import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.xq.web.goods.entity.Goods;

public interface GoodsMapper extends BaseMapper<Goods> {
}
```

```
<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="com.xq.web.goods.mapper.GoodsMapper">

</mapper>
```

13.2.3 创建service接口及实现类

```
package com.xq.web.goods.service;

import com.baomidou.mybatisplus.extension.service.IService;
import com.xq.web.goods.entity.Goods;

public interface GoodsService extends IService<Goods> {
}
```

```

package com.xq.web.goods.service.impl;

import
com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
import com.xq.web.goods.entity.Goods;
import com.xq.web.goods.mapper.GoodsMapper;
import com.xq.web.goods.service.GoodsService;
import org.springframework.stereotype.Service;

@Service
public class GoodsServiceImpl extends ServiceImpl<GoodsMapper,
Goods> implements GoodsService {
}

```

13.2.4 创建Controller

```

package com.xq.web.goods.controller;

import
com.baomidou.mybatisplus.core.conditions.query.QueryWrapper;
import com.baomidou.mybatisplus.core.metadata.IPage;
import
com.baomidou.mybatisplus.extension.plugins.pagination.Page;
import com.xq.utils.ResultUtils;
import com.xq.utils.ResultVo;
import com.xq.web.goods.entity.Goods;
import com.xq.web.goods.entity.GoodsParm;
import com.xq.web.goods.service.GoodsService;
import org.apache.commons.lang.StringUtils;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/goods")
public class GoodsController {
    @Autowired
    private GoodsService goodsService;

    //新增
    @PostMapping
    public ResultVo add(@RequestBody Goods goods){
        if(goodsService.save(goods)){
            return ResultUtils.success("新增成功");
        }
    }
}

```



```

    }
    return ResultUtils.error("新增失败!");
}

//编辑
@PutMapping
public ResultVo edit(@RequestBody Goods goods){
    if(goodsService.updateById(goods)){
        return ResultUtils.success("编辑成功");
    }
    return ResultUtils.error("编辑失败!");
}

//删除
@DeleteMapping("/{goodsId}")
public ResultVo delete(@PathVariable("goodsId") Long goodsId)
{
    if(goodsService.removeById(goodsId)){
        return ResultUtils.success("删除成功");
    }
    return ResultUtils.error("删除失败!");
}

//列表
@GetMapping("/list")
public ResultVo list(GoodsParm parm){
    //构造分页对象
    IPage<Goods> page = new Page<>
(parm.getCurrentPage(),parm.getPageSize());
    //构造查询条件
    QueryWrapper<Goods> query = new QueryWrapper<>();
    if(StringUtils.isEmpty(parm.getName())){
        query.lambda().like(Goods::getName,parm.getName());
    }
    IPage<Goods> list = goodsService.page(page, query);
    return ResultUtils.success("查询成功",list);
}
}

```

13.3 商品管理模块前端代码实现

13.3.1 新增商品页面制作与数据对接

13.3.1.1 修改MenuBar.vue组件

```
{
  path: "/goodsRoot",
  component: "Layout",
  name: "goodsRoot",
  meta: {
    title: "商品管理",
    icon: "Calendar",
    roles: ["sys:goodsRoot"],
  },
  children: [
    {
      path: "/goodsList",
      component: "/goods/GoodsList",
      name: "goodsList",
      meta: {
        title: "商品列表",
        icon: "UserFilled",
        roles: ["sys:goodsList"],
      },
    },
  ],
},
```

13.3.1.2 修改路由文件

```
{
  path: "/goodsRoot",
  component: Layout,
  name: "goodsRoot",
  meta: {
    title: "商品管理",
    icon: "Document",
    roles: ["sys:goodsRoot"],
  },
  children: [
    {
      path: "/goodsList",
      component: () => import('@/views/goods/GoodsList.vue'),
      name: "goodsList",
    },
  ],
}
```

```

        meta: {
            title: "商品列表",
            icon: "UserFilled",
            roles: ["sys:goodsList"],
        },
    },
]
},

```

13.3.1.3 api下新建GoodsModel.ts和index.ts

- GoodsModel.ts

```

//列表查询参数类型
export type GoodsParm = {
    currentPage:number,
    pageSize:number,
    name:string
}
//商品数据类型
export type GoodsType = {
    type:string,
    goodsId:string,
    name:string,
    image:string,
    details:string,
    unit:string,
    specs:string,
    price:number,
    store:number,
}

```

- index.ts

```

import { GoodsType } from "../GoodsModel"
import http from "@/http"
//新增
export const addApi = (parm:GoodsType)=>{
    return http.post('/api/goods',parm)
}

```

13.3.1.4 composables下新建goods目录，然后新建useGoods.ts和useTable.ts

- useGoods.ts

```
import { ref } from "vue"

export default function useGoods(){
  const addRef = ref<{show:()=>void}>()
  //新增
  const addBtn = ()=>{
    addRef.value?.show()
  }
  //编辑
  const editBtn = ()=>{

  }
  //删除
  const deleteBtn = ()=>{

  }
  return{
    addBtn,
    editBtn,
    deleteBtn,
    addRef
  }
}
```

- useTable.ts

```
import { GoodsParm } from "@api/goods/GoodsModel";
import { reactive } from "vue";

export default function useTable(){
  //列表查询的参数
  const listParm = reactive<GoodsParm>({
    pageSize:10,
    currentPage:1,
    name:''
  })
  //搜索
  const searchBtn = ()=>{

  }
  //重置
```

```

const resetBtn = ()=>{

}

return{
  listParm,
  searchBtn,
  resetBtn
}
}

```

13.3.1.5 创建GoodsList.vue和AddGoods.vue组件

- AddGoods.vue

```

<template>
  <SysDialog
    :title="dialog.title"
    :width="dialog.width"
    :height="dialog.height"
    :visible="dialog.visible"
    @onClose="onClose"
    @onConfirm="commit"
  >
    <template v-slot:content>
      <el-form
        :model="addModel"
        ref="addFormRef"
        :rules="rules"
        label-width="80px"
        size="default"
      >
        <el-row>
          <el-col :span="12" :offset="0">
            <el-form-item prop="name" label="商品名称">
              <el-input v-model="addModel.name"></el-input>
            </el-form-item>
          </el-col>
          <el-col :span="12" :offset="0">
            <el-form-item prop="price" label="商品价格">
              <el-input v-model="addModel.price"></el-input>
            </el-form-item>
          </el-col>
        </el-row>
        <el-row>
          <el-col :span="12" :offset="0">

```

```

        <el-form-item prop="unit" label="单位">
            <el-input v-model="addModel.unit"></el-input>
        </el-form-item>
    </el-col>
    <el-col :span="12" :offset="0">
        <el-form-item prop="specs" label="规格">
            <el-input v-model="addModel.specs"></el-input>
        </el-form-item>
    </el-col>
</el-row>
<el-row>
    <el-col :span="12" :offset="0">
        <el-form-item prop="store" label="库存">
            <el-input v-model="addModel.store"></el-input>
        </el-form-item>
    </el-col>
</el-row>
<el-form-item prop="image" label="商品图片">
    <el-upload
        ref="uploadRef"
        action="#"
        :on-change="uploadFile"
        list-type="picture-card"
        :auto-upload="false"
        :file-list="fileList"
        :limit="1"
    >
        <el-icon><Plus /></el-icon>

    <template #file="{ file }">
        <div>
            
            <span class="el-upload-list__item-actions">
                <span
                    class="el-upload-list__item-preview"
                    @click="handlePictureCardPreview(file)"
                >
                    <el-icon><zoom-in /></el-icon>
                </span>
                <span
                    v-if="!disabled"

```

```

        class="el-upload-list__item-delete"
        @click="handleRemove(file)"
      >
        <el-icon><Delete /></el-icon>
      </span>
    </span>
  </div>
</template>
</el-upload>

<el-dialog v-model="dialogVisible">
  
</el-dialog>
</el-form-item>
<el-form-item prop="details" label="商品详情">
  <div style="border: 1px solid #ccc">
    <Toolbar
      style="border-bottom: 1px solid #ccc"
      :editor="editorRef"
      :defaultConfig="toolbarConfig"
      :mode="mode"
    />
    <Editor
      style="height: 300px; overflow-y: hidden"
      v-model="valueHtml"
      :defaultConfig="editorConfig"
      :mode="mode"
      @onCreated="handleCreated"
    />
  </div>
</el-form-item>
</el-form>
</template>
</SysDialog>
</template>

<script setup lang="ts">
import "@wangeditor/editor/dist/css/style.css"; // 引入 css
import { Editor, Toolbar } from "@wangeditor/editor-for-vue";
import { GoodsType } from "@api/goods/GoodsModel";
import SysDialog from "@components/SysDialog.vue";
import useDialog from "@hooks/useDialog";
import { ElMessage, FormInstance } from "element-plus";
import { reactive, ref } from "vue";

```

```

import useUpload from "@/composables/course/useUpload";
import useEditor from "@/composables/course/useEditor";
import { addApi } from "@/api/goods/index";
//图片上传
const {
  uploadRef,
  dialogImageUrl,
  dialogVisible,
  disabled,
  handleRemove,
  handlePictureCardPreview,
  fileList,
  uploadFile,
  imgurl,
} = useUpload();
//文本编辑器
const {
  editorRef,
  handleCreated,
  mode,
  editorConfig,
  valueHtml,
  toolbarConfig,
} = useEditor();
//表单ref属性
const addFormRef = ref<FormInstance>();
//弹框属性
const { dialog, onClose, onConfirm, onShow } = useDialog();
//定义外部使用的方法
const show = () => {
  dialog.width = 900;
  dialog.height = 500;
  onShow();
};
//暴露出去
defineExpose({
  show,
});
//表单绑定的对象
const addModel = reactive<GoodsType>({
  type: "",
  goodsId: "",
  name: "",
  image: "",
  details: "",

```



```
    unit: "",
    specs: "",
    price: 0,
    store: 0,
  });
  const validatePrice = (rule: any, value: any, callback: any) =>
  {
    if (value === 0 || value < 0) {
      callback(new Error("请填写商品价格"));
    } else {
      callback();
    }
  };
  const validateStore = (rule: any, value: any, callback: any) =>
  {
    if (value === 0 || value < 0) {
      callback(new Error("请填写商品库存"));
    } else {
      callback();
    }
  };
  // 表单验证规则
  const rules = reactive({
    name: [
      {
        required: true,
        trigger: "blur",
        message: "请输入商品名称",
      },
    ],
    image: [
      {
        required: true,
        trigger: "blur",
        message: "请上传商品图片",
      },
    ],
    unit: [
      {
        required: true,
        trigger: "blur",
        message: "请填写单位",
      },
    ],
    details: [
```

```

        {
            required: true,
            trigger: "blur",
            message: "请填写商品详情",
        },
    ],
    specs: [
        {
            required: true,
            trigger: "blur",
            message: "请填写商品规格",
        },
    ],
    price: [
        {
            required: true,
            validator: validatePrice,
            trigger: "blur",
        },
    ],
    store: [
        {
            required: true,
            validator: validateStore,
            trigger: "blur",
        },
    ],
});
//提交表单
const commit = () => {
    //封面图地址
    addModel.image = imgUrl.value;
    //商品详情
    addModel.details = valueHtml.value;
    addFormRef.value?.validate(async (valid) => {
        if (valid) {
            let res = await addApi(addModel);
            if (res && res.code == 200) {
                ElMessage.success(res.msg);
                onClose();
            }
        }
    });
};
</script>

```

```
<style scoped></style>
```

- **GoodsList.vue**

```
<template>
  <el-main>
    <!-- 搜索栏 -->
    <el-form :model="listParm" :inline="true" size="default">
      <el-form-item>
        <el-input v-model="listParm.name" label="请输入商品名称">
</el-input>
      </el-form-item>
      <el-form-item>
        <el-button :icon="Search" @click="searchBtn">搜索</el-
button>
        <el-button :icon="Close" type="danger" plain
@click="resetBtn">重置</el-button>
        <el-button :icon="Plus" type="primary" @click="addBtn"
>新增</el-button>
      </el-form-item>
    </el-form>
    <!-- 新增弹框 -->
    <AddGoods ref="addRef"></AddGoods>
  </el-main>
</template>

<script setup lang="ts">
import AddGoods from "../AddGoods.vue";
import { Plus, Edit, Delete, Search, Close } from "@element-
plus/icons-vue";
import useTable from "@composables/goods/useTable";
import useGoods from "@composables/goods/useGoods";
// 表格
const { listParm, searchBtn, resetBtn } = useTable();
// 新增、编辑
const { addBtn, editBtn, deleteBtn, addRef } = useGoods();
</script>

<style scoped></style>
```

13.3.2 商品列表页面制作与对接

13.3.2.1 api/goods添加listApi()方法

```
import { GoodsParm, GoodsType } from "../GoodsModel"
import http from "@/http"
//新增
export const addApi = (parm:GoodsType)=>{
  return http.post('/api/goods',parm)
}
//列表
export const listApi = (parm:GoodsParm)=>{
  return http.get("/api/goods/list",parm)
}
```

13.3.2.2 composables/goods的useTable.ts改为如下

```
import { GoodsParm } from "@/api/goods/GoodsModel";
import { nextTick, onMounted, reactive, ref } from "vue";
import { listApi } from "@/api/goods";
export default function useTable(){
  //表格高度
  const tableHeight = ref(0)
  //表格数据
  const tableData = reactive({
    list:[]
  })
  //列表查询的参数
  const listParm = reactive<GoodsParm>({
    pageSize:10,
    currentPage:1,
    name:'',
    total:0
  })
  //商品列表
  const getList = async()=>{
    let res = await listApi(listParm)
    if(res && res.code == 200){
      console.log(res)
      //设置表格数据
      tableData.list = res.data.records;
      //设置分页总数
      listParm.total = res.data.total
    }
  }
}
```

```

//搜索
const searchBtn = ()=>{
  getList()
}
//重置
const resetBtn = ()=>{
  listParm.name = ''
  listParm.currentPage = 1
  getList()
}
//页容量改变触发
const sizeChange = (size:number)=>{
  listParm.pageSize = size;
  getList()
}
//页数改变触发
const currentChange = (page:number)=>{
  listParm.currentPage =page;
  getList()
}
//刷新表格
const reFresh = ()=>{
  getList()
}
onMounted(()=>{
  nextTick(()=>{
    tableHeight.value = window.innerHeight - 230
  })
  getList()
})
return{
  listParm,
  searchBtn,
  resetBtn,
  tableData,
  getList,
  sizeChange,
  currentChange,
  tableHeight,
  reFresh
}
}

```

13.3.2.3 调整GoodsList.vue组件

```
<template>
  <el-main>
    <!-- 搜索栏 -->
    <el-form :model="listParm" :inline="true" size="default">
      <el-form-item>
        <el-input v-model="listParm.name" label="请输入商品名称">
</el-input>
      </el-form-item>
      <el-form-item>
        <el-button :icon="Search" @click="searchBtn">搜索</el-
button>
        <el-button :icon="Close" type="danger" plain
@click="resetBtn">重置</el-button>
        <el-button :icon="Plus" type="primary" @click="addBtn"
>新增</el-button
      >
    </el-form-item>
  </el-form>
  <!-- 表格 -->
  <el-table :height="tableHeight" :data="tableData.list" border
stripe>
    <el-table-column prop="image" width="90" label="商品图片"
align="center">
      <template #default="scope">
        <el-image
          style="width: 60px; height: 60px; border-radius: 50%"
          :src="scope.row.image"
        ></el-image>
      </template>
    </el-table-column>
    <el-table-column prop="name" label="商品名称"></el-table-
column>
    <el-table-column prop="price" label="商品价格"></el-table-
column>
    <el-table-column prop="store" label="商品库存"></el-table-
column>
    <el-table-column prop="unit" label="单位"></el-table-column>
    <el-table-column prop="specs" label="规格"></el-table-
column>
    <el-table-column label="操作" align="center" width="220">
      <template #default="scope">
        <el-button
          type="primary"

```

```

        :icon="Edit"
        size="default"
        @click="editBtn(scope.row)"
      >编辑</el-button>
    >
    <el-button
      type="danger"
      :icon="Delete"
      size="default"
      @click="deleteBtn(scope.row)"
    >删除</el-button>
  >
</template>
</el-table-column>
</el-table>
<!-- 分页 -->
<el-pagination
  @size-change="sizeChange"
  @current-change="currentChange"
  :current-page.sync="listParm.currentPage"
  :page-sizes="[10,20, 40, 80, 100]"
  :page-size="listParm.pageSize"
  layout="total, sizes, prev, pager, next, jumper"
  :total="listParm.total" background>
</el-pagination>

<!-- 新增弹框 -->
<AddGoods ref="addRef" @reFresh="reFresh"></AddGoods>
</el-main>
</template>

<script setup lang="ts">
import AddGoods from "../AddGoods.vue";
import { Plus, Edit, Delete, Search, Close } from "@element-plus/icons-vue";
import useTable from "@/composables/goods/useTable";
import useGoods from "@/composables/goods/useGoods";

//表格
const { tableHeight, listParm, searchBtn,
resetBtn, getList, tableData, sizeChange, currentChange, reFresh } =
useTable();
//新增、编辑
const { addBtn, editBtn, deleteBtn, addRef } = useGoods();
</script>

```

```
<style scoped></style>
```

13.3.2.4 调整AddGoods.vue组件

```
<template>
  <SysDialog
    :title="dialog.title"
    :width="dialog.width"
    :height="dialog.height"
    :visible="dialog.visible"
    @onClose="onClose"
    @onConfirm="commit"
  >
    <template v-slot:content>
      <el-form
        :model="addModel"
        ref="addFormRef"
        :rules="rules"
        label-width="80px"
        size="default"
      >
        <el-row>
          <el-col :span="12" :offset="0">
            <el-form-item prop="name" label="商品名称">
              <el-input v-model="addModel.name"></el-input>
            </el-form-item>
          </el-col>
          <el-col :span="12" :offset="0">
            <el-form-item prop="price" label="商品价格">
              <el-input v-model="addModel.price"></el-input>
            </el-form-item>
          </el-col>
        </el-row>
        <el-row>
          <el-col :span="12" :offset="0">
            <el-form-item prop="unit" label="单位">
              <el-input v-model="addModel.unit"></el-input>
            </el-form-item>
          </el-col>
          <el-col :span="12" :offset="0">
            <el-form-item prop="specs" label="规格">
              <el-input v-model="addModel.specs"></el-input>
            </el-form-item>
          </el-col>
        </el-row>
      </el-form>
    </template>
  </SysDialog>
</template>
```



```

</el-row>
<el-row>
  <el-col :span="12" :offset="0">
    <el-form-item prop="store" label="库存">
      <el-input v-model="addModel.store"></el-input>
    </el-form-item>
  </el-col>
</el-row>
<el-form-item prop="image" label="商品图片">
  <el-upload
    ref="uploadRef"
    action="#"
    :on-change="uploadFile"
    list-type="picture-card"
    :auto-upload="false"
    :file-list="fileList"
    :limit="1"
  >
    <el-icon><Plus /></el-icon>

    <template #file="{ file }">
      <div>
        
        <span class="el-upload-list__item-actions">
          <span
            class="el-upload-list__item-preview"
            @click="handlePictureCardPreview(file)"
          >
            <el-icon><zoom-in /></el-icon>
          </span>
          <span
            v-if="!disabled"
            class="el-upload-list__item-delete"
            @click="handleRemove(file)"
          >
            <el-icon><Delete /></el-icon>
          </span>
        </span>
      </div>
    </template>
  </el-upload>

```

```

        <el-dialog v-model="dialogVisible">
            
        </el-dialog>
    </el-form-item>
    <el-form-item prop="details" label="商品详情">
        <div style="border: 1px solid #ccc">
            <Toolbar
                style="border-bottom: 1px solid #ccc"
                :editor="editorRef"
                :defaultConfig="toolbarConfig"
                :mode="mode"
            />
            <Editor
                style="height: 300px; overflow-y: hidden"
                v-model="valueHtml"
                :defaultConfig="editorConfig"
                :mode="mode"
                @onCreated="handleCreated"
            />
        </div>
    </el-form-item>
</el-form>
</template>
</SysDialog>
</template>

<script setup lang="ts">
import "@wangeditor/editor/dist/css/style.css"; // 引入 css
import { Editor, Toolbar } from "@wangeditor/editor-for-vue";
import { GoodsType } from "@/api/goods/GoodsModel";
import SysDialog from "@/components/SysDialog.vue";
import useDialog from "@/hooks/useDialog";
import { ElMessage, FormInstance } from "element-plus";
import { reactive, ref } from "vue";
import useUpload from "@/composables/course/useUpload";
import useEditor from "@/composables/course/useEditor";
import { addApi } from "@/api/goods/index";
// 图片上传
const {
    dialogImageUrl,
    dialogVisible,
    disabled,
    handleRemove,

```

```

    handlePictureCardPreview,
    uploadFile,
    uploadRef,
    imgUrl,
    fileList,
  } = useUpload();
  //文本编辑器
  const {
    editorRef,
    handleCreated,
    mode,
    editorConfig,
    valueHtml,
    toolbarConfig,
  } = useEditor();
  //表单ref属性
  const addFormRef = ref<FormInstance>();
  //弹框属性
  const { dialog, onClose, onConfirm, onShow } = useDialog();
  //定义外部使用的方法
  const show = () => {
    dialog.width = 900;
    dialog.height = 500;
    onShow();
  };
  //暴露出去
  defineExpose({
    show,
  });
  //表单绑定的对象
  const addModel = reactive<GoodsType>({
    type: "",
    goodsId: "",
    name: "",
    image: "",
    details: "",
    unit: "",
    specs: "",
    price: 0,
    store: 0,
  });
  const validatePrice = (rule: any, value: any, callback: any) =>
  {
    if (value === 0 || value < 0) {
      callback(new Error("请填写商品价格"));
    }
  }

```

```
    } else {
      callback();
    }
  };
const validateStore = (rule: any, value: any, callback: any) =>
{
  if (value === 0 || value < 0) {
    callback(new Error("请填写商品库存"));
  } else {
    callback();
  }
};
//表单验证规则
const rules = reactive({
  name: [
    {
      required: true,
      trigger: "blur",
      message: "请输入商品名称",
    },
  ],
  image: [
    {
      required: true,
      trigger: "blur",
      message: "请上传商品图片",
    },
  ],
  unit: [
    {
      required: true,
      trigger: "blur",
      message: "请填写单位",
    },
  ],
  details: [
    {
      required: true,
      trigger: "blur",
      message: "请填写商品详情",
    },
  ],
  specs: [
    {
      required: true,
```

```

        trigger: "blur",
        message: "请填写商品规格",
    },
],
price: [
    {
        required: true,
        validator: validatePrice,
        trigger: "blur",
    },
],
store: [
    {
        required: true,
        validator: validateStore,
        trigger: "blur",
    },
],
});
//注册事件
const emits = defineEmits(['reFresh'])
//提交表单
const commit = () => {
    //封面图地址
    addModel.image = imgUrl.value;
    //商品详情
    addModel.details = valueHtml.value;
    addFormRef.value?.validate(async (valid) => {
        if (valid) {
            let res = await addApi(addModel);
            if (res && res.code == 200) {
                ElMessage.success(res.msg);
                emits('reFresh')
                onClose();
            }
        }
    });
};
</script>

<style scoped></style>

```

13.3.3 商品编辑、删除接口开发

13.3.3.1 api/goods/index.ts添加editApi()和deleteApi()

```
//编辑
export const editApi = (parm:GoodsType)=>{
  return http.put('/api/goods',parm)
}
//删除
export const deleteApi = (goodsId:string)=>{
  return http.delete(`/api/goods/${goodsId}`)
}
```

13.3.3.2 修改composables/goods的用useGoods.ts

```
import { GoodsType } from "@api/goods/GoodsModel"
import { EditType } from "@type/BaseEnum"
import { ref } from "vue"
import { deleteApi } from "@api/goods"
import useInstance from "@hooks/useInstance"
import { ElMessage } from "element-plus"
import { FuncList } from "@type/BaseType"
export default function useGoods(getList:FuncList){
  const {global} = useInstance()
  const addRef = ref<{show:(type:string,row?:GoodsType)=>void}>()
  //新增
  const addBtn = ()=>{
    addRef.value?.show(EditType.ADD)
  }
  //编辑
  const editBtn = (row:GoodsType)=>{
    addRef.value?.show(EditType.EDIT,row)
  }
  //删除
  const deleteBtn = async(row:GoodsType)=>{
    const confirm = await global.$myconfirm('确定删除该数据吗?')
    if(confirm){
      let res = await deleteApi(row.goodsId)
      if(res && res.code == 200){
        ElMessage.success(res.msg)
        getList()
      }
    }
  }
}
```

```

    return{
      addBtn,
      editBtn,
      deleteBtn,
      addRef
    }
  }
}

```

13.3.3.3 调整AddGoods.vue组件

```

<template>
  <SysDialog
    :title="dialog.title"
    :width="dialog.width"
    :height="dialog.height"
    :visible="dialog.visible"
    @onClose="onClose"
    @onConfirm="commit"
  >
    <template v-slot:content>
      <el-form
        :model="addModel"
        ref="addFormRef"
        :rules="rules"
        label-width="80px"
        size="default"
      >
        <el-row>
          <el-col :span="12" :offset="0">
            <el-form-item prop="name" label="商品名称">
              <el-input v-model="addModel.name"></el-input>
            </el-form-item>
          </el-col>
          <el-col :span="12" :offset="0">
            <el-form-item prop="price" label="商品价格">
              <el-input v-model="addModel.price"></el-input>
            </el-form-item>
          </el-col>
        </el-row>
        <el-row>
          <el-col :span="12" :offset="0">
            <el-form-item prop="unit" label="单位">
              <el-input v-model="addModel.unit"></el-input>
            </el-form-item>
          </el-col>
        </el-row>
      </el-form>
    </template>
  </SysDialog>
</template>

```

```

    <el-col :span="12" :offset="0">
      <el-form-item prop="specs" label="规格">
        <el-input v-model="addModel.specs"></el-input>
      </el-form-item>
    </el-col>
  </el-row>
  <el-row>
    <el-col :span="12" :offset="0">
      <el-form-item prop="store" label="库存">
        <el-input v-model="addModel.store"></el-input>
      </el-form-item>
    </el-col>
  </el-row>
  <el-form-item prop="image" label="商品图片">
    <el-upload
      ref="uploadRef"
      action="#"
      :on-change="uploadFile"
      list-type="picture-card"
      :auto-upload="false"
      :file-list="fileList"
      :limit="1"
    >
      <el-icon><Plus /></el-icon>

      <template #file="{ file }">
        <div>
          
          <span class="el-upload-list__item-actions">
            <span
              class="el-upload-list__item-preview"
              @click="handlePictureCardPreview(file)"
            >
              <el-icon><zoom-in /></el-icon>
            </span>
            <span
              v-if="!disabled"
              class="el-upload-list__item-delete"
              @click="handleRemove(file)"
            >
              <el-icon><Delete /></el-icon>
            </span>
          </span>
        </div>
      </template>
    </el-upload>
  </el-form-item>

```



```

        </span>
      </span>
    </div>
  </template>
</el-upload>

<el-dialog v-model="dialogVisible">
  
</el-dialog>
</el-form-item>
<el-form-item prop="details" label="商品详情">
  <div style="border: 1px solid #ccc">
    <Toolbar
      style="border-bottom: 1px solid #ccc"
      :editor="editorRef"
      :defaultConfig="toolbarConfig"
      :mode="mode"
    />
    <Editor
      style="height: 300px; overflow-y: hidden"
      v-model="valueHtml"
      :defaultConfig="editorConfig"
      :mode="mode"
      @onCreated="handleCreated"
    />
  </div>
</el-form-item>
</el-form>
</template>
</SysDialog>
</template>

<script setup lang="ts">
import "@wangeditor/editor/dist/css/style.css"; // 引入 css
import { Editor, Toolbar } from "@wangeditor/editor-for-vue";
import { GoodsType } from "@api/goods/GoodsModel";
import SysDialog from "@components/SysDialog.vue";
import useDialog from "@hooks/useDialog";
import { ElMessage, FormInstance } from "element-plus";
import { nextTick, reactive, ref } from "vue";
import useUpload from "@composables/course/useUpload";
import useEditor from "@composables/course/useEditor";
import { addApi, editApi } from "@api/goods/index";
import { EditType, Title } from "@type/BaseEnum";

```

```
import useInstance from "@/hooks/useInstance";
const { global } = useInstance();
//图片上传
const {
  dialogImageUrl,
  dialogVisible,
  disabled,
  handleRemove,
  handlePictureCardPreview,
  uploadFile,
  uploadRef,
  imgUrl,
  fileList,
} = useUpload();
//文本编辑器
const {
  editorRef,
  handleCreated,
  mode,
  editorConfig,
  valueHtml,
  toolbarConfig,
} = useEditor();
//表单ref属性
const addFormRef = ref<FormInstance>();
//弹框属性
const { dialog, onClose, onConfirm, onShow } = useDialog();
//定义外部使用的方法
const show = (type: string, row?: GoodsType) => {
  //清空图片数据
  fileList.value = [];
  //设置弹框属性
  type == EditType.ADD
    ? (dialog.title = Title.ADD)
    : (dialog.title = Title.EDIT);
  dialog.width = 900;
  dialog.height = 500;
  //清空图片和文本编辑器
  const upload = uploadRef.value;
  if (upload) {
    upload.clearFiles();
  }
  // addModel.image = "";
  if (type == EditType.ADD) {
    const editor = editorRef.value;
```

```

        if (editor) {
            editor.clear();
        }
    }
    // addModel.details = "";
    //编辑数据回显
    if (type == EditType.EDIT) {
        nextTick(() => {
            global.$objCopy(row, addModel);
            //图片回显
            if (row?.image) {
                //图片回显
                let img = {
                    name: "",
                    url: "",
                };
                imgUrl.value = addModel.image
                img.url = addModel.image;
                fileList.value.push(img);
            }
        });
    }
    if (row && row.details) {
        //文本编辑器的回显
        valueHtml.value = row.details;
    }
    onShow();
    addFormRef.value?.resetFields()
    addModel.type = type;
};
//暴露出去
defineExpose({
    show,
});
//表单绑定的对象
const addModel = reactive<GoodsType>({
    type: "",
    goodsId: "",
    name: "",
    image: "",
    details: "",
    unit: "",
    specs: "",
    price: 0,
    store: 0,

```

```
});  
const validatePrice = (rule: any, value: any, callback: any) =>  
{  
  if (value === 0 || value < 0) {  
    callback(new Error("请填写商品价格"));  
  } else {  
    callback();  
  }  
};  
const validateStore = (rule: any, value: any, callback: any) =>  
{  
  if (value === 0 || value < 0) {  
    callback(new Error("请填写商品库存"));  
  } else {  
    callback();  
  }  
};  
//表单验证规则  
const rules = reactive({  
  name: [  
    {  
      required: true,  
      trigger: "blur",  
      message: "请输入商品名称",  
    },  
  ],  
  image: [  
    {  
      required: true,  
      trigger: "blur",  
      message: "请上传商品图片",  
    },  
  ],  
  unit: [  
    {  
      required: true,  
      trigger: "blur",  
      message: "请填写单位",  
    },  
  ],  
  details: [  
    {  
      required: true,  
      trigger: "blur",  
      message: "请填写商品详情",  
    },  
  ],  
});
```

```

    },
  ],
  specs: [
    {
      required: true,
      trigger: "blur",
      message: "请填写商品规格",
    },
  ],
  price: [
    {
      required: true,
      validator: validatePrice,
      trigger: "blur",
    },
  ],
  store: [
    {
      required: true,
      validator: validateStore,
      trigger: "blur",
    },
  ],
});
//注册事件
const emits = defineEmits(["reFresh"]);
//提交表单
const commit = () => {
  // if(addModel.type == EditType.ADD){

  // }
  //封面图地址
  addModel.image = imgUrl.value;
  //商品详情
  addModel.details = valueHtml.value;
  addFormRef.value?.validate(async (valid) => {
    if (valid) {
      let res = null;
      if (addModel.type == EditType.ADD) {
        res = await addApi(addModel);
      } else {
        res = await editApi(addModel);
      }
      if (res && res.code == 200) {
        ElMessage.success(res.msg);
      }
    }
  });

```

```

        emits("reFresh");
        onClose();
    }
}
});
};
</script>

```

```
<style scoped></style>
```

13.3.3.4 调整GoodsList.vue组件

```

<template>
  <el-main>
    <!-- 搜索栏 -->
    <el-form :model="listParm" :inline="true" size="default">
      <el-form-item>
        <el-input v-model="listParm.name" label="请输入商品名称">
</el-input>
      </el-form-item>
      <el-form-item>
        <el-button :icon="Search" @click="searchBtn">搜索</el-
button>
        <el-button :icon="Close" type="danger" plain
@click="resetBtn"
>重置</el-button>
      </el-form-item>
      <el-button
v-permission="['sys:goodsList:add']"
:icon="Plus"
type="primary"
@click="addBtn"
>新增</el-button>
    </el-form>
    <!-- 表格 -->
    <el-table :height="tableHeight" :data="tableData.list" border
stripe>
      <el-table-column prop="image" width="90" label="商品图片"
align="center">
        <template #default="scope">
          <el-image
            style="width: 60px; height: 60px; border-radius: 50%"
            :src="scope.row.image"

```

```

        ></el-image>
    </template>
</el-table-column>
<el-table-column prop="name" label="商品名称"></el-table-
column>
<el-table-column prop="price" label="商品价格"></el-table-
column>
<el-table-column prop="store" label="商品库存"></el-table-
column>
<el-table-column prop="unit" label="单位"></el-table-column>
<el-table-column prop="specs" label="规格"></el-table-
column>
<el-table-column label="操作" align="center" width="220">
    <template #default="scope">
        <el-button
            v-permission="['sys:goodsList:edit']"
            type="primary"
            :icon="Edit"
            size="default"
            @click="editBtn(scope.row)"
        >编辑</el-button>
    >
    <el-button
        v-permission="['sys:goodsList:delete']"
        type="danger"
        :icon="Delete"
        size="default"
        @click="deleteBtn(scope.row)"
    >删除</el-button>
    >
    </template>
</el-table-column>
</el-table>
<!-- 分页 -->
<el-pagination
    @size-change="sizeChange"
    @current-change="currentChange"
    :current-page.sync="listParm.currentPage"
    :page-sizes="[10, 20, 40, 80, 100]"
    :page-size="listParm.pageSize"
    layout="total, sizes, prev, pager, next, jumper"
    :total="listParm.total"
    background
>
</el-pagination>

```

```
      <!-- 新增弹框 -->
      <AddGoods ref="addRef" @reFresh="reFresh"></AddGoods>
    </el-main>
  </template>

  <script setup lang="ts">
    import AddGoods from "../AddGoods.vue";
    import { Plus, Edit, Delete, Search, Close } from "@element-plus/icons-vue";
    import useTable from "@/composables/goods/useTable";
    import useGoods from "@/composables/goods/useGoods";

    //表格
    const {
      tableHeight,
      listParm,
      searchBtn,
      resetBtn,
      getList,
      tableData,
      sizeChange,
      currentChange,
      reFresh,
    } = useTable();
    //新增、编辑
    const { addBtn, editBtn, deleteBtn, addRef } = useGoods(getList);
  </script>

  <style scoped></style>
```

14 失物招领模块功能设计

14.1 数据表设计


```

create table lost
(
    lost_id          int not null auto_increment comment '主
键',
    lost_name        varchar(64) comment '名称',
    found_time       varchar(36) comment '捡到时间',
    found_addres     varchar(128) comment '捡到地址',
    found_person     varchar(36) comment '捡到人',
    found_phone      varchar(15) comment '捡到人电话',
    status           varchar(2) comment '认领状态 0: 未认领 1: 已
认领',
    lost_person      varchar(36) comment '认领人',
    primary key (lost_id)
);

```

14.2 失物招领管理模块后台接口开发

14.2.1 创建实体类

```

package com.xq.web.lost.entity;

import com.baomidou.mybatisplus.annotation.IdType;
import com.baomidou.mybatisplus.annotation.TableId;
import com.baomidou.mybatisplus.annotation.TableName;
import lombok.Data;

@Data
@TableName("lost")
public class Lost {
    @TableId(type = IdType.AUTO)
    private Long lostId;
    private String lostName;
    private String foundTime;
    private String foundAddres;
    private String foundPerson;
    private String foundPhone;
    private String status;
    private String lostPerson;
}

```

```

package com.xq.web.lost.entity;

import lombok.Data;

@Data
public class LostParm {
    private Long currentPage;
    private Long pageSize;
    private String lostName;
}

```

14.2.2 创建mapper接口和映射文件

```

package com.xq.web.lost.mapper;

import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.xq.web.lost.entity.Lost;

public interface LostMapper extends BaseMapper<Lost> {
}

```

```

<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="com.xq.web.lost.mapper.LostMapper">

</mapper>

```

14.2.3 创建业务接口和实现类

```

package com.xq.web.lost.service;

import com.baomidou.mybatisplus.extension.service.IService;
import com.xq.web.lost.entity.Lost;

public interface LostService extends IService<Lost> {
}

```

业务接口实现类：

```

package com.xq.web.lost.service.impl;

import
com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
import com.xq.web.lost.entity.Lost;
import com.xq.web.lost.mapper.LostMapper;
import com.xq.web.lost.service.LostService;
import org.springframework.stereotype.Service;

@Service
public class LostServiceImpl extends ServiceImpl<LostMapper,
Lost> implements LostService {
}

```

14.2.4 创建controller

```

package com.xq.web.lost.controller;

import
com.baomidou.mybatisplus.core.conditions.query.QueryWrapper;
import com.baomidou.mybatisplus.core.metadata.IPage;
import
com.baomidou.mybatisplus.extension.plugins.pagination.Page;
import com.xq.utils.ResultUtils;
import com.xq.utils.ResultVo;
import com.xq.web.lost.entity.Lost;
import com.xq.web.lost.entity.LostParm;
import com.xq.web.lost.service.LostService;
import org.apache.commons.lang.StringUtils;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/lost")
public class LostController {
    @Autowired
    private LostService lostService;

    //新增
    @PostMapping
    public ResultVo add(@RequestBody Lost lost){
        if(lostService.save(lost)){
            return ResultUtils.success("插入成功!");
        }
    }
}

```

```

        return ResultUtils.error("插入失败!");
    }

    //编辑
    @PutMapping
    public ResultVo edit(@RequestBody Lost lost){
        if(lostService.updateById(lost)){
            return ResultUtils.success("编辑成功!");
        }
        return ResultUtils.error("编辑失败!");
    }

    //删除
    @DeleteMapping("/{lostId}")
    public ResultVo delete(@PathVariable("lostId") Long lostId){
        if(lostService.removeById(lostId)){
            return ResultUtils.success("删除成功!");
        }
        return ResultUtils.error("删除失败!");
    }

    //列表查询
    @GetMapping("/list")
    public ResultVo list(LostParm parm){
        //构造分页对象
        IPage<Lost> page = new Page<>
(parm.getCurrentPage(),parm.getPageSize());
        //构造查询条件
        QueryWrapper<Lost> query = new QueryWrapper<>();
        if(StringUtils.isEmpty(parm.getLostName())){

            query.lambda().like(Lost::getLostName,parm.getLostName());
        }
        IPage<Lost> list = lostService.page(page, query);
        return ResultUtils.success("查询成功",list);
    }
}

```

14.3 失物招领模块前端功能开发

14.3.1 新增失物招领页面制作及数据对接

14.3.1.1 修改MenuBar.vue组件

```
{
  path: "/lostRoot",
  component: "Layout",
  name: "lostRoot",
  meta: {
    title: "失物招领",
    icon: "Document",
    roles: ["sys:lostRoot"],
  },
  children: [
    {
      path: "/lostList",
      component: "/goods/LostList",
      name: "lostList",
      meta: {
        title: "失物列表",
        icon: "UserFilled",
        roles: ["sys:lostList"],
      },
    },
  ],
},
```

14.3.1.2 修改路由文件

```
{
  path: "/lostRoot",
  component: Layout,
  name: "lostRoot",
  meta: {
    title: "失物招领",
    icon: "Document",
    roles: ["sys:lostRoot"],
  },
  children: [
    {
      path: "/lostList",
      component: () => import('@/views/lost/LostList.vue'),
      name: "lostList",
      meta: {
        title: "失物列表",
```

```

        icon: "UserFilled",
        roles: ["sys:lostList"],
      },
    }
  ]
},

```

14.3.1.3 api下新建lost目录，然后新建LostModel.ts和index.ts

- **LostModel.ts**

```

//列表参数类型
export type LostParm = {
  currentPage:number,
  pageSize:number,
  lostName:string,
  total:number
}
//失物数据类型
export type LostType = {
  type:string,
  lostId:string,
  lostName:string,
  foundTime:string,
  foundAddres:string,
  foundPerson:string,
  foundPhone:string,
  status:string,
  lostPerson:string,
}

```

- **index.ts**

```

import http from "@/http";
import { LostType } from "./LostModel";
//新增
export const addApi = (parm:LostType)=>{
  return http.post('/api/lost',parm)
}

```

14.3.1.4 composables下新建lost目录，然后新建useLost.ts和useTable.ts

- useLost.ts

```
import { ref } from "vue"

export default function useLost() {
  const addRef = ref<{show:()=>void}>()
  //新增
  const addBtn = () => {
    addRef.value?.show()
  }
  //编辑
  const editBtn = () => {

  }
  //删除
  const deleteBtn = () => {

  }
  const lostBtn = ()=>{

  }
  return {
    addBtn,
    editBtn,
    deleteBtn,
    lostBtn,
    addRef
  }
}
```

- useTable.ts

```
import { reactive } from "vue";
import { LostParm } from '@api/lost/LostModel'
export default function useTable() {
  //列表参数
  const listParm = reactive<LostParm>({
    currentPage:1,
    pageSize:10,
    lostName:''
  })
  //重置
  const resetBtn = ()=>{
```

```

    }
    //搜索
    const searchBtn = ()=>{

    }
    return{
        listParm,
        resetBtn,
        searchBtn
    }
}

```

14.3.1.5 views目录下新建lost目录，然后新建AddLost.vue和LostList.vue组件

- AddLost.vue

```

<template>
  <SysDialog
    :title="dialog.title"
    :height="dialog.height"
    :width="dialog.width"
    :visible="dialog.visible"
    @onConfirm="commit"
    @onClose="onClose"
  >
    <template v-slot:content>
      <el-form
        :model="addModel"
        ref="addFormRef"
        :rules="rules"
        label-width="80px"
        size="default"
      >
        <el-row>
          <el-col :span="12" :offset="0">
            <el-form-item prop="lostName" label="物品名称">
              <el-input v-model="addModel.lostName"></el-input>
            </el-form-item>
          </el-col>
          <el-col :span="12" :offset="0">
            <el-form-item prop="foundTime" label="捡到时间">
              <el-date-picker
                style="width: 100%"
                v-model="addModel.foundTime"
              >

```



```

        type="date"
        placeholder="请选择捡到时间"
        size="default"
        value-format="YYYY-MM-DD"
        format="YYYY-MM-DD"
      />
    </el-form-item>
  </el-col>
</el-row>
<el-row>
  <el-col :span="12" :offset="0">
    <el-form-item prop="foundAddress" label="捡到地址">
      <el-input v-model="addModel.foundAddress"></el-
input>
    </el-form-item>
  </el-col>
  <el-col :span="12" :offset="0">
    <el-form-item prop="foundPerson" label="捡到人">
      <el-input v-model="addModel.foundPerson"></el-
input>
    </el-form-item>
  </el-col>
</el-row>
<el-row>
  <el-col :span="12" :offset="0">
    <el-form-item prop="foundPhone" label="联系电话">
      <el-input v-model="addModel.foundPhone"></el-input>
    </el-form-item>
  </el-col>
  <el-col :span="12" :offset="0">
    <el-form-item prop="status" label="状态">
      <el-radio-group v-model="addModel.status">
        <el-radio :label="'0'">未认领</el-radio>
        <el-radio :label="'1'">已认领</el-radio>
      </el-radio-group>
    </el-form-item>
  </el-col>
</el-row>
</el-form>
</template>
</sysDialog>
</template>

<script setup lang="ts">
import { addApi } from "@api/lost/index";

```

```
import { LostType } from "@/api/lost/LostModel";
import SysDialog from "@/components/SysDialog.vue";
import useDialog from "@/hooks/useDialog";
import { ElMessage, FormInstance } from "element-plus";
import { reactive, ref } from "vue";
const addFormRef = ref<FormInstance>();
//弹框属性
const { dialog, onClose, onConfirm, onShow } = useDialog();
//定义外部使用的方法
const show = () => {
  onShow();
};
defineExpose({
  show,
});
//定义表单的数据对象
const addModel = reactive<LostType>({
  type: "",
  lostId: "",
  lostName: "",
  foundTime: "",
  foundAddres: "",
  foundPerson: "",
  foundPhone: "",
  status: "",
  lostPerson: "",
});
//表单验证规则
const rules = reactive({
  lostName: [
    {
      trigger: "blur",
      message: "请填写物品名称",
      required: true,
    },
  ],
  foundTime: [
    {
      trigger: "blur",
      message: "请填写捡到时间",
      required: true,
    },
  ],
  foundAddres: [
    {
```

```

        trigger: "blur",
        message: "请填写捡到地址",
        required: true,
    },
],
foundPerson: [
    {
        trigger: "blur",
        message: "请填写捡到人",
        required: true,
    },
],
foundPhone: [
    {
        trigger: "blur",
        message: "请填写捡到人电话",
        required: true,
    },
],
status: [
    {
        trigger: "blur",
        message: "请选择认领状态",
        required: true,
    },
],
});
//表单提交
const commit = () => {
    addFormRef.value?.validate(async(valid) => {
        if (valid) {
            let res = await addApi(addModel)
            if(res && res.code == 200){
                //信息提示
                ElMessage.success(res.msg)
                //刷新表格
                onClose();
            }
        }
    });
};
</script>

<style scoped></style>

```

- LostList.vue

```
<template>
  <el-main>
    <!-- 搜索栏 -->
    <el-form :model="listParm" :inline="true" size="default">
      <el-form-item>
        <el-input v-model="listParm.lostName"></el-input>
      </el-form-item>
      <el-form-item>
        <el-button :icon="Search" @click="searchBtn">搜索</el-
button>
        <el-button type="danger" plain :icon="Close"
@click="resetBtn"
        >重置</el-button
        >
        <el-button :icon="Plus" type="primary" @click="addBtn">新
增</el-button>
      </el-form-item>
    </el-form>
    <!-- 新增编辑 -->
    <AddLost ref="addRef"></AddLost>
  </el-main>
</template>

<script setup lang="ts">
import AddLost from "../AddLost.vue";
import useLost from "../../composables/lost/useLost";
import { Plus, Edit, Delete, Search, Close } from "@element-
plus/icons-vue";
import useTable from "../../composables/lost/useTable";
// 表格
const { listParm, searchBtn, resetBtn } = useTable();
// 新增 编辑
const { addBtn, editBtn, deleteBtn, addRef } = useLost();
</script>

<style scoped></style>
```

14.3.2 失物招领列表页面制作与对接

14.3.2.1 api/lost/index.ts添加getListApi()方法

```
import http from "@http";
import { LostType, LostParm } from "../LostModel";
//新增
export const addApi = (parm:LostType)=>{
    return http.post('/api/lost',parm)
}
//列表
export const getListApi = (parm:LostParm)=>{
    return http.get("/api/lost/list",parm)
}
```

14.3.2.2 修改composables/lost/useTable.ts

```
import { nextTick, onMounted, reactive, ref } from "vue";
import { LostParm } from '@api/lost/LostModel'
import { getListApi } from "@api/lost";
export default function useTable() {
    //表格高度
    const tableHeight = ref(0)
    //定义表格数据
    const tableData = reactive({
        list:[]
    })
    //列表参数
    const listParm = reactive<LostParm>({
        currentPage:1,
        pageSize:10,
        lostName:'',
        total:0
    })
    //重置
    const resetBtn = ()=>{
        listParm.lostName = ''
        listParm.currentPage = 1;
        getList()
    }
    //搜索
    const searchBtn = ()=>{
        getList()
    }
    //列表
    const getList = async()=>{
        let res = await getListApi(listParm)
```

```

        if(res && res.code == 200){
            console.log(res)
            tableData.list = res.data.records;
            listParm.total = res.data.total
        }
    }
    //页容量改变时触发
    const sizeChange = (size:number)=>{
        listParm.pageSize = size;
        getList()
    }
    //页数改变时触发
    const currentChange = (page:number)=>{
        listParm.currentPage = page;
        getList()
    }
    //刷新表格
    const reFresh = ()=>{
        getList()
    }
    onMounted(()=>{
        nextTick(()=>{
            tableHeight.value = window.innerHeight - 230
        })
        getList()
    })
    return{
        listParm,
        resetBtn,
        searchBtn,
        getList,
        tableData,
        sizeChange,
        currentChange,
        tableHeight,
        reFresh
    }
}

```

14.3.2.3 修改LostList.vue组件

```

<template>
  <el-main>
    <!-- 搜索栏 -->
    <el-form :model="listParm" :inline="true" size="default">

```

```

    <el-form-item>
      <el-input v-model="listParm.lastName"></el-input>
    </el-form-item>
    <el-form-item>
      <el-button :icon="Search" @click="searchBtn">搜索</el-
button>
      <el-button type="danger" plain :icon="Close"
@click="resetBtn"
      >重置</el-button>
    >
      <el-button :icon="Plus" type="primary" @click="addBtn">新
增</el-button>
    </el-form-item>
  </el-form>
  <!-- 表格 -->
  <el-table :height="tableHeight" :data="tableData.list" border
stripe>
    <el-table-column prop="lastName" label="物品名称"></el-table-
column>
    <el-table-column prop="foundPerson" label="捡到人"></el-
table-column>
    <el-table-column prop="foundAddres" label="捡到地址"></el-
table-column>
    <el-table-column prop="foundTime" label="捡到时间"></el-
table-column>
    <el-table-column prop="foundPhone" label="捡到人电话"></el-
table-column>
    <el-table-column prop="status" label="认领状态">
      <template #default="scope">
        <el-tag v-if="scope.row.status == '1'" type="success"
size="default">已认领</el-tag>
        <el-tag v-if="scope.row.status == '0'" type="danger"
size="default">未认领</el-tag>
      </template>
    </el-table-column>
    <el-table-column label="操作" align="center" width="290">
      <template #default="scope">
        <el-button type="primary" :icon="Edit" size="default"
@click="editBtn(scope.row)">编辑</el-button>
        <el-button type="danger" :icon="Delete" size="default"
@click="deleteBtn(scope.row)">删除</el-button>
        <el-button type="success" :icon="Edit" size="default"
@click="lostBtn(scope.row)">认领</el-button>
      </template>
    </el-table-column>

```

```

</el-table>
<!-- 分页 -->
<el-pagination
  @size-change="sizeChange"
  @current-change="currentChange"
  :current-page.sync="listParm.currentPage"
  :page-sizes="[10,20, 40, 80, 100]"
  :page-size="listParm.pageSize"
  layout="total, sizes, prev, pager, next, jumper"
  :total="listParm.total" background>
</el-pagination>

<!-- 新增编辑 -->
<AddLost ref="addRef" @reFresh="reFresh"></AddLost>
</el-main>
</template>

<script setup lang="ts">
import AddLost from "@/AddLost.vue";
import useLost from "@/composables/lost/useLost";
import { Plus, Edit, Delete, Search, Close } from "@element-plus/icons-vue";
import useTable from "@/composables/lost/useTable";
//表格
const { listParm, searchBtn,
resetBtn,tableData,getList,sizeChange,currentChange
,tableHeight,reFresh} = useTable();
//新增 编辑
const { addBtn, editBtn, deleteBtn,addRef,lostBtn } = useLost();
</script>

<style scoped></style>

```

14.3.3 编辑、删除、认领接口对接

14.3.3.1 api/lost/index.ts添加editApi()和deleteApi()方法

```

import http from "@/http";
import { LostParm, LostType } from "@/LostModel";
//新增
export const addApi = (parm:LostType)=>{
  return http.post('/api/lost',parm)
}
//列表
export const getListApi = (parm:LostParm)=>{

```



```

    return http.get("/api/lost/list", parm)
  }
  //编辑
  export const editApi = (parm: LostType) => {
    return http.put('/api/lost', parm)
  }
  //删除
  export const deleteApi = (lostId: string) => {
    return http.delete(`/api/lost/${lostId}`)
  }

```

14.3.3.2 在composables/lost/useLost.ts添加认领方法

```

import { LostType } from "@api/lost/LostModel"
import { EditType } from "@type/BaseEnum"
import { ref } from "vue"
import { deleteApi } from "@api/lost"
import useInstance from "@hooks/useInstance"
import { ElMessage } from "element-plus"
import { FuncList } from "@type/BaseType"
export default function useLost(getList: FuncList) {
  const { global } = useInstance()
  const addRef = ref<{ show: (type: string, row?: LostType) => void }>()
  const lostPersonRef = ref<{ show: (row: LostType) => void }>()
  ()
  //新增
  const addBtn = () => {
    addRef.value?.show(EditType.ADD)
  }
  //编辑
  const editBtn = (row: LostType) => {
    addRef.value?.show(EditType.EDIT, row)
  }
  //删除
  const deleteBtn = async(row: LostType) => {
    const cofirm = await global.$myconfirm('确定删除该数据吗?')
    if(cofirm){
      let res = await deleteApi(row.lostId)
      if(res && res.code == 200){
        //信息提示
        ElMessage.success(res.msg)
        //刷新表格
        getList()
      }
    }
  }
}

```

```

    }
  }
  //认领
  const lostBtn = (row: LostType) => {
    if(row.status == '1'){
      ElMessage.warning('该物品已经被认领!')
      return;
    }
    lostPersonRef.value?.show(row)
  }
  return {
    addBtn,
    editBtn,
    deleteBtn,
    addRef,
    lostBtn,
    lostPersonRef
  }
}

```

14.3.3.3 修改AddLost.vue组件

```

<template>
  <SysDialog
    :title="dialog.title"
    :height="dialog.height"
    :width="dialog.width"
    :visible="dialog.visible"
    @onConfirm="commit"
    @onClose="onClose"
  >
    <template v-slot:content>
      <el-form
        :model="addModel"
        ref="addFormRef"
        :rules="rules"
        label-width="80px"
        size="default"
      >
        <el-row>
          <el-col :span="12" :offset="0">
            <el-form-item prop="lostName" label="物品名称">
              <el-input v-model="addModel.lostName"></el-input>
            </el-form-item>
          </el-col>

```

```

    <el-col :span="12" :offset="0">
      <el-form-item prop="foundTime" label="捡到时间">
        <el-date-picker
          style="width: 100%"
          v-model="addModel.foundTime"
          type="date"
          placeholder="请选择捡到时间"
          size="default"
          value-format="YYYY-MM-DD"
          format="YYYY-MM-DD"
        />
      </el-form-item>
    </el-col>
  </el-row>
  <el-row>
    <el-col :span="12" :offset="0">
      <el-form-item prop="foundAddress" label="捡到地址">
        <el-input v-model="addModel.foundAddress"></el-
input>
      </el-form-item>
    </el-col>
    <el-col :span="12" :offset="0">
      <el-form-item prop="foundPerson" label="捡到人">
        <el-input v-model="addModel.foundPerson"></el-
input>
      </el-form-item>
    </el-col>
  </el-row>
  <el-row>
    <el-col :span="12" :offset="0">
      <el-form-item prop="foundPhone" label="联系电话">
        <el-input v-model="addModel.foundPhone"></el-input>
      </el-form-item>
    </el-col>
    <el-col :span="12" :offset="0">
      <el-form-item prop="status" label="状态">
        <el-radio-group v-model="addModel.status">
          <el-radio :label="'0'">未认领</el-radio>
          <el-radio :label="'1'">已认领</el-radio>
        </el-radio-group>
      </el-form-item>
    </el-col>
  </el-row>
</el-form>
</template>

```

```

    </SysDialog>
</template>

<script setup lang="ts">
import { addApi, editApi } from "@api/lost/index";
import { LostType } from "@api/lost/LostModel";
import SysDialog from "@components/SysDialog.vue";
import useDialog from "@hooks/useDialog";
import { EditType, Title } from "@type/BaseEnum";
import { ElMessage, FormInstance } from "element-plus";
import { nextTick, reactive, ref } from "vue";
import useInstance from "@hooks/useInstance";
const { global } = useInstance();
const addFormRef = ref<FormInstance>();
//弹框属性
const { dialog, onClose, onConfirm, onShow } = useDialog();
//定义外部使用的方法
const show = (type: string, row?: LostType) => {
    //设置弹框属性
    type == EditType.ADD
        ? (dialog.title = Title.ADD)
        : (dialog.title = Title.EDIT);
    dialog.height = 180;
    //如果是编辑，设置回显的数据
    if (type == EditType.EDIT) {
        nextTick(() => {
            global.$objCopy(row, addModel);
        });
    }
    onShow();
    addModel.type = type;
    //清空表单
    addFormRef.value?.resetFields();
};
defineExpose({
    show,
});
//定义表单的数据对象
const addModel = reactive<LostType>({
    type: "",
    lostId: "",
    lostName: "",
    foundTime: "",
    foundAddres: "",
    foundPerson: "",

```

```
    foundPhone: "",
    status: "",
    lostPerson: "",
  });
  // 表单验证规则
  const rules = reactive({
    lostName: [
      {
        trigger: "blur",
        message: "请填写物品名称",
        required: true,
      },
    ],
    foundTime: [
      {
        trigger: "blur",
        message: "请填写捡到时间",
        required: true,
      },
    ],
    foundAddres: [
      {
        trigger: "blur",
        message: "请填写捡到地址",
        required: true,
      },
    ],
    foundPerson: [
      {
        trigger: "blur",
        message: "请填写捡到人",
        required: true,
      },
    ],
    foundPhone: [
      {
        trigger: "blur",
        message: "请填写捡到人电话",
        required: true,
      },
    ],
    status: [
      {
        trigger: "blur",
        message: "请选择认领状态",
      },
    ],
  });
```

```

        required: true,
      },
    ],
  });
  //注册事件
  const emits = defineEmits(["reFresh"]);
  //表单提交
  const commit = () => {
    addFormRef.value?.validate(async (valid) => {
      if (valid) {
        let res = null;
        if (addModel.type == EditType.ADD) {
          res = await addApi(addModel);
        } else {
          res = await editApi(addModel);
        }
        if (res && res.code == 200) {
          //信息提示
          ElMessage.success(res.msg);
          //刷新表格
          emits("reFresh");
          onClose();
        }
      }
    });
  };
</script>

<style scoped></style>

```

14.3.3.4 添加失物认领组件LostPerson.vue

```

<template>
  <SysDialog
    :title="dialog.title"
    :height="dialog.height"
    :width="dialog.width"
    :visible="dialog.visible"
    @onClose="onClose"
    @onConfirm="commit"
  >
    <template v-slot:content>
      <el-form
        :model="lostPerson"
        ref="lostRef"

```

```

        :rules="rules"
        label-width="80px"
        :inline="false"
        size="default"
      >
        <el-form-item prop="lostPerson" label="认领人">
          <el-input
            v-model="lostPerson.lostPerson"
            placeholder="请输入认领人"
          />
        </el-form-item>
      </el-form>
    </template>
  </SysDialog>
</template>

<script setup lang="ts">
import { LostType } from "@/api/lost/LostModel";
import SysDialog from "@/components/SysDialog.vue";
import useDialog from "@/hooks/useDialog";
import { ElMessage, FormInstance } from "element-plus";
import { reactive, ref } from "vue";
import { editApi } from "@/api/lost";
const lostRef = ref<FormInstance>();
const lostPerson = reactive<LostType>({
  type: "",
  lostId: "",
  lostName: "",
  foundTime: "",
  foundAddres: "",
  foundPerson: "",
  foundPhone: "",
  status: "1",
  lostPerson: "",
});
//弹框属性
const { dialog, onClose, onConfirm, onShow } = useDialog();
//显示弹框
const show = (row:LostType) => {
  dialog.title = "认领";
  dialog.height = 150;
  lostPerson.lostId = row.lostId
  onShow();
  lostRef.value?.resetFields()
};

```

```

defineExpose({
  show,
});
// 表单验证规则
const rules = reactive({
  lostPerson: [
    {
      required: true,
      message: "请录入认领人",
      trigger: "blur",
    },
  ],
});
// 注册事件
const emits = defineEmits(['reFresh'])
// 表单提交
const commit = () => {
  lostRef.value?.validate(async(valid) => {
    if(valid){
      let res = await editApi(lostPerson)
      if(res && res.code == 200){
        ElMessage.success('认领成功')
        emits('reFresh')
        onClose()
      }
    }
  });
};
</script>

<style scoped></style>

```

14.3.3.5 修改LostList.vue组件

```

<template>
  <el-main>
    <!-- 搜索栏 -->
    <el-form :model="listParm" :inline="true" size="default">
      <el-form-item>
        <el-input v-model="listParm.lostName"></el-input>
      </el-form-item>
      <el-form-item>
        <el-button :icon="Search" @click="searchBtn">搜索</el-
button>

```



```

        <el-button type="danger" plain :icon="Close"
@click="resetBtn"
        >重置</el-button>
    >
    <el-button v-permission="['sys:lostList:add']"
:icon="Plus" type="primary" @click="addBtn">新增</el-button>
    </el-form-item>
</el-form>
<!-- 表格 -->
<el-table :height="tableHeight" :data="tableData.list" border
stripe>
    <el-table-column prop="lostName" label="物品名称"></el-table-
column>
    <el-table-column prop="foundPerson" label="捡到人"></el-
table-column>
    <el-table-column prop="foundAddres" label="捡到地址"></el-
table-column>
    <el-table-column prop="foundTime" label="捡到时间"></el-
table-column>
    <el-table-column prop="foundPhone" label="捡到人电话"></el-
table-column>
    <el-table-column prop="lostPerson" label="认领人"></el-
table-column>
    <el-table-column prop="status" label="认领状态">
        <template #default="scope">
            <el-tag v-if="scope.row.status == '1'" type="success"
size="default">已认领</el-tag>
            <el-tag v-if="scope.row.status == '0'" type="danger"
size="default">未认领</el-tag>
        </template>
    </el-table-column>
    <el-table-column label="操作" align="center" width="290">
        <template #default="scope">
            <el-button v-permission="['sys:lostList:edit']"
type="primary" :icon="Edit" size="default"
@click="editBtn(scope.row)">编辑</el-button>
            <el-button v-permission="['sys:lostList:delete']"
type="danger" :icon="Delete" size="default"
@click="deleteBtn(scope.row)">删除</el-button>
            <el-button v-permission="['sys:lostList:get']"
type="success" :icon="Edit" size="default"
@click="lostBtn(scope.row)">认领</el-button>
        </template>
    </el-table-column>
</el-table>

```

```

<!-- 分页 -->
<el-pagination
  @size-change="sizeChange"
  @current-change="currentChange"
  :current-page.sync="listParm.currentPage"
  :page-sizes="[10,20, 40, 80, 100]"
  :page-size="listParm.pageSize"
  layout="total, sizes, prev, pager, next, jumper"
  :total="listParm.total" background>
</el-pagination>

<!-- 新增编辑 -->
<AddLost ref="addRef" @reFresh="reFresh"></AddLost>
<!-- 认领弹框 -->
<LostPerson ref="lostPersonRef" @reFresh="reFresh">
</LostPerson>
</el-main>
</template>

<script setup lang="ts">
import AddLost from "@/AddLost.vue";
import LostPerson from "@/LostPerson.vue";
import useLost from "@/composables/lost/useLost";
import { Plus, Edit, Delete, Search, Close } from "@element-plus/icons-vue";
import useTable from "@/composables/lost/useTable";
import useInstance from "@/hooks/useInstance";
const { global } = useInstance();
//表格
const { listParm, searchBtn,
resetBtn,tableData,getList,sizeChange,currentChange
,tableHeight,reFresh} = useTable();
//新增 编辑
const { addBtn, editBtn, deleteBtn,addRef,lostBtn,lostPersonRef }
= useLost(getList);
</script>

<style scoped></style>

```

15 反馈管理功能设计

15.1 数据表设计

```
create table suggest
(
    id                int not null auto_increment comment '主
键',
    title             varchar(200) comment '标题',
    content           text comment '反馈内容',
    date_time         datetime comment '反馈时间',
    primary key (id)
);
```

15.2 反馈管理后台代码开发

15.2.1 新建实体类

```
package com.xq.web.suggest.entity;

import com.baomidou.mybatisplus.annotation.IdType;
import com.baomidou.mybatisplus.annotation.TableId;
import com.baomidou.mybatisplus.annotation.TableName;
import lombok.Data;

import java.util.Date;

@Data
@TableName("suggest")
public class Suggest {
    @TableId(type = IdType.AUTO)
    private Long id;
    private String title;
    private String content;
    private Date dateTime;
}
```

```
package com.xq.web.suggest.entity;

import lombok.Data;

@Data
public class SuggestParm {
    private Long currentPage;
    private Long pageSize;
    private String title;
}
```

15.2.2 新建mapper接口及映射文件

```
package com.xq.web.suggest.mapper;

import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.xq.web.suggest.entity.Suggest;

public interface SuggestMapper extends BaseMapper<Suggest> {
}
```

```
<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="com.xq.web.suggest.mapper.SuggestMapper">

</mapper>
```

15.2.3 新建service接口及实现类

```
package com.xq.web.suggest.service;

import com.baomidou.mybatisplus.extension.service.IService;
import com.xq.web.suggest.entity.Suggest;

public interface SuggestService extends IService<Suggest> {
}
```

```

package com.xq.web.suggest.service.impl;

import
com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
import com.xq.web.suggest.entity.Suggest;
import com.xq.web.suggest.mapper.SuggestMapper;
import com.xq.web.suggest.service.SuggestService;
import org.springframework.stereotype.Service;

@Service
public class SuggestServiceImpl extends
ServiceImpl<SuggestMapper, Suggest> implements SuggestService {
}

```

15.3.4 创建controller

```

package com.xq.web.suggest.controller;

import
com.baomidou.mybatisplus.core.conditions.query.QueryWrapper;
import com.baomidou.mybatisplus.core.metadata.IPage;
import
com.baomidou.mybatisplus.extension.plugins.pagination.Page;
import com.xq.utils.ResultUtils;
import com.xq.utils.ResultVo;
import com.xq.web.suggest.entity.Suggest;
import com.xq.web.suggest.entity.SuggestParm;
import com.xq.web.suggest.service.SuggestService;
import org.apache.commons.lang.StringUtils;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.Date;

@RestController
@RequestMapping("/api/suggest")
public class SuggestController {
    @Autowired
    private SuggestService suggestService;

    //新增
    @PostMapping
    public ResultVo add(@RequestBody Suggest suggest){
        suggest.setDateTime(new Date());
    }
}

```

```

        if(suggestService.save(suggest)){
            return ResultUtils.success("反馈成功!");
        }
        return ResultUtils.error("反馈失败!");
    }

    //编辑
    @PutMapping
    public ResultVo edit(@RequestBody Suggest suggest){
        if(suggestService.updateById(suggest)){
            return ResultUtils.success("编辑成功!");
        }
        return ResultUtils.error("编辑失败!");
    }

    //删除
    @DeleteMapping("/{id}")
    public ResultVo delete(@PathVariable("id") Long id){
        if(suggestService.removeById(id)){
            return ResultUtils.success("删除成功!");
        }
        return ResultUtils.error("删除失败!");
    }

    //列表
    @GetMapping("/list")
    public ResultVo list(SuggestParm parm){
        //构造分页对象
        IPage<Suggest> page = new Page<>
(parm.getCurrentPage(),parm.getPageSize());
        //构造查询条件
        QueryWrapper<Suggest> query = new QueryWrapper<>();
        if(StringUtils.isNotEmpty(parm.getTitle())){
            query.lambda().like(Suggest::getTitle,parm.getTitle());
        }
        query.lambda().orderByDesc(Suggest::getDateTime);
        IPage<Suggest> list = suggestService.page(page, query);
        return ResultUtils.success("查询成功",list);
    }
}

```

15.3 反馈管理前端代码开发

15.3.1 新增反馈页面制作与对接

15.3.1.1 修改MenuBar.vue组件

```
{
  path: "/suggestRoot",
  component: "Layout",
  name: "suggestRoot",
  meta: {
    title: "反馈管理",
    icon: "Document",
    roles: ["sys:suggestRoot"],
  },
  children: [
    {
      path: "/suggestList",
      component: "/suggest/SuggestList",
      name: "suggestList",
      meta: {
        title: "反馈列表",
        icon: "UserFilled",
        roles: ["sys:suggestList"],
      },
    },
  ],
}
```

15.3.1.2 修改路由文件

```
{
  path: "/suggestRoot",
  component: Layout,
  name: "suggestRoot",
  meta: {
    title: "反馈管理",
    icon: "Document",
    roles: ["sys:suggestRoot"],
  },
  children: [
    {
      path: "/suggestList",
      component: () =>
import('@/views/suggest/SuggestList.vue'),
    },
  ],
}
```

```

        name: "suggestList",
        meta: {
            title: "反馈列表",
            icon: "UserFilled",
            roles: ["sys:suggestList"],
        },
    },
]
}

```

15.3.1.3 api下新建suggest目录，然后新建SuggestModel.ts和index.ts

- SuggestModel.ts

```

//列表查询的参数类型
export type SuggestParm = {
    currentPage:number,
    pageSize:number,
    title:string,
    total:number
}
//表单提交的数据类型
export type SuggestType = {
    type:string,
    id:string,
    title:string,
    content:string
}

```

- index.ts

```

import http from "@/http";
import { SuggestType } from "../SuggestModel";
//新增
export const addApi = (parm:SuggestType)=>{
    return http.post("/api/suggest",parm)
}

```

15.3.1.4 composables下新建suggest目录，然后新建useSuggest.ts和useTable.ts

useSuggest.ts

```

import { ref } from "vue"
export default function useSuggest(){

```



```

const addRef = ref<{show:()=>void}>()
//新增
const addBtn = ()=>{
  addRef.value?.show()
}
//编辑
const editBtn = ()=>{

}
//删除
const deleteBtn = ()=>{

}

return{
  addBtn,
  editBtn,
  deleteBtn,
  addRef
}
}

```

useTable.ts

```

import { SuggestParm } from "@/api/suggest/SuggestModel";
import { reactive } from "vue";

export default function useTable(){
  //列表查询的参数
  const listParm = reactive<SuggestParm>({
    currentPage:1,
    pageSize:10,
    title:'',
    total:0
  })
  //搜索
  const searchBtn = ()=>{

  }
  //重置
  const resetBtn = ()=>{

  }

  return{
    listParm,
    searchBtn,

```

```

        resetBtn
      }
    }
  }
}

```

15.3.1.5 新建AddSuggest.vue组件

```

<template>
  <SysDialog
    :title="dialog.title"
    :height="dialog.height"
    :width="dialog.width"
    :visible="dialog.visible"
    @onClose="onClose"
    @onConfirm="confirm"
  >
    <template v-slot:content>
      <el-form
        :model="addModel"
        ref="addFormRef"
        :rules="rules"
        label-width="80px"
        size="default"
      >
        <el-form-item prop="title" label="标题">
          <el-input v-model="addModel.title"></el-input>
        </el-form-item>
        <el-form-item prop="content" label="内容">
          <el-input type="textarea" v-model="addModel.content">
</el-input>
        </el-form-item>
      </el-form>
    </template>
  </SysDialog>
</template>

<script setup lang="ts">
import { SuggestType } from "@api/suggest/SuggestModel";
import SysDialog from "@components/SysDialog.vue";
import useDialog from "@hooks/useDialog";
import { ElMessage, FormInstance } from "element-plus";
import { reactive, ref } from "vue";
import { addApi } from "@api/suggest/index";
const addFormRef = ref<FormInstance>();
//弹框属性
const { dialog, onClose, onConfirm, onShow } = useDialog();

```

```
//显示弹框
const show = () => {
  dialog.height = 160;
  onShow();
};
defineExpose({
  show,
});
//表单对象
const addModel = reactive<SuggestType>({
  type: "",
  id: "",
  title: "",
  content: "",
});
//表单验证规则
const rules = reactive({
  title: [
    {
      required: true,
      trigger: "blur",
      message: "请输入标题",
    },
  ],
  content: [
    {
      required: true,
      trigger: "blur",
      message: "请输入内容",
    },
  ],
});
//表单提交
const commit = () => {
  addFormRef.value?.validate(async (valid) => {
    if (valid) {
      let res = await addApi(addModel);
      if (res && res.code == 200) {
        ElMessage.success(res.msg)
        onClose();
      }
    }
  });
};
</script>
```

```
<style scoped></style>
```

15.3.1.6 修改SuggestList.vue组件

```
<template>
  <el-main>
    <!-- 搜索栏 -->
    <el-form :model="listParm" :inline="true" size="default">
      <el-form-item>
        <el-input
          v-model="listParm.title"
          placeholder="请输入反馈标题"
        ></el-input>
      </el-form-item>
      <el-form-item>
        <el-button :icon="Search" @click="searchBtn">搜索</el-
button>
        <el-button :icon="Close" type="danger" plain
@click="resetBtn">重置</el-button>
        <el-button :icon="Plus" type="primary" @click="addBtn">新
增</el-button>
      </el-form-item>
    </el-form>
    <!-- 新增编辑 -->
    <AddSuggest ref="addRef"></AddSuggest>
  </el-main>
</template>

<script setup lang="ts">
import AddSuggest from "../AddSuggest.vue";
import { Plus, Edit, Delete, Search, Close } from "@element-
plus/icons-vue";
import useTable from "@composables/suggest/useTable";
import useSuggest from "@composables/suggest/useSuggest";
// 表格
const { listParm, searchBtn, resetBtn } = useTable();
// 新增编辑
const { addBtn, editBtn, deleteBtn, addRef } = useSuggest();
</script>

<style scoped></style>
```

15.3.2 反馈列表页面制作与对接

15.3.2.1 api/suggest/index.ts添加listApi()方法

```
//列表
export const listApi = (parm:SuggestParm)=>{
  return http.get("/api/suggest/list",parm)
}
```

15.3.2.2 修改composables/suggest下的useTable.ts

```
import { SuggestParm } from "@api/suggest/SuggestModel";
import { nextTick, onMounted, reactive, ref } from "vue";
import { listApi } from "@api/suggest";
export default function useTable(){
  //表格高度
  const tableHeight = ref(0)
  //表格数据
  const tableData = reactive({
    list:[]
  })
  //列表查询的参数
  const listParm = reactive<SuggestParm>({
    currentPage:1,
    pageSize:10,
    title:'',
    total:0
  })
  //搜索
  const searchBtn = ()=>{
    getList()
  }
  //重置
  const resetBtn = ()=>{
    listParm.title = ''
    getList()
  }
  //列表
  const getList = async()=>{
    let res = await listApi(listParm)
    if(res && res.code == 200){
      tableData.list = res.data.records;
      listParm.total = res.data.total
    }
  }
}
```

```

//页容量改变时触发
const sizeChange = (size:number)=>{
  listParm.pageSize = size;
  getList()
}
//页数改变时触发
const currentChange = (page:number)=>{
  listParm.currentPage = page
  getList()
}
//刷新
const reFresh = ()=>{
  getList()
}
onMounted(()=>{
  nextTick(()=>{
    tableHeight.value = window.innerHeight - 230
  })
  getList()
})
return{
  listParm,
  searchBtn,
  resetBtn,
  getList,
  tableData,
  sizeChange,
  currentChange,
  tableHeight,
  reFresh
}
}

```

15.3.2.3 修改SuggestList.vue组件

```

<template>
  <el-main>
    <!-- 搜索栏 -->
    <el-form :model="listParm" :inline="true" size="default">
      <el-form-item>
        <el-input
          v-model="listParm.title"
          placeholder="请输入反馈标题"
        ></el-input>
      </el-form-item>
    </el-form>
  </el-main>
</template>

```

```

        <el-form-item>
            <el-button :icon="Search" @click="searchBtn">搜索</el-
button>
            <el-button :icon="Close" type="danger" plain
@click="resetBtn">重置</el-button>
            <el-button :icon="Plus" type="primary" @click="addBtn">新
增</el-button>
        </el-form-item>
    </el-form>
    <!-- 表格 -->
    <el-table :height="tableHeight" :data="tableData.list" border
stripe>
        <el-table-column prop="title" label="标题"></el-table-
column>
        <el-table-column prop="content" label="内容"></el-table-
column>
        <el-table-column label="操作" align="center" width="210">
            <template #default="scope">
                <el-button type="primary" :icon="Edit" size="default"
@click="editBtn(scope.row)">编辑</el-button>
                <el-button type="danger" :icon="Delete" size="default"
@click="deleteBtn(scope.row)">删除</el-button>
            </template>
        </el-table-column>
    </el-table>
    <!-- 分页 -->
    <el-pagination
        @size-change="sizeChange"
        @current-change="currentChange"
        :current-page.sync="listParm.currentPage"
        :page-sizes="[10,20, 40, 80, 100]"
        :page-size="listParm.pageSize"
        layout="total, sizes, prev, pager, next, jumper"
        :total="listParm.total" background>
    </el-pagination>

    <!-- 新增编辑 -->
    <AddSuggest ref="addRef" @reFresh="reFresh"></AddSuggest>
</el-main>
</template>

<script setup lang="ts">
import AddSuggest from "../AddSuggest.vue";
import { Plus, Edit, Delete, Search, Close } from "@element-
plus/icons-vue";

```

```
import useTable from "@/composables/suggest/useTable";
import useSuggest from "@/composables/suggest/useSuggest";
//表格
const { listParm
,searchBtn,resetBtn,getList,tableData,sizeChange,currentChange,tableHeight,refresh} = useTable();
//新增编辑
const { addBtn, editBtn, deleteBtn,addRef } = useSuggest();
</script>

<style scoped></style>
```

15.3.3 反馈编辑、删除对接

15.3.3.1 api/suggest/index.ts添加editApi()和deleteApi()方法

```
//编辑
export const editApi = (parm:SuggestType)=>{
  return http.put('/api/suggest',parm)
}
//删除
export const deleteApi = (id:string)=>{
  return http.delete(`/api/suggest/${id}`)
}
```

15.3.3.2 修改composables/suggest/useSuggest.ts

```
import { SuggestType } from "@/api/suggest/SuggestModel"
import { EditType } from "@/type/BaseEnum"
import { ref } from "vue"
import { deleteApi } from "@/api/suggest"
import useInstance from "@/hooks/useInstance"
import { ElMessage } from "element-plus"
import { FuncList } from "@/type/BaseType"
export default function useSuggest(getList:FuncList) {
  const { global } = useInstance()
  const addRef = ref<{ show: (type: string, row?: SuggestType)
=> void }>()
  //新增
  const addBtn = () => {
    addRef.value?.show(EditType.ADD)
  }
  //编辑
  const editBtn = (row: SuggestType) => {
    addRef.value?.show(EditType.EDIT, row)
  }
}
```



```

    }
    //删除
    const deleteBtn = async(row: SuggestType) => {
      const confrim = await global.$myconfirm('确定删除该数据吗?')
      if(confrim){
        let res = await deleteApi(row.id)
        if(res && res.code == 200){
          ElMessage.success(res.msg)
          getList()
        }
      }
    }
  }
  return {
    addBtn,
    editBtn,
    deleteBtn,
    addRef
  }
}

```

15.3.3.3 修改AddSuggest.vue组件

```

<template>
  <SysDialog
    :title="dialog.title"
    :height="dialog.height"
    :width="dialog.width"
    :visible="dialog.visible"
    @onClose="onClose"
    @onConfirm="commit"
  >
    <template v-slot:content>
      <el-form
        :model="addModel"
        ref="addFormRef"
        :rules="rules"
        label-width="80px"
        size="default"
      >
        <el-form-item prop="title" label="标题">
          <el-input v-model="addModel.title"></el-input>
        </el-form-item>
        <el-form-item prop="content" label="内容">
          <el-input type="textarea" v-model="addModel.content">
</el-input>

```

```

        </el-form-item>
      </el-form>
    </template>
  </SysDialog>
</template>

<script setup lang="ts">
import { SuggestType } from "@api/suggest/SuggestModel";
import SysDialog from "@components/SysDialog.vue";
import useDialog from "@hooks/useDialog";
import { ElMessage, FormInstance } from "element-plus";
import { nextTick, reactive, ref } from "vue";
import { addApi, editApi } from "@api/suggest/index";
import { EditType, Title } from "@type/BaseEnum";
import useInstance from "@hooks/useInstance";
const { global } = useInstance();
const addFormRef = ref<FormInstance>();
//弹框属性
const { dialog, onClose, onConfirm, onShow } = useDialog();
//显示弹框
const show = (type: string, row?: SuggestType) => {
  console.log(type)
  console.log(row)
  dialog.height = 160;
  type == EditType.ADD
    ? (dialog.title = Title.ADD)
    : (dialog.title = Title.EDIT);
  //编辑数据回显
  if (type == EditType.EDIT) {
    nextTick(()=>{
      global.$objCopy(row, addModel);
    })
  }
  onShow();
  addFormRef.value?.resetFields();
  addModel.type = type;
};
defineExpose({
  show,
});
//表单对象
const addModel = reactive<SuggestType>({
  type: "",
  id: "",
  title: "",

```

```

        content: "",
    });
    // 表单验证规则
    const rules = reactive({
        title: [
            {
                required: true,
                trigger: "blur",
                message: "请输入标题",
            },
        ],
        content: [
            {
                required: true,
                trigger: "blur",
                message: "请输入内容",
            },
        ],
    });
    // 注册事件
    const emits = defineEmits(["reFresh"]);
    // 表单提交
    const commit = () => {
        addFormRef.value?.validate(async (valid) => {
            if (valid) {
                let res = null;
                if (addModel.type == EditType.ADD) {
                    res = await addApi(addModel);
                } else {
                    res = await editApi(addModel);
                }
                if (res && res.code == 200) {
                    ElMessage.success(res.msg);
                    emits("reFresh");
                    onClose();
                }
            }
        });
    };
</script>

<style scoped></style>

```

15.3.3.4 修改SuggestList.vue组件

```
<template>
  <el-main>
    <!-- 搜索栏 -->
    <el-form :model="listParm" :inline="true" size="default">
      <el-form-item>
        <el-input
          v-model="listParm.title"
          placeholder="请输入反馈标题"
        ></el-input>
      </el-form-item>
      <el-form-item>
        <el-button :icon="Search" @click="searchBtn">搜索</el-
button>
        <el-button :icon="Close" type="danger" plain
@click="resetBtn">重置</el-button>
        <el-button v-permission="['sys:suggestList:add']"
:icon="Plus" type="primary" @click="addBtn">新增</el-button>
      </el-form-item>
    </el-form>
    <!-- 表格 -->
    <el-table :height="tableHeight" :data="tableData.list" border
stripe>
      <el-table-column prop="title" label="标题"></el-table-
column>
      <el-table-column prop="content" label="内容"></el-table-
column>
      <el-table-column label="操作" align="center" width="210">
        <template #default="scope">
          <el-button v-permission="['sys:suggestList:edit']"
type="primary" :icon="Edit" size="default"
@click="editBtn(scope.row)">编辑</el-button>
          <el-button v-permission="['sys:suggestList:delete']"
type="danger" :icon="Delete" size="default"
@click="deleteBtn(scope.row)">删除</el-button>
        </template>
      </el-table-column>
    </el-table>
    <!-- 分页 -->
    <el-pagination
      @size-change="sizeChange"
      @current-change="currentChange"
      :current-page.sync="listParm.currentPage"
      :page-sizes="[10,20, 40, 80, 100]"
```

```

      :page-size="listParm.pageSize"
      layout="total, sizes, prev, pager, next, jumper"
      :total="listParm.total" background>
    </el-pagination>

    <!-- 新增编辑 -->
    <AddSuggest ref="addRef" @reFresh="reFresh"></AddSuggest>
  </el-main>
</template>

<script setup lang="ts">
import AddSuggest from "../AddSuggest.vue";
import { Plus, Edit, Delete, Search, Close } from "@element-
plus/icons-vue";
import useTable from "@/composables/suggest/useTable";
import useSuggest from "@/composables/suggest/useSuggest";
//表格
const { listParm
, searchBtn, resetBtn, getList, tableData, sizeChange, currentChange, ta
bleHeight, reFresh } = useTable();
//新增编辑
const { addBtn, editBtn, deleteBtn, addRef } =
useSuggest(getList);
</script>

<style scoped></style>

```

16 登录功能实现

16.1 登录页面制作

16.1.1 修改路由文件

在/router/index.ts里面定义如下：

```

{
  path: "/login",
  component: () => import('@views/login/Login.vue'),
  name: "login"
}

```

16.1.2 定义Login.vue

在views/login目录下面定义Login.vue

```
<template>
  <div class="logincontainer">
    <el-form
      class="loginForm"
      :model="loginModel"
      ref="loginForm"
      :rules="rules"
      size="default"
    >
      <el-form-item style="display: flex; justify-content:
center">
        <div class="loginTitle">系统登录</div>
      </el-form-item>
      <el-form-item>
        <el-input
          size="large"
          v-model="loginModel.username"
          placeholder="请输入账号"
        ></el-input>
      </el-form-item>
      <el-form-item>
        <el-input
          size="large"
          v-model="loginModel.password"
          placeholder="请输入密码"
        ></el-input>
      </el-form-item>
      <el-form-item>
        <el-row :gutter="20">
          <el-col :span="16" :offset="0">
            <el-input
              size="large"
              v-model="loginModel.code"
              placeholder="请输入验证码"
            ></el-input>
          </el-col>
          <el-col :span="8" :offset="0">
            <el-input
              size="large"
              v-model="loginModel.code"
              placeholder="验证码"
            ></el-input>
          </el-col>
        </el-row>
      </el-form-item>
    </el-form>
  </div>
</template>
```

```

        ></el-input>
      </el-col>
    </el-row>
  </el-form-item>
  <el-form-item label="用户类型">
    <el-radio-group v-model="loginModel.userType">
      <el-radio :label="1">会员</el-radio>
      <el-radio :label="2">员工</el-radio>
    </el-radio-group>
  </el-form-item>
  <el-form-item>
    <el-row style="width:100%;">
      <el-col :span="12" :offset="0" style="padding-right:
10px; padding-left: 0px">
        <el-button style="width:100%;" size="large"
type="primary" @click="onSubmit">登录</el-button>
      </el-col>
      <el-col :span="12" :offset="0" style="padding-right:
0px; padding-left: 10px">
        <el-button type="danger" plain style="width:100%;"
size="large">重置</el-button>
      </el-col>
    </el-row>
  </el-form-item>
</el-form>
</div>
</template>

```

```

<script setup lang="ts">
  import { reactive } from "vue";

  // 表单对象
  const loginModel = reactive({
    username: "",
    password: "",
    code: "",
  });

  // 表单验证规则
  const rules = reactive({});

  // 登录提交
  const onSubmit = () => {};
</script>

```

```

<style scoped lang="scss">
  .logincontainer {

```

```

        background-color: #fff;
        height: 100%;
        display: flex;
        justify-content: center;
        align-items: center;
        background-image: url("../assets/bg.png");
        background-size: 100% 100%;
        .loginForm {
            border-radius: 10px;
            background-color: #fff;
            width: 430px;
            height: 340px;
            padding: 20px 35px;
            .loginTitle {
                width: 100%;
                font-size: 24px;
                font-weight: 600;
                display: flex;
                justify-content: center;
                align-items: center;
            }
        }
    }
}
</style>

```

16.2 登录验证码生成与数据对接

16.2.1 后端web模块的pom.xml里面引入验证码依赖

```

<!-- 图片验证码 -->
<dependency>
    <groupId>com.github.penggle</groupId>
    <artifactId>kaptcha</artifactId>
</dependency>

```

16.2.2 配置验证码配置类

```

package com.xq.config.imagecode;

import com.google.code.kaptcha.Constants;
import com.google.code.kaptcha.impl.DefaultKaptcha;
import com.google.code.kaptcha.util.Config;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

```



```
import java.util.Properties;

@Configuration
public class ImageConfig {
    @Bean
    public DefaultKaptcha getDefauItKaptcha(){
        DefaultKaptcha defaultKaptcha = new DefaultKaptcha();
        Properties properties = new Properties();
        //验证码是否有边框
        properties.setProperty(Constants.KAPTCHA_BORDER, "yes");
        //边框颜色
        properties.setProperty(Constants.KAPTCHA_BORDER_COLOR,
"105,179,90");
        //字体颜色

        properties.setProperty(Constants.KAPTCHA_TEXTPRODUCER_FONT_COLOR
, "blue");
        //验证码图片宽度
        properties.setProperty(Constants.KAPTCHA_IMAGE_WIDTH,
"200");
        //验证码图片高度
        properties.setProperty(Constants.KAPTCHA_IMAGE_HEIGHT,
"36");
        //生成验证码的字符

        properties.setProperty(Constants.KAPTCHA_TEXTPRODUCER_CHAR_STRIN
G, "0123456789");
        //去掉干扰线

        properties.setProperty(Constants.KAPTCHA_NOISE_IMPL, "com.google.
code.kaptcha.impl.NoNoise");
        //字体大小

        properties.setProperty(Constants.KAPTCHA_TEXTPRODUCER_FONT_SIZE,
"34");
        //字体样式

        properties.setProperty(Constants.KAPTCHA_TEXTPRODUCER_FONT_NAMES
, "楷体");
        //验证码位数

        properties.setProperty(Constants.KAPTCHA_TEXTPRODUCER_CHAR LENGT
H, "4");
        // 图片效果
```

```

        properties.setProperty(Constants.KAPTCHA_OBSCURIFICATOR_IMPL,
            "com.google.code.kaptcha.impl.ShadowGimpy");
        Config config = new Config(properties);
        defaultKaptcha.setConfig(config);
        return defaultKaptcha;
    }
}

```

16.2.3 创建login模块，并建LoginController

```

package com.xq.web.login.entity;

```

```

import lombok.Data;

```

```

@Data

```

```

public class LoginParm {
    private String username;
    private String password;
    private String code;
    private String userType;
}

```

```

package com.xq.web.login.controller;

```

```

import com.google.code.kaptcha.impl.DefaultKaptcha;
import com.xq.utils.ResultVo;
import com.xq.web.login.entity.LoginParm;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import sun.misc.BASE64Encoder;

```

```

import javax.imageio.ImageIO;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpSession;
import java.awt.image.BufferedImage;
import java.io.ByteArrayOutputStream;
import java.io.IOException;

```

```

@RestController
@RequestMapping("/api/login")
public class LoginController {
    @Autowired
    private DefaultKaptcha defaultKaptcha;

    // 生成验证码
    @PostMapping("/image")
    public ResultVo imageCode(HttpServletRequest request) throws
    IOException {
        //获取验证码字符
        String text = defaultKaptcha.createText();
        //存储验证码到session
        HttpSession session = request.getSession();
        session.setAttribute("code", text);
        //生成图片
        BufferedImage bufferedImage =
defaultKaptcha.createImage(text);
        ByteArrayOutputStream outputStream = null;
        try {
            outputStream = new ByteArrayOutputStream();
            ImageIO.write(bufferedImage, "jpg", outputStream);
            BASE64Encoder encoder = new BASE64Encoder();
            String base64 =
encoder.encode(outputStream.toByteArray());
            String captchaBase64 = "data:image/jpeg;base64," +
base64.replaceAll("\r\n", "");
            ResultVo result = new ResultVo("生成成功", 200,
captchaBase64);
            return result;
        } catch (IOException e) {
            e.printStackTrace();
        } finally {
            try {
                if (outputStream != null) {
                    outputStream.close();
                }
            } catch (IOException e) {
                e.printStackTrace();
            }
        }
        return null;
    }

    // 登录

```

```

    @PostMapping("/login")
    public ResultVo login(HttpServletRequest request,
    @RequestBody LoginParm loginParm) {
        HttpSession session = request.getSession();
        String code = (String) session.getAttribute("code");
        return null;
    }
}

```

16.2.4 api下新建login目录，然后新建LoginModel.ts和index.ts

```

//登录参数类型
export type LoginParm = {
    username:string,
    password:string,
    code:string
    userType:string
}

```

```

import http from "@http/index";
import { LoginParm } from "../LoginModel";
//获取验证码
export const getImageApi = ()=>{
    return http.post("/api/login/image")
}
//登录
export const loginApi = (parm:LoginParm)=>{
    return http.post("/api/login/login",parm)
}

```

16.2.5 composables下新建login目录，并建useImage.ts

```

import { onMounted, ref } from "vue";
import { getImageApi } from "@api/login";
export default function useImage(){
    //定义图片src
    const imgSrc = ref()
    //获取图片验证码
    const getImage = async()=>{
        let res = await getImageApi()
        console.log(res)
        if(res && res.code == 200){
            imgSrc.value = res.data;
        }
    }
}

```

```

    }
  }
  onMounted(()=>{
    getImage()
  })
  return{
    imgSrc,
    getImage
  }
}

```

16.2.6 Login.vue页面

```

<template>
  <div class="logincontainer">
    <el-form
      class="loginForm"
      :model="loginModel"
      ref="loginForm"
      :rules="rules"
      size="default"
    >
      <el-form-item style="display: flex; justify-content:
center">
        <div class="loginTitle">系统登录</div>
      </el-form-item>
      <el-form-item>
        <el-input
          size="large"
          v-model="loginModel.username"
          placeholder="请输入账号"
        ></el-input>
      </el-form-item>
      <el-form-item>
        <el-input
          size="large"
          v-model="loginModel.password"
          placeholder="请输入密码"
        ></el-input>
      </el-form-item>
      <el-form-item>
        <el-row :gutter="20">
          <el-col :span="16" :offset="0">
            <el-input
              size="large"

```

```

        v-model="loginModel.code"
        placeholder="请输入验证码"
    ></el-input>
</el-col>
<el-col :span="8" :offset="0">
    
</el-col>
</el-row>
</el-form-item>
<el-form-item label="用户类型">
    <el-radio-group v-model="loginModel.userType">
        <el-radio :label="1">会员</el-radio>
        <el-radio :label="2">员工</el-radio>
    </el-radio-group>
</el-form-item>
<el-form-item>
    <el-row style="width: 100%">
        <el-col
            :span="12"
            :offset="0"
            style="padding-right: 10px; padding-left: 0px"
        >
            <el-button
                style="width: 100%"
                size="large"
                type="primary"
                @click="onSubmit"
            >登录</el-button>
        >
    </el-col>
    <el-col
        :span="12"
        :offset="0"
        style="padding-right: 0px; padding-left: 10px"
    >
        <el-button type="danger" plain style="width: 100%"
size="large"
        >重置</el-button>
    >
    </el-col>
</el-row>
</el-form-item>
</el-form>
</div>
</template>

```

```

<script setup lang="ts">
import { reactive } from "vue";
import useImage from "@/composables/login/useImage";
import { loginApi } from "@/api/login/index";
const { imgSrc, getImage } = useImage();
//表单对象
const loginModel = reactive({
  username: "",
  password: "",
  code: "",
  userType: ''
});
//表单验证规则
const rules = reactive({});
//登录提交
const onSubmit = async () => {
  const res = await loginApi(loginModel);
};
</script>

```

```

<style scoped lang="scss">
.logincontainer {
  background-color: #ffff;
  height: 100%;
  display: flex;
  justify-content: center;
  align-items: center;
  background-image: url("../assets/bg.png");
  background-size: 100% 100%;
  .loginForm {
    border-radius: 10px;
    background-color: #fff;
    width: 430px;
    height: 340px;
    padding: 20px 35px;
    .loginTitle {
      width: 100%;
      font-size: 24px;
      font-weight: 600;
      display: flex;
      justify-content: center;
      align-items: center;
    }
  }
  .image {

```

```
        height: 40px;
        width: 100%;
        display: flex;
        justify-content: center;
        align-items: center;
    }
}
}
</style>
```

此时我们debug测试，发现并没有获取到对应的code，这是因为session不一致导致的，所以我们需要解决session不一致的问题。

16.2.7 解决session不一致的问题

```
//axios配置
const config = {
    //接口请求的地址，真实上线是一个域名
    baseURL: process.env.BASE_API,
    timeout: 10000,
    withCredentials: true //解决session不一致的问题
}
```

16.3 登录接口开发与数据对接

16.3.1 登录后台接口

第一步：配置JWT

在springboot配置文件中定义如下：

```
#jwt配置
jwt:
    #颁发者
    issuer: krisswen
    #密钥
    secret: com.xq
    #30分钟过期
    expiration: 30
```

第二步：定义jwt工具类

```
package com.xq.jwt;
```



```

import com.auth0.jwt.JWT;
import com.auth0.jwt.JWTCreator;
import com.auth0.jwt.algorithms.Algorithm;
import com.auth0.jwt.exceptions.AlgorithmMismatchException;
import com.auth0.jwt.exceptions.JWTVerificationException;
import com.auth0.jwt.exceptions.SignatureVerificationException;
import com.auth0.jwt.exceptions.TokenExpiredException;
import com.auth0.jwt.interfaces.DecodedJWT;
import lombok.Data;
import
org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.stereotype.Component;

import java.util.Calendar;
import java.util.Date;
import java.util.Map;

@Component
@Data
@ConfigurationProperties(prefix = "jwt")
public class JwtUtils {
    //颁发者
    private String issuer;
    //密钥
    private String secret;
    // 过期时间 分钟
    private int expiration;

    /**
     * @param map 自定义参数
     */
    public String generateToken(Map<String, String> map) {
        //设置令牌的过期时间
        Calendar instance = Calendar.getInstance();
        //设置失效时间
        instance.add(Calendar.MINUTE, expiration);
        //创建JWT builder
        JWTCreator.Builder builder = JWT.create();
        //payload
        map.forEach((k, v) -> {
            builder.withClaim(k, v);
        });
    }
}

```

```

        String token =
builder.withIssuer(issuer).withIssuedAt(new
Date()).withExpiresAt(instance.getTime()) //指定令牌过期时间
        .sign(Algorithm.HMAC256(secret));
        return token;
    }

    /**
     * 验证令牌是否合法
     *
     * @param token
     */
    public boolean verify(String token) {
        try {

JWT.require(Algorithm.HMAC256(secret)).build().verify(token);
        } catch (JWTVerificationException e) {
            return false;
        }
        return true;
    }

    /**
     * 解析token
     *
     * @param token
     * @return
     */
    public DecodedJWT jwtDecode(String token) {
        try {
            return
JWT.require(Algorithm.HMAC256(secret)).build().verify(token);
        } catch (SignatureVerificationException e) {
            throw new RuntimeException("token签名错误!");
        } catch (AlgorithmMismatchException e) {
            throw new RuntimeException("token算法不匹配!");
        } catch (TokenExpiredException e) {
            throw new RuntimeException("token过期!");
        } catch (Exception e) {
            throw new RuntimeException("token解析失败!");
        }
    }
}

```

第三步：定义登录的控制器类

```
package com.xq.web.login.controller;

import
com.baomidou.mybatisplus.core.conditions.query.QueryWrapper;
import com.google.code.kaptcha.impl.DefaultKaptcha;
import com.xq.jwt.JwtUtils;
import com.xq.utils.ResultUtils;
import com.xq.utils.ResultVo;
import com.xq.web.login.entity.LoginParam;
import com.xq.web.login.entity.LoginResult;
import com.xq.web.member.entity.Member;
import com.xq.web.member.service.MemberService;
import com.xq.web.sys_user.entity.SysUser;
import com.xq.web.sys_user.service.SysUserService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.util.DigestUtils;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import sun.misc.BASE64Encoder;

import javax.imageio.ImageIO;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpSession;
import java.awt.image.BufferedImage;
import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.util.HashMap;
import java.util.Map;

@RestController
@RequestMapping("/api/login")
public class LoginController {
    @Autowired
    MemberService memberService;

    @Autowired
    SysUserService sysUserService;

    @Autowired
    JwtUtils jwtUtils;

    //登录
    @PostMapping("/login")
```

```

    public ResultVo login(HttpServletRequest request,
@RequestBody LoginParam loginParam){
        //获取sessoion
        HttpSession session = request.getSession();
        //获取验证码
        String code = (String) session.getAttribute("code");
        //验证验证码
        if(!code.equals(loginParam.getCode())){
            return ResultUtils.error("验证码错误!");
        }
        String password =
DigestUtils.md5DigestAsHex(loginParam.getPassword().getBytes());
        //用户类型判断
        if(loginParam.getUserType().equals("1")){ //会员
            //构造查询条件
            QueryWrapper<Member> query = new QueryWrapper<>();

            query.lambda().eq(Member::getUsername,loginParam.getUsername()).
            eq(Member::getPassword,loginParam.getPassword());
            Member one = memberService.getOne(query);
            if(one == null){
                return ResultUtils.error("用户名或密码错误!");
            }
            // 生成token
            Map<String,String> map = new HashMap<>();
            map.put("userId",Long.toString(one.getMemberId()));
            map.put("username",one.getUsername());
            String token = jwtUtils.generateToken(map);
            System.out.println("token:" + token);
            // 返回登录成功信息
            LoginResult result = new LoginResult();
            result.setToken(token);
            result.setUserId(one.getMemberId());
            result.setUsername(one.getName());
            result.setUserType(loginParam.getUserType());
            return ResultUtils.success("登录成功",result);
        }else if(loginParam.getUserType().equals("2")){ //员工
            QueryWrapper<SysUser> query = new QueryWrapper<>();

            query.lambda().eq(SysUser::getPassword,password).eq(SysUser::get
            Username,loginParam.getUsername());
            SysUser one = sysUserService.getOne(query);
            if(one == null){
                return ResultUtils.error("用户名或密码错误!");
            }
        }
    }

```

```

        // 生成token
        Map<String,String> map = new HashMap<>();
        map.put("userId",Long.toString(one.getUserId()));
        map.put("username",one.getUsername());
        String token = jwtUtils.generateToken(map);
        System.out.println("token:" + token);
        // 返回登录成功信息
        LoginResult result = new LoginResult();
        result.setToken(token);
        result.setUserId(one.getUserId());
        result.setUsername(one.getNickName());
        result.setUserType(loginParam.getUserType());
        return ResultUtils.success("登录成功",result);
    }else{
        return ResultUtils.error("用户类型错误!");
    }
}
}

```

16.3.2 登录前端数据对接

16.3.2.1 安装pinia持久化插件

```
npm install pinia-plugin-persist
```

16.3.2.2 持久化插件引入main.ts

```

import piniaPersist from 'pinia-plugin-persist'

// 实例化 Pinia
const pinia = createPinia()
//使用持久化插件
pinia.use(piniaPersist)

```

16.3.2.3 store/tabs中的index.ts定义如下

```

import { defineStore } from 'pinia'
//定义选项卡数据类型
export type Tab = {
  title: string,
  path: string
}
//定义state的类型
export type TabState = {
  tabList: Tab[]
}

```

```

}
//定义store
export const tabStore = defineStore('tabStore', {
  state: (): TabState => {
    return {
      tabList: []
    }
  },
  getters: {
    getTabs(state) {
      return state.tabList
    }
  },
  actions: {
    addTab(tab: Tab) {
      //判断是否已经添加数据
      if (this.tabList.some(item => item.path ===
tab.path)) return;
      this.tabList.push(tab)
    }
  },
  persist: {
    enabled: true,
    strategies: [
      { storage: localStorage, paths: ['tabList'] },
    ],
  }
})

```

16.3.2.4 store模块新建user目录

```

import { defineStore } from "pinia";
//定义store
export const userStore = defineStore('userStore', {
  state: () => {
    return {
      userId: '',
      token: ''
    }
  },
  getters: {
    getUserId(state) {
      return state.userId
    },
    getToken(state) {

```

```

        return state.token
    },
    actions: {
        setUserId(userId: string) {
            this.userId = userId
        },
        setToken(token: string) {
            this.token = token
        }
    },
    persist: {
        enabled: true,
        strategies: [
            { storage: localStorage, paths: ['userId', 'token'] }
        ]
    }
})

```

16.3.2.5 修改登录页面

```

<template>
  <div class="logincontainer">
    <el-form
      class="loginForm"
      :model="loginModel"
      ref="loginForm"
      :rules="rules"
      size="default"
    >
      <el-form-item style="display: flex; justify-content:
center">
        <div class="loginTitle">系统登录</div>
      </el-form-item>
      <el-form-item>
        <el-input
          size="large"
          v-model="loginModel.username"
          placeholder="请输入账号"
        ></el-input>
      </el-form-item>
      <el-form-item>
        <el-input
          size="large"

```

```
        v-model="loginModel.password"
        placeholder="请输入密码"
    ></el-input>
</el-form-item>
<el-form-item>
    <el-row :gutter="20">
        <el-col :span="16" :offset="0">
            <el-input
                size="large"
                v-model="loginModel.code"
                placeholder="请输入验证码"
            ></el-input>
        </el-col>
        <el-col :span="8" :offset="0">
            
        </el-col>
    </el-row>
</el-form-item>
<el-form-item label="用户类型">
    <el-radio-group v-model="loginModel.userType">
        <el-radio :label="1">会员</el-radio>
        <el-radio :label="2">员工</el-radio>
    </el-radio-group>
</el-form-item>
<el-form-item>
    <el-row style="width: 100%">
        <el-col
            :span="12"
            :offset="0"
            style="padding-right: 10px; padding-left: 0px"
        >
            <el-button
                style="width: 100%"
                size="large"
                type="primary"
                @click="onSubmit"
            >登录</el-button>
        >
    </el-col>
    <el-col
        :span="12"
        :offset="0"
        style="padding-right: 0px; padding-left: 10px"
    >
```



```

        <el-button type="danger" plain style="width: 100%"
size="large"
        >重置</el-button
    >
    </el-col>
</el-row>
</el-form-item>
</el-form>
</div>
</template>

<script setup lang="ts">
import { reactive } from "vue";
import useImage from "@/composables/login/useImage";
import { loginApi } from "@api/login/index";
import { useRouter } from "vue-router";
import { userStore } from "@store/user";
const store = userStore()
const router = useRouter();
const { imgSrc, getImage } = useImage();
//表单对象
const loginModel = reactive({
    username: "",
    password: "",
    code: "",
    userType: "",
});
//表单验证规则
const rules = reactive({});
//登录提交
const onSubmit = async () => {
    const res = await loginApi(loginModel);
    if (res && res.code == 200) {
        store.setToken(res.data.token)
        store.setUserId(res.data.userId)
        //跳转到首页
        router.push({ path: "/" });
    }
};
</script>

<style scoped lang="scss">
.logincontainer {
    background-color: #ffff;
    height: 100%;

```

```
display: flex;
justify-content: center;
align-items: center;
background-image: url("../assets/bg.png");
background-size: 100% 100%;
.loginForm {
  border-radius: 10px;
  background-color: #fff;
  width: 430px;
  height: 340px;
  padding: 20px 35px;
  .loginTitle {
    width: 100%;
    font-size: 24px;
    font-weight: 600;
    display: flex;
    justify-content: center;
    align-items: center;
  }
  .image {
    height: 40px;
    width: 100%;
    display: flex;
    justify-content: center;
    align-items: center;
  }
}
}
</style>
```

17 分配权限树功能模块

17.1 角色分配权限树回显接口开发

17.1.1 新建实体类RoleAssignParm和RolePermissionVo

在com.xq.web.sys_role.entity目录下新建实体

```

package com.xq.web.sys_role.entity;

import lombok.Data;

@Data
public class RoleAssignParam {
    //用户id
    private Long userId;
    //角色id
    private Long roleId;
}

```

```

package com.xq.web.sys_role.entity;

import com.xq.web.sys_menu.entity.SysMenu;
import lombok.Data;

import java.util.ArrayList;
import java.util.List;

@Data
public class RolePermissionVo {
    //当前登录系统用户的菜单数据
    List<SysMenu> listmenu = new ArrayList<>();
    //角色原来分配的菜单
    private Object[] checkList;
}

```

17.1.2 SysMenuMapper接口添加如下方法

```

public interface SysMenuMapper extends BaseMapper<SysMenu> {

    //根据员工id查询菜单
    List<SysMenu> getMenuByUserId(@Param("userId") Long userId);
    //根据会员id查询菜单
    List<SysMenu> getMenuByMemberId(@Param("userId") Long
userId);
    //根据角色id查询菜单
    List<SysMenu> getMenuByRoleId(@Param("roleId") Long roleId);
}

```

17.1.3 修改SysMenuMapper映射文件

```
<mapper namespace="com.xq.web.sys_menu.mapper.SysMenuMapper">

    <select id="getMenuByUserId" parameterType="long"
resultType="com.xq.web.sys_menu.entity.SysMenu">
        select m.* from sys_user_role as ur
                        left join sys_role as r on ur.role_id
= r.role_id
                        left join sys_role_menu as rm on
r.role_id = rm.role_id
                        left join sys_menu as m on rm.menu_id
= m.menu_id
        where ur.user_id =#{userId}
        order by m.order_num asc
    </select>
    <select id="getMenuByMemberId" parameterType="long"
resultType="com.xq.web.sys_menu.entity.SysMenu">
        select m.* from member_role as ur
                        left join sys_role as r on ur.role_id
= r.role_id
                        left join sys_role_menu as rm on
r.role_id = rm.role_id
                        left join sys_menu as m on rm.menu_id
= m.menu_id
        where ur.member_id =#{userId}
        order by m.order_num asc
    </select>
    <select id="getMenuByRoleId" parameterType="long"
resultType="com.xq.web.sys_menu.entity.SysMenu">
        select m.* from sys_role_menu as rm,sys_menu as m
        where rm.menu_id = m.menu_id
        and rm.role_id =#{roleId}
    </select>
</mapper>
```

17.1.4 SysMenuService接口添加如下方法

```

public interface SysMenuService extends IService<SysMenu> {
    //根据员工id查询菜单
    List<SysMenu> getMenuByUserId(Long userId);
    //根据会员id查询菜单
    List<SysMenu> getMenuByMemberId(Long userId);
    //根据角色id查询菜单
    List<SysMenu> getMenuByRoleId(Long roleId);
}

```

修改实现类:

```

@Service
public class SysMenuServiceImpl extends
ServiceImpl<SysMenuMapper, SysMenu> implements SysMenuService {
    @Override
    public List<SysMenu> getMenuByUserId(Long userId) {
        return this.baseMapper.getMenuByUserId(userId);
    }

    @Override
    public List<SysMenu> getMenuByMemberId(Long userId) {
        return this.baseMapper.getMenuByMemberId(userId);
    }

    @Override
    public List<SysMenu> getMenuByRoleId(Long roleId) {
        return this.baseMapper.getMenuByRoleId(roleId);
    }
}

```

17.1.5 SysRoleService接口添加getMenuTree()方法

```

public interface SysRoleService extends IService<SysRole> {
    //查询权限树数据回显
    RolePermissionVo getMenuTree(RoleAssignParam param);
}

```

实现类:

```

@Service
public class SysRoleServiceImpl extends
ServiceImpl<SysRoleMapper, SysRole> implements SysRoleService {
    @Autowired
    SysUserService sysUserService;
}

```

```

@Autowired
SysMenuService sysMenuService;

@Override
public RolePermissionVo getMenuTree(RoleAssignParam param) {
    //查询用户信息
    SysUser user = sysUserService.getById(param.getUserId());
    List<SysMenu> list = null;
    //如果是超级管理员，直接查询所有的菜单
    if(StringUtils.isEmpty(user.getIsAdmin()) &&
user.getIsAdmin().equals("1")){
        list = sysMenuService.list();
    }else{
        list =
sysMenuService.getMenuByUserId(user.getUserId());
    }
    //组装树形数据
    List<SysMenu> menuList = MakeMenuTree.makeTree(list, 0L);
    //查询角色原来的数据
    List<SysMenu> roleList =
sysMenuService.getMenuByRoleId(param.getRoleId());
    List<Long> ids = new ArrayList<>();
    Optional.ofNullable(roleList).orElse(new ArrayList<>())
        .stream()
        .filter(item -> item != null)
        .forEach(item -> {
            ids.add(item.getMenuId());
        });
    //组装数据
    RolePermissionVo vo = new RolePermissionVo();
    vo.setListmenu(menuList);
    vo.setCheckList(ids.toArray());
    return vo;
}
}

```

17.1.6 SysRoleController添加getMenuTree()方法

```

@RestController
@RequestMapping("/api/role")
public class SysRoleController {
    @Autowired
    private SysRoleService sysRoleService;

    //分配权限树数据回显查询

```

```

    @GetMapping("/getMenuTree")
    public ResultVo getMenuTree(RoleAssignParam param){
        RolePermissionVo tree =
sysRoleService.getMenuTree(param);
        return ResultUtils.success("查询成功",tree);
    }
}

```

17.2 分配权限树制作与对接

17.2.1 RoleModel.ts添加AssignParam

在api/role目录下面的RoleModel.ts里面添加：

```

export type AssignParam = {
    roleId:string,
    userId:string
}

```

17.2.2 api/role/index.ts添加getMenuTreeApi()方法

```

import http from "@/http";
import { AddRoleModel,AssignParam,ListParam } from "../RoleModel";
//查询权限树数据
export const getMenuTreeApi =(parm:AssignParam)=>{
    return http.get('/api/role/getMenuTree',parm)
}

```

17.2.3 composables/role添加useAssign.ts

```

import { getMenuTreeApi } from "@/api/role";
import { AssignParam } from "@/api/role/RoleModel";
import { reactive } from "vue";
export default function useAssign() {
    //树的属性配置
    const defaultProps = reactive({
        children: 'children',
        label: 'title',
    })
    //权限树数据
    const assignTreeData = reactive({
        list: [],
        assignTreeChecked: [] //原来分配的权限id的集合
    })
}

```

```

    })
    //获取权限树数据
    const getMenuTree = async (parm: AssignParam) => {
      let res = await getMenuTreeApi(parm)
      if (res && res.code == 200) {
        //设置权限树数据
        assignTreeData.list = res.data.listmenu
        //设置角色原来的权限Id
        assignTreeData.assignTreeChecked = res.data.checkList
        //数据回显，判断角色原来是否已经分配过权限，如果有，回显
        if (assignTreeData.assignTreeChecked.length > 0) {
          let newArr: any = [];
          assignTreeData.assignTreeChecked.forEach((item =>
{
              checked(item, assignTreeData.list, newArr)
            })))
          assignTreeData.assignTreeChecked = newArr;
        }
      }
    }
    const checked = (id: number, data: any, newArr: any) => {
      data.forEach((item: any) => {
        if (item.menuId == id) {
          if (item.children && item.children.length == 0) {
            newArr.push(item.menuId)
          }
        } else {
          if (item.children && item.children.length != 0) {
            //递归调用
            checked(id, item.children, newArr)
          }
        }
      })
    }
    return{
      defaultProps,
      assignTreeData,
      getMenuTree
    }
  }
}

```


17.2.4 新建AssignRole.vue组件

在views/system目录下面创建

```
<template>
  <SysDialog
    :title="dialog.title"
    :width="dialog.width"
    :height="dialog.height"
    :visible="dialog.visible"
    @onClose="onClose"
    @onConfirm="onConfirm"
  >
    <template v-slot:content>
      <el-tree
        ref="assignTree"
        :data="assignTreeData.list"
        node-key="menuId"
        :props="defaultProps"
        empty-text="暂无数据"
        :show-checkbox="true"
        default-expand-all
        :highlight-current="true"
        :default-checked-
keys="assignTreeData.assignTreeChecked"
      ></el-tree>
    </template>
  </SysDialog>
</template>

<script setup lang="ts">
  import SysDialog from "@/components/SysDialog.vue";
  import useDialog from "@/hooks/useDialog";
  import useAssign from "@/composables/role/useAssign";
  import { userStore } from "@/store/user";
  const store = userStore();
  //获取权限树
  const { defaultProps, assignTreeData, getMenuTree } =
useAssign();
  //弹框属性
  const { dialog, onClose, onConfirm, onShow } = useDialog();
  //显示弹框
  const show = (roleId: string, name: string) => {
    let parm = {
      roleId: roleId,
```

```

        userId: store.getUserId,
      };
      getMenuTree(parm);
      //设置弹框属性
      dialog.width = 300;
      dialog.height = 450;
      dialog.title = "为【" + name + "】分配权限";
      onShow();
    };
    defineExpose({
      show,
    });
  }
</script>

<style scoped></style>

```

17.2.5 useRole添加assignBtn()方法

```

export default function useRole(getList: FuncList) {
  //分配权限弹框ref属性
  const assignRoleRef = ref<{ show: (roleId: string, name:
string) => void }>()
  //分配权限按钮
  const assignBtn = (row: AddRoleModel) => {
    assignRoleRef.value?.show(row.roleId, row.roleName)
  }
  return {
    addBtn,
    editBtn,
    deleteBtn,
    assignBtn,
    addRef,
    assignRoleRef
  }
}

```

17.2.6 修改RoleList.vue组件

```

<template>
  <el-main>
    <!-- 搜索栏 -->
    <el-form :model="listParm" :inline="true" size="default">
      <el-form-item>
        <el-input

```

```

        v-model="listParm.roleName"
        placeholder="请输入角色名称"
    ></el-input>
</el-form-item>
<el-form-item>
    <el-button @click="searchBtn" :icon="Search">搜索</el-
button>
    <el-button @click="resetBtn" :icon="Close" type="danger"
plain>重置</el-button>
    <el-button type="primary" @click="addBtn" :icon="Plus">新
增</el-button>
</el-form-item>
</el-form>
<!-- 表格 -->
<el-table :height="tableHeight" :data="tableList.list" border
stripe>
    <el-table-column prop="roleName" label="角色名称"></el-table-
column>
    <el-table-column prop="remark" label="角色描述"></el-table-
column>
    <el-table-column label="操作" width="320" align="center">
        <template #default="scope">
            <el-button type="primary" :icon="Edit" size="default"
@click="editBtn(scope.row)">编辑</el-button>
            <el-button type="success" :icon="Edit" size="default"
@click="assignBtn(scope.row)">分配权限</el-button>
            <el-button type="danger" :icon="Delete"
size="default" @click="deleteBtn(scope.row)">删除</el-button>
        </template>
    </el-table-column>
</el-table>
<!-- 分页 -->
<el-pagination
    @size-change="sizeChange"
    @current-change="currentChange"
    :current-page.sync="listParm.currentPage"
    :page-sizes="[10,20, 40, 80, 100]"
    :page-size="listParm.pageSize"
    layout="total, sizes, prev, pager, next, jumper"
    :total="listParm.total" background>
</el-pagination>

<!-- 新增、编辑弹框 -->
<role-add ref="addRef" @refresh="refresh"></role-add>
<!-- 分配权限 -->

```

```

        <AssignRole ref="assignRoleRef"></AssignRole>
    </el-main>
</template>

<script setup lang="ts">
import RoleAdd from "../RoleAdd.vue";
import AssignRole from "../AssignRole.vue";
import { Plus, Edit, Delete, Search, Close } from "@element-plus/icons-vue";
import useTable from "@/composables/role/useTable";
import useRole from "@/composables/role/useRole";
//表格相关的业务
const { listParm, getList, searchBtn,
resetBtn, tableList, sizeChange, currentChange, tableHeight, refresh }
= useTable();
//增删改查业务
const { addBtn, editBtn, deleteBtn, addRef, assignBtn, assignRoleRef
} = useRole(getList);
</script>

<style scoped></style>

```

17.3 分配权限树保存与前端对接

17.3.1 新建实体类

```

package com.xq.web.sys_role_menu.entity;

import com.baomidou.mybatisplus.annotation.IdType;
import com.baomidou.mybatisplus.annotation.TableId;
import com.baomidou.mybatisplus.annotation.TableName;
import lombok.Data;

@Data
@TableName("sys_role_menu")
public class RoleMenu {
    @TableId(type = IdType.AUTO)
    private Long roleMenuId;
    private Long roleId;
    private Long menuId;
}

```

在sys_role/entity下面创建

```

@Data
public class SaveMenuParam {
    private Long roleId;
    private List<Long> list;
}

```

17.3.2 新建mapper和映射文件

```

package com.xq.web.sys_role_menu.mapper;

import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.xq.web.sys_role_menu.entity.RoleMenu;
import org.apache.ibatis.annotations.Param;

import java.util.List;

public interface RoleMenuMapper extends BaseMapper<RoleMenu> {
    //保存
    boolean saveRoleMenu(@Param("roleId") Long roleId,
        @Param("menuIds") List<Long> menuIds);
}

```

```

<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper
    namespace="com.xq.web.sys_role_menu.mapper.RoleMenuMapper">
    <insert id="saveRoleMenu">
        insert into sys_role_menu(role_id,menu_id) values
        <foreach collection="menuIds" separator="," item="item"
            index="index">
            (#{roleId},#{item})
        </foreach>
    </insert>
</mapper>

```

17.3.3 新建service和实现类

```

package com.xq.web.sys_role_menu.service;

import com.baomidou.mybatisplus.extension.service.IService;
import com.xq.web.sys_role.entity.SaveMenuParam;
import com.xq.web.sys_role_menu.entity.RoleMenu;

public interface RoleMenuService extends IService<RoleMenu> {
    //保存角色权限
    void saveMenu(SaveMenuParam param);
}

```

```

package com.xq.web.sys_role_menu.service.impl;

import
com.baomidou.mybatisplus.core.conditions.query.QueryWrapper;
import
com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
import com.xq.web.sys_role.entity.SaveMenuParam;
import com.xq.web.sys_role_menu.entity.RoleMenu;
import com.xq.web.sys_role_menu.mapper.RoleMenuMapper;
import com.xq.web.sys_role_menu.service.RoleMenuService;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class RoleMenuServiceImpl extends
ServiceImpl<RoleMenuMapper, RoleMenu> implements RoleMenuService
{
    @Transactional
    @Override
    public void saveMenu(SaveMenuParam parm) {
        //先删除角色原来的权限
        QueryWrapper<RoleMenu> query = new QueryWrapper<>();
        query.lambda().eq(RoleMenu::getRoleId, parm.getRoleId());
        this.baseMapper.delete(query);
        //重新保存

        this.baseMapper.saveRoleMenu(parm.getRoleId(), parm.getList());
    }
}

```

17.3.4 RoleController控制器添加saveRoleMenu()方法

```
package com.xq.web.sys_role.controller;

import com.xq.web.sys_role_menu.service.RoleMenuService;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;

@RestController
@RequestMapping("/api/role")
public class SysRoleController {

    @Autowired
    private RoleMenuService roleMenuService;

    //分配权限保存
    @PostMapping("/saveRoleMenu")
    public ResultVo saveRoleMenu(@RequestBody SaveMenuParam param){
        roleMenuService.saveMenu(param);
        return ResultUtils.success("分配成功");
    }

}
```

17.3.5 RoleModel.ts添加SaveAssignParm

```
export type SaveAssignParam = {
    roleId:string,
    list:Array<string>
}
```

17.3.6 api/role/index.ts添加saveRoleMenuApi()方法

```
//保存权限
export const saveRoleMenuApi =(parm:SaveAssignParam)=>{
    return http.post('/api/role/saveRoleMenu',parm)
}
```

17.3.7 修改AssignRole.vue组件

```
<template>
  <SysDialog
    :title="dialog.title"
    :width="dialog.width"
    :height="dialog.height"
    :visible="dialog.visible"
    @onClose="onClose"
    @onConfirm="commit"
  >
    <template v-slot:content>
      <el-tree
        ref="assignTree"
        :data="assignTreeData.list"
        node-key="menuId"
        :props="defaultProps"
        empty-text="暂无数据"
        :show-checkbox="true"
        default-expand-all
        :highlight-current="true"
        :default-checked-keys="assignTreeData.assignTreeChecked"
      ></el-tree>
    </template>
  </SysDialog>
</template>

<script setup lang="ts">
import SysDialog from "@/components/SysDialog.vue";
import useDialog from "@/hooks/useDialog";
import useAssign from "@/composables/role/useAssign";
import { userStore } from "@/store/user";
import { reactive, ref } from "vue";
import { ElMessage, ElTree } from "element-plus";
import { saveRoleMenuApi } from "@/api/role/index";
import { SaveAssignParam } from "@/api/role/RoleModel";
//树的ref属性
const assignTree = ref<InstanceType<typeof ElTree>>();
const store = userStore();
//提交参数
const saveParm = reactive<SaveAssignParam>({
  roleId: "",
  list: [],
});
//权限树
```



```
const { assignTreeData, getMenuTree, defaultProps } =
useAssign();
//弹框属性
const { dialog, onClose, onConfirm, onShow } = useDialog();
//弹框显示
const show = (roleId: string, name: string) => {
  saveParm.roleId = roleId;
  let parm = {
    roleId: roleId,
    userId: store.getUserId,
  };
  getMenuTree(parm);
  //设置弹框属性
  dialog.title = "为【" + name + "】分配权限";
  dialog.width = 300;
  dialog.height = 450;
  onShow();
};
defineExpose({
  show,
});
//提交保存
const commit = async () => {
  //获取选择的菜单数据id
  // console.log(assignTree.value)
  let checkIds = assignTree.value?.getCheckedKeys() as string[];
  let hlfIds = assignTree.value?.getHalfCheckedKeys() as
string[];
  let list = checkIds?.concat(hlfIds);
  if (list.length == 0) {
    ElMessage.warning("请勾选权限信息!");
    return;
  }
  saveParm.list = list;
  console.log(checkIds);
  console.log(hlfIds);
  console.log(list);
  //提交保存
  let res = await saveRoleMenuApi(saveParm);
  if (res && res.code == 200) {
    ElMessage.success(res.msg);
    onClose();
  }
};
</script>
```

```
<style scoped></style>
```

18 用户菜单功能模块

18.1 获取用户信息和菜单信息接口开发

18.1.1 创建实体类

在com.xq.web.login.entity创建实体UserInfo:

```
@Data
public class UserInfo {
    private Long userId;
    private String name;
    private Object[] permissions;
}
```

```
package com.xq.web.login.entity;

import lombok.Data;

@Data
public class InfoParam {
    private Long userId;
    private String userType;
}
```

18.1.2 定义获取用户信息的方法

在LoginController添加getInfo()方法

```
@Autowired
SysMenuService sysMenuService;

//查询用户信息
@GetMapping("/getInfo")
public ResultVo getInfo(InfoParam param){
    UserInfo userInfo = new UserInfo();
    if(param.getUserType().equals("1")){ //会员
        //根据会员id查询权限字段
        List<SysMenu> menuList =
            sysMenuService.getMenuByMemberId(param.getUserId());
        //获取全部的code字段
    }
}
```

```

        List<String> collect =
Optional.ofNullable(menuList).orElse(new ArrayList<>())
        .stream()
        .map(item -> item.getCode())
        .filter(item -> item != null)
        .collect(Collectors.toList());
//转为数组
String[] strings = collect.toArray(new
String[collect.size()]);
//查询用户信息
Member member = memberService.getById(param.getUserId());
//设置返回信息
userInfo.setName(member.getName());
userInfo.setUserId(member.getMemberId());
userInfo.setPermissions(strings);
return ResultUtils.success("查询成功",userInfo);
}else if(param.getUserType().equals("2")){ //员工
//查询用户信息
SysUser user = sysUserService.getById(param.getUserId());
List<SysMenu> menuList = null;
if(StringUtils.isEmpty(user.getIsAdmin()) &&
user.getIsAdmin().equals("1")){ //超级管理员
    menuList = sysMenuService.list();
}else{
    menuList =
sysMenuService.getMenuByUserId(user.getUserId());
}
//获取全部的code字段
List<String> collect =
Optional.ofNullable(menuList).orElse(new ArrayList<>())
        .stream()
        .map(item -> item.getCode())
        .filter(item -> item != null)
        .collect(Collectors.toList());
//转为数组
String[] strings = collect.toArray(new
String[collect.size()]);

//设置返回信息
userInfo.setName(user.getNickName());
userInfo.setUserId(user.getUserId());
userInfo.setPermissions(strings);
return ResultUtils.success("查询成功",userInfo);
}else{
return ResultUtils.error("用户类型错误!");
}

```

```
}  
}
```

18.1.3 添加获取菜单信息的方法

在LoginController添加 getMenuList()

```
@GetMapping("/getMenuList")  
public ResultVo getMenuList(InfoParam parm) {  
    if (parm.getUserType().equals("1")) { //会员  
        List<SysMenu> menus =  
sysMenuService.getMenuByMemberId(parm.getUserId());  
        //获取菜单和目录  
        List<SysMenu> collect =  
Optional.ofNullable(menus).orElse(new ArrayList<>())  
            .stream()  
            .filter(item -> item != null &&  
!item.getType().equals("2")).collect(Collectors.toList());  
        List<RouterVO> rourer = MakeMenuTree.makeRourer(collect,  
0L);  
        return ResultUtils.success("查询成功", rourer);  
    } else if (parm.getUserType().equals("2")) { //员工  
        SysUser user = sysUserService.getById(parm.getUserId());  
        List<SysMenu> menuList = null;  
        if  
(org.apache.commons.lang3.StringUtils.isEmpty(user.getIsAdmin()  
) && user.getIsAdmin().equals("1")) {  
            menuList = sysMenuService.list();  
        } else {  
            menuList =  
sysMenuService.getMenuByUserId(parm.getUserId());  
        }  
        //获取菜单和目录  
        List<SysMenu> collect =  
Optional.ofNullable(menuList).orElse(new ArrayList<>())  
            .stream()  
            .filter(item -> item != null &&  
!item.getType().equals("2")).collect(Collectors.toList());  
        List<RouterVO> rourer = MakeMenuTree.makeRourer(collect,  
0L);  
        return ResultUtils.success("查询成功", rourer);  
    } else {  
        return ResultUtils.error("用户类型错误!");  
    }  
}
```

18.2 菜单和动态路由生成

18.2.1 修改api/login下面的index.ts和LoginModel.ts文件

修改LoginModel.ts

```
export type InfoParm = {  
  userId:string,  
  userType:string  
}
```

修改index.ts

```
//获取菜单数据  
export const getMenuListApi = (parm:InfoParm)=>{  
  return http.get("/api/login/getMenuList",parm)  
}
```

18.2.2 修改路由文件

修改router/index.ts文件

```
import { createRouter, createWebHistory, RouteRecordRaw } from  
'vue-router'  
import Layout from '@/layout/Index.vue'  
export const constantRoutes: Array<RouteRecordRaw> = [  
  {  
    path: "/login",  
    component: () => import('@/views/login/Login.vue'),  
    name: "login"  
  },  
  {  
    path: '/',  
    component: Layout,  
    redirect: '/dashboard',  
    children: [  
      {  
        path: '/dashboard',  
        component: () => import('@/layout/dashboard/Index.vue'),  
        name: 'dashboard',  
        meta: {  
          title: '首页',  
          icon: '#icondashboard'  
        }  
      }  
    ]  
  }  
]
```

```

    ]
  }
]
/* const routes: Array<RouteRecordRaw> = [
  {
    path: "/login",
    component: () => import('@views/login/Login.vue'),
    name: "login"
  },
  {
    path: '/',
    component: Layout,
    redirect: '/dashboard',
    children: [
      {
        path: '/dashboard',
        component: () =>
import('@layout/dashboard/Index.vue'),
        name: 'dashboard',
        meta: {
          title: '首页',
          icon: '#icondashboard'
        }
      }
    ]
  },
  {
    path: "/system",
    component: Layout,
    name: "system",
    meta: {
      title: "系统管理",
      icon: "el-icon-menu",
      roles: ["sys:manage"]
    },
    children: [
      {
        path: "/userList",
        component: () =>
import('@views/system/UserList.vue'),
        name: "userList",
        meta: {
          title: "员工管理",

          icon: "el-icon-s-custom",

```

```
        roles: ["sys:user"]
      },
    },
    {
      path: "/roleList",
      component: () =>
import('@/views/system/RoleList.vue'),
      name: "roleList",
      meta: {
        title: "角色管理",
        icon: "el-icon-s-tools",
        roles: ["sys:role"]
      },
    },
    {
      path: "/menuList",
      component: () =>
import('@/views/system/MenuList.vue'),
      name: "menuList",
      meta: {
        title: "权限管理",
        icon: "el-icon-document",
        roles: ["sys:menu"]
      },
    },
  ]
},
{
  path: "/memberRoot",
  component: Layout,
  name: "memberRoot",
  meta: {
    title: "会员管理",
    icon: "Setting",
    roles: ["sys:memberRoot"],
  },
  children: [
    {
      path: "/cardType",
      component: () =>
import('@/views/member/CardType.vue'),
      name: "cardType",
      meta: {
        title: "会员卡类型",
        icon: "UserFilled",
```

```
        roles: ["sys:cardType"],
      },
    },
    {
      path: "/memberList",
      component: () =>
import('@/views/member/MemberList.vue'),
      name: "memberList",
      meta: {
        title: "会员管理",
        icon: "wallet",
        roles: ["sys:memberList"],
      },
    },
    {
      path: "/myFee",
      component: () => import('@/views/member/MyFee.vue'),
      name: "myFee",
      meta: {
        title: "我的充值",
        icon: "Menu",
        roles: ["sys:myFee"],
      },
    },
  ],
},
{
  path: "/courseRoot",
  component: Layout,
  name: "courseRoot",
  meta: {
    title: "课程管理",
    icon: "Document",
    roles: ["sys:courseRoot"],
  },
  children: [
    {
      path: "/courseList",
      component: () =>
import('@/views/course/CourseList.vue'),
      name: "courseList",
      meta: {
        title: "课程列表",
        icon: "UserFilled",
        roles: ["sys:courseList"],
```



```

        },
        },
        {
            path: "/mycourse",
            component: () =>
import('@/views/mycourse/mycourse.vue'),
            name: "mycourse",
            meta: {
                title: "我的课程",
                icon: "wallet",
                roles: ["sys:mycourse"],
            },
        }
    ]
},
{
    path: "/materialRoot",
    component: Layout,
    name: "materialRoot",
    meta: {
        title: "器材管理",
        icon: "Document",
        roles: ["sys:materialRoot"],
    },
    children: [
        {
            path: "/materialList",
            component: () =>
import('@/views/material/MaterialList.vue'),
            name: "materialList",
            meta: {
                title: "器材列表",
                icon: "UserFilled",
                roles: ["sys:materialList"],
            },
        }
    ]
},
{
    path: "/goodsRoot",
    component: Layout,
    name: "goodsRoot",
    meta: {
        title: "商品管理",
        icon: "Document",

```

```
    roles: ["sys:goodsRoot"],
  },
  children: [
    {
      path: "/goodsList",
      component: () => import('@/views/goods/GoodsList.vue'),
      name: "goodsList",
      meta: {
        title: "商品列表",
        icon: "UserFilled",
        roles: ["sys:goodsList"],
      },
    },
  ],
},
{
  path: "/lostRoot",
  component: Layout,
  name: "lostRoot",
  meta: {
    title: "失物招领",
    icon: "Document",
    roles: ["sys:lostRoot"],
  },
  children: [
    {
      path: "/lostList",
      component: () => import('@/views/lost/LostList.vue'),
      name: "lostList",
      meta: {
        title: "失物列表",
        icon: "UserFilled",
        roles: ["sys:lostList"],
      },
    },
  ],
},
{
  path: "/suggestRoot",
  component: Layout,
  name: "suggestRoot",
  meta: {
    title: "反馈管理",
    icon: "Document",
    roles: ["sys:suggestRoot"],
  },
}
```

```

    },
    children: [
      {
        path: "/suggestList",
        component: () => import('@/views/suggest/SuggestList.vue'),
        name: "suggestList",
        meta: {
          title: "反馈列表",
          icon: "UserFilled",
          roles: ["sys:suggestList"],
        },
      },
    ]
  }
]
] */
const router = createRouter({
  history: createWebHistory(),
  routes: constantRoutes
})
export default router

```

18.2.3 layout下新建center，并建center.vue组件

```

<template>
  <router-view></router-view>
</template>
<script setup lang='ts'>
</script>
<style scoped lang='scss'>
</style>

```

18.2.4 store/menu下新建index.ts

```

import { defineStore } from "pinia";
import { getMenuListApi } from "@/api/login";
import { RouteRecordRaw } from "vue-router";
import Layout from '@/layout/Index.vue'
import center from '@/layout/center/center.vue'
import { InfoParm } from "@/api/login/LoginModel";
//获取views下面的所有页面
const modules = import.meta.glob('../..views/**/*.vue')
//定义store
export const menuStore = defineStore('menuStore', {
  state: () => {

```

```

        return {
            menuList: []
        }
    },
    getters: {
        getMenuList(state) {
            return state.menuList
        }
    },
    actions: {
        getMenu(router: any, parm: InfoParm) {
            return new Promise((resolve, reject) => {
                getMenuListApi(parm).then((res) => {
                    let accessRoute;
                    if (res && res.code == 200) {
                        //动态生成路由
                        accessRoute = generateRoutes(res.data,
router)

                        const desk = [
                            {
                                path: "/dashboard",
                                component: "Layout",
                                name: "dashboard",
                                meta: {
                                    title: "首页",
                                    icon: "HomeFilled",
                                    roles: ["sys:dashboard"],
                                },
                                children: []
                            }
                        ] as any
                        this.menuList = desk.concat(accessRoute)
                    }
                    resolve(this.menuList)
                }).catch((error) => {
                    reject(error)
                })
            })
        },
        persist: {
            enabled: true, //开启持久化操作,默认全部字段存储,存储在
sessionSotrage里面
            strategies: [
                { storage: localStorage, paths: ['menuList'] },
            ]
        }
    }
}

```

```

    ],
  }
})
//动态生成路由
export function generateRoutes(routes: RouteRecordRaw[], router:
any) {
  //定义接收生成的菜单
  const res: Array<RouteRecordRaw> = [];
  routes.forEach((route: any) => {
    //把route里面数据放到新的tmp里面
    const tmp = { ...route }
    const component = tmp.component;
    if (route.component) {
      if (component == 'Layout') {
        tmp.component = Layout;
      } else {
        tmp.component =
modules[`${...}/../views${component}.vue`]
      }
    }
    //如果存在下级
    if (tmp.children && tmp.children.length > 0) {
      if (route.component != 'Layout') {
        tmp.component = center;
      }
      //递归调用
      tmp.children = generateRoutes(tmp.children, router)
    }
    //动态添加路由
    router.addRoute(tmp)
    res.push(tmp)
  })
  return res;
}

```

18.2.5 用户信息接口对接

18.2.5.1 LoginModel.ts添加InfoParm类型

```

export type InfoParm = {
  userId:string,
  userType:string
}

```

18.2.5.2 api/login/index.ts添加getInfoApi()方法

```
//获取用户信息
export const getInfoApi = (parm:InfoParm)=>{
  return http.get("/api/login/getInfo",parm)
}
```

18.2.5.3 修改userStore.ts

```
import { defineStore } from "pinia";
import { getInfoApi } from "@/api/login";
//定义store
export const userStore = defineStore('userStore', {
  state: () => {
    return {
      userId: '',
      token: '',
      userType: '',
      codeList: []
    }
  },
  getters: {
    getUserId(state) {
      return state.userId
    },
    getToken(state) {
      return state.token
    },
    getUserType(state) {
      return state.userType
    }
  },
  actions: {
    setUserId(userId: string) {
      this.userId = userId
    },
    setToken(token: string) {
      this.token = token
    },
    setUserType(userType: string) {
      this.userType = userType;
    },
    getInfo() {
      return new Promise((resolve, reject) => {
```

```

        getInfoApi({ userType: this.userType, userId:
this.userId }).then((res) => {
            if (res && res.code == 200) {
                this.codeList = res.data.permissions
            }
            resolve(this.codeList)
        }).catch((error) => {
            reject(error)
        })
    })
}
},
persist: {
    enabled: true,
    strategies: [
        { storage: localStorage, paths: ['userId',
'token', 'codeList', 'name', 'userType'] },
    ],
}
})

```

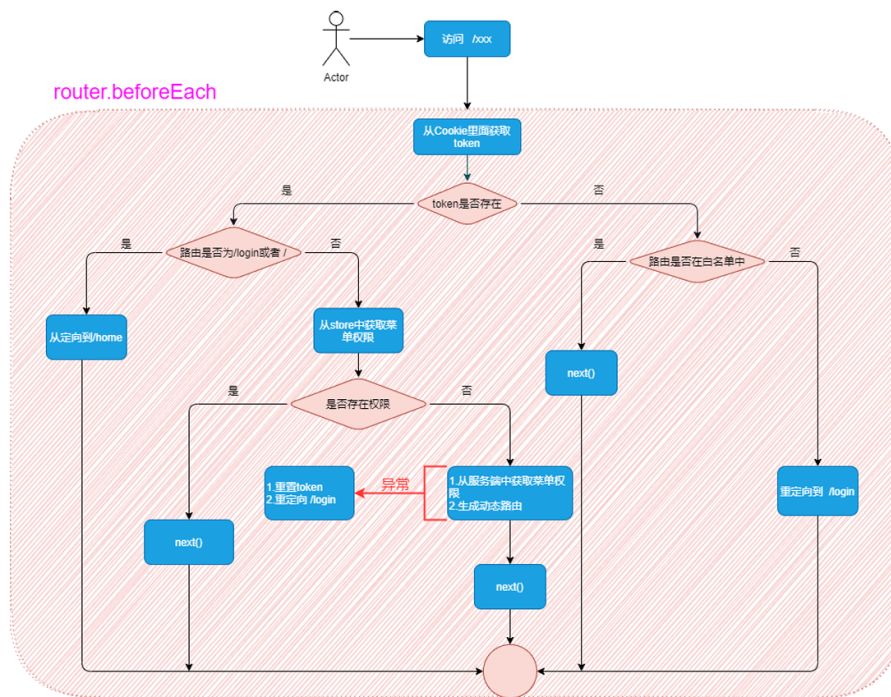
18.3 权限验证流程

注意：

先将资料中的数据库脚本导入到数据库中。

> 此电脑 > 本地磁盘 (G:) > 录课视频 > springboot+vue项目 > 2.资料				
名称	修改日期	类型	大小	
前端项目用到的图片	2024-05-10 15:06	文件夹		
minio.exe	2024-05-19 14:42	应用程序	101,545 KB	
node-v20.13.0-x64.msi	2024-05-08 12:26	Windows Install...	25,984 KB	
数据库脚本.sql	2024-03-11 0:50	SQL Text File	58 KB	

权限验证流程图：



18.3.1 在src目录下创建permission.ts

在src下新建permission.ts:

```

import router from "../router";
import { userStore } from "../store/user";
import { menuStore } from "../store/menu";
//白名单 不需要权限可以访问
const whiteList = ['/login']
//全局守卫路由
router.beforeEach(async (to, from, next) => {
  //获取用户的store
  const ustore = userStore()
  //获取菜单的store
  const mstore = menuStore()
  //获取token
  const token = ustore.getToken
  //判断token是否存在
  if (token) { //存在
    if (to.path === '/login' || to.path === '/') {
      next({ path: '/' })
    } else {
      const menuList = mstore.getMenuList
      if (menuList.length > 0) {
        next()
      } else {
        try {
          //获取用户信息
          await ustore.getInfo()
        } catch {
          //异常处理
        }
      }
    }
  } else {
    //token不存在
    if (whiteList.includes(to.path)) {
      next()
    } else {
      //重定向到login
      next('/login')
    }
  }
})

```



```

        //获取菜单信息,动态生成路由
        await mstore.getMenu(router, { userId:
ustore.getUserId, userType: ustore.getUserType })
        //等待路由全部挂载
        next({ ...to, replace: true })
      } catch (error) {
        localStorage.clear()
        next({ path: '/login' })
      }
    }
  } else { //不存在
    if (whiteList.indexOf(to.path) !== -1) { //存在白名单,直接放
行
      next()
    } else {
      localStorage.clear()
      //去登录
      next({ path: '/login' })
    }
  }
})

```

在main.ts中导入permission.ts, 让其全局生效

```
import './permission'
```

18.3.2 修改MenuBar.vue组件

```

<template>
  <menu-logo :isCollapsed="isCollapse"></menu-logo>
  <el-menu
    :collapse="isCollapse"
    :default-active="activeIndex"
    class="el-menu-vertical-demo"
    unique-opened
    background-color="#304156"
    router
  >
    <menu-item :menuList="menuList"></menu-item>
  </el-menu>
</template>
<script setup lang="ts">
import { ref, reactive, computed } from "vue";
import { useRoute } from "vue-router";

```

```
import MenuItem from "@/layout/menu/MenuItem.vue";
import MenuLogo from "@/layout/menu/MenuLogo.vue";
import { collapseStore } from "@/store/collapse/index";
import { menuStore } from "@/store/menu";
const colstore = collapseStore()
const mstore = menuStore()
//collapse
const isCollapse = computed(()=>{
  return colstore.getCollapse
})
//获取当前的路由
const route = useRoute();
//当前激活的菜单index
const activeIndex = computed(() => {
  const { path } = route;
  return path;
});
//树形的菜单数据
const menuList = computed(()=>{
  return mstore.getMenuList
})
console.log(menuList)
//reactive:定义响应式的数据（复杂类型，对象）
//ref:定义响应式的数据（基本类型） let count = ref(0)
// let menuList = reactive([
//   {
//     path: "/dashboard",
//     component: "Layout",
//     name: "dashboard",
//     meta: {
//       title: "首页",
//       icon: "HomeFilled",
//       roles: ["sys:dashboard"],
//     },
//   },
//   {
//     path: "/system",
//     component: "Layout",
//     name: "system",
//     meta: {
//       title: "系统管理",
//       icon: "Document",
//       roles: ["sys:manage"],
//     },
//     children: [
```

```
//      {
//          path: "/userList",
//          component: "/system/user/UserList",
//          name: "userList",
//          meta: {
//              title: "员工管理",
//              icon: "UserFilled",
//              roles: ["sys:user"],
//          },
//      },
//      {
//          path: "/roleList",
//          component: "/system/role/RoleList",
//          name: "roleList",
//          meta: {
//              title: "角色管理",
//              icon: "wallet",
//              roles: ["sys:role"],
//          },
//      },
//      {
//          path: "/menuList",
//          component: "/system/menu/MenuList",
//          name: "menuList",
//          meta: {
//              title: "菜单管理",
//              icon: "Menu",
//              roles: ["sys:menu"],
//          },
//      },
//  ],
//  },
//  {
//      path: "/memberRoot",
//      component: "Layout",
//      name: "memberRoot",
//      meta: {
//          title: "会员管理",
//          icon: "Stamp",
//          roles: ["sys:memberRoot"],
//      },
//      children: [
//          {
//              path: "/cardType",
//              component: "/member/type/CardType",
```

```
//      name: "cardType",
//      meta: {
//          title: "会员卡类型",
//          icon: "UserFilled",
//          roles: ["sys:cardType"],
//      },
//  },
//  {
//      path: "/memberList",
//      component: "/member/list/MemberList",
//      name: "memberList",
//      meta: {
//          title: "会员管理",
//          icon: "wallet",
//          roles: ["sys:memberList"],
//      },
//  },
//  {
//      path: "/myFee",
//      component: "/member/fee/FeeList",
//      name: "myFee",
//      meta: {
//          title: "我的充值",
//          icon: "Menu",
//          roles: ["sys:myFee"],
//      },
//  },
//  ],
//  },
//  {
//      path: "/courseRoot",
//      component: "Layout",
//      name: "courseRoot",
//      meta: {
//          title: "课程管理",
//          icon: "ScaleToOriginal",
//          roles: ["sys:courseRoot"],
//      },
//      children: [
//          {
//              path: "/courseList",
//              component: "/course/CourseList",
//              name: "courseList",
//              meta: {
//                  title: "课程列表",
```

```
//      icon: "UserFilled",
//      roles: ["sys:courseList"],
//    },
//  },
//  {
//    path: "/mycourse",
//    component: "/mycourse/mycourse",
//    name: "mycourse",
//    meta: {
//      title: "我的课程",
//      icon: "wallet",
//      roles: ["sys:mycourse"],
//    },
//  }
// ],
// },
// {
//   path: "/materialRoot",
//   component: "Layout",
//   name: "materialRoot",
//   meta: {
//     title: "器材管理",
//     icon: "KnifeFork",
//     roles: ["sys:materialRoot"],
//   },
//   children: [
//     {
//       path: "/materialList",
//       component: "/material/MaterialList",
//       name: "materialList",
//       meta: {
//         title: "器材列表",
//         icon: "UserFilled",
//         roles: ["sys:materialList"],
//       },
//     }
//   ],
// },
// {
//   path: "/goodsRoot",
//   component: "Layout",
//   name: "goodsRoot",
//   meta: {
//     title: "商品管理",
//     icon: "Calendar",
```

```
//      roles: ["sys:goodsRoot"],
//    },
//    children: [
//      {
//        path: "/goodsList",
//        component: "/goods/GoodsList",
//        name: "goodsList",
//        meta: {
//          title: "商品列表",
//          icon: "UserFilled",
//          roles: ["sys:goodsList"],
//        },
//      },
//    ],
//    {
//      path: "/orderList",
//      component: "/order/OrderList",
//      name: "orderList",
//      meta: {
//        title: "订单管理",
//        icon: "UserFilled",
//        roles: ["sys:orderList"],
//      },
//    }
//  ],
// },
// {
//   path: "/lostRoot",
//   component: "Layout",
//   name: "lostRoot",
//   meta: {
//     title: "失物招领",
//     icon: "Document",
//     roles: ["sys:lostRoot"],
//   },
//   children: [
//     {
//       path: "/lostList",
//       component: "/goods/LostList",
//       name: "lostList",
//       meta: {
//         title: "失物列表",
//         icon: "UserFilled",
//         roles: ["sys:lostList"],
//       },
//     },
//   ]
// }
```

```

//    ],
//  },
//  {
//    path: "/suggestRoot",
//    component: "Layout",
//    name: "suggestRoot",
//    meta: {
//      title: "反馈管理",
//      icon: "Document",
//      roles: ["sys:suggestRoot"],
//    },
//    children: [
//      {
//        path: "/suggestList",
//        component: "/suggest/SuggestList",
//        name: "suggestList",
//        meta: {
//          title: "反馈列表",
//          icon: "UserFilled",
//          roles: ["sys:suggestList"],
//        },
//      },
//    ]
//  },
//  ],
// }
// ]);
</script>
<style scoped lang="scss">
.el-menu-vertical-demo: not(.el-menu--collapse) {
  width: 230px;
  min-height: 400px;
}
.el-menu {
  border-right: none;
}

:deep(.el-sub-menu .el-sub-menu__title) {
  color: #f4f4f5 !important;
}

:deep(.el-menu .el-menu-item) {
  color: #bfc9d9;
}
/* 菜单点中文字的颜色 */

:deep(.el-menu-item.is-active) {

```

```

    color: #409eff !important;
  }
  /* 当前打开菜单的所有子菜单颜色 */

:deep(.is-opened .el-menu-item) {
  background-color: #1f2d3d !important;
}
/* 鼠标移动菜单的颜色 */

:deep(.el-menu-item:hover) {
  background-color: #001528 !important;
}
</style>

```

18.3.3 用户退出登录讲解

18.3.3.1 layout/header下新建LoginOut.vue组件

```

<template>
  <el-dropdown>
    <span class="el-dropdown-link">
      
    </span>
    <template #dropdown>
      <el-dropdown-menu>
        <el-dropdown-item @click="loginOut">退出登录</el-dropdown-item>
      </el-dropdown-menu>
    </template>
  </el-dropdown>
</template>

<script setup lang="ts">
import useInstance from "@/hooks/useInstance";
import { userStore } from "@/store/user";
const { global } = useInstance();
const store = userStore();
const loginOut = async () => {
  const confirm = await global.$myconfirm("确定退出登录吗? ");
  if (confirm) {
    //清空用户信息
    store.setToken("");
    store.setUserId("");
    store.setUserType("");
    window.location.href = "/login";
  }
}

```



```

    }
  };
</script>

<style scoped>
.userimg {
  height: 38px;
  width: 38px;
  border-radius: 50%;
}
</style>

```

18.3.3.2 Header.vue组件使用

```

<template>
  <div class="header-container">
    <Collapse></Collapse>
    <bread-crumb></bread-crumb>
  </div>
  <LoginOut></LoginOut>
</template>
<script setup lang="ts">
import { ref, reactive } from "vue";
import LoginOut from "./LoginOut.vue";
import Collapse from "./Collapse.vue";
import BreadCrumb from "./BreadCrumb.vue";
</script>
<style scoped lang="scss">
.header-container {
  display: flex;
  align-items: center;
}
</style>

```

18.3.3.3 Index.vue组件的header类样式修改为如下

```

.header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  background-color: #009688;
}

```

19 我的充值功能模块设计

19.1 我的充值后台接口开发

19.1.1 修改实体类

修改com.xq.web.member_recharge.entity.MemberRecharge

```
package com.xq.web.member_recharge.entity;

import com.baomidou.mybatisplus.annotation.IdType;
import com.baomidou.mybatisplus.annotation.TableField;
import com.baomidou.mybatisplus.annotation.TableId;
import com.baomidou.mybatisplus.annotation.TableName;
import lombok.Data;

import java.math.BigDecimal;
import java.util.Date;

@Data
@TableName("member_recharge")
public class MemberRecharge {
    @TableId(type = IdType.AUTO)
    private Long rechargeId;
    private Long memberId;
    private BigDecimal money;
    private Date createTime;
    private String createUser;

    @TableField(exist = false)
    private String name;
    @TableField(exist = false)
    private String username;
}
```

```

package com.xq.web.member.entity;

import lombok.Data;
@Data
public class RechargeParamList {
    private Long currentPage;
    private Long pageSize;
    private Long memberId;
    private String userType;
}

```

19.1.2 MemberRechargeMapper接口添加如下方法

```

package com.xq.web.member_recharge.mapper;

import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.baomidou.mybatisplus.core.metadata.IPage;
import com.xq.web.member_recharge.entity.MemberRecharge;
import org.apache.ibatis.annotations.Param;

import java.util.List;

public interface MemberRechargeMapper extends
BaseMapper<MemberRecharge> {
    IPage<MemberRecharge> getRecherList(IPage<MemberRecharge>
page);
    IPage<MemberRecharge> getRecherByMember(IPage<MemberRecharge>
page,@Param("memberId") Long memberId);
}

```

MemberRechargeMapper.xml映射文件

```

<!DOCTYPE mapper
PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper
namespace="com.xq.web.member_recharge.mapper.MemberRechargeMapper"
">
    <select id="getRecherList"
resultType="com.itmk.web.member_recharge.entity.MemberRecharge">
        SELECT mr.*,m.`name`,m.username FROM member_recharge as
mr

```

```

        left join member as m  on m.member_id = mr.member_id
    </select>
    <select id="getRecherByMember" parameterType="long"
resultType="com.xq.web.member_recharge.entity.MemberRecharge">
        SELECT mr.*,m.`name`,m.username FROM member_recharge as
mr
        left join member as m  on m.member_id = mr.member_id
        where m.member_id =#{memberId}
    </select>
</mapper>

```

19.1.3 MemberRechargeService接口添加如下方法

```

package com.itmk.web.member_recharge.service;

import com.baomidou.mybatisplus.core.metadata.IPage;
import com.baomidou.mybatisplus.extension.service.IService;
import com.xq.web.member.entity.RechargeParmList;
import com.xq.web.member_recharge.entity.MemberRecharge;

public interface MemberRechargeService extends
IService<MemberRecharge> {
    IPage<MemberRecharge> getRecherList(RechargeParmList
parmList);
    IPage<MemberRecharge> getRecherByMember(RechargeParmList
parmList);
}

```

实现类

```

package com.xq.web.member_recharge.service.impl;

import com.baomidou.mybatisplus.core.metadata.IPage;
import
com.baomidou.mybatisplus.extension.plugins.pagination.Page;
import
com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
import com.xq.web.member.entity.RechargeParmList;
import com.xq.web.member_recharge.entity.MemberRecharge;
import com.xq.web.member_recharge.mapper.MemberRechargeMapper;
import com.xq.web.member_recharge.service.MemberRechargeService;
import org.springframework.stereotype.Service;

import java.util.List;

```

```

@Service
public class MemberRechargeServiceImpl extends
ServiceImpl<MemberRechargeMapper, MemberRecharge> implements
MemberRechargeService {
    @Override
    public IPage<MemberRecharge> getRecherList(RechargeParmList
parmList) {
        //构造分页对象
        IPage<MemberRecharge> page = new Page<>
(parmList.getCurrentPage(), parmList.getPageSize());
        return this.baseMapper.getRecherList(page);
    }

    @Override
    public IPage<MemberRecharge>
getRecherByMember(RechargeParmList parmList) {
        //构造分页对象
        IPage<MemberRecharge> page = new Page<>
(parmList.getCurrentPage(), parmList.getPageSize());
        return
this.baseMapper.getRecherByMember(page, parmList.getMemberId());
    }
}

```

19.1.4 MemberController控制器添加getMyRecharge()方法

```

//我的充值查询
@GetMapping("/getMyRecharge")
public ResultVo getMyRecharge(RechargeParmList parm){
    //判断是会员还是员工
    if(parm.getUserType().equals("1")){//会员
        IPage<MemberRecharge> list =
memberRechargeService.getRecherByMember(parm);
        return ResultUtils.success("查询成功", list);
    }else if(parm.getUserType().equals("2")){ //员工
        IPage<MemberRecharge> list =
memberRechargeService.getRecherList(parm);
        return ResultUtils.success("查询成功", list);
    }else{
        return ResultUtils.error("用户类型不存在!");
    }
}
}

```

19.2 我的充值页面制作与对接

19.2.1 api/member/MemberModel.ts添加MemberRecharge

```
export type MemberRecharge = {  
  currentPage:number,  
  pageSize:number,  
  total:number,  
  memberId:string,  
  userType:string  
}
```

19.2.2 api/member/index.ts添加getMyRechargeApi()方法

```
//我的充值  
export const getMyRechargeApi = (parm:MemberRecharge)=>{  
  return http.get("/api/member/getMyRecharge",parm)  
}
```

19.2.3 创建MyFee.vue组件

```
<template>  
  <el-main>  
    <el-table :height="tableHeight" :data="tableData.list" border  
stripe>  
      <el-table-column label="会员卡号" prop="username"></el-table-  
column>  
      <el-table-column label="会员姓名" prop="name"></el-table-  
column>  
      <el-table-column label="充值金额" prop="money"></el-table-  
column>  
      <el-table-column label="充值时间" prop="createTime"></el-  
table-column>  
      <el-table-column label="操作人" prop="createUser"></el-  
table-column>  
    </el-table>  
    <el-pagination  
      @size-change="sizeChange"  
      @current-change="currentChange"  
      :current-page.sync="listParm.currentPage"  
      :page-sizes="[10, 20, 40, 80, 100]"  
      :page-size="listParm.pageSize"  
      layout="total, sizes, prev, pager, next, jumper"  
      :total="listParm.total"  
    ></el-pagination>  
  </el-main>  
</template>
```

```

        background
      >
    </el-pagination>
  </el-main>
</template>

<script setup lang="ts">
import { MemberRecharge } from "@api/member/MemberModel";
import { nextTick, onMounted, reactive, ref } from "vue";
import { getMyRechargeApi } from "@api/member";
import { userStore } from "@store/user";
//表格高度
const tableHeight = ref(0);
const store = userStore();
//表格查询的参数
const listParm = reactive<MemberRecharge>({
  currentPage: 1,
  pageSize: 10,
  total: 0,
  memberId: "",
  userType: "",
});
//表格数据
const tableData = reactive({
  list: [],
});
//查询表格数据
const getList = async () => {
  listParm.memberId = store.getUserId();
  listParm.userType = store.getUserTyp;
  let res = await getMyRechargeApi(listParm);
  if (res && res.code == 200) {
    console.log(res);
    tableData.list = res.data.records;
    listParm.total = res.data.total;
  }
};
//页容量改变时触发
const sizeChange = (size: number) => {
  listParm.pageSize = size;
  getList();
};
//页数改变时触发
const currentChange = (page: number) => {
  listParm.currentPage = page;

```

```

    getList();
};
onMounted(() => {
    nextTick(() => {
        tableHeight.value = window.innerHeight - 230;
    });
    getList();
});
</script>

<style scoped></style>

```

20 课程报名选课功能模块设计

20.1 数据表设计

第一步：课程表添加teacher_id字段

列名	数据类型	长度	默认	主键?	非空?	Unsigned	自增?	Zerofill?	注释
course_id	int	11		☒	☒	☐	☒	☐	课程id
course_name	varchar	128		☐	☐	☐	☐	☐	课程名称
image	varchar	255		☐	☐	☐	☐	☐	封面图
teacher_name	varchar	64		☐	☐	☐	☐	☐	授课教师
course_hour	int	11		☐	☐	☐	☐	☐	课时
course_details	text			☐	☐	☐	☐	☐	课程详情
course_price	decimal	18,2		☐	☐	☐	☐	☐	课程价格
teacher_id	int	11		☐	☐	☐	☐	☐	教师id

对应的课程(Course)实体也需要发生变化：

```

@Data
@Table("course")
public class Course {
    no usages
    @TableId(type = IdType.AUTO)
    private Long courseId;
    no usages
    private String courseName;
    no usages
    private String image;
    no usages
    private String teacherName;
    no usages
    private Integer courseHour;
    no usages
    private String courseDetails;
    no usages
    private BigDecimal coursePrice;
    no usages
    private Long teacherId;
}

```

第二步：创建报名表

```

create table member_course
(

```



```

    member_course_id    int not null auto_increment comment '主
    键',
    course_id            int comment '课程id',
    course_name          varchar(128) comment '课程名称',
    image                varchar(255) comment '封面图',
    teacher_name         varchar(64) comment '授课教师',
    course_hour          int comment '课时',
    course_details       text comment '课程详情',
    course_price          decimal(18,2) comment '课程价格',
    teacher_id           int comment '教师id',
    member_id            int comment '会员id',
    primary key (member_course_id)
);

```

20.2 修改前端页面，将课程和教师相关联

修改CourseModel.ts

```

//课程数据类型
export type CourseType = {
  type:string,
  courseId:string,
  courseName:string,
  image:string,
  teacherName:string,
  teacherId:string,
  courseHour:number,
  courseDetails:string,
  coursePrice:number
}

```

修改后端代码

```

// 查询课程教师
no usages
@GetMapping(value = "getTeacher")
public ResultVo getTeacher(){
    QueryWrapper<SysUser> query = new QueryWrapper<>();
    query.lambda().eq(SysUser::getUserType, val: "2");
    List<SysUser> list = sysUserService.list(query);
    // 组装数据
    List<SelectType> selectTypeList = new ArrayList<>();
    if(list.size() > 0){
        list.stream().forEach(item ->{
            SelectType selectType = new SelectType();
            selectType.setLabel(item.getNickName());
            selectType.setValue(item.getUserId());
            selectTypeList.add(selectType);
        });
    }
    return ResultUtils.success(msg: "查询成功", selectTypeList);
}

```

调整AddCourse.vue

```

<template>
  <SysDialog
    :title="dialog.title"
    :height="dialog.height"

```

```

:width="dialog.width"
:visible="dialog.visible"
@onClose="onClose"
@onConfirm="commit"
>
<template v-slot:content>
  <el-form
    :model="addModel"
    ref="addFormRef"
    :rules="rules"
    label-width="80px"
    size="default"
  >
    <el-row>
      <el-col :span="12" :offset="0">
        <el-form-item prop="courseName" label="课程名称">
          <el-input v-model="addModel.courseName"></el-input>
        </el-form-item>
      </el-col>
      <el-col :span="12" :offset="0">
        <el-form-item prop="courseHour" label="课程课时">
          <el-input v-model="addModel.courseHour"></el-input>
        </el-form-item>
      </el-col>
    </el-row>
    <el-row>
      <el-col :span="12" :offset="0">
        <el-form-item prop="coursePrice" label="课程价格">
          <el-input v-model="addModel.coursePrice"></el-
input>
        </el-form-item>
      </el-col>
      <el-col :span="12" :offset="0">
        <el-form-item prop="teacherName" label="教练">
          <el-select
            style="width: 100%"
            v-model="addModel.teacherName"
            class="m-2"
            placeholder="请选择教练"
            size="default"
            @change="selectTeach"
          >
            <el-option
              v-for="item in teacherData.list"
              :key="item.label"

```

```

        :label="item.label"
        :value="{value:item.value,label:item.label}"
      />
    </el-select>
  </el-form-item>
</el-col>
</el-row>
<el-form-item prop="image" label="课程图片">
  <el-upload
    ref="uploadRef"
    action="#"
    :on-change="uploadFile"
    list-type="picture-card"
    :auto-upload="false"
    :file-list="fileList"
    :limit="1"
  >
    <el-icon><Plus /></el-icon>

    <template #file="{ file }">
      <div>
        
        <span class="el-upload-list__item-actions">
          <span
            class="el-upload-list__item-preview"
            @click="handlePictureCardPreview(file)"
          >
            <el-icon><zoom-in /></el-icon>
          </span>
          <span
            v-if="!disabled"
            class="el-upload-list__item-delete"
            @click="handleRemove(file)"
          >
            <el-icon><Delete /></el-icon>
          </span>
        </span>
      </div>
    </template>
  </el-upload>

```

```

        <el-dialog v-model="dialogVisible">
            
        </el-dialog>
    </el-form-item>
    <el-form-item prop="courseDetails" label="课程详情">
        <div style="border: 1px solid #ccc">
            <Toolbar
                style="border-bottom: 1px solid #ccc"
                :editor="editorRef"
                :defaultConfig="toolbarConfig"
                :mode="mode"
            />
            <Editor
                style="height: 300px; overflow-y: hidden"
                v-model="valueHtml"
                :defaultConfig="editorConfig"
                :mode="mode"
                @onCreated="handleCreated"
            />
        </div>
    </el-form-item>
</el-form>
</template>
</SysDialog>
</template>

```

```

<script setup lang="ts">
import "@wangeditor/editor/dist/css/style.css"; // 引入 css
import { Editor, Toolbar } from "@wangeditor/editor-for-vue";
import { CourseType } from "@api/course/CourseModel";
import SysDialog from "@components/SysDialog.vue";
import useDialog from "@hooks/useDialog";
import { ElMessage, FormInstance } from "element-plus";
import { nextTick, reactive, ref } from "vue";
import useUpload from "@composables/course/useUpload";
import useEditor from "@composables/course/useEditor";
import useSelectTeacher from
"@composables/course/useSelectTeacher";
import { EditType, Title } from "@type/BaseEnum";
import { addApi, editApi } from "@api/course/index";
import useInstance from "@hooks/useInstance";
const { global } = useInstance();
// 表单的ref属性
const addFormRef = ref<FormInstance>();

```

```
//弹框属性
const { dialog, onClose, onShow, onConfirm } = useDialog();
//教练选择
const { teacherData, listTeacher } = useSelectTeacher();
//图片上传
const {
  dialogImageUrl,
  dialogVisible,
  disabled,
  handleRemove,
  handlePictureCardPreview,
  uploadFile,
  uploadRef,
  imgurl,
  fileList,
} = useUpload();
//文本编辑器
const {
  editorRef,
  handleCreated,
  mode,
  editorConfig,
  valueHtml,
  toolbarConfig,
} = useEditor();
//显示弹框
const show = async (type: string, row?: CourseType) => {
  //清空图片数据
  fileList.value = [];
  //获取教练数据列表
  await listTeacher();
  //弹框属性
  type == EditType.ADD
    ? (dialog.title = Title.ADD)
    : (dialog.title = Title.EDIT);
  dialog.width = 900;
  dialog.height = 500;
  //清空图片和文本编辑器
  const upload = uploadRef.value;
  if (upload) {
    upload.clearFiles();
  }
  addModel.image = "";
  if (type == EditType.ADD) {
    const editor = editorRef.value;
```

```

        if (editor) {
            editor.clear();
        }
    }
    addModel.courseDetails = "";
    //编辑数据回显
    if (type == EditType.EDIT) {
        nextTick(() => {
            //把要编辑的数据复制到表单绑定的对象
            global.$objCopy(row, addModel);
            //文本编辑器的回显
            valueHtml.value = addModel.courseDetails;
            //图片回显
            if (row?.image) {
                //图片回显
                let img = {
                    name: "",
                    url: "",
                };
                imgUrl.value = addModel.image;
                img.url = addModel.image;
                fileList.value.push(img);
            }
        });
    }
    if (row && row.courseDetails) {
        //文本编辑器的回显
        valueHtml.value = row.courseDetails;
    }
    onShow();
    //表单清空
    addFormRef.value?.resetFields();
    addModel.type = type;
};
//暴露出去，给外部组件使用
defineExpose({
    show,
});
//表单绑定的数据对象
const addModel = reactive<CourseType>({
    type: "", //区分 新增 编辑
    courseId: "",
    courseName: "",
    image: "",
    teacherName: "",

```

```
teacherId: "",
courseHour: 0,
courseDetails: "",
coursePrice: 0,
});
const validateCourseHour = (rule: any, value: any, callback: any)
=> {
  if (value === 0 || value < 0) {
    callback(new Error("请填写课程课时"));
  } else {
    callback();
  }
};
const validateCoursePrice = (rule: any, value: any, callback:
any) => {
  if (value === 0 || value < 0) {
    callback(new Error("请填写课程价格"));
  } else {
    callback();
  }
};
//表单验证规则
const rules = reactive({
  courseName: [
    {
      required: true,
      trigger: "blur",
      message: "请输入课程名称",
    },
  ],
  image: [
    {
      required: true,
      trigger: "blur",
      message: "请上传课程图片",
    },
  ],
  teacherName: [
    {
      required: true,
      trigger: "blur",
      message: "请选择课程教练",
    },
  ],
  courseDetails: [
```

```

    {
      required: true,
      trigger: "blur",
      message: "请填写课程详情",
    },
  ],
  coursePrice: [
    {
      required: true,
      validator: validateCoursePrice,
      trigger: "blur",
    },
  ],
  courseHour: [
    {
      required: true,
      validator: validateCourseHour,
      trigger: "blur",
    },
  ],
});
//注册事件
const emits = defineEmits(["reFresh"]);
//表单提交
const commit = () => {
  //封面图地址
  addModel.image = imgUrl.value;
  //课程详情
  addModel.courseDetails = valueHtml.value;
  addFormRef.value?.validate(async (valid) => {
    if (valid) {
      let res = null;
      if (addModel.type == EditType.ADD) {
        res = await addApi(addModel);
      } else {
        res = await editApi(addModel);
      }
      if (res && res.code == 200) {
        ElMessage.success(res.msg);
        emits("reFresh");
        onClose();
      }
    }
  });
};

```



```
//选择教练的change
const selectTeach = (val:any)=>{
  console.log('obj:',val)
  addModel.teacherId = val.value;
  addModel.teacherName = val.label;
}
</script>

<style scoped></style>
```

重新启动项目测试，查看数据表：

course_name	image	teacher_name	course_hour	course_details	course_price	teacher_id
跆拳道	http://localhost:8089/images/cdb847d1-0503-4b24-b99d-9f37ae65a4e5.j	李四	50	非常实用的333	23B	500.00 (NULL)
三人制篮球	http://localhost:8089/images/7c01a1f5-alc6-4bd1-9508-e8590d2c7041.j	李四	60	<p>非常精彩的技术拼抢。演示。实际训练11</p><p>实际训练。非常好！</p><p style="text-align: center;"><1...	178B	666.00 (NULL)
排球	http://localhost:8089/images/1ef1fe21-be21-48c2-aecb-clcd2094b7e3.j	李四	100	测试	13B	2000.00 (NULL)
皮划艇	http://127.0.0.1:9000/gymnasium/gymnasium_1716789021493.jpg	李四	20	<p>是一个好的项目 {		
}
```

```
<!DOCTYPE mapper
 PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
 "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper
namespace="com.xq.web.member_course.mapper.MemberCourseMapper">

</mapper>
```

## 20.3.3 创建service

```
package com.xq.web.member_course.service;

import com.baomidou.mybatisplus.extension.service.IService;
import com.xq.web.member_course.entity.MemberCourse;

public interface MemberCourseService extends
IService<MemberCourse> {
 void joinCourse(MemberCourse memberCourse);
}
```

```
package com.xq.web.member_course.service.impl;

import
com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
import com.xq.web.course.entity.Course;
import com.xq.web.course.service.CourseService;
import com.xq.web.member.entity.RechargeParm;
import com.xq.web.member.mapper.MemberMapper;
import com.xq.web.member_course.entity.MemberCourse;
import com.xq.web.member_course.mapper.MemberCourseMapper;
import com.xq.web.member_course.service.MemberCourseService;
import org.springframework.beans.BeanUtils;
```

```

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import javax.annotation.Resource;

@Service
public class MemberCourseServiceImpl extends
ServiceImpl<MemberCourseMapper, MemberCourse> implements
MemberCourseService {
 @Autowired
 private CourseService courseService;
 @Resource
 private MemberMapper memberMapper;

 @Override
 @Transactional
 public void joinCourse(MemberCourse memberCourse) {
 //根据课程id查询课程信息
 Course course =
courseService.getById(memberCourse.getCourseId());
 BeanUtils.copyProperties(course,memberCourse);
 //插入报名表
 int insert = this.baseMapper.insert(memberCourse);
 if(insert > 0){
 RechargeParm parm = new RechargeParm();
 parm.setMemberId(memberCourse.getMemberId());
 parm.setMoney(course.getCoursePrice());
 memberMapper.subMoney(parm);
 }
 }
}

```

### 20.3.4 MemberMapper接口添加subMoney()方法

```

public interface MemberMapper extends BaseMapper<Member> {

 int addMoney(@Param("parm") RechargeParm parm);

 int subMoney(@Param("parm") RechargeParm parm);
}

```

```

<!DOCTYPE mapper
PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"

```

```

"http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="com.xq.web.member.mapper.MemberMapper">

 <update id="addMoney">
 update member set money = money + #{parm.money} where
 member_id =#{parm.memberId}
 </update>

 <update id="subMoney">
 update member set money = money - #{parm.money} where
 member_id =#{parm.memberId}
 </update>
</mapper>

```

### 20.3.5 CourseController控制器添加joinCourse()方法

```

@Autowired
MemberCourseService memberCourseService;

@Autowired
MemberService memberService;

//报名课程
@PostMapping("/joinCourse")
public ResultVo joinCourse(@RequestBody MemberCourse
memberCourse){
 //查询是否报名过该课程
 QueryWrapper<MemberCourse> query = new QueryWrapper<>();

 query.lambda().eq(MemberCourse::getCourseId,memberCourse.getCour
seId())

 .eq(MemberCourse::getMemberId,memberCourse.getMemberId());
 MemberCourse one = memberCourseService.getOne(query);
 if(one != null){
 return ResultUtils.error("您已经报名该课程!");
 }
 //判断余额是否充足
 Course course =
courseService.getById(memberCourse.getCourseId());
 Member member =
memberService.getById(memberCourse.getMemberId());
 int flag =
member.getMoney().compareTo(course.getCoursePrice());
 if(flag == -1){

```

```

 return ResultUtils.error("您的余额不足，请充值!");
 }
 memberCourseService.joinCourse(memberCourse);
 return ResultUtils.success("报名成功!");
}

```

## 20.4 选课接口前端对接

### 20.4.1 调整api/course目录下面的CourseModel.ts和index.ts

- CourseModel.ts

```

//选课参数类型
export type SelectCourse = {
 courseId:string,
 memberId:string
}

```

- index.ts

```

//选课
export const joinCourseApi = (parm:SelectCourse)=>{
 return http.post("/api/course/joinCourse",parm)
}

```

### 20.4.2 useCourse.ts添加joinBtn()方法

```

import { deleteApi,joinCourseApi } from "@api/course"
import { userStore } from "@store/user"

export default function useCourse(getList:FuncList){

 const store = userStore()

 //选课
 const joinBtn = async(row: CourseType)=>{
 //信息确定
 let confirm = await global.$myconfirm('确定选课该课程吗?')
 if(confirm){
 let res = await joinCourseApi({
 courseId:row.courseId,
 memberId:store.getUserId
 })
 if(res && res.code == 200){

```

```

 ElMessage.success(res.msg)
 }
 }
 }

 return {
 addBtn,
 editBtn,
 deleteBtn,
 addRef,
 joinBtn
 }
}

```

### 20.4.3 调整CourseList.vue组件

```

<template>
 <el-main>
 <!-- 搜索栏 -->
 <el-form :model="listParam" :inline="true" size="default">
 <el-form-item>
 <el-input
 v-model="listParam.courseName"
 placeholder="课程名称"
 ></el-input>
 </el-form-item>
 <el-form-item>
 <el-input v-model="listParam.teacherName" placeholder="教
 练"></el-input>
 </el-form-item>
 <el-form-item>
 <el-button :icon="Search" @click="searchBtn">搜索</el-
 button>
 <el-button :icon="Close" @click="resetBtn">重置</el-
 button>
 <el-button
 :icon="Plus"
 type="primary"
 @click="addBtn"
 >新增</el-button>
 </el-form-item>
 </el-form>
 <!-- 表格 -->

```

```
<el-table :height="tableHeight" :data="tableData.list" border
stripe>
 <el-table-column prop="image" width="90" label="课程封面"
align="center">
 <template #default="scope">
 <el-image
 style="width: 60px; height: 60px; border-radius: 50%"
 :src="scope.row.image"
 ></el-image>
 </template>
 </el-table-column>
 <el-table-column prop="courseName" label="课程名称"></el-
table-column>
 <el-table-column prop="courseHour" label="课时"></el-table-
column>
 <el-table-column prop="coursePrice" label="价格"></el-table-
column>
 <el-table-column prop="teacherName" label="授课教师"></el-
table-column>
 <el-table-column label="操作" align="center" width="290">
 <template #default="scope">
 <el-button
 type="primary"
 :icon="Edit"
 size="default"
 @click="editBtn(scope.row)"
 >编辑</el-button>
 >
 <el-button
 @click="joinBtn(scope.row)"
 type="success"
 :icon="Edit"
 size="default"
 >报名</el-button>
 >
 <el-button
 type="danger"
 :icon="Delete"
 size="default"
 @click="deleteBtn(scope.row)"
 >删除</el-button>
 >
 </template>
</el-table-column>
</el-table>
```

```

<!-- 分页 -->
<el-pagination
 @size-change="sizeChange"
 @current-change="currentChange"
 :current-page.sync="listParam.currentPage"
 :page-sizes="[10, 20, 40, 80, 100]"
 :page-size="listParam.pageSize"
 layout="total, sizes, prev, pager, next, jumper"
 :total="listParam.total"
 background
>
</el-pagination>

<!-- 新增、编辑 -->
<AddCourse ref="addRef" @reFresh="reFresh"></AddCourse>
</el-main>
</template>

<script setup lang="ts">
import AddCourse from "../AddCourse.vue";
import { Plus, Edit, Delete, Search, Close } from "@element-plus/icons-vue";
import useTable from "../../composables/course/useTable";
import useCourse from "../../composables/course/useCourse";
//列表
const {
 listParam,
 tableData,
 getList,
 sizeChange,
 currentChange,
 tableHeight,
 searchBtn,
 resetBtn,
 reFresh,
} = useTable();
//新增、编辑
const { addBtn, editBtn, deleteBtn, addRef, joinBtn } =
useCourse(getList);
</script>

<style scoped></style>

```

## 21 我的课程功能模块开发



## 21.1 CourseController添加getMyCourseList()方法

在course/entity目录下创建实体类

```
package com.xq.web.course.entity;
```

```
import lombok.Data;
```

```
@Data
```

```
public class PageParm {
 private Long currentPage;
 private Long pageSize;
 private String userType;
 private Long userId;
}
```

```
//我的课程列表
```

```
@GetMapping("/getMyCourseList")
```

```
public ResultVo getMyCourseList(PageParm pageParm){
 if(pageParm.getUserType().equals("1")){ //会员
 IPage<MemberCourse> page = new Page<>
(pageParm.getCurrentPage(),pageParm.getPageSize());
 QueryWrapper<MemberCourse> query = new QueryWrapper<>();

 query.lambda().eq(MemberCourse::getMemberId,pageParm.getUserId()
);
 IPage<MemberCourse> list =
memberCourseService.page(page,query);
 return ResultUtils.success("查询成功",list);
 }else{
 IPage<Course> page = new Page<>
(pageParm.getCurrentPage(),pageParm.getPageSize());
 QueryWrapper<Course> query = new QueryWrapper<>();

 query.lambda().eq(Course::getTeacherId,pageParm.getUserId());
 IPage<Course> list = courseService.page(page, query);
 return ResultUtils.success("查询成功",list);
 }
}
```

## 21.2 我的课程模块前端数据对接

### 21.2.1 api/course/CourseModel.ts添加MyCourseListParm

```
export type MyCourseListParm = {
 userType:string,
 userId:string,
 currentPage:number,
 pageSize:number,
 total:number
}
```

### 21.2.2 api/course/index.ts添加getMyCourseListApi()方法

```
//我的课程
export const getMyCourseListApi = (parm:MyCourseListParm)=>{
 return http.get("/api/course/getMyCourseList",parm)
}
```

### 21.2.3 composables下新建mycourse目录，然后新建useMyCourseTable.ts文件

```
import { nextTick, onMounted, reactive, ref } from "vue";
import { MyCourseListParm } from '@api/course/CourseModel'
import { getMyCourseListApi } from "@api/course";
import { userStore } from "@store/user";
export default function useMyCourseTable(){
 const store = userStore()
 //表格高度
 const tableHeight = ref(0)
 //定义表格数据
 const tableDate = reactive({
 list:[]
 })
 //列表查询参数
 const listParm = reactive<MyCourseListParm>({
 userId:'',
 userType:'',
 currentPage:1,
 pageSize:10,
 total:0
 })
 //查询列表
 const getList = async()=>{
```

```

 listParm.userId = store.getUserId
 listParm.userType = store.getUserType
 let res = await getMyCourseListApi(listParm)
 if(res && res.code == 200){
 console.log(res)
 tableDate.list = res.data.records;
 listParm.total = res.data.total;
 }
}
//页容量改变时触发
const sizeChange = (size:number)=>{
 listParm.pageSize = size;
 getList()
}
//页数改变时触发
const currentChange = (page:number)=>{
 listParm.currentPage = page;
 getList()
}
onMounted(()=>{
 nextTick(()=>{
 tableHeight.value = window.innerHeight - 230
 })
 getList()
})
return{
 listParm,
 tableDate,
 getList,
 sizeChange,
 currentChange,
 tableHeight
}
}

```

## 21.2.4 调整mycourse.vue组件

```

<template>
 <el-main>
 <!-- 表格 -->
 <el-table :height="tableHeight" :data="tableDate.list"
border stripe>
 <el-table-column prop="image" width="90" label="课程封面"
align="center">
 <template #default="scope">

```

```

 <el-image
 style="width: 60px; height: 60px; border-radius:
50%"
 :src="scope.row.image"
 ></el-image>
 </template>
 </el-table-column>
 <el-table-column prop="courseName" label="课程名称"></el-
table-column>
 <el-table-column prop="courseHour" label="课时"></el-
table-column>
 <el-table-column prop="coursePrice" label="价格"></el-
table-column>
 <el-table-column prop="teacherName" label="授课教师"></el-
table-column>
 <el-table-column label="操作" align="center" width="290">
 <template #default="scope">
 <el-button
 v-permission="['sys:mycourse:export']"
 type="primary"
 :icon="Edit"
 size="default"
 @click="exportBtn(scope.row)"
 >导出学生</el-button>
 </template>
 </el-table-column>
 </el-table>
 <!-- 分页 -->
 <el-pagination
 @size-change="sizeChange"
 @current-change="currentChange"
 :current-page.sync="listParm.currentPage"
 :page-sizes="[10, 20, 40, 80, 100]"
 :page-size="listParm.pageSize"
 layout="total, sizes, prev, pager, next, jumper"
 :total="listParm.total"
 background
 >
 </el-pagination>
</el-main>
</template>

<script setup lang="ts">
 import { Edit } from "@element-plus/icons-vue";

```

```

import useMyCourseTable from
"../../composables/mycourse/useMyCourseTable";
import { CourseType } from "@api/course/CourseModel";
import useInstance from "@hooks/useInstance";
const { global } = useInstance();
//列表
const {
 listParm,
 tableDate,
 sizeChange,
 currentChange,
 tableHeight,
} = useMyCourseTable();
//导出
const exportBtn = (row:CourseType)=>{

}
</script>
<style scoped></style>

```

## 21.3 改进会员查询的功能

### 21.3.1 调整MemberController中的list方法

调整PageParm实体类:

```

@Data
public class PageParm {
 private Long currentPage;
 private Long pageSize;
 private String name;
 private String phone;
 private String username;
 private String userType;
 private String memberId;
}

```

```

//查询
@GetMapping("/list")
public ResultVo list(PageParm pageParm) {
 if (pageParm.getUserType().equals("1")) {
 //构造分页对象
 IPage<Member> page = new Page<>
(pageParm.getCurrentPage(), pageParm.getPageSize());

```

```

 //构造查询条件
 QueryWrapper<Member> query = new QueryWrapper<>();
 if (StringUtils.isEmpty(pageParm.getName())) {
 query.lambda().like(Member::getName,
pageParm.getName());
 }
 if (StringUtils.isEmpty(pageParm.getPhone())) {
 query.lambda().like(Member::getPhone,
pageParm.getPhone());
 }
 if (StringUtils.isEmpty(pageParm.getUsername())) {
 query.lambda().like(Member::getUsername,
pageParm.getUsername());
 }

 query.lambda().eq(Member::getMemberId, pageParm.getMemberId());
 query.lambda().orderByDesc(Member::getJoinTime);
 IPage<Member> list = memberService.page(page, query);
 return ResultUtils.success("查询成功", list);
 } else {
 //构造分页对象
 IPage<Member> page = new Page<>
(pageParm.getCurrentPage(), pageParm.getPageSize());
 //构造查询条件
 QueryWrapper<Member> query = new QueryWrapper<>();
 if (StringUtils.isEmpty(pageParm.getName())) {
 query.lambda().like(Member::getName,
pageParm.getName());
 }
 if (StringUtils.isEmpty(pageParm.getPhone())) {
 query.lambda().like(Member::getPhone,
pageParm.getPhone());
 }
 if (StringUtils.isEmpty(pageParm.getUsername())) {
 query.lambda().like(Member::getUsername,
pageParm.getUsername());
 }
 query.lambda().orderByDesc(Member::getJoinTime);
 IPage<Member> list = memberService.page(page, query);
 return ResultUtils.success("查询成功", list);
 }
}

```

## 21.3.2 调整前端数据模型

调整MemberModel.ts中的MemberParam:

```
//定义列表参数类型
export type MemberParam = {
 name:string,
 phone:string,
 username:string,
 currentPage:number,
 pageSize:number,
 total:number,
 userType:string,
 memberId:string
}
```

调整composables/member目录下面的useTable.ts

```
//参数列表
const listParam = reactive<MemberParam>({
 name: '',
 phone: '',
 username: '',
 pageSize: 10,
 currentPage: 1,
 memberId: '',
 userType: '',
 total: 0
})
```

## 22 订单管理模块功能设计

### 22.1 前端订单页面设计

#### 22.1.1 api下新建order目录，然后新建OrderModel.ts

```
//列表查询参数类型
export type OrderListParm = {
 name:string,
 currentPage:number,
 pageSize:number,
 total:number,
 controlName:string
}
```

## 22.1.2 composables下新建order目录，然后新建 useOrderTable.ts、 useFaDrawer.ts、 useChildDrawer.ts

useOrderTable.ts

```
import { OrderListParm } from "@/api/order/OrderModel";
import { reactive } from "vue";

export default function useOrderTable() {
 //列表参数
 const listParm = reactive<OrderListParm>({
 name: '',
 currentPage: 1,
 pageSize: 10,
 total: 0,
 controlName: ''
 })
 //搜索
 const searchBtn = ()=>{

 }
 //重置
 const resetBtn = ()=>{

 }
 //下单
 const orderBtn = ()=>{

 }
 return{
 listParm,
 searchBtn,
 resetBtn,
 orderBtn
 }
}
```

useFaDrawer.ts

```
import { ref } from "vue";
export default function useFaDrawer() {
 const faDrawer = ref(false);
 const cancelClick = () => {
 faDrawer.value = false;
 };
}
```



```

const confirmClick = () => {
 faDrawer.value = false;
};
const order = () => {
 faDrawer.value = true;
};
return {
 faDrawer,
 cancelClick,
 confirmClick,
 order
}
}

```

useChildDrawer.ts

```

import { ref } from "vue";

export default function useChildDrawer(){
 const innerDrawer = ref(false)
 const innerCloseClick = () => {
 innerDrawer.value = false;
 };
 return{
 innerDrawer,
 innerCloseClick
 }
}

```

### 22.1.3 OrderList.vue组件

```

<template>
 <el-main>
 <!--搜索栏 -->
 <el-form :model="listParm" :inline="true" size="default">
 <el-form-item>
 <el-input
 v-model="listParm.name"
 placeholder="请输入商品名称"
 ></el-input>
 </el-form-item>
 <el-form-item>
 <el-button :icon="Search" @click="searchBtn">搜索</el-
button>

```

```

 <el-button type="danger" plain :icon="Close"
@click="resetBtn"
 >重置</el-button>
 >
 <el-button :icon="Plus" type="primary" @click="order">下单
</el-button>
 </el-form-item>
</el-form>
<!-- 购物车 -->
<el-drawer
 :modal="false"
 custom-class="faDrawer"
 :append-to-body="true"
 :modal-append-to-body="false"
 size="30%"
 title="购物车"
 :close-on-click-modal="false"
 v-model="faDrawer"
 direction="rtl"
 >
 <template #title>
 <div class="car-title">
 <el-icon size="18px"><ShoppingCartFull /></el-icon>
 购物车
 </div>
 <el-button size="small" @click="innerDrawer = true">选择商
品</el-button>
 </template>
 <template #default>
 <el-drawer
 custom-class="childDrawer"
 size="30%"
 v-model="innerDrawer"
 title="商品列表"
 :append-to-body="true"
 :before-close="innerCloseClick"
 >
 <div>商品列表</div>
 </el-drawer>
 </template>
 <template #footer>
 <div style="flex: auto">
 <el-button type="danger" plain @click="cancelClick">取消
 </el-button>
 </template>
 </el-drawer>

```

```

 <el-button type="primary" @click="confirmClick">确定
 </el-button>
 </div>
</template>
</el-drawer>
</el-main>
</template>

<script setup lang="ts">
import { Plus, Search, Close } from "@element-plus/icons-vue";
import useOrderTable from "@/composables/order/useOrderTable";
import useFaDrawer from "@/composables/order/useFaDrawer";
import useChildDrawer from "@/composables/order/useChildDrawer";
//表格
const { listParm, searchBtn, resetBtn, orderBtn } =
useOrderTable();
//选择商品
const { innerDrawer, innerCloseClick } = useChildDrawer();
//购物车抽屉
const { faDrawer, cancelClick, confirmClick, order } =
useFaDrawer();
</script>

<style lang="scss">
.faDrawer {
 .el-drawer__header {
 font-size: 16px;
 padding: 12.5px;
 margin-bottom: 0px !important;
 background-color: #009688 !important;
 .el-drawer__close {
 color: #fff !important;
 }
 }
}
.car-title {
 color: #fff;
 font-size: 16px;
 display: flex;
 align-items: center;
}
.el-drawer__footer {
 border-top: 1px solid #e8eae6 !important;
 padding: 10px;
}
}

```

```

.childDrawer {
 .el-drawer__header {
 color: #fff;
 font-size: 16px;
 padding: 12.5px;
 margin-bottom: 0px !important;
 background-color: #009688 !important;
 .el-drawer__close {
 color: #fff !important;
 }
 }
 .el-drawer__body {
 padding: 5px;
 }
}
</style>

```

## 22.1.4 商品选择列表制作与对接

### 22.1.4.1 api/goods/GoodsModel.ts添加AddCar

```

export type AddCar = {
 list: GoodsType[]
}

```

### 22.1.4.2 composables/useChildDrawer.ts修改为如下

```

import { nextTick, onMounted, reactive, ref } from "vue";
import { listApi } from "@/api/goods";
import { AddCar, GoodsParm, GoodsType } from
"@/api/goods/GoodsModel";
import { ElMessage } from "element-plus";
export default function useChildDrawer() {
 const innerDrawer = ref(false)
 const innerCloseClick = () => {
 innerDrawer.value = false;
 };
 //表格高度
 const goodsHeight = ref(0)
 //选择的商品数据
 const selectGoods = reactive<AddCar>({
 list: []
 })
 //商品列表参数
 const goodsParm = reactive<GoodsParm>({

```

```

 name: '',
 pageSize: 20,
 currentPage: 1,
 total: 0
 })
 //商品表格数据
 const tableList = reactive({
 list: []
 })
 //获取商品列表
 const getList = async () => {
 let res = await listApi(goodsParm)
 if (res && res.code == 200) {
 tableList.list = res.data.records;
 goodsParm.total = res.data.total;
 }
 }
 //搜索
 const searchGoodsBtn = ()=>{
 getList()
 }
 //重置
 const resetGoodsBtn = ()=>{
 goodsParm.name = ''
 getList()
 }
 //页容量改变时触发
 const goodsSizeChange= (size:number)=>{
 goodsParm.pageSize = size;
 getList()
 }
 //页数改变时触发
 const goodsCurrentChange = (page:number)=>{
 goodsParm.currentPage = page;
 getList()
 }
 //加入购物车按钮
 const addCarBtn = (row: GoodsType) => {
 const flag = selectGoods.list.some(item => item.goodsId
=== row.goodsId)
 if (flag) {
 ElMessage.warning('该商品已经添加到购物车!');
 return;
 }
 selectGoods.list.push(row)
 }

```

```

 ElMessage.success('加入成功!');
 }
 onMounted(() => {
 nextTick(()=>{
 goodsHeight.value = window.innerHeight - 180
 })
 })
 return {
 innerDrawer,
 innerCloseClick,
 goodsParm,
 tableList,
 addCarBtn,
 selectGoods,
 searchGoodsBtn,
 resetGoodsBtn,
 goodsSizeChange,
 goodsCurrentChange,
 goodsHeight,
 getList
 }
}

```

#### 22.1.4.3 useFaDrawer.ts

```

import { FuncList } from "@/type/BaseType";
import { ref } from "vue";
export default function useFaDrawer(getList:FuncList) {
 const faDrawer = ref(false);
 const cancelClick = () => {
 faDrawer.value = false;
 };
 const confirmClick = () => {
 faDrawer.value = false;
 };
 const order = () => {
 getList()
 faDrawer.value = true;
 };
 return {
 faDrawer,
 cancelClick,
 confirmClick,
 order
 }
}

```

```
}
```

#### 22.1.4.4 OrderList.vue组件

```
<template>
 <el-main>
 <!-- 搜索栏 -->
 <el-form :model="listParm" :inline="true" size="default">
 <el-form-item>
 <el-input
 v-model="listParm.name"
 placeholder="请输入商品名称"
 ></el-input>
 </el-form-item>
 <el-form-item>
 <el-button :icon="Search" @click="searchBtn">搜索</el-
button>
 <el-button type="danger" plain :icon="Close"
@click="resetBtn"
 >重置</el-button
 >
 <el-button :icon="Plus" type="primary" @click="order">下单
</el-button>
 </el-form-item>
 </el-form>
 <!-- 购物车 -->
 <el-drawer
 :modal="false"
 custom-class="faDrawer"
 :append-to-body="true"
 :modal-append-to-body="false"
 size="30%"
 title="购物车"
 :close-on-click-modal="false"
 v-model="faDrawer"
 direction="rtl"
 >
 <template #title>
 <div class="car-title">
 <el-icon size="18px"><ShoppingCartFull /></el-icon>
 购物车
 </div>
 </template>
 </el-drawer>
 </el-main>
</template>
```

```

 <el-button size="small" @click="innerDrawer = true">选择商
品</el-button>
 </template>
 <template #default>
 <el-drawer
 custom-class="childDrawer"
 size="30%"
 v-model="innerDrawer"
 title="商品列表"
 :append-to-body="true"
 :before-close="innerCloseClick"
 >
 <div>
 <el-form style="margin-top:10px;" :model="goodsParm"
:inline="true" size="default">
 <el-form-item>
 <el-input
 v-model="goodsParm.name"
 placeholder="请输入商品名称"
 ></el-input>
 </el-form-item>
 <el-form-item>
 <el-button :icon="Search"
@click="searchGoodsBtn">搜索</el-button>
 <el-button type="danger" plain :icon="Close"
@click="resetGoodsBtn"
>重置</el-button>
 </el-form-item>
 </el-form>
 <el-table :height="goodsHeight"
:data="tableList.list" border stripe>
 <el-table-column
 prop="image"
 width="90"
 label="商品图片"
 align="center"
 >
 <template #default="scope">
 <el-image
 style="width: 40px; height: 40px; border-
radius: 50%"
 :src="scope.row.image"
 ></el-image>
 </template>
 </el-table-column>
 </el-table>
 </div>
 </el-drawer>
 </template>

```



```

 </el-table-column>
 <el-table-column prop="name" label="商品名称"></el-
table-column>
 <el-table-column prop="price" label="商品价格"></el-
table-column>
 <el-table-column prop="store" label="商品库存"></el-
table-column>
 <el-table-column label="操作" align="center">
 <template #default="scope">
 <el-button
 :icon="Plus"
 type="success"
 size="default"
 @click="addCarBtn(scope.row)"
 >加入</el-button>
 </template>
 </el-table-column>
 </el-table>
 <!-- 分页 -->
 <el-pagination
 @size-change="goodsSizeChange"
 @current-change="goodsCurrentChange"
 :current-page.sync="goodsParm.currentPage"
 :page-sizes="[20, 40, 80, 100]"
 :page-size="goodsParm.pageSize"
 layout="total, sizes, prev, pager, next, jumper"
 :total="goodsParm.total" background>
 </el-pagination>

 </div>
 </el-drawer>
</template>
<template #footer>
 <div style="flex: auto">
 <el-button type="danger" plain @click="cancelClick">取消
 </el-button>
 <el-button type="primary" @click="confirmClick">确定
 </el-button>
 </div>
</template>
</el-drawer>
</el-main>
</template>

```

```

<script setup lang="ts">
import { Plus, Search, Close } from "@element-plus/icons-vue";
import useOrderTable from "@/composables/order/useOrderTable";
import useFaDrawer from "@/composables/order/useFaDrawer";
import useChildDrawer from "@/composables/order/useChildDrawer";
//表格
const { listParm, searchBtn, resetBtn, orderBtn } =
useOrderTable();
//选择商品
const {
 innerDrawer,
 innerCloseClick,
 tableList,
 goodsParm,
 addCarBtn,
 selectGoods,
 resetGoodsBtn,
 searchGoodsBtn,
 goodsSizeChange,
 goodsCurrentChange,
 goodsHeight,
 getList
} = useChildDrawer();
//购物车抽屉
const { faDrawer, cancelClick, confirmClick, order } =
useFaDrawer(getList);
</script>

```

```

<style lang="scss">
.faDrawer {
 .el-drawer__header {
 font-size: 16px;
 padding: 12.5px;
 margin-bottom: 0px !important;
 background-color: #009688 !important;
 .el-drawer__close {
 color: #fff !important;
 }
 }
}
.car-title {
 color: #fff;
 font-size: 16px;
 display: flex;
 align-items: center;
}

```

```

.el-drawer__footer {
 border-top: 1px solid #e8eaec !important;
 padding: 10px;
}
}
.childDrawer {
 .el-drawer__header {
 color: #fff;
 font-size: 16px;
 padding: 12.5px;
 margin-bottom: 0px !important;
 background-color: #009688 !important;
 .el-drawer__close {
 color: #fff !important;
 }
 }
 .el-drawer__body {
 padding: 5px;
 }
}
}
</style>

```

## 22.1.5 商品购物车的实现

### 22.1.5.1 GoodsType添加num字段

```

//列表查询参数类型
export type GoodsParm = {
 currentPage:number,
 pageSize:number,
 name:string,
 total:number
}
//商品数据类型
export type GoodsType = {
 type:string,
 goodsId:string,
 name:string,
 image:string,
 details:string,
 unit:string,
 specs:string,
 price:number,
 store:number,
 num?:number
}

```

```

}
export type AddCar = {
 list:GoodsType[]
}

```

### 22.1.5.2 useFaDrawer.ts修改为如下

```

import { AddCar, GoodsType } from "@api/goods/GoodsModel";
import { ref } from "vue";

export default function useFaDrawer(selectGoods:AddCar) {
 const faDrawer = ref(false)
 const cancelClick = () => {
 faDrawer.value = false;
 };
 const confirmClick = () => {
 faDrawer.value = false;
 };
 const order = ()=>{
 selectGoods.list = []
 faDrawer.value = true;
 }
 //购物车删除
 const deleteCarBtn = (row:GoodsType)=>{
 selectGoods.list = selectGoods.list.filter(item =>
item.goodsId !== row.goodsId)
 }
 return {
 faDrawer,
 cancelClick,
 confirmClick,
 order,
 deleteCarBtn
 }
}

```

### 22.1.5.3 useChildDrawer.ts修改为如下

```

import { onMounted, reactive, ref } from "vue";
import { listApi } from "@api/goods";
import { ElMessage } from "element-plus";
import { GoodsParm, GoodsType, AddCar } from
"@api/goods/GoodsModel";
export default function useChildDrawer() {

```

```
const innerDrawer = ref(false)
const innerCloseClick = () => {
 innerDrawer.value = false;
};
//表格高度
const goodsHeight = ref(0)
//列表参数
const goodsParm = reactive<GoodsParm>({
 currentPage: 1,
 pageSize: 20,
 name: '',
 total: 0
})
//定义表格数据
const tableList = reactive({
 list: []
})
//获取表格数据
const getList = async() => {
 let res = await listApi(goodsParm)
 if(res && res.code == 200){
 tableList.list = res.data.records;
 goodsParm.total = res.data.total;
 }
}
const innerDrawerShow = ()=>{
 getList()
 innerDrawer.value = true;
}
//页容量改变时触发
const goodsSizeChange = (size:number)=>{
 goodsParm.pageSize = size;
 getList()
}
//页数改变时触发
const goodsCurrentChange = (page:number)=>{
 goodsParm.currentPage = page;
 getList()
}
//搜索按钮
const searchGoodsBtn = ()=>{
 getList()
}
//重置
const resetGoodsBtn = ()=>{
```

```

 goodsParm.name = ''
 getList()
 }
 //选择的商品数据
 const selectGoods = reactive<AddCar>({
 list: []
 })
 //加入购物车按钮
 const addCarBtn = (row: GoodsType) => {
 row.num = 1;
 //判断商品是否已经加入购物车
 const flag = selectGoods.list.some(item => item.goodsId
=== row.goodsId)
 if (flag) {
 ElMessage.warning('该商品已经添加到购物车!');
 return;
 }
 selectGoods.list.push(row)
 ElMessage.success('加入成功!');
 }
 onMounted(()=>{
 goodsHeight.value = window.innerHeight - 180
 })
 return {
 innerDrawer,
 innerCloseClick,
 goodsParm,
 getList,
 tableList,
 innerDrawerShow,
 goodsSizeChange,
 goodsCurrentChange,
 searchGoodsBtn,
 resetGoodsBtn,
 goodsHeight,
 addCarBtn,
 selectGoods
 }
}

```

#### 22.1.5.4 调整OrderList.vue组件

```

<template>
 <el-main>
 <!--搜索栏 -->

```

```

 <el-form :model="listParm" :inline="true" size="default">
 <el-form-item>
 <el-input v-model="listParm.name" label="请输入商品名称">
</el-input>
 </el-form-item>
 <el-form-item>
 <el-button :icon="Search" @click="searchBtn">搜索</el-
button>
 <el-button :icon="Close" type="danger" plain
@click="resetBtn"
 >重置</el-button
 >
 <el-button :icon="Plus" type="primary" @click="order">下单
</el-button>
 </el-form-item>
 </el-form>
 <!-- 下单 -->
 <el-drawer
 :modal="false"
 custom-class="faDrawer"
 :append-to-body="true"
 :modal-append-to-body="false"
 size="50%"
 title="购物车"
 :close-on-click-modal="false"
 v-model="faDrawer"
 direction="rtl"
 >
 <template #title>
 <div class="car-title">
 <el-icon size="18px"><ShoppingCartFull /></el-icon>
 购物车
 </div>
 <el-button size="small" @click="innerDrawerShow">选择商品
</el-button>
 <el-drawer
 custom-class="childDrawer"
 size="50%"
 v-model="innerDrawer"
 title="商品列表"
 :append-to-body="true"
 :before-close="innerCloseClick"
 >
 <el-form
 style="margin-top: 10px"

```

```

 :model="goodsParm"
 :inline="true"
 size="default"
 >
 <el-form-item>
 <el-input
 v-model="goodsParm.name"
 placeholder="请输入商品名称"
 ></el-input>
 </el-form-item>
 <el-form-item>
 <el-button :icon="Search" @click="searchGoodsBtn">搜索</el-button>
 <el-button
 type="danger"
 plain
 :icon="Close"
 @click="resetGoodsBtn"
 >重置</el-button>
 >
 </el-form-item>
</el-form>
<el-table :height="goodsHeight" :data="tableList.list"
border stripe>
 <el-table-column
 prop="image"
 width="90"
 label="商品图片"
 align="center"
 >
 <template #default="scope">
 <el-image
 style="width: 40px; height: 40px; border-
radius: 50%"
 :src="scope.row.image"
 ></el-image>
 </template>
 </el-table-column>
 <el-table-column prop="name" label="商品名称"></el-
table-column>
 <el-table-column prop="price" label="商品价格"></el-
table-column>
 <el-table-column prop="store" label="商品库存"></el-
table-column>
 <el-table-column label="操作" align="center">

```



```

 <template #default="scope">
 <el-button
 @click="addCarBtn(scope.row)"
 type="primary"
 :icon="Plus"
 size="default"
 >加入</el-button>
 >
 </template>
</el-table-column>
</el-table>
<el-pagination
 @size-change="goodsSizeChange"
 @current-change="goodsCurrentChange"
 :current-page.sync="goodsParm.currentPage"
 :page-sizes="[20, 40, 80, 100]"
 :page-size="goodsParm.pageSize"
 layout="total, sizes, prev, pager, next, jumper"
 :total="goodsParm.total"
 background
>
</el-pagination>
</el-drawer>
</template>
<template #default>
 <el-table :data="selectGoods.list" border stripe>
 <el-table-column prop="name" label="商品名称"></el-table-
column>
 <el-table-column prop="price" label="单价"></el-table-
column>
 <el-table-column prop="num" label="数量" width="180">
 <template #default="scope">
 <el-input-number
 @change="change"
 size="default"
 v-model="scope.row.num"
 :min="1"
 label="描述文字"
 ></el-input-number>
 </template>
 </el-table-column>
 <el-table-column label="金额">
 <template #default="scope">
 {{ (scope.row.price * scope.row.num).toFixed(2) }}
 </template>
 </el-table-column>
 </el-table>

```

```

 </el-table-column>
 <el-table-column label="操作" align="center"
width="120">
 <template #default="scope">
 <el-button
 :icon="Delete"
 type="danger"
 size="default"
 @click="deleteCarBtn(scope.row)"
 >删除</el-button>
 >
 </template>
 </el-table-column>
 </el-table>
</template>
<template #footer>
 <div
 style="
 display: flex;
 align-items: center;
 justify-content: space-between;
 "
 >
 <div>
 <span
 style="
 font-size: 17px;
 font-weight: 600;
 padding-left: 50px;
 color: #009688;
 "
 >合计:
 >{{ totalPrice }}
 </div>
 <div>
 <el-button type="danger" plain @click="cancelClick">取消</el-button>
 <el-button type="primary" @click="confirmClick">确定
 </el-button>
 </div>
 </div>
</template>
</el-drawer>
</el-main>
</template>

```

```

<script setup lang="ts">
import { Plus, Edit, Delete, Search, Close } from "@element-plus/icons-vue";
import useOrderTable from "@/composables/order/useOrderTable";
import useFaDrawer from "@/composables/order/useFaDrawer";
import useChildDrawer from "@/composables/order/useChildDrawer";
import { ref, watch } from "vue";
const { listParm, searchBtn, resetBtn } = useOrderTable();
const {
 innerCloseClick,
 innerDrawer,
 addCarBtn,
 goodsParm,
 tableList,
 innerDrawerShow,
 goodsSizeChange,
 goodsCurrentChange,
 searchGoodsBtn,
 resetGoodsBtn,
 goodsHeight,
 selectGoods,
} = useChildDrawer();
const { faDrawer, cancelClick, confirmClick, order, deleteCarBtn } =
 useFaDrawer(selectGoods);
//总价格
const totalPrice = ref<string | number>(0);
//数量按钮点击事件
const change = () => {
 if (selectGoods.list.length > 0) {
 let sumPrice = selectGoods.list
 .map((row) => (row.num as number) * row.price)
 .reduce((total: number, num: number) => total + num, 0)
 .toFixed(2);
 totalPrice.value = sumPrice;
 } else {
 totalPrice.value = 0;
 }
};
watch(
 () => [...selectGoods.list],
 () => {
 change();
 }
)

```

```
);
</script>

<style lang="scss">
.faDrawer {
 .el-drawer__header {
 font-size: 16px;
 padding: 12.5px;
 margin-bottom: 0px !important;
 background-color: #009688 !important;
 .el-drawer__close {
 color: #fff !important;
 }
 }
 .car-title {
 color: #fff;
 font-size: 16px;
 display: flex;
 align-items: center;
 }
 .el-drawer__footer {
 border-top: 1px solid #e8eaec !important;
 padding: 10px;
 }
}
.childDrawer {
 .el-drawer__header {
 color: #fff;
 font-size: 16px;
 padding: 12.5px;
 margin-bottom: 0px !important;
 background-color: #009688 !important;
 .el-drawer__close {
 color: #fff !important;
 }
 }
 .el-drawer__body {
 padding: 5px;
 }
}
</style>
```

## 22.1.6 订单后端接口开发与数据对接

### 22.1.6.1 数据表设计

```
create table goods_order
(
 order_id int not null auto_increment comment '订单id',
 goods_id int comment '商品id',
 name varchar(200) comment '名称',
 price decimal(18,2) comment '价格',
 image varchar(220) comment '图片',
 details text comment '描述',
 unit varchar(36) comment '单位',
 specs varchar(64) comment '规格',
 num int comment '数量',
 total_price decimal(18,2) comment '总价',
 control_user varchar(36) comment '操作人',
 primary key (order_id)
);
```

### 22.1.6.2 新建实体类

```
package com.xq.web.goods_order.entity;

import com.baomidou.mybatisplus.annotation.IdType;
import com.baomidou.mybatisplus.annotation.TableId;
import com.baomidou.mybatisplus.annotation.TableName;
import lombok.Data;

import java.math.BigDecimal;

@Data
@TableName("goods_order")
public class GoodsOrder {
 @TableId(type = IdType.AUTO)
 private Long orderId;
 private Long goodsId;
 private String name;
 private String image;
 private String details;
 private String unit;
 private String specs;
 private BigDecimal price;
 private BigDecimal totalPrice;
```

```
 private Integer num;
 private String controlUser;
}
```

```
package com.xq.web.goods_order.entity;

import lombok.Data;

import java.util.List;

@Data
public class OrderParm {
 private Long userId;
 private List<OrderItem> orderList;
}
```

```
@Data
public class OrderItem {
 private Long goodsId;
 private Integer num;
}
```

### 22.1.6.3 创建mapper接口和映射文件

```
package com.xq.web.goods_order.mapper;

import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.xq.web.goods_order.entity.GoodsOrder;

public interface GoodsOrderMapper extends BaseMapper<GoodsOrder>
{
}
}
```

```
<!DOCTYPE mapper
 PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
 "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper
 namespace="com.xq.web.goods_order.mapper.GoodsOrderMapper">

</mapper>
```

#### 22.1.6.4 创建service和实现类

```
package com.xq.web.goods_order.service;

import com.baomidou.mybatisplus.extension.service.IService;
import com.xq.web.goods_order.entity.GoodsOrder;

public interface GoodsOrderService extends IService<GoodsOrder> {

}
```

```
package com.xq.web.goods_order.service.impl;

import
com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
import com.xq.web.goods_order.entity.GoodsOrder;
import com.xq.web.goods_order.mapper.GoodsOrderMapper;
import com.xq.web.goods_order.service.GoodsOrderService;
import org.springframework.stereotype.Service;

@Service
public class GoodsOrderServiceImpl extends
ServiceImpl<GoodsOrderMapper, GoodsOrder> implements
GoodsOrderService {

}
```

#### 22.1.6.5 创建controller

```
package com.xq.web.goods_order.controller;

import com.xq.utils.ResultUtils;
import com.xq.utils.ResultVo;
import com.xq.web.goods.entity.Goods;
import com.xq.web.goods.service.GoodsService;
import com.xq.web.goods_order.entity.GoodsOrder;
import com.xq.web.goods_order.entity.OrderItem;
import com.xq.web.goods_order.entity.OrderParm;
import com.xq.web.goods_order.service.GoodsOrderService;
import com.xq.web.sys_user.entity.SysUser;
import com.xq.web.sys_user.service.SysUserService;
import org.springframework.beans.BeanUtils;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.PostMapping;
```

```

import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;

@RestController
@RequestMapping("/api/order")
public class GoodsOrderController {
 @Autowired
 private GoodsOrderService goodsOrderService;
 @Autowired
 private SysUserService sysUserService;
 @Autowired
 private GoodsService goodsService;
 //下单
 @PostMapping("/down")
 public ResultVo down(@RequestBody OrderParm parm){
 //查询用户信息
 SysUser user = sysUserService.getById(parm.getUserId());
 List<OrderItem> list = parm.getOrderList();
 List<GoodsOrder> orderList = new ArrayList<>();
 for(int i=0;i<list.size();i++){
 Long goodsId = list.get(i).getGoodsId();
 Integer num = list.get(i).getNum();
 //查询商品信息
 Goods goods = goodsService.getById(goodsId);
 GoodsOrder order = new GoodsOrder();
 BeanUtils.copyProperties(goods,order);
 order.setNum(list.get(i).getNum());
 BigDecimal number =
BigDecimal.valueOf(list.get(i).getNum());
 BigDecimal price = goods.getPrice();
 BigDecimal total = number.multiply(price);
 BigDecimal totalPrice = total.setScale(2,
BigDecimal.ROUND_HALF_UP);
 order.setTotalPrice(totalPrice);
 order.setControlUser(user.getNickName());
 orderList.add(order);
 }
 if(orderList.size() > 0){
 goodsOrderService.saveBatch(orderList);
 }
 }
}

```



```

 return ResultUtils.success("下单成功!");
 }
}

```

#### 22.1.6.6 定义api/order/index.ts

```

import http from "@/http";
import { OrderListParm } from '@/api/order/OrderModel'
//下单
export const downApi = (parm:any)=>{
 return http.post("/api/order/down",parm)
}

```

#### 22.1.6.7 改造OrderList.vue

```

<template>
 <el-main>
 <!--搜索栏 -->
 <el-form :model="listParm" :inline="true" size="default">
 <el-form-item>
 <el-input v-model="listParm.name" label="请输入商品名称">
</el-input>
 </el-form-item>
 <el-form-item>
 <el-button :icon="Search" @click="searchBtn">搜索</el-
button>
 <el-button :icon="Close" type="danger" plain
@click="resetBtn"
 >重置</el-button>
 >
 <el-button v-permission="['sys:orderList:down']"
:icon="Plus" type="primary" @click="order">下单</el-button>
 </el-form-item>
 </el-form>
 <el-table :height="orderTableHeight" border stripe>
 <el-table-column prop="image" width="90" label="商品图片"
align="center">
 <template #default="scope">
 <el-image
 style="width: 60px; height: 60px; border-radius:
50%"
 :src="scope.row.image"
 ></el-image>
 </template>
 </el-table-column>

```

```

 <el-table-column prop="name" label="商品名称"></el-table-
column>
 <el-table-column prop="price" label="单价"></el-table-
column>
 <el-table-column prop="num" label="数量"></el-table-
column>
 <el-table-column prop="totalPrice" label="价格"></el-
table-column>
 <el-table-column prop="controlUser" label="操作人"></el-
table-column>
 </el-table>
 <!-- 分页 -->
 <el-pagination
 @size-change="orderSizeChange"
 @current-change="orderCurrentChange"
 :current-page.sync="listParm.currentPage"
 :page-sizes="[20, 40, 80, 100]"
 :page-size="listParm.pageSize"
 layout="total, sizes, prev, pager, next, jumper"
 :total="listParm.total" background>
 </el-pagination>

 <!-- 下单 -->
 <el-drawer
 :modal="false"
 custom-class="faDrawer"
 :append-to-body="true"
 :modal-append-to-body="false"
 size="50%"
 title="购物车"
 :close-on-click-modal="false"
 v-model="faDrawer"
 direction="rtl"
 >
 <template #title>
 <div class="car-title">
 <el-icon size="18px"><ShoppingCartFull /></el-icon>
 购物车
 </div>
 <el-button size="small" @click="innerDrawerShow">选择商品
 </el-button>
 <el-drawer
 custom-class="childDrawer"
 size="50%"
 v-model="innerDrawer"

```

```

 title="商品列表"
 :append-to-body="true"
 :before-close="innerCloseClick"
 >
 <el-form
 style="margin-top: 10px"
 :model="goodsParm"
 :inline="true"
 size="default"
 >
 <el-form-item>
 <el-input
 v-model="goodsParm.name"
 placeholder="请输入商品名称"
 ></el-input>
 </el-form-item>
 <el-form-item>
 <el-button :icon="Search"
@click="searchGoodsBtn">搜索</el-button>
 <el-button
 type="danger"
 plain
 :icon="Close"
 @click="resetGoodsBtn"
 >重置</el-button>
 >
 </el-form-item>
</el-form>
<el-table :height="goodsHeight"
:data="tableList.list" border stripe>
 <el-table-column
 prop="image"
 width="90"
 label="商品图片"
 align="center"
 >
 <template #default="scope">
 <el-image
 style="width: 40px; height: 40px; border-
radius: 50%"
 :src="scope.row.image"
 ></el-image>
 </template>
 </el-table-column>

```

```

 <el-table-column prop="name" label="商品名称"></el-
table-column>
 <el-table-column prop="price" label="商品价格"></el-
table-column>
 <el-table-column prop="store" label="商品库存"></el-
table-column>
 <el-table-column label="操作" align="center">
 <template #default="scope">
 <el-button
 @click="addCarBtn(scope.row)"
 type="primary"
 :icon="Plus"
 size="default"
 >加入</el-button>
 </template>
 </el-table-column>
 </el-table>
 <el-pagination
 @size-change="goodsSizeChange"
 @current-change="goodsCurrentChange"
 :current-page.sync="goodsParm.currentPage"
 :page-sizes="[20, 40, 80, 100]"
 :page-size="goodsParm.pageSize"
 layout="total, sizes, prev, pager, next, jumper"
 :total="goodsParm.total"
 background
 >
 </el-pagination>
 </el-drawer>
</template>
<template #default>
 <el-table :data="selectGoods.list" border stripe>
 <el-table-column prop="name" label="商品名称"></el-
table-column>
 <el-table-column prop="price" label="单价"></el-table-
column>
 <el-table-column prop="num" label="数量" width="180">
 <template #default="scope">
 <el-input-number
 @change="change"
 size="default"
 v-model="scope.row.num"
 :min="1"
 label="数量"

```

```

 ></el-input-number>
 </template>
 </el-table-column>
 <el-table-column label="金额">
 <template #default="scope">
 {{ (scope.row.price * scope.row.num).toFixed(2)
}}

 </template>
 </el-table-column>
 <el-table-column label="操作" align="center">
 <template #default="scope">
 <el-button
 @click="deleteCarBtn(scope.row)"
 type="danger"
 :icon="Delete"
 size="default"
 >删除</el-button>
 </template>
 </el-table-column>
 </el-table>
</template>
<template #footer>
 <div
 style="
 display: flex;
 align-items: center;
 justify-content: space-between;
 "
 >
 <div>
 <span
 style="
 font-size: 17px;
 font-weight: 600;
 padding-left: 50px;
 color: #009688;
 "
 >合计:
 >{{ totalPrice }}
 </div>
 <div style="flex: auto">
 <el-button type="danger" plain
 @click="cancelClick">取消</el-button>

```

```

 <el-button type="primary" @click="commit">确定</el-
button>

 </div>
 </div>
</template>
</el-drawer>
</el-main>
</template>

<script setup lang="ts">
import { ref, watch } from "vue";
import { Plus, Edit, Delete, Search, Close } from "@element-
plus/icons-vue";
import useOrderTable from "@/composables/order/useOrderTable";
import useFaDrawer from "@/composables/order/useFaDrawer";
import useChildDrawer from
"@/composables/order/useChildDrawer";
import { ElMessage } from "element-plus";
import { userStore } from "@/store/user";
import { downApi } from "@/api/order";
const store = userStore();
const { listParm, searchBtn, resetBtn } = useOrderTable();
const {
 innerCloseClick,
 innerDrawer,
 addCarBtn,
 goodsParm,
 tableList,
 innerDrawerShow,
 goodsSizeChange,
 goodsCurrentChange,
 searchGoodsBtn,
 resetGoodsBtn,
 goodsHeight,
 selectGoods,
} = useChildDrawer();
const { faDrawer, cancelClick, confirmClick, order,
deleteCarBtn } =
 useFaDrawer(selectGoods);
//总价格
const totalPrice = ref<string | number>(0);
//购物车数量点击事件
const change = () => {
 if (selectGoods.list.length > 0) {
 let sumPrice = selectGoods.list

```

```

 .map((row) => (row.num as number) * row.price)
 .reduce((total, num) => total + num, 0)
 .toFixed(2);
 totalPrice.value = sumPrice;
 } else {
 totalPrice.value = 0;
 }
};

//监听表格数据的变化
watch(
 () => [...selectGoods.list],
 () => {
 change();
 }
);

//提交
const commit = async () => {
 if (selectGoods.list.length == 0) {
 ElMessage.warning("请选择商品!");
 return;
 }
 //组装数据
 let list = selectGoods.list.map((item) => {
 return { goodsId: item.goodsId, num: item.num };
 });
 let parm = {
 userId: store.getUserId,
 orderList: list,
 };
 let res = await downApi(parm);
 if (res && res.code == 200) {
 ElMessage.success(res.msg);
 selectGoods.list = [];
 faDrawer.value = false;
 }
};
</script>

<style lang="scss">
 .faDrawer {
 .el-drawer__header {
 font-size: 16px;
 padding: 12.5px;
 margin-bottom: 0px !important;
 background-color: #009688 !important;
 }
 }
</style>

```

```

 .el-drawer__close {
 color: #fff !important;
 }
 }
 .car-title {
 color: #fff;
 font-size: 16px;
 display: flex;
 align-items: center;
 }
 .el-drawer__footer {
 border-top: 1px solid #e8eaec !important;
 padding: 10px;
 }
}
.childDrawer {
 .el-drawer__header {
 color: #fff;
 font-size: 16px;
 padding: 12.5px;
 margin-bottom: 0px !important;
 background-color: #009688 !important;
 .el-drawer__close {
 color: #fff !important;
 }
 }
 .el-drawer__body {
 padding: 5px;
 }
}
</style>

```

## 22.1.7 订单列表页面制作与对接

### 22.1.7.1 定义查询订单列表的接口



```

//列表
@GetMapping("/list")
public ResultVo list(GoodsParam parm){
 //构造分页对象
 IPage<GoodsOrder> page = new Page<>
(parm.getCurrentPage(),parm.getPageSize());
 //构造查询条件
 QueryWrapper<GoodsOrder> query = new QueryWrapper<>();
 if(StringUtils.isNotEmpty(parm.getName())){
 query.lambda().like(GoodsOrder::getName,parm.getName());
 }
 IPage<GoodsOrder> list = goodsOrderService.page(page, query);
 return ResultUtils.success("查询成功",list);
}

```

### 22.1.7.2 改造index.ts

在/api/order/index.ts添加listApi()方法

```

//列表
export const listApi = (parm:OrderListParm)=>{
 return http.get("/api/order/list",parm)
}

```

### 22.1.7.3 修改useOrderTable.ts

```

import { nextTick, onMounted, reactive, ref } from "vue";
import { OrderListParm } from '@/api/order/OrderModel'
import { listApi } from "@/api/order";
export default function useOrderTable() {
 //表格高度
 const orderTableHeight = ref(0)
 //表格数据
 const tableData = reactive({
 list:[]
 })
 //列表查询的参数
 const listParm = reactive<OrderListParm>({
 name: '',
 currentPage: 1,
 pageSize: 20,
 total: 0,
 controlName: ''
 })
 //搜索按钮

```

```

const searchBtn = () => {
 getList()
}
//重置按钮
const resetBtn = () => {
 listParm.name = ''
 getList()
}
//页容量改变是触发
const ordersSizeChange = (size:number)=>{
 listParm.pageSize = size;
 getList()
}
//页数改变时触发
const orderCurrentChange =(page:number)=>{
 listParm.currentPage = page;
 getList()
}
//获取列表
const getList = async()=>{
 let res = await listApi(listParm)
 if(res && res.code == 200){
 tableData.list = res.data.records;
 listParm.total = res.data.total;
 }
}
onMounted(()=>{
 getList()
 nextTick(()=>{
 orderTableHeight.value = window.innerHeight - 230
 })
})
return {
 listParm,
 searchBtn,
 resetBtn,
 tableData,
 getList,
 ordersSizeChange,
 orderCurrentChange,
 orderTableHeight
}
}

```

#### 22.1.7.4 改造OrderList.vue组件

```
<template>
 <el-main>
 <!-- 搜索栏 -->
 <el-form :model="listParm" :inline="true" size="default">
 <el-form-item>
 <el-input v-model="listParm.name" label="请输入商品名称">
</el-input>
 </el-form-item>
 <el-form-item>
 <el-button :icon="Search" @click="searchBtn">搜索</el-
button>
 <el-button :icon="Close" type="danger" plain
@click="resetBtn"
 >重置</el-button>
 >
 <el-button v-permission="['sys:orderList:down']"
:icon="Plus" type="primary" @click="order">下单</el-button>
 </el-form-item>
 </el-form>
 <el-table :height="orderTableHeight" :data="tableData.list"
border stripe>
 <el-table-column prop="image" width="90" label="商品图片"
align="center">
 <template #default="scope">
 <el-image
 style="width: 60px; height: 60px; border-radius:
50%"
 :src="scope.row.image"
 ></el-image>
 </template>
 </el-table-column>
 <el-table-column prop="name" label="商品名称"></el-table-
column>
 <el-table-column prop="price" label="单价"></el-table-
column>
 <el-table-column prop="num" label="数量"></el-table-
column>
 <el-table-column prop="totalPrice" label="价格"></el-
table-column>
 <el-table-column prop="controlUser" label="操作人"></el-
table-column>
 </el-table>
 <!-- 分页 -->
 </el-main>
</template>
```

```

<el-pagination
 @size-change="orderSizeChange"
 @current-change="orderCurrentChange"
 :current-page.sync="listParm.currentPage"
 :page-sizes="[20, 40, 80, 100]"
 :page-size="listParm.pageSize"
 layout="total, sizes, prev, pager, next, jumper"
 :total="listParm.total" background>
</el-pagination>

<!-- 下单 -->
<el-drawer
 :modal="false"
 custom-class="faDrawer"
 :append-to-body="true"
 :modal-append-to-body="false"
 size="50%"
 title="购物车"
 :close-on-click-modal="false"
 v-model="faDrawer"
 direction="rtl"
>
 <template #title>
 <div class="car-title">
 <el-icon size="18px"><ShoppingCartFull /></el-icon>
 购物车
 </div>
 <el-button size="small" @click="innerDrawersShow">选择商品
 </el-button>
 <el-drawer
 custom-class="childDrawer"
 size="50%"
 v-model="innerDrawer"
 title="商品列表"
 :append-to-body="true"
 :before-close="innerCloseClick"
 >
 <el-form
 style="margin-top: 10px"
 :model="goodsParm"
 :inline="true"
 size="default"
 >
 <el-form-item>
 <el-input

```

```

 v-model="goodsParm.name"
 placeholder="请输入商品名称"
 ></el-input>
</el-form-item>
<el-form-item>
 <el-button :icon="Search"
@click="searchGoodsBtn">搜索</el-button>
 <el-button
 type="danger"
 plain
 :icon="Close"
 @click="resetGoodsBtn"
 >重置</el-button>
 >
</el-form-item>
</el-form>
<el-table :height="goodsHeight"
:data="tableList.list" border stripe>
 <el-table-column
 prop="image"
 width="90"
 label="商品图片"
 align="center"
 >
 <template #default="scope">
 <el-image
 style="width: 40px; height: 40px; border-
radius: 50%"
 :src="scope.row.image"
 ></el-image>
 </template>
 </el-table-column>
 <el-table-column prop="name" label="商品名称"></el-
table-column>
 <el-table-column prop="price" label="商品价格"></el-
table-column>
 <el-table-column prop="store" label="商品库存"></el-
table-column>
 <el-table-column label="操作" align="center">
 <template #default="scope">
 <el-button
 @click="addCarBtn(scope.row)"
 type="primary"
 :icon="Plus"
 size="default"

```

```

 >加入</el-button>
 >
 </template>
 </el-table-column>
</el-table>
<el-pagination
 @size-change="goodsSizeChange"
 @current-change="goodsCurrentChange"
 :current-page.sync="goodsParm.currentPage"
 :page-sizes="[20, 40, 80, 100]"
 :page-size="goodsParm.pageSize"
 layout="total, sizes, prev, pager, next, jumper"
 :total="goodsParm.total"
 background
>
</el-pagination>
</el-drawer>
</template>
<template #default>
 <el-table :data="selectGoods.list" border stripe>
 <el-table-column prop="name" label="商品名称"></el-
table-column>
 <el-table-column prop="price" label="单价"></el-table-
column>
 <el-table-column prop="num" label="数量" width="180">
 <template #default="scope">
 <el-input-number
 @change="change"
 size="default"
 v-model="scope.row.num"
 :min="1"
 label="数量"
 ></el-input-number>
 </template>
 </el-table-column>
 <el-table-column label="金额">
 <template #default="scope">
 {{ (scope.row.price * scope.row.num).toFixed(2)
}}

 </template>
 </el-table-column>
 <el-table-column label="操作" align="center">
 <template #default="scope">
 <el-button
 @click="deleteCarBtn(scope.row)"

```

```

 type="danger"
 :icon="Delete"
 size="default"
 >删除</el-button>
 >
 </template>
</el-table-column>
</el-table>
</template>
<template #footer>
 <div
 style="
 display: flex;
 align-items: center;
 justify-content: space-between;
 "
 >
 <div>
 <span
 style="
 font-size: 17px;
 font-weight: 600;
 padding-left: 50px;
 color: #009688;
 "
 >合计:
 >{{ totalPrice }}
 </div>
 <div style="flex: auto">
 <el-button type="danger" plain
@click="cancelClick">取消</el-button>
 <el-button type="primary" @click="commit">确定</el-
button>
 </div>
 </div>
</template>
</el-drawer>
</el-main>
</template>

<script setup lang="ts">
import { ref, watch } from "vue";
import { Plus, Edit, Delete, Search, Close } from "@element-
plus/icons-vue";
import useOrderTable from "@/composables/order/useOrderTable";

```

```

import useFaDrawer from "@composables/order/useFaDrawer";
import useChildDrawer from
"@composables/order/useChildDrawer";
import { ElMessage } from "element-plus";
import { userStore } from "@store/user";
import { downApi } from "@api/order";
const store = userStore();
const { listParm, searchBtn, resetBtn, tableData,
getList, ordersSizeChange, orderCurrentChange, orderTableHeight } =
useOrderTable();
const {
 innerCloseClick,
 innerDrawer,
 addCarBtn,
 goodsParm,
 tableList,
 innerDrawersShow,
 goodsSizeChange,
 goodsCurrentChange,
 searchGoodsBtn,
 resetGoodsBtn,
 goodsHeight,
 selectGoods,
} = useChildDrawer();
const { faDrawer, cancelClick, confirmClick, order,
deleteCarBtn } =
 useFaDrawer(selectGoods);
//总价格
const totalPrice = ref<string | number>(0);
//购物车数量点击事件
const change = () => {
 if (selectGoods.list.length > 0) {
 let sumPrice = selectGoods.list
 .map((row) => (row.num as number) * row.price)
 .reduce((total, num) => total + num, 0)
 .toFixed(2);
 totalPrice.value = sumPrice;
 } else {
 totalPrice.value = 0;
 }
};
//监听表格数据的变化
watch(
 () => [...selectGoods.list],
 () => {

```



```

 change();
 }
);
//提交
const commit = async () => {
 if (selectGoods.list.length == 0) {
 ElMessage.error("请选择商品!");
 return;
 }
 //组装数据
 let list = selectGoods.list.map((item) => {
 return { goodsId: item.goodsId, num: item.num };
 });
 let parm = {
 userId: store.getUserId,
 orderList: list,
 };
 let res = await downApi(parm);
 if (res && res.code == 200) {
 getList()
 ElMessage.success(res.msg);
 faDrawer.value = false;
 }
};
</script>

```

```

<style lang="scss">
.faDrawer {
 .el-drawer__header {
 font-size: 16px;
 padding: 12.5px;
 margin-bottom: 0px !important;
 background-color: #009688 !important;
 .el-drawer__close {
 color: #fff !important;
 }
 }
 .car-title {
 color: #fff;
 font-size: 16px;
 display: flex;
 align-items: center;
 }
 .el-drawer__footer {
 border-top: 1px solid #e8eaec !important;
 }
}

```

```

 padding: 10px;
 }
}
.childDrawer {
 .el-drawer__header {
 color: #fff;
 font-size: 16px;
 padding: 12.5px;
 margin-bottom: 0px !important;
 background-color: #009688 !important;
 .el-drawer__close {
 color: #fff !important;
 }
 }
 .el-drawer__body {
 padding: 5px;
 }
}
</style>

```

## 23 项目整合echarts

echarts官网: [Apache ECharts](https://echarts.apache.org/)

### 23.1 在项目中安装echarts环境

```
npm install echarts --save
```

在main.js中全局挂载echarts

```

import * as echarts from 'echarts'
// 挂载echarts
app.config.globalProperties.$echarts = echarts;

```

### 23.2 修改首页

修改layout\dashboard\Index.vue

```

<template>
 <el-main :style="{ height: mianHeight + 'px' }">
 <!-- 数据统计 -->
 <el-row
 :gutter="20"
 type="flex"
 >

```

```

class="row-bg"
justify="center"
style="margin-bottom: 80px"
>
<el-col :span="6">
 <div class="show-header" style="background: rgb(45, 183,
245)">
 <div class="show-num">0</div>
 <div class="bottom-text">会员总数</div>
 </div>
</el-col>
<el-col :span="6">
 <div class="show-header" style="background: rgb(237, 64,
20)">
 <div class="show-num">0</div>
 <div class="bottom-text">员工总数</div>
 </div>
</el-col>
<el-col :span="6">
 <div class="show-header">
 <div class="show-num">0</div>
 <div class="bottom-text">器材总数</div>
 </div>
</el-col>
<el-col :span="6">
 <div class="show-header" style="background: rgb(255, 153,
0)">
 <div class="show-num">0</div>
 <div class="bottom-text">今日订单</div>
 </div>
</el-col>
</el-row>
<div style="display: flex">
 <el-card style="flex: 1">
 <template #header>
 <div class="card-header">
 热销商品
 </div>
 </template>
 <div ref="myChart" :style="{ width: '400px', height:
'250px' }"></div>
 </el-card>
 <el-card style="margin-left: 20px; flex: 1">
 <template #header>
 <div class="card-header">

```

```

 热销卡
 </div>
 </template>
 <div ref="myChart1" :style="{ width: '400px', height:
'250px' }"></div>
 </el-card>
 <el-card style="margin-left: 20px; flex: 1">
 <template #header>
 <div class="card-header">
 热销课程
 </div>
 </template>
 <div ref="myChart2" :style="{ width: '400px', height:
'250px' }"></div>
 </el-card>
</div>
<el-card class="box-card" style="margin-top: 30px">
 <div slot="header" class="clearfix">
 <span style="color: #000000; font-weight: 600; margin-
bottom: 10px">
 >反馈列表
 >
 <el-divider></el-divider>
 </div>
 <div class="text item">
 放假通知

 ><span style="margin-left: 30px; font-size: 14px"
 >放假时间为 2022-01-10放到 2022-3-03-01，大家关好门窗。
 >
 <el-divider></el-divider>
 </div>
 <div class="text item">
 放假通知

 ><span style="margin-left: 30px; font-size: 14px"
 >放假时间为 2022-01-10放到 2022-3-03-01，大家关好门窗。
 >
 <el-divider></el-divider>
 </div>
 <div class="text item">
 放假通知

 ><span style="margin-left: 30px; font-size: 14px"
 >放假时间为 2022-01-10放到 2022-3-03-01，大家关好门窗。
 </div>

```

```

 >
 <el-divider></el-divider>
 </div>
</el-card>
</el-main>
</template>

<script setup lang="ts">
import { onMounted, reactive, nextTick, ref } from "vue";
import useInstance from "@/hooks/useInstance";
const { global } = useInstance();
const myChart = ref<HTMLElement>();
const myChart1 = ref<HTMLElement>();
const myChart2 = ref<HTMLElement>();
const mianHeight = ref(0);
//柱状图
const charts1 = () => {
 const echartInstance = global.$echarts.init(myChart.value);
 let option = reactive({
 xAxis: {
 type: "category",
 data: ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"],
 axisLabel: {
 //x轴文字的配置
 show: true,
 interval: 0, //使x轴文字显示全
 },
 },
 yAxis: {
 type: "value",
 },
 series: [
 {
 data: [120, 200, 150, 80, 70, 110, 130],
 type: "bar",
 },
],
 });
 echartInstance.setOption(option);
};
//饼图
const charts2 = () => {
 const myChart = global.$echarts.init(myChart1.value);
 let option = reactive({
 title: {

```

```

 subtext: "Fake Data",
 left: "center",
 },
 tooltip: {
 trigger: "item",
 },
 legend: {
 orient: "vertical",
 left: "left",
 },
 series: [
 {
 name: "Access From",
 type: "pie",
 radius: "50%",
 data: [
 { value: 1048, name: "Search Engine" },
 { value: 735, name: "Direct" },
 { value: 580, name: "Email" },
 { value: 484, name: "Union Ads" },
 { value: 300, name: "Video Ads" },
],
 emphasis: {
 itemStyle: {
 shadowBlur: 10,
 shadowOffsetX: 0,
 shadowColor: "rgba(0, 0, 0, 0.5)",
 },
 },
 },
],
});
myChart.setOption(option);
};
//环图
const charts3 = () => {
 const myChart = global.$echarts.init(myChart2.value);
 let option = reactive({
 tooltip: {
 trigger: "item",
 },
 legend: {
 top: "5%",
 left: "center",
 },
 },

```

```

series: [
 {
 name: "Access From",
 type: "pie",
 radius: ["40%", "70%"],
 avoidLabelOverlap: false,
 label: {
 show: false,
 position: "center",
 },
 emphasis: {
 label: {
 show: true,
 fontSize: "40",
 fontWeight: "bold",
 },
 },
 labelLine: {
 show: false,
 },
 data: [
 { value: 1048, name: "Search Engine" },
 { value: 735, name: "Direct" },
 { value: 580, name: "Email" },
 { value: 484, name: "Union Ads" },
 { value: 300, name: "Video Ads" },
],
 },
],
});
myChart.setOption(option);
};

onMounted(() => {
 charts1();
 charts2();
 charts3();
 nextTick(() => {
 mianHeight.value = window.innerHeight - 90;
 });
});
</script>

<style lang="scss" scoped>
.bottom-text {

```

```

 bottom: 0;
 width: 100%;
 background: rgba(0, 0, 0, 0.1);
 height: 25px;
 line-height: 25px;
 text-align: center;
 position: absolute;
 font-weight: 600;
}
.show-header {
 background: #00c0ef;
 color: #fff;
 height: 80px;
 border-radius: 5px;
 position: relative;
}
.show-num {
 font-size: 38px;
 font-weight: 600;
 padding: 5px;
}
</style>

```

## 23.3 首页总数统计接口开发与对接

### 23.3.1 创建实体类

反馈时间字段添加如下注解：

```

@Data
@Table("suggest")
public class Suggest {
 no usages
 @TableId(type = IdType.AUTO)
 private Long id;
 no usages
 private String title;
 no usages
 private String content;
 no usages
 @JsonFormat(pattern = "yyyy-MM-dd HH:mm:ss", timezone = "GMT+8")
 private Date dateTime;
}

```

新建实体类



```

package com.xq.web.home.entity;

import lombok.Data;

@Data
public class TotalCount {
 private int memberCount;
 private int userCount;
 private int materCount;
 private int orderCount;
}

```

### 23.3.2 创建控制器

新建HomeController控制器

```

package com.xq.web.home.controller;

import
com.baomidou.mybatisplus.core.conditions.query.QueryWrapper;
import com.xq.utils.ResultUtils;
import com.xq.utils.ResultVo;
import com.xq.web.goods_order.entity.GoodsOrder;
import com.xq.web.goods_order.service.GoodsOrderService;
import com.xq.web.equipment.service.MaterialService;
import com.xq.web.home.entity.TotalCount;
import com.xq.web.member.service.MemberService;
import com.xq.web.suggest.entity.Suggest;
import com.xq.web.suggest.service.SuggestService;
import com.xq.web.sys_user.service.SysUserService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RequestMapping("/api/home")
@RestController
public class HomeController {
 @Autowired
 private MemberService memberService;
 @Autowired
 private SysUserService sysUserService;
}

```

```

@Autowired
private GoodsOrderService goodsOrderService;
@Autowired
private MaterialService materialService;
@Autowired
private SuggestService suggestService;

//统计总数
@GetMapping("/getTotal")
public ResultVo getTotal(){
 TotalCount totalCount = new TotalCount();
 int memberCount = memberService.count();
 totalCount.setMemberCount(memberCount);
 int userCount = sysUserService.count();
 totalCount.setUserCount(userCount);
 int materCount = materialService.count();
 totalCount.setMaterCount(materCount);
 int orderCount = goodsOrderService.count();
 totalCount.setOrderCount(orderCount);
 return ResultUtils.success("查询成功",totalCount);
}

@GetMapping("/getSuggestList")
public ResultVo getSuggestList(){
 QueryWrapper<Suggest> query = new QueryWrapper<>();
 query.lambda().orderByDesc(Suggest::getDateTime).last("
limit 3");
 List<Suggest> list = suggestService.list(query);
 return ResultUtils.success("查询成功",list);
}
}

```

## 23.4 echarts图表接口开发

### 23.4.1 新建实体类

```
package com.xq.web.home.entity;

import lombok.Data;

@Data
public class EchartItem {
 private String name;
 private Integer value;
}
```

```
package com.xq.web.home.entity;

import lombok.Data;

import java.util.ArrayList;
import java.util.List;

@Data
public class Echart {
 private List<String> names = new ArrayList<>();
 private List<Integer> values = new ArrayList<>();
}
```

### 23.4.2 GoodsOrderMapper接口添加如下方法

```
package com.xq.web.goods_order.mapper;

import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.xq.web.goods_order.entity.GoodsOrder;
import com.xq.web.home.entity.EchartItem;

import java.util.List;

public interface GoodsOrderMapper extends BaseMapper<GoodsOrder>
{

 //热销商品
 List<EchartItem> hotGoods();
 //热销卡
 List<EchartItem> hotCards();
 //热销课程
 List<EchartItem> hotCourse();
}
```

```

<!DOCTYPE mapper
 PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
 "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper
namespace="com.xq.web.goods_order.mapper.GoodsOrderMapper">

 <select id="hotGoods"
resultType="com.xq.web.home.entity.EchartItem">
 select g.`name`,sum(gd.num) as value from goods as g
 inner join goods_order as gd on gd.goods_id =
g.goods_id
 GROUP BY g.goods_id
 order by value desc
 limit 7
 </select>
 <select id="hotCards"
resultType="com.xq.web.home.entity.EchartItem">
 select ma.card_type as name,count(ma.apply_id) as value
from member_apply as ma
 GROUP BY ma.card_type,ma.card_day
 order by value desc
 limit 7
 </select>
 <select id="hotCourse"
resultType="com.xq.web.home.entity.EchartItem">
 select c.course_name as name,count(mc.member_course_id)
as value from course as c
 left join member_course as mc on c.course_id =
mc.course_id
 GROUP BY c.course_id
 </select>
</mapper>

```

### 23.4.3 GoodsOrderService添加如下方法

```

package com.xq.web.goods_order.service;

import com.baomidou.mybatisplus.extension.service.IService;
import com.xq.web.goods_order.entity.GoodsOrder;
import com.xq.web.home.entity.EchartItem;

import java.util.List;

public interface GoodsOrderService extends IService<GoodsOrder> {

```

```
//热销商品
List<EchartItem> hotGoods();
//热销卡
List<EchartItem> hotCards();
//热销课程
List<EchartItem> hotCourse();
}
```

```
package com.xq.web.goods_order.service.impl;

import
com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
import com.xq.web.goods_order.entity.GoodsOrder;
import com.xq.web.goods_order.mapper.GoodsOrderMapper;
import com.xq.web.goods_order.service.GoodsOrderService;
import com.xq.web.home.entity.EchartItem;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class GoodsOrderServiceImpl extends
ServiceImpl<GoodsOrderMapper, GoodsOrder> implements
GoodsOrderService {

 @Override
 public List<EchartItem> hotGoods() {
 return this.baseMapper.hotGoods();
 }

 @Override
 public List<EchartItem> hotCards() {
 return this.baseMapper.hotCards();
 }

 @Override
 public List<EchartItem> hotCourse() {
 return this.baseMapper.hotCourse();
 }
}
```

## 23.4.4 修改HomeController

```
package com.xq.web.home.controller;

import
com.baomidou.mybatisplus.core.conditions.query.QueryWrapper;
import com.xq.utils.ResultUtils;
import com.xq.utils.ResultVo;
import com.xq.web.goods_order.entity.GoodsOrder;
import com.xq.web.goods_order.service.GoodsOrderService;
import com.xq.web.equipment.service.MaterialService;
import com.xq.web.home.entity.Echart;
import com.xq.web.home.entity.EchartItem;
import com.xq.web.home.entity.TotalCount;
import com.xq.web.member.service.MemberService;
import com.xq.web.suggest.entity.Suggest;
import com.xq.web.suggest.service.SuggestService;
import com.xq.web.sys_user.service.SysUserService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RequestMapping("/api/home")
@RestController
public class HomeController {

 @Autowired
 private GoodsOrderService goodsOrderService;

 //热销商品
 @GetMapping("/getHotGoods")
 public ResultVo getHotGoods(){
 List<EchartItem> echartItems =
goodsOrderService.hotGoods();
 Echart echart = new Echart();
 if(echartItems.size() >0){
 for(int i=0;i<echartItems.size();i++){

 echart.getNames().add(echartItems.get(i).getName());

 echart.getValues().add(echartItems.get(i).getValue());
 }
 }
 }
}
```

```

 }
 return ResultUtils.success("查询成功",echart);
}

//热销卡
@GetMapping("/getHotCards")
public ResultVo getHotCards(){
 List<EchartItem> echartItems =
goodsOrderService.hotCards();
 return ResultUtils.success("查询成功",echartItems);
}

//热销课程
@GetMapping("/getHotCourse")
public ResultVo getHotCourse(){
 List<EchartItem> echartItems =
goodsOrderService.hotCourse();
 return ResultUtils.success("查询成功",echartItems);
}
}

```

## 23.4.5 echarts动态数据对接

### 23.4.5.1 创建api/home/index.ts

```

import http from "@/http";
//总数统计
export const getTotalApi = ()=>{
 return http.get("/api/home/getTotal")
}
//反馈列表
export const getSuggestListApi = ()=>{
 return http.get("/api/home/getSuggestList")
}
//热销商品
export const getHotGoodstApi = ()=>{
 return http.get("/api/home/getHotGoods")
}
//热销卡
export const getHotCardstApi = ()=>{
 return http.get("/api/home/getHotCards")
}
//热销课程
export const getHotCourseApi = ()=>{

```

```
 return http.get("/api/home/getHotCourse")
 }
```

### 23.4.5.2 调整Index.vue首页

```
<template>
 <el-main :style="{ height: mianHeight + 'px' }">
 <!-- 数据统计 -->
 <el-row
 :gutter="20"
 type="flex"
 class="row-bg"
 justify="center"
 style="margin-bottom: 80px"
 >
 <el-col :span="6">
 <div class="show-header" style="background: rgb(45, 183,
245)">
 <div class="show-num">{{ total.memberCount }}</div>
 <div class="bottom-text">会员总数</div>
 </div>
 </el-col>
 <el-col :span="6">
 <div class="show-header" style="background: rgb(237, 64,
20)">
 <div class="show-num">{{ total.userCount }}</div>
 <div class="bottom-text">员工总数</div>
 </div>
 </el-col>
 <el-col :span="6">
 <div class="show-header">
 <div class="show-num">{{ total.materCount }}</div>
 <div class="bottom-text">器材总数</div>
 </div>
 </el-col>
 <el-col :span="6">
 <div class="show-header" style="background: rgb(255,
153, 0)">
 <div class="show-num">{{ total.orderCount }}</div>
 <div class="bottom-text">昨日订单</div>
 </div>
 </el-col>
 </el-row>
 <div style="display: flex">
 <el-card style="flex: 1">
```



```

 <template #header>
 <div class="card-header">
 热销商品
 </div>
 </template>
 <div ref="myChart" :style="{ width: '400px', height:
'250px' }"></div>
 </el-card>
 <el-card style="margin-left: 20px; flex: 1">
 <template #header>
 <div class="card-header">
 热销卡
 </div>
 </template>
 <div ref="myChart1" :style="{ width: '400px', height:
'250px' }"></div>
 </el-card>
 <el-card style="margin-left: 20px; flex: 1">
 <template #header>
 <div class="card-header">
 热销课程
 </div>
 </template>
 <div ref="myChart2" :style="{ width: '400px', height:
'250px' }"></div>
 </el-card>
</div>
<el-card class="box-card" style="margin-top: 30px">
 <div slot="header" class="clearfix">
 <span style="color: #000000; font-weight: 600; margin-
bottom: 10px">
 >反馈列表
 >
 </div>
 <div v-for="item in suggestList" class="text item">
 {{
item.title }}
 >{{
 item.content
 }}
 {{
 item.dateTime
 }}
 </div>
</el-card>

```

```
 </div>
 </el-card>
</el-main>
</template>
```

```
<script setup lang="ts">
import { getTotalApi,
getSuggestListApi, getHotCardstApi, getHotCourseApi, getHotGoodstApi
} from "@/api/home";
import { onMounted, reactive, nextTick, ref } from "vue";
import useInstance from "@/hooks/useInstance";
const { global } = useInstance();
const myChart = ref<HTMLElement>();
const myChart1 = ref<HTMLElement>();
const myChart2 = ref<HTMLElement>();
const mianHeight = ref(0);
//柱状图
const charts1 = async() => {
 const echartInstance = global.$echarts.init(myChart.value);
 let option = reactive({
 xAxis: {
 type: "category",
 data: [],
 axisLabel: {
 //x轴文字的配置
 show: true,
 interval: 0, //使x轴文字显示全
 },
 },
 yAxis: {
 type: "value",
 },
 series: [
 {
 data: [],
 type: "bar",
 },
],
 });
 //获取数据
 const res = await getHotGoodstApi()
 if(res && res.code == 200){
 console.log(res.data)
 option.xAxis.data = res.data.names
 option.series[0].data = res.data.values
 }
}
```

```

 }
 echartInstance.setOption(option);
};
//饼图
const charts2 = async() => {
 const myChart = global.$echarts.init(myChart1.value);
 let option = reactive({
 title: {
 left: "center",
 },
 tooltip: {
 trigger: "item",
 },
 legend: {
 orient: "vertical",
 left: "left",
 },
 series: [
 {
 name: "Access From",
 type: "pie",
 radius: "50%",
 data: [],
 emphasis: {
 itemStyle: {
 shadowBlur: 10,
 shadowOffsetX: 0,
 shadowColor: "rgba(0, 0, 0, 0.5)",
 },
 },
 },
],
 });
 //获取数据
 const res = await getHotCardstApi()
 if(res && res.code == 200){
 console.log(res)
 option.series[0].data = res.data;
 }
 myChart.setOption(option);
};
//环图
const charts3 = async() => {
 const myChart = global.$echarts.init(myChart2.value);
 let option = reactive({

```

```

 tooltip: {
 trigger: "item",
 },
 legend: {
 top: "5%",
 left: "center",
 },
 series: [
 {
 name: "Access From",
 type: "pie",
 radius: ["40%", "70%"],
 avoidLabelOverlap: false,
 label: {
 show: false,
 position: "center",
 },
 emphasis: {
 label: {
 show: true,
 fontSize: "40",
 fontWeight: "bold",
 },
 },
 labelLine: {
 show: false,
 },
 data: [],
 },
],
 });
const res = await getHotCourseApi()
if(res && res.code == 200){
 option.series[0].data = res.data;
}
myChart.setOption(option);
};
//总数
const total = reactive({
 memberCount: 0,
 userCount: 0,
 materCount: 0,
 orderCount: 0,
});

```

```

//获取总数
const getTotal = async () => {
 let res = await getTotalApi();
 if (res && res.code == 200) {
 total.memberCount = res.data.memberCount;
 total.userCount = res.data.userCount;
 total.materCount = res.data.materCount;
 total.orderCount = res.data.orderCount;
 }
};

type Suggest = {
 id: string;
 title: string;
 content: string;
 dateTime: string;
};

const suggestList = ref<Suggest[]>([]);
//反馈列表
const suggest = async () => {
 let res = await getSuggestListApi();
 if (res && res.code == 200) {
 suggestList.value = res.data;
 }
};

onMounted(() => {
 getTotal();
 suggest();
 charts1();
 charts2();
 charts3();
 nextTick(() => {
 mianHeight.value = window.innerHeight - 90;
 });
});
</script>

```

```

<style lang="scss" scoped>
.bottom-text {
 bottom: 0;
 width: 100%;
 background: rgba(0, 0, 0, 0.1);
 height: 25px;
 line-height: 25px;
 text-align: center;
 position: absolute;

```

```
font-weight: 600;
}
.show-header {
background: #00c0ef;
color: #fff;
height: 80px;
border-radius: 5px;
position: relative;
}
.show-num {
font-size: 38px;
font-weight: 600;
padding: 5px;
}
</style>
```

## 24 会员、员工密码重置与修改

### 24.1 密码重置

#### 24.1.1 新建实体类

```
package com.xq.web.home.entity;

import lombok.Data;

@Data
public class ResetPassword {
 private Long userId;
 private String userType;
 private String password;
 private String oldPassword;
}
```

#### 24.1.2 HomeController添加resetPassword()方法

```
package com.xq.web.home.controller;

import org.springframework.web.bind.annotation.RequestMapping("/api/home")
import org.springframework.web.bind.annotation.RestController

public class HomeController {
 @Autowired
 private MemberService memberService;
 @Autowired
```

```

private SysUserService sysUserService;

//修改密码
@PostMapping("/resetPassword")
public ResultVo resetPassword(@RequestBody ResetPassword
resetPassword){
 if(resetPassword.getUserType().equals("1")){
 Member member = new Member();
 member.setMemberId(resetPassword.getUserId());
 member.setPassword("666666");
 memberService.updateById(member);
 return ResultUtils.success("密码重置成功!");
 }else if(resetPassword.getUserType().equals("2")){
 SysUser user = new SysUser();
 user.setUserId(resetPassword.getUserId());
 String password =
DigestUtils.md5DigestAsHex("666666".getBytes());
 user.setPassword(password);
 sysUserService.updateById(user);
 return ResultUtils.success("密码重置成功!");
 }else{
 return ResultUtils.error("用户类型错误!");
 }
}
}

```

### 24.1.3 api/home下新建HomeModel.ts

```

export type ResetPasswordParam = {
 userId:string,
 userType:string,
 password?:string,
 oldPassword?:string
}

```

```

import http from "@/http";
import { ResetPasswordParam } from "../HomeModel";
//重置密码
export const resetPasswordApi = (parm:ResetPasswordParam)=>{
 return http.post("/api/home/resetPassword",parm)
}

```

## 24.1.4 useUser.ts添加resetPasBtn()方法

```
import { resetPasswordApi } from "@/api/home"
export default function useUser(getList: FuncList) {

 //重置密码
 const resetPasBtn = async (row: AddUserModel) => {
 const confirm = await global.$myconfirm('确定重置密码吗, 重置后密码为【666666】?')
 if (confirm) {
 let parm = {
 userId: row.userId,
 userType: "2"

 }
 let res = await resetPasswordApi(parm)
 if (res && res.code == 200) {
 ElMessage.success(res.msg)
 }
 }
 }

 return {
 resetPasBtn
 }
}
```

修改UserList.vue组件:

```
<el-table-column label="操作" align="center" width="320">
 <template #default="scope">
 <el-button v-permission="['sys:user:edit']" type="primary" :icon="Edit" size="default" @click="editBtn(scope.row)">编辑</el-button>
 <el-button v-permission="['sys:user:delete']" type="danger" :icon="Delete" size="default" @click="deleteBtn(scope.row)">删除</el-button>
 <el-button v-permission="['sys:user:resetPassword']" type="warning" :icon="Edit" size="default" @click="resetPasBtn(scope.row)">重置密码</el-button>
 </template>
</el-table-column>

<script setup lang="ts">
import AddUser from "@/components/AddUser.vue";
import { Plus, Edit, Delete, Search, Close } from "@element-plus/icons-vue";
import useTable from "@/composables/user/useTable";
import useUser from "@/composables/user/useUser";
//表格
const { listParam, getList, searchBtn, resetBtn, tableList, sizeChange, currentChange, tableHeight, refresh } = useTable();
//新增、编辑
const { addBtn, editBtn, deleteBtn, addRef, resetPasBtn } = useUser(getList);
</script>

<style scoped></style>
```

## 24.1.5 useMember.ts添加resetPasBtn()方法

```
import { resetPasswordApi } from "@/api/home"

export default function useMember(getList: FuncList){
 //重置密码
```



```

const resetPasBtn = async (row: MemberType) => {
 const confirm = await global.$myconfirm('确定重置密码吗, 重置后密码为【666666】?')
 if (confirm) {
 let parm = {
 userId: row.memberId,
 userType: "1"
 }
 let res = await resetPasswordApi(parm)
 if (res && res.code == 200) {
 ElMessage.success(res.msg)
 }
 }
}

return {
 resetPasBtn,
}
}

```

修改MemberList.vue组件:

```

<template #dropdown>
 <el-dropdown-menu>
 <el-dropdown-item :icon="ChatLineSquare" @click="rechargeBtn(scope.row)">充值</el-dropdown-item>
 <el-dropdown-item :icon="Edit" @click="editBtn(scope.row)">编辑</el-dropdown-item>
 <el-dropdown-item type="danger" :icon="Delete" @click="deleteBtn(scope.row)">删除</el-dropdown-item>
 <el-dropdown-item :icon="Edit" @click="resetPasBtn(scope.row)">重置密码</el-dropdown-item>
 </el-dropdown-menu>
</template>

```

```

//新增、编辑操作
const { addBtn, editBtn, deleteBtn, addRef, resetPasBtn } = useMember(getList);
//办卡
const { joinRef, joinBtn } = useJoin();
//充值
const { rechargeRef, rechargeBtn } = useRecharge();
</script>

```

## 24.2 密码修改

### 24.2.1 HomeController添加updatePassword()方法

```

//修改密码
@PostMapping("/updatePassword")
public ResultVo updatePassword(@RequestBody ResetPassword resetPassword) {
 if (resetPassword.getUserType().equals("1")) {
 //验证原密码是否正确
 Member member1 =
 memberService.getById(resetPassword.getUserId());

```

```

 if
(!member1.getPassword().equals(resetPassword.getOldPassword())) {
 return ResultUtils.error("原密码不正确!");
 }
 //修改
 Member member = new Member();
 member.setMemberId(resetPassword.getUserId());
 member.setPassword(resetPassword.getPassword());
 memberService.updateById(member);
 return ResultUtils.success("修改密码成功!");
 } else if (resetPassword.getUserType().equals("2")) {
 //验证原密码
 SysUser sysUser =
sysUserService.getById(resetPassword.getUserId());
 String oldPas =
DigestUtils.md5DigestAsHex(resetPassword.getOldPassword().getBytes());
 if(!oldPas.equals(sysUser.getPassword())){
 return ResultUtils.error("原密码不正确!");
 }
 SysUser user = new SysUser();
 user.setUserId(resetPassword.getUserId());
 String password =
DigestUtils.md5DigestAsHex(resetPassword.getPassword().getBytes());
 user.setPassword(password);
 sysUserService.updateById(user);
 return ResultUtils.success("修改密码成功!");
 } else {
 return ResultUtils.error("用户类型错误!");
 }
}

```

## 24.2.2 api/home/index.ts添加updatePasswordApi()方法

```

//修改密码
export const updatePasswordApi = (parm:ResetPasswordParam)=>{
 return http.post("/api/home/updatePassword",parm)
}

```

## 24.2.3 调整LoginOut.vue组件

调整layout/header/下面的LoginOut.vue组件

```
<template>
 <el-dropdown>

 <template #dropdown>
 <el-dropdown-menu>
 <el-dropdown-item @click="updateBtn">修改密码</el-dropdown-
item>
 <el-dropdown-item @click="loginOut">退出登录</el-dropdown-
item>
 </el-dropdown-menu>
 </template>
 </el-dropdown>
 <!-- 修改密码弹框 -->
 <SysDialog
 :title="dialog.title"
 :width="dialog.width"
 :height="dialog.height"
 :visible="dialog.visible"
 @onClose="onClose"
 @onConfirm="commit"
 >
 <template v-slot:content>
 <el-form
 :model="parm"
 ref="upForm"
 :rules="rules"
 label-width="80px"
 size="default"
 >
 <el-form-item prop="oldPassword" label="原密码">
 <el-input v-model="parm.oldPassword"></el-input>
 </el-form-item>
 <el-form-item prop="password" label="新密码">
 <el-input v-model="parm.password"></el-input>
 </el-form-item>
 </el-form>
 </template>
 </SysDialog>
</template>
```

```

<script setup lang="ts">
import { ResetPasswordParm } from "@/api/home/HomeModel";
import SysDialog from "@/components/SysDialog.vue";
import useDialog from "@/hooks/useDialog";
import useInstance from "@/hooks/useInstance";
import { userStore } from "@/store/user";
import { ElMessage, FormInstance } from "element-plus";
import { reactive, ref } from "vue";
import { updatePasswordApi } from "@/api/home";
const { dialog, onClose, onConfirm, onShow } = useDialog();
const { global } = useInstance();
const upForm = ref<FormInstance>();
const store = userStore();
const loginOut = async () => {
 const confirm = await global.$myconfirm("确定退出登录吗? ");
 if (confirm) {
 //清空用户信息
 store.setToken("");
 store.setUserId("");
 store.setUserType("");
 localStorage.clear();
 window.location.href = "/login";
 }
};
const updateBtn = () => {
 dialog.height = 150;
 dialog.title = "修改密码";
 onShow();
};
const parm = reactive<ResetPasswordParm>({
 userId: store.getUserId,
 userType: store.getUserType,
 oldPassword: "",
 password: "",
});
const rules = reactive([
 oldPassword: [
 {
 required: true,
 trigger: "blur",
 message: "原密码不能为空!",
 },
],
 password: [

```

```

 {
 required: true,
 trigger: "blur",
 message: "新密码不能为空!",
 },
],
});
const commit = () => {
 upForm.value?.validate(async (valid) => {
 if (valid) {
 let res = await updatePasswordApi(parm);
 if (res && res.code == 200) {
 ElMessage.success(res.msg);
 //清空用户信息
 store.setToken("");
 store.setUserId("");
 store.setUserType("");
 localStorage.clear();
 window.location.href = "/login";
 }
 }
 });
};
</script>

<style scoped>
.userimg {
 height: 38px;
 width: 38px;
 border-radius: 50%;
}
</style>

```

## 25 认证和授权

Spring Security是Spring提供的一个安全框架，提供认证( Authentication)和授权( Authorization)功能，核心技术使用了servlet、ioc和Aop。

### 认证:

认证简单的说就是登录，当用户访问系统时，需要到数据库查看，有没有这个用户，有用户了才允许进入系统。

### 授权:

授权指的就是用户拥有哪些权限，如用户可以操作的菜单、按钮、数据等权限。

## 25.1 在项目中引入SpringSecurity环境

### 25.1.1 在web模块引入如下依赖

```
<!-- spring security 启动器-->
<dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

### 25.1.2 新建spring security配置类SpringSecurityConfig

```
package com.xq.config;
import org.springframework.context.annotation.Configuration;
import
org.springframework.security.config.annotation.method.configurati
on.EnableGlobalMethodSecurity;
import
org.springframework.security.config.annotation.web.configuration.
EnablewebSecurity;
import
org.springframework.security.config.annotation.web.configuration.
WebSecurityConfigurerAdapter;

@Configuration
@EnablewebSecurity
@EnableGlobalMethodSecurity(prePostEnabled = true) //开启
springSecurity注解的使用权限
public class SpringSecurityConfig extends
WebSecurityConfigurerAdapter {

}
```

此时启动项目访问前端页面会报错：

```
● GET http://localhost:8089/api/login/getInfo?userId=3 net::ERR_FAILED 302 (Found) index.ts:124
● Access to XMLHttpRequest at 'http://localhost:8089/api/login/getInfo?userId=3' from origin 'http://localhost:8080' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource. courseList:1
 进入错误 index.ts:57
 {transitional: {…}, adapter: Array(2), transformRequest: Array(1), transformResponse: Array(1), timeout: 10000, …} index.ts:39
● GET http://localhost:8089/api/login/getInfo?userId=3 net::ERR_FAILED 302 (Found) index.ts:124
● Access to XMLHttpRequest at 'http://localhost:8089/api/login/getInfo?userId=3' from origin 'http://localhost:8080' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource. courseList:1
 进入错误 index.ts:57
```

解决方法，在spring security配置类重写configure()方法，并配置跨域

```

@Configuration
@EnableWebSecurity
@EnableGlobalMethodSecurity(prePostEnabled = true)
public class SpringSecurityConfig extends
WebSecurityConfigurerAdapter {
 @Override
 protected void configure(HttpSecurity http) throws Exception
 {
 //解决跨域问题
 http.cors().and().headers().frameOptions().disable();
 }
}

```

此时访问登录页面，又出现了验证码无法显示的问题，我们需要进一步配置，关闭跨域请求伪造：

```

@Configuration
@EnableWebSecurity
@EnableGlobalMethodSecurity(prePostEnabled = true)
public class SpringSecurityConfig extends
WebSecurityConfigurerAdapter {
 @Override
 protected void configure(HttpSecurity http) throws Exception
 {
 //解决跨域问题
 http.cors().and().headers().frameOptions().disable();
 //关闭跨域请求伪造
 http.csrf().disable().authorizeRequests()
 //放行登录、验证码请求,其他的所有请求都要认证

 .antMatchers("/api/login/image", "/api/login/login").permitAll()
 //
 .anyRequest().authenticated();
 }
}

```

## 25.2 用户认证

先将数据库重新导入：

此电脑 > 本地磁盘 (G:) > 录课视频 > springboot+vue项目 > 2.资料

名称	修改日期	类型	大小
前端项目用到的图片	2024-05-10 15:06	文件夹	
minio.exe	2024-05-19 14:42	应用程序	101,545 KB
node-v20.13.0-x64.msi	2024-05-08 12:26	Windows Install...	25,984 KB
springSecurity数据库脚本.sql	2024-03-11 0:50	SQL Text File	58 KB
数据库脚本.sql	2024-05-27 10:26	SQL Text File	25 KB

## 25.2.1 SpringSecurity认证的基本原理

Spring Security登录认证主要涉及两个重要的接口 UserDetails和UserDetailsService接口。

- UserDetails接口主要定义了一个方法 loadUserByUsername(String username)用于完成用户信息的查询，其中username就是登录时的登录名称，登录认证时，需要自定义一个实现类实现UserDetailsService接口，完成数据库查询，该接口返回UserDetail。
- UserDetails主要用于封装认证成功时的用户信息，即UserDetailsService返回的用户信息，可以用Spring Security自己的User对象，但是最好是实现UserDetail接口，自定义用户对象。

### Spring Security认证步骤:

- 自定义UserDetails 实体对象字段不满足时需要自定义UserDetails，实际开发中一般都要自定义UserDetails
- 自定义UserDetailsService 主要用于从数据库查询用户信息
- 用户认证失败的处理器 认证失败需要返回JSON数据，给前端判断
- 用户无权限访问的处理器 无权限访问需要返回JSON数据，给前端判断
- 自定义异常处理器 发生异常的处理
- 配置Spring Security配置类 把上面自定义的处理器交给Spring Security

#### 25.2.1.1 自定义UserDetails

UserDetails属于spring security自己的用户实体对象，但是他里面的属性字段很少，不符合实际开发需求,因此实际开发中需要扩展该实体；我们看看SpringSecurity设计的UserDetails:

```
public interface UserDetails extends Serializable {
 //用户权限字段的集合
 Collection<? extends GrantedAuthority> getAuthorities();
 //登录密码
 String getPassword();
 //登录账户
 String getUsername();
 //帐户是否过期(1 未过期, 0已过期)
 boolean isAccountNonExpired();
 //帐户是否被锁定(1 未锁定, 0已锁定)
 boolean isAccountNonLocked();
 //密码是否过期(1 未过期, 0已过期)
 boolean isCredentialsNonExpired();
 //帐户是否可用(1 可用, 0 删除用户)
 boolean isEnabled();
}
```



## 调整用户表和会员表:

列名	数据类型	长度	默认	主键?	非空?	Unsigned	自增?	Zerofill?	注释
member_id	int	11		<input checked="" type="checkbox"/>	<input checked="" type="checkbox"/>	<input type="checkbox"/>	<input checked="" type="checkbox"/>	<input type="checkbox"/>	会员id
name	varchar	64		<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	姓名
sex	varchar	2		<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	性别
phone	varchar	15		<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	电话
age	int	11		<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	年龄
birthday	varchar	36		<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	生日
height	int	11		<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	身高
weight	int	11		<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	体重
waist	int	11		<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	腰围
join_time	varchar	36		<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	加入时间
end_time	varchar	36		<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	到期时间
username	varchar	64		<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	会员卡号
password	varchar	128		<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	密码
status	varchar	2		<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	状态 1: 启用 0: 停用
money	decimal	18,2	0.00	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	充值金额
card_type	varchar	64		<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	会员类型
card_day	int	11		<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	天数
price	decimal	18,2	0.00	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	会员价格
is_account_non_expired	tinyint	4	0	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	帐户是否过期 (1未过期 0已过期)
is_account_non_locked	tinyint	4	0	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	帐户是否被锁定 (1未锁定 0已锁定)
is_credentials_non_expired	tinyint	4	0	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	密码是否过期 (1未过期 0已过期)
is_enabled	tinyint	4	0	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	帐户是否可用 (1可用 0删除用户)

列名	数据类型	长度	默认	主键?	非空?	Unsigned	自增?	Zerofill?	注释
user_id	int	11		<input checked="" type="checkbox"/>	<input checked="" type="checkbox"/>	<input type="checkbox"/>	<input checked="" type="checkbox"/>	<input type="checkbox"/>	员工id
username	varchar	64		<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	帐户(员工编号)
password	varchar	128		<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	密码
phone	varchar	15		<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	电话
email	varchar	36		<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	邮箱
sex	varchar	2		<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	性别 0: 男 1: 女
salary	decimal	18,2		<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	工资
user_type	varchar	2		<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	类型 (1: 员工 2: 教练)
status	varchar	2		<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	状态 0: 停用 1: 启用
is_admin	varchar	2		<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	是否是管理员 (1: 是 0: 否)
is_account_non_expired	tinyint	4		<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	帐户是否过期 (1 未过期, 0已过期)
is_account_non_locked	tinyint	4		<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	帐户是否被锁定 (1 未锁定, 0已锁定)
is_credentials_non_expired	tinyint	4		<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	密码是否过期 (1 未过期, 0已过期)
is_enabled	tinyint	4		<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	帐户是否可用 (1 可用, 0 删除用户)
nick_name	varchar	64		<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	姓名
create_time	datetime			<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	创建时间
update_time	datetime			<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	<input type="checkbox"/>	更新时间

## 用户实体类和会员实体类修改为如下

```
package com.xq.web.sys_user.entity;

import com.baomidou.mybatisplus.annotation.IdType;
import com.baomidou.mybatisplus.annotation.TableField;
import com.baomidou.mybatisplus.annotation.TableId;
import com.baomidou.mybatisplus.annotation.TableName;
import lombok.Data;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;

import java.math.BigDecimal;
import java.util.Collection;
import java.util.Date;

@Data
@TableName("sys_user")
public class SysUser implements UserDetails {
 @TableId(type = IdType.AUTO)
 private Long userId;
 //表明roleId字段不属于sys_user表, 需要排除
 @TableField(exist = false)
```

```

private Long roleId;
private String username;
private String password;
private String phone;
private String email;
private String sex;
private String isAdmin;
//类型 (1: 员工 2: 教练)
private String userType;
//状态 0: 停用 1: 启用
private String status;
private BigDecimal salary;
private String nickName;
//创建时间
private Date createTime;
//更新时间
private Date updateTime;
//帐户是否过期(1 未过期, 0已过期)
private boolean isAccountNonExpired = true;
//帐户是否被锁定(1 未锁定, 0已锁定)
private boolean isAccountNonLocked = true;
//密码是否过期(1 未过期, 0已过期)
private boolean isCredentialsNonExpired = true;
//帐户是否可用(1 可用, 0 删除用户)
private boolean isEnabled = true;
//用户权限字段的集合 authorities字段不属于sys_user表, 需要排除
@TableField(exist = false)
Collection<? extends GrantedAuthority> authorities;
}

```

```

package com.xq.web.member.entity;

import com.baomidou.mybatisplus.annotation.IdType;
import com.baomidou.mybatisplus.annotation.TableField;
import com.baomidou.mybatisplus.annotation.TableId;
import com.baomidou.mybatisplus.annotation.TableName;
import lombok.Data;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;

import java.math.BigDecimal;
import java.util.Collection;

@Data
@TableName("member")

```

```

public class Member implements UserDetails {
 @TableId(type = IdType.AUTO)
 private Long memberId;
 //排除roleId字段
 @TableField(exist = false)
 private Long roleId;
 private String name;
 private String sex;
 private String phone;
 private Integer age;
 private String birthday;
 private Integer height;
 private Integer weight;
 private Integer waist;
 private String joinTime;
 private String endTime;
 private String username;
 private String password;
 private String status;

 private String cardType;
 private Integer cardDay;
 private BigDecimal money;
 private BigDecimal price;

 //帐户是否过期(1 未过期, 0已过期)
 private boolean isAccountNonExpired = true;
 //帐户是否被锁定(1 未锁定, 0已锁定)
 private boolean isAccountNonLocked = true;
 //密码是否过期(1 未过期, 0已过期)
 private boolean isCredentialsNonExpired = true;
 //帐户是否可用(1 可用, 0 删除用户)
 private boolean isEnabled = true;
 //用户权限字段的集合
 //表明authorities字段不属于sys_user表, 需要排除
 @TableField(exist = false)
 Collection<? extends GrantedAuthority> authorities;
}

```

**SysUserService和MemberService接口添加loadUser()方法**

```
//根据用户名查用户
SysUser loadUser(String username);

// 实现类
@Override
public SysUser loadUser(String username) {
 QueryWrapper<SysUser> query = new QueryWrapper<>();
 query.lambda().eq(SysUser::getUsername, username);
 return this.baseMapper.selectOne(query);
}
```

```
//根据用户名查用户
Member loadUser(String username);

// 实现类
@Override
public Member loadUser(String username) {
 QueryWrapper<Member> query = new QueryWrapper<>();
 query.lambda().eq(Member::getUsername, username);
 return this.baseMapper.selectOne(query);
}
```

### 25.2.1.2 自定义UserDetailsService

自定义UserDetailsService的目的：也就是用户认证的时候调用，告诉spring security到哪里查询用户信息。

```
package com.xq.config.springSecurity;

import com.xq.web.member.service.MembersService;
import com.xq.web.sys_user.service.SysUserService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Component;

@Component("customizedUserDetailsService")
public class CustomizedUserDetailsService implements
UserDetailsService {
 @Autowired
```

```

 private SysUserService sysUserService;
 @Autowired
 private MemberService memberService;
 @Override
 public UserDetails loadUserByUsername(String s) throws
UsernameNotFoundException {
 //认证
 int index = s.indexOf(":");
 String username = s.substring(0, index);
 String userType = s.substring(index+1,s.length());
 UserDetails user = null;
 if(userType.equals("1")){ //会员
 user = memberService.loadUser(username);
 }else if(userType.equals("2")) { //员工
 user = sysUserService.loadUser(username);
 }else{
 throw new UsernameNotFoundException("用户类型不存在!");
 }
 if(user == null){
 throw new UsernameNotFoundException("用户名或密码错
误!");
 }
 //授权
 return user;
 }
 }
}

```

### 25.2.1.3 调整登录接口

#### (1) SpringSecurityConfig配置类添加密码加密bean

```

@Bean
public PasswordEncoder passwordEncoder(){
 return new BCryptPasswordEncoder();
}

```

#### (2) 配置我们自定义的UserDetailService

```

package com.xq.config.springSecurity;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import
org.springframework.security.config.annotation.authentication.bui
lders.AuthenticationManagerBuilder;

```

```
import
org.springframework.security.config.annotation.method.configurati
on.EnableGlobalMethodSecurity;
import
org.springframework.security.config.annotation.web.builders.HttpS
ecurity;
import
org.springframework.security.config.annotation.web.configuration.
EnableWebSecurity;
import
org.springframework.security.config.annotation.web.configuration.
WebSecurityConfigurerAdapter;
import
org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import
org.springframework.security.crypto.password.PasswordEncoder;
import
org.springframework.security.authentication.AuthenticationManager
;

@Configuration
@EnableWebSecurity
@EnableGlobalMethodSecurity(prePostEnabled = true)
public class SpringSecurityConfig extends
WebSecurityConfigurerAdapter {
 @Override
 protected void configure(HttpSecurity http) throws Exception
 {
 //解决跨域问题
 http.cors().and().headers().frameOptions().disable();
 //关闭跨域请求伪造
 http.csrf().disable().authorizeRequests()
 //放行登录、验证码请求,其他的所有请求都要认证

 .antMatchers("/api/login/image", "/api/login/login").permitAll()
 //其他的任何接口访问都需要认证
 .anyRequest().authenticated();
 }

 @Bean
 public PasswordEncoder passwordEncoder(){
 return new BCryptPasswordEncoder();
 }
}
```

```

@Autowired
CustomizedUserDetailService customizedUserDetailService;

//配置我们自定义的UserDetailsService
@Override
protected void configure(AuthenticationManagerBuilder auth)
throws Exception {
 auth.userDetailsService(customizedUserDetailService);
}

//注入AuthenticationManager
@Bean
@Override
public AuthenticationManager authenticationManagerBean()
throws Exception {
 return super.authenticationManagerBean();
}
}

```

### (3) 修改登录方法

```

@Autowired
PasswordEncoder passwordEncoder;

@Autowired
AuthenticationManager authenticationManager;

@PostMapping("/login")
public ResultVo login(HttpServletRequest request, @RequestBody
LoginParam param) {
 if(StringUtils.isEmpty(param.getUsername()) ||
StringUtils.isEmpty(param.getPassword()) ||
StringUtils.isEmpty(param.getUserType()) ||
StringUtils.isEmpty(param.getCode())){
 return ResultUtils.error("用户名、密码、验证码或用户类型不能为
空!");
 }
 //获取session
 HttpSession session = request.getSession();
 //获取验证码
 String code = (String) session.getAttribute("code");
 if(StringUtils.isEmpty(code)){
 return ResultUtils.error("验证码过期!");
 }
 //验证验证码

```

```

 if (!code.equals(param.getCode())) {
 return ResultUtils.error("验证码错误!");
 }
 //String password =
 DigestUtils.md5DigestAsHex(param.getPassword().getBytes());
 String password =
 passwordEncoder.encode(param.getPassword());
 //构造spring security需要的token
 UsernamePasswordAuthenticationToken
 usernamePasswordAuthenticationToken = new
 UsernamePasswordAuthenticationToken(param.getUsername()+":"+param
 .getUserType(),param.getPassword());
 Authentication authenticate =
 authenticationManager.authenticate(usernamePasswordAuthentication
 Token);

 SecurityContextHolder.getContext().setAuthentication(authenticat
 e);
 //用户类型判断
 if (param.getUserType().equals("1")) { //会员
 Member user = (Member)authenticate.getPrincipal();
 //生成token
 Map<String, String> map = new HashMap<>();
 map.put("userId", Long.toString(user.getMemberId()));
 map.put("username", user.getUsername());
 map.put("userType", param.getUserType());
 String token = jwtUtils.generateToken(map);
 //返回登录成功信息
 LoginResult result = new LoginResult();
 result.setToken(token);
 result.setUserId(user.getMemberId());
 result.setUsername(user.getName());
 result.setUserType(param.getUserType());
 return ResultUtils.success("登录成功", result);
 } else if (param.getUserType().equals("2")) { //员工
 SysUser user = (SysUser)authenticate.getPrincipal();
 //生成token
 Map<String, String> map = new HashMap<>();
 map.put("userId", Long.toString(user.getUserId()));
 map.put("username", user.getUsername());
 map.put("userType", param.getUserType());
 String token = jwtUtils.generateToken(map);
 //返回登录成功信息
 LoginResult result = new LoginResult();
 result.setToken(token);

```



```

 result.setUserId(user.getUserId());
 result.setUsername(user.getNickName());
 result.setUserType(param.getUserType());
 return ResultUtils.success("登录成功", result);
 } else {
 return ResultUtils.error("用户类型错误!");
 }
}

```

测试：使用admin用户登录，密码是123456,用户类型是员工。

## 25.3 授权

### 25.3.1 开启SpringSecurity注解的使用

```

@Configuration
@EnableWebSecurity
@EnableGlobalMethodSecurity(prePostEnabled = true) //开启springSecurity注解的使用权限
public class SpringSecurityConfig extends WebSecurityConfigurerAdapter {
 @Override
 protected void configure(HttpSecurity http) throws Exception {
 //解决跨域问题
 http.cors().and().headers().frameOptions().disable();
 //关闭跨域请求伪造
 http.csrf().disable().authorizeRequests()
 //放行登录、验证码请求,其他的所有请求都要认证
 .antMatchers(...antPatterns: "/api/login/image", "/api/login/login").permitAll()
 //其他的任何接口访问都需要认证
 .anyRequest().authenticated();
 }
}

```

### 25.3.2 定义授权逻辑

修改CustomizedUserService类中的loadUserByUsername方法：

```

@Autowired
SysMenuService sysMenuService;

public UserDetails loadUserByUsername(String s) throws
UsernameNotFoundException {
 //admin:2 ls:1
 int index = s.indexOf(":");
 String username = s.substring(0, index);
 String userType = s.substring(index + 1);
 //认证:登录
 if (userType.equals("1")) {
 Member user = memberService.loadUser(username);
 if (user == null) {
 throw new UsernameNotFoundException("用户名或密码错误!");
 }
 //授权:把该用户拥有的按钮权限,交给spring security进行管理
 }
}

```

```

 //获取会员的按钮权限
 List<SysMenu> menuList =
sysMenuService.getMenuByMemberId(user.getMemberId());
 //取出code字段
 List<String> collect =
 menuList.stream().map(item ->
item.getCode()).filter(item -> item != null &&
StringUtils.isNotEmpty(item)).collect(Collectors.toList());
 String[] strings = collect.toArray(new
String[collect.size()]);
 List<GrantedAuthority> authorityList =
AuthorityUtils.createAuthorityList(strings);
 //授权
 user.setAuthorities(authorityList);
 return user;
 } else if (userType.equals("2")) {
 SysUser user = sysUserService.loadUser(username);
 if (user == null) {
 throw new UsernameNotFoundException("用户名或密码错
误!");
 }
 //授权:把该用户拥有的按钮权限,交给spring security进行管理
 List<SysMenu> menuList = null;
 if(StringUtils.isNotEmpty(user.getIsAdmin()) &&
user.getIsAdmin().equals("1")){
 menuList = sysMenuService.list();
 }else{
 menuList =
sysMenuService.getMenuByUserId(user.getUserId());
 }
 //取出code字段
 List<String> collect =
 menuList.stream().map(item ->
item.getCode()).filter(item -> item != null &&
StringUtils.isNotEmpty(item)).collect(Collectors.toList());
 String[] strings = collect.toArray(new
String[collect.size()]);
 List<GrantedAuthority> authorityList =
AuthorityUtils.createAuthorityList(strings);
 //授权
 user.setAuthorities(authorityList);
 return user;
 } else {
 throw new UsernameNotFoundException("用户类型错误!");
 }
}

```

```
}
```

### 25.3.3 使用SpringSecurity注解

```
@RestController
@RequestMapping("/api/role")
public class SysRoleController {

 6 usages
 @Autowired
 private SysRoleService sysRoleService;

 // 新增角色
 no usages
 @PostMapping
 @PreAuthorize("hasAuthority('sys:role:add')")
 public ResultVo addRole(@RequestBody SysRole role){
 role.setCreateTime(new Date());
 boolean save = sysRoleService.save(role);
 if(save){
 return ResultUtils.success(msg: "新增成功!");
 }
 return ResultUtils.error(msg: "新增失败!");
 }
}
```

此时我们发现新增成功，是因为当前用户具备sys:role:add的权限。如果我们将权限修改为sys:role:add1.因为这个权限压根不存在，在新增操作的时候，就会失败。

## 25.4 自定义认证失败处理器

### 25.4.1 自定义异常处理器

```
package com.xq.config.springSecurity;

import org.springframework.security.core.AuthenticationException;

public class CustomerAuthenionException extends
AuthenticationException {
 public CustomerAuthenionException(String msg) {
 super(msg);
 }
}
```

### 25.4.2 自定义无权限访问的处理器

```
package com.xq.config.springSecurity;

import com.alibaba.fastjson.JSONObject;
import com.alibaba.fastjson.serializer.SerializerFeature;
import com.xq.utils.ResultVo;
import org.springframework.security.access.AccessDeniedException;
import
org.springframework.security.web.access.AccessDeniedHandler;
import org.springframework.stereotype.Component;
```

```

import javax.servlet.ServletException;
import javax.servlet.ServletOutputStream;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.io.IOException;

@Component("customAccessDeineHandler")
public class CustomAccessDeineHandler implements
AccessDeniedHandler {
 @Override
 public void handle(HttpServletRequest httpServletRequest,
HttpServletResponse httpServletResponse, AccessDeniedException e)
throws IOException, ServletException {
 String res = JSONObject.toJSONString(new ResultVo("无权限
访问,请联系管理员!", 700, null),
SerializerFeature.DisableCircularReferenceDetect);
 //设置返回格式

 httpServletResponse.setContentType("application/json;charset=UTF
-8");

 ServletOutputStream out =
httpServletResponse.getOutputStream();
 out.write(res.getBytes("UTF-8"));
 out.flush();
 out.close();
 }
}

```

### 25.4.3 自定义认证失败处理器

```

package com.xq.config.springSecurity;

import com.alibaba.fastjson.JSONObject;
import com.alibaba.fastjson.serializer.SerializerFeature;
import com.xq.config.springSecurity.CustomerAuthenionException;
import com.xq.utils.ResultVo;
import org.springframework.security.authentication.*;
import org.springframework.security.core.AuthenticationException;
import org.springframework.security.web.AuthenticationEntryPoint;
import org.springframework.stereotype.Component;

import javax.servlet.ServletException;
import javax.servlet.ServletOutputStream;
import javax.servlet.http.HttpServletRequest;

```

```

import javax.servlet.http.HttpServletResponse;
import java.io.IOException;

@Component("loginFailureHandler")
public class LoginFailureHandler implements
AuthenticationEntryPoint {
 @Override
 public void commence(HttpServletRequest httpServletRequest,
HttpServletResponse httpServletResponse, AuthenticationException
e) throws IOException, ServletException {
 int code = 500;
 String str = "";
 if(e instanceof AccountExpiredException){
 str = "账户过期，登录失败!";
 }else if(e instanceof BadCredentialsException){
 str = "用户名或密码错误，登录失败!";
 }else if(e instanceof CredentialsExpiredException){
 str = "密码过期，登录失败!";
 }else if(e instanceof DisabledException){
 str = "账户被禁用，登录失败!";
 }else if(e instanceof LockedException){
 str = "账户被锁，登录失败!";
 }else if(e instanceof
InternalAuthenticationServiceException){
 str = "账户不存在，登录失败!";
 }else if(e instanceof CustomerAuthenionException){
 code = 600;
 str = e.getMessage();
 }else if(e instanceof
InsufficientAuthenticationException){
 str = "无权限访问资源!";
 }
 else{
 str = "登录失败!";
 }
 String res = JSONObject.toJSONString(new
ResultVo(str,code,null),
SerializerFeature.DisableCircularReferenceDetect);
 //设置返回格式

 httpServletResponse.setContentType("application/json;charset=UTF
-8");
 ServletOutputStream out =
httpServletResponse.getOutputStream();
 out.write(res.getBytes("UTF-8"));
 }
}

```

```

 out.flush();
 out.close();
 }
}

```

## 25.4.4 在Spring Security配置类配置上面自定义处理器

```

package com.xq.config.springSecurity;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.authentication.builders.AuthenticationManagerBuilder;
import org.springframework.security.config.annotation.method.configuration.EnableGlobalMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configuration.WebSecurityConfigurerAdapter;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.authentication.AuthenticationManager;

@Configuration
@EnableWebSecurity
@EnableGlobalMethodSecurity(prePostEnabled = true) //开启
springSecurity注解的使用权限
public class SpringSecurityConfig extends
WebSecurityConfigurerAdapter {

 @Autowired
 private LoginFailureHandler loginFailureHandler;
 @Autowired
 private CustomAccessDeineHandler customAccessDeineHandler;
 @Override

```

```

protected void configure(HttpSecurity http) throws Exception
{
 //解决跨域问题
 http.cors().and().headers().frameOptions().disable();
 //关闭跨域请求伪造
 http.csrf().disable().authorizeRequests()
 //放行登录、验证码请求,其他的所有请求都要认证

.antMatchers("/api/login/image", "/api/login/login").permitAll()
 //其他的任何接口访问都需要认证
 .anyRequest().authenticated()
 .and().exceptionHandling()
 .authenticationEntryPoint(loginFailureHandler)
 .accessDeniedHandler(customAccessDeineHandler);;
}

@Bean
public PasswordEncoder passwordEncoder(){
 return new BCryptPasswordEncoder();
}

@Autowired
CustomizedUserDetailsService customizedUserDetailsService;

//配置我们自定义的UserService
@Override
protected void configure(AuthenticationManagerBuilder auth)
throws Exception {
 auth.userDetailsService(customizedUserDetailsService);
}

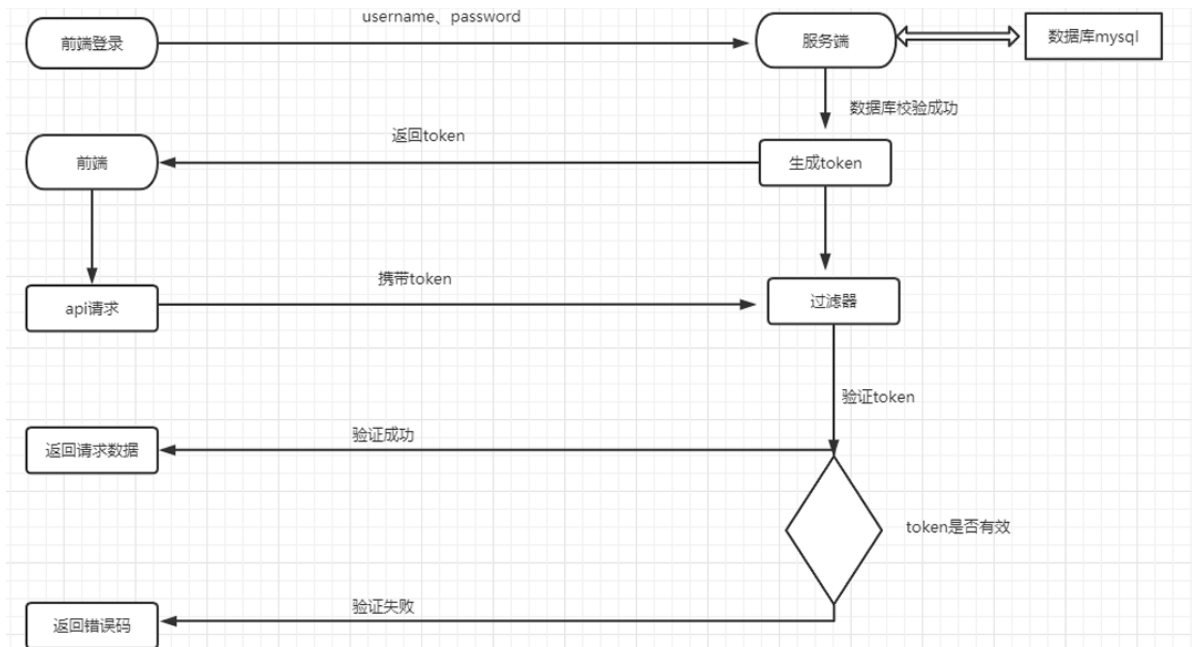
//注入AuthenticationManager
@Bean
@Override
public AuthenticationManager authenticationManagerBean()
throws Exception {
 return super.authenticationManagerBean();
}
}

```

## 25.5 token验证

### 25.5.1 token验证流程

Token是服务端生成的一串字符串，以作客户端进行请求的一个令牌，当第一次登录后，服务器生成一个Token便将此Token返回给客户端，以后客户端只需带上这个Token前来请求数据即可，无需再次带上用户名和密码。



### 25.5.2 编写token过滤器

```
package com.xq.config.springSecurity;

import com.auth0.jwt.interfaces.Claim;
import com.auth0.jwt.interfaces.DecodedJWT;

import com.xq.jwt.JwtUtils;
import lombok.Data;
import org.apache.commons.lang.StringUtils;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.AuthenticationException;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
```



```

import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import javax.servlet.FilterChain;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.util.Collections;
import java.util.List;
import java.util.Map;

@Data
@Component("checkTokenFilter")
public class CheckTokenFilter extends OncePerRequestFilter {
 @Value("#{${ignore.url}'.split(',')}")
 private List<String> ignoreUrl = Collections.emptyList();
 @Autowired
 private JwtUtils jwtUtils;
 @Autowired
 private CustomizedUserDetailsService
customizedUserDetailsService;
 @Autowired
 private LoginFailureHandler loginFailureHandler;

 @Override
 protected void doFilterInternal(HttpServletRequest request,
HttpServletResponse response, FilterChain filterChain) throws
ServletException, IOException {
 try {
 //获取请求的url
 String url = request.getRequestURI();
 //判断是否放行
 if (!ignoreUrl.contains(url) &&
!url.startsWith("/images/")) {
 validateToken(request);
 }
 } catch (AuthenticationException e) {
 loginFailureHandler.commence(request, response, e);
 return;
 }
 // 放行
 filterChain.doFilter(request, response);
 }
}

```

```

private void validateToken(HttpServletRequest request) {
 //获取token数据
 String token = request.getHeader("token");
 //如果从头部获取token失败，那么从参数获取
 if (StringUtils.isEmpty(token)) {
 token = request.getParameter("token");
 }
 //如果没有获取到token
 if (StringUtils.isEmpty(token)) {
 throw new CustomerAuthenionException("token获取失
败! ");
 }
 //token验证
 if(!jwtUtils.verify(token)){
 throw new CustomerAuthenionException("token验证失
败! ");
 }
 DecodedJWT decodedJWT = jwtUtils.jwtDecode(token);
 Map<String, Claim> claims = decodedJWT.getClaims();
 String username = claims.get("username").asString();
 String userType = claims.get("userType").asString();
 //查询用户信息,交给spring security
 UserDetails details =
customizedUserDetailService.loadUserByUsername(username + ":" +
userType);
 //构造token
 UsernamePasswordAuthenticationToken authenticationToken =
new
UsernamePasswordAuthenticationToken(details,null,details.getAutho
rities());
 authenticationToken.setDetails(new
WebAuthenticationDetailsSource().buildDetails(request));
 //设置到spring security上下文

 SecurityContextHolder.getContext().setAuthentication(authenticat
ionToken);
}
}

```

在application-dev.yml添加token放行的配置

```

#不需要token验证的请求，逗号分隔配置
ignore:
 url: /api/login/login,/api/login/image

```

## 25.5.3 SpringSecurityConfig配置类配置token过滤器

```
@Autowired
CheckTokenFilter checkTokenFilter;

@Override
protected void configure(HttpSecurity http) throws Exception {
 // 解决跨域问题
 http.cors().and().headers().frameOptions().disable();
 // 关闭跨域请求伪造
 http.csrf().disable().authorizeRequests() ExpressionUrlAuthorizationConfigurer<...>.ExpressionInterceptUrlRegistry
 // 放行登录、验证码请求, 其他的所有请求都要认证
 .antMatchers(...antPatterns: "/api/login/image", "/api/login/login").permitAll()
 // 其他的任何接口访问都需要认证
 .anyRequest().authenticated()
 .and().exceptionHandling() ExceptionHandlingConfigurer<HttpSecurity>
 .authenticationEntryPoint(LoginFailureHandler)
 .accessDeniedHandler(customAccessDeineHandler);
 // 配置token过滤器
 http.addFilterBefore(checkTokenFilter, UsernamePasswordAuthenticationFilter.class);
}
```

## 25.5.4 前端http/index.ts的头部添加token, 携带给后端验证

```
import axios, { AxiosInstance, AxiosRequestConfig, AxiosResponse, AxiosRequestHeaders } from "axios";
import { ElMessage } from 'element-plus';
import { userStore } from "@/store/user";

// 拦截器
private interceptors() {
 // axios发送请求之前的处理
 this.instance.interceptors.request.use((config: AxiosRequestConfig) => {
 // 在请求头部携带token
 // let token = sessionStorage.getItem('token');
 // let token = 'eyJhbGciOiJIUzUxMiJ9.eyJleHAiOjE2NzE5ODMwMzksInN1YiI6ImFkbWluIiwiaWF0IjoxNjUzOTgzMDM5NTQzfQ.Ai_Lcq0tE7-G
 const store = userStore()
 // 在请求的头部添加token
 let token = store.getToken; // 可以从cookies、sessionStorage里面
 if (token) {
 config.headers!['token'] = token
 // 把token放到headers里面
 // (config.headers as AxiosRequestHeaders).token = token
 }
 console.log(config)
 return config;
 });
}
```

测试：看看验证token过滤器是否生效，发送请求是否携带token。

## 25.6 密码加密，退出登录改进

### 25.6.1 修改MemberController

修改MemberController新增的方法：

```
@Autowired
PasswordEncoder passwordEncoder;

// 新增
@PostMapping
public ResultVo add(@RequestBody Member member){
 // 判断卡号是否重复
 QueryWrapper<Member> query = new QueryWrapper<>();
 query.lambda().eq(Member::getUsername, member.getUsername());
 Member one = memberService.getOne(query);
 if(one != null){
 return ResultUtils.error(msg: "会员卡号被占用!");
 }
 member.setPassword(passwordEncoder.encode(member.getPassword()));
 memberService.addMember(member);
 return ResultUtils.success(msg: "新增成功!");
}
```

## 25.6.2 修改SysUserController

```
@Autowired
PasswordEncoder passwordEncoder;

// 新增员工
no usages
@PostMapping
public ResultVo addUser(@RequestBody SysUser sysUser){
 // 判断用户名是否存在
 QueryWrapper<SysUser> query = new QueryWrapper<>();
 query.lambda().eq(SysUser::getUsername, sysUser.getUsername());
 SysUser one = sysUserService.getOne(query);
 if(one != null){
 return ResultUtils.error(msg: "账户已经被占用");
 }

 // 密码加密
 /*if(StringUtils.isEmpty(sysUser.getPassword())){
 sysUser.setPassword(DigestUtils.md5DigestAsHex(sysUser.getPassword().getBytes()));
 }*/
 sysUser.setPassword(passwordEncoder.encode(sysUser.getPassword()));
 sysUser.setIsAdmin("0");
 sysUser.setCreateTime(new Date());
 // 存入数据库
 boolean save = sysUserService.save(sysUser);
 if(save){
 return ResultUtils.success(msg: "新增用户成功!");
 }
 return ResultUtils.error(msg: "新增用户失败!");
}
```

## 25.6.3 修改HomeController

修改HomeController里面的重置密码和修改密码的方法

```
@Autowired
PasswordEncoder passwordEncoder;

// 重置密码
no usages
@PostMapping("/resetPassword")
public ResultVo resetPassword(@RequestBody ResetPassword resetPassword){
 if(resetPassword.getUserType().equals("1")){
 Member member = new Member();
 member.setMemberId(resetPassword.getUserId());
 member.setPassword(passwordEncoder.encode(charSequence: "666666"));
 memberService.updateById(member);
 return ResultUtils.success(msg: "密码重置成功!");
 }else if(resetPassword.getUserType().equals("2")){
 SysUser user = new SysUser();
 user.setUserId(resetPassword.getUserId());
 //String password = DigestUtils.md5DigestAsHex("666666".getBytes());
 user.setPassword(passwordEncoder.encode(charSequence: "666666"));
 sysUserService.updateById(user);
 return ResultUtils.success(msg: "密码重置成功!");
 }else{
 return ResultUtils.error(msg: "用户类型错误!");
 }
}
```

```
//修改密码
no usages
@PostMapping("/updatePassword")
public ResultVo updatePassword(@RequestBody ResetPassword resetPassword) {
 if (resetPassword.getUserType().equals("1")) {
 //验证原密码是否正确
 Member member1 = memberService.getById(resetPassword.getUserId());
 //获取数据库原密码
 String dbPassword = member1.getPassword();
 /*if (!member1.getPassword().equals(resetPassword.getOldPassword())) {
 return ResultUtils.error("原密码不正确!");
 }*/
 if (!passwordEncoder.matches(resetPassword.getOldPassword(), dbPassword)) {
 return ResultUtils.error(msg: "原密码不正确!");
 }
 //修改
 Member member = new Member();
 member.setMemberId(resetPassword.getUserId());
 //member.setPassword(resetPassword.getPassword());
 member.setPassword(passwordEncoder.encode(resetPassword.getPassword()));
 memberService.updateById(member);
 return ResultUtils.success(msg: "修改密码成功!");
 } else if (resetPassword.getUserType().equals("2")) {
 //验证原密码
 SysUser sysUser = sysUserService.getById(resetPassword.getUserId());
 /*String oldPas = DigestUtils.md5DigestAsHex(resetPassword.getOldPassword().getBytes());
 if(!oldPas.equals(sysUser.getPassword())){
 return ResultUtils.error("原密码不正确!");
 }*/
 String dbPassword = sysUser.getPassword();
 if (!passwordEncoder.matches(resetPassword.getOldPassword(), dbPassword)) {
 return ResultUtils.error(msg: "原密码不正确!");
 }
 SysUser user = new SysUser();
 user.setUserId(resetPassword.getUserId());
 //String password = DigestUtils.md5DigestAsHex(resetPassword.getPassword().getBytes());
 user.setPassword(passwordEncoder.encode(resetPassword.getPassword()));
 sysUserService.updateById(user);
 return ResultUtils.success(msg: "修改密码成功!");
 } else {
 return ResultUtils.error(msg: "用户类型错误!");
 }
}
```

在HomeController里面定义退出登录的方法：

```
/**
 * 退出登录
 */
no usages
@PostMapping("/loginOut")
public ResultVo loginOut(HttpServletRequest request, HttpServletResponse response){
 Authentication authentication = SecurityContextHolder.getContext().getAuthentication();
 if(authentication != null){
 new SecurityContextLogoutHandler().logout(request, response, authentication);
 }
 return ResultUtils.success(msg: "退出登录成功!");
}
```

## 25.6.4 退出登录--改造前端

(1) 在api/home/index.ts里面定义如下：

```
//退出登录
export const logiOutApi = ()=>{
 return http.post("/api/home/loginOut")
}
```

## (2) 调整Logout.vue组件:

```
import { updatePasswordApi, logoutApi } from "@/api/home";
const { dialog, onClose, onConfirm, onShow } = useDialog();
const { global } = useInstance();
const upForm = ref<FormInstance>();
const store = userStore();
const loginOut = async () => {
 const confirm = await global.$myconfirm("确定退出登录吗? ");
 if (confirm) {
 let res = await logoutApi()
 if(res && res.code == 200){
 //清空用户信息
 store.setToken("");
 store.setUserId("");
 store.setUserType("");
 localStorage.clear();
 window.location.href = "/login";
 }
 }
}
```

分别使用会员、员工账号登录，测试重置密码、修改密码、退出登录的功能。